

PwC Technology Interview Preparation

Modern Data Systems, SQL, Databases, APIs & Data Integration

A complete, interview-oriented study guide built on the seven modules of the Tekstac *Modern Data Systems* course. Each session is re-taught in simple language, with diagrams, PostgreSQL examples that were actually run, interview questions with model answers, practice sets and a final revision kit.

PART I	Introduction to Modern Data Systems
PART II	Data Modeling Fundamentals
PART III	Basics of SQL (Foundational) · SQL Interview Patterns
PART IV	Cloud Databases & Storage Systems
PART V	Data Integration & APIs
PART VI	Integration Patterns in the Enterprise
PART VII	Modern Trends & Future Directions
FINAL	Revision Kit · 100 Questions · SQL Bank · One-Day Sheet

399

INTERVIEW Q&AS

202

SQL EXAMPLES RUN ON
POSTGRESQL

40

DIAGRAMS

125

PRACTICE QUESTIONS

Contents

START HERE	How to Use This Handbook	4
	What Is Inside	4
	How Every Session Is Built	4
	Labels You Will See	5
	Study Plans	5
	About the Source Material	6
START HERE	Course Coverage Map	7
START HERE	PwC Interview Orientation	11
	Four Kinds of Information — Keep Them Separate	11
	A. Common Software and Database Questions	11
	B. Reported Candidate Experiences	11
	C. Likely Topics for Technology and Data Roles	12
	D. Recommended Preparation	13
PART I • MODULE 1	Introduction to Modern Data Systems	14
	Session 1.0 — Recommended Prerequisite: Data and Database Basics	15
	Session 1.1 — Modern Data Systems & Integration (Part 1): What a Data System Is	18
	Session 1.2 — Modern Data Systems & Integration (Part 2): How Data Moves Between Systems	23
	Session 1.3 — Modern Data Systems & Integration (Part 3): Processing, OLTP vs OLAP, and Why One Database Is Not Enough	26
	Session 1.4 — Module 1 Summary & Rapid Revision	32
PART II • MODULE 2	Data Modeling Fundamentals	33
	Session 2.1 — What Is Data Modeling?	34
	Session 2.2 — Entities, Attributes and Relationships (ER Modeling)	37
	Session 2.3 — Keys and Constraints	40
	Session 2.4 — The Relational Model, Schema and Table Design	46
	Session 2.5 — Normalization	51
	Session 2.6 — Denormalization: When and Why	58
	Session 2.7 — Data Warehousing and Dimensional Modeling	62
	Session 2.8 — NoSQL Data Modeling	70
	Session 2.9 — Module 2 Summary & Rapid Revision	76
PART III • MODULE 3	Basics of SQL (Foundational)	77
	Session 3.0 — Meet the Sample Database	78
	Session 3.1 — PostgreSQL Basics (Part 1)	81
	Session 3.2 — PostgreSQL Basics (Part 2)	90
	Session 3.3 — DDL: Data Definition Language	102
	Session 3.4 — DML: Data Manipulation Language (plus DCL and TCL)	108
	Session 3.5 — Grouping and Aggregation: GROUP BY and HAVING	116
	Session 3.6 — Joins	123
	Session 3.7 — Subqueries and CTEs	137
	Session 3.8 — Window Functions & Indexing (A): Window Functions	151
	Session 3.9 — Window Functions & Indexing (B): Indexing	164
	Session 3.10 — Transactions and ACID	172
	Session 3.11 — SQL Interview Patterns (Practice Programs)	180
	Pattern 1 — Filtering	181
	Pattern 2 — Aggregation	181
	Pattern 3 — GROUP BY + HAVING	182

Pattern 4 — Joining Multiple Tables	182
Pattern 5 — Find Duplicates	183
Pattern 6 — Remove Duplicates	184
Pattern 7 — Nth Highest Salary	185
Pattern 8 — Second Highest Salary	186
Pattern 9 — Top N per Group	187
Pattern 10 — Rows Above Their Group's Average	188
Pattern 11 — Customers With No Orders	189
Pattern 12 — Latest Record per User	190
Pattern 13 — Finding Missing Records	191
Pattern 14 — Ranking	191
Pattern 15 — Running Totals	192
Pattern 16 — Previous / Next Row (LAG and LEAD)	193
Pattern 17 — Date-Based Analysis	194
Pattern 18 — Conditional Aggregation	195
Pattern 19 — Subquery Problems	196
Pattern 20 — CTE Problems (Multi-Step Logic)	196
Pattern 21 — Self Joins	197
Pattern 22 — Window-Function Problems (Gaps and Islands)	199
Session 3.12 — Module 3 Summary & Rapid Revision	205
PART IV • MODULE 4 Cloud Databases & Storage Systems	207
Session 4.1 — Introduction to Cloud Databases	208
Session 4.2 — SQL vs NoSQL: Types of Databases	214
Session 4.3 — NoSQL Fundamentals and the CAP Theorem	219
Session 4.4 — MongoDB: Core Concepts	224
Session 4.5 — MongoDB: CRUD Operations	228
Session 4.6 — MongoDB: Indexing, Aggregation and Data Modeling	234
Session 4.7 — Data Storage Systems: Database, Warehouse, Lake, Lakehouse	240
Session 4.8 — Module 4 Summary & Rapid Revision	246
PART V • MODULE 5 Data Integration & APIs	248
Session 5.1 — What Is an API? Client–Server and Request/Response	249
Session 5.2 — HTTP and REST Fundamentals	252
Session 5.3 — Designing and Consuming REST APIs	260
Session 5.4 — API Security: Authentication and Authorization	267
Session 5.5 — API Integration and Reliability	273
Session 5.6 — Data Integration	279
Session 5.7 — Module 5 Summary & Rapid Revision	287

START HERE

How to Use This Handbook

This handbook replaces the course videos for interview preparation. It follows the seven modules of the Tekstac *Modern Data Systems* course, re-teaches every topic in plain English, and adds what interviewers usually ask on top of the course.

You do not need to watch the videos first. Read a session, run through its interview questions out loud, try the practice set, and tick the checklist at the end of the module.

What Is Inside

Part	Module	What you will be able to do after it
I	Introduction to Modern Data Systems	Explain how an app stores, moves and analyses data; OLTP vs OLAP; batch vs real-time
II	Data Modeling Fundamentals	Design tables, choose keys, normalize to 3NF, design a star schema, model documents
III	Basics of SQL (Foundational)	Write SELECT, joins, GROUP BY, subqueries, CTEs, window functions; explain indexes and transactions; solve 22 classic SQL interview patterns
IV	Cloud Databases & Storage Systems	Choose SQL vs NoSQL, explain CAP, use MongoDB CRUD, compare warehouse, lake and lakehouse
V	Data Integration & APIs	Explain a REST request end to end, HTTP methods and status codes, auth, retries, idempotency
VI	Integration Patterns in the Enterprise	Compare APIs vs queues, pub/sub, event-driven design, ETL vs ELT, batch vs streaming
VII	Modern Trends & Future Directions	Discuss data mesh, serverless and AI-ready data critically, with trade-offs
Final	Final Interview Revision	100 key questions, a 50-question SQL bank, scenario questions, rapid fire, a one-day revision sheet and an answering strategy

How Every Session Is Built

Every session follows the same rhythm, so you always know where to look:

1. **The idea in simple words**, then *why it exists*, *how it works*, an example and where it is used in real systems.
2. **Diagrams and tables** wherever a picture is faster than a paragraph.
3. **Code with real output**. Every SQL query marked with an *Output* panel was actually run on PostgreSQL while this PDF was built. Important queries are followed by a **Line by Line** explanation.
4. **How I Would Explain This in an Interview**: a natural 30–60 second spoken answer.
5. **Common Traps & Mistakes**: the confusions that cost candidates marks.
6. **Interview Questions** in four groups (Basic, Intermediate, Scenario-based, Follow-up/Trap), each with a short model answer.
7. **Practice Questions** at three levels, with the answers kept in a separate **Answer / Explanation** box so you can test yourself first.
8. At the end of each module: a **Module Summary** and a **Rapid Revision Checklist**.

Labels You Will See

Label	Meaning
DEFINITION	"What is X?" questions. Answer in one or two clean sentences.
WHY	"Why do we need X?" Talk about the problem it solves.
HOW	"How does X work?" Walk through the mechanism step by step.
COMPARISON	"X vs Y". Use a small table in your head: purpose, how, when to use.
SCENARIO	A real situation. Ask a clarifying question, then propose and justify.
SQL PROBLEM	Write a query. Think aloud, start simple, then refine.
DESIGN QUESTION	Design a small system or schema. Cover data, flow and trade-offs.
TRAP QUESTION	A question designed to catch a common misunderstanding.
EASY MEDIUM HARD	Difficulty of a SQL pattern or practice question.
HIGH PRIORITY MEDIUM PRIORITY LOWER PRIORITY	How much interview time a topic usually gets. Spend your time accordingly.
INTERVIEW EXTENSION	Useful knowledge that goes beyond the course content.

INTERVIEW EXTENSION BOXES

Purple boxes and the **INTERVIEW EXTENSION** label mark material that is **not** part of the original course but that interviewers commonly expect (for example, isolation levels, SCD types or idempotency keys). Everything else follows the course topics.

RECOMMENDED PREREQUISITE BOXES

Grey boxes mark background knowledge you need before a topic makes sense, such as "what is a database?" before data modeling.

Study Plans

Pick the plan that matches the time you have. Sessions tagged **HIGH PRIORITY** come first in every plan.

If you have...	Do this
1 day (about 8 hours)	One-Day Revision Sheet (1 h) → Module 3 Sessions 3.1–3.8 skim plus SQL Patterns 1–12 (3 h) → Sessions 2.3–2.5 keys and normalization (1 h) → Sessions 4.2–4.5 SQL vs NoSQL and MongoDB (1 h) → Sessions 5.2–5.5 REST and HTTP (45 min) → Sessions 6.2, 6.3 and 6.6 (30 min) → Module 7 summary (15 min) → 100 Most Important Questions, read aloud (30 min)
3 days	Day 1: Modules 1–2 plus their practice sets. Day 2: all of Module 3 with every SQL example typed out once. Day 3: Modules 4–7 summaries and questions, the SQL Question Bank, then the One-Day Sheet in the evening.
7 days	One module per day (Module 3 gets two days), finishing every practice set. Day 7: the Final Interview Revision part, a mock interview with a friend, and your project story.

THE SINGLE BEST HABIT

Say answers **out loud**. Reading an answer and being able to say it clearly are different skills. The "How I Would Explain This" boxes exist for that reason.

About the Source Material

This handbook was built from the module and topic structure of the Tekstac *Modern Data Systems* course as described in the preparation brief: seven modules and their listed sessions, quizzes, practice programs and subtopics. The course screenshots themselves were not available when this edition was produced, so:

- Session titles that were named explicitly (for example *PostgreSQL Basics – Part 1, DDL, DML and Window Functions & Indexing*) are used as they are.
- Where a module's sessions were described only by topic, the session titles were rebuilt from those topics and are marked † in the Course Coverage Map. The topics themselves are all covered; only the exact titles or the split into parts may differ from the videos.
- Nothing is presented as course content unless it was part of the course topic list. Extra material is labelled

INTERVIEW EXTENSION .

If your course shows a session that is not listed in the coverage map, look it up in the index-like map on the next pages by topic. Every topic from the brief is covered somewhere.

START HERE

Course Coverage Map

Use this map to check that every course session and topic is covered, and to jump straight to it. Titles marked † were reconstructed from the module's topic list (see "About the Source Material").

MODULE 1 – INTRODUCTION TO MODERN DATA SYSTEMS

Course session / topic	Covered in	Page
(Prerequisite) Data, databases and DBMS basics	Session 1.0 INTERVIEW EXTENSION	p. 15
Modern Data Systems & Integration — Part 1: data systems, how applications store data †	Session 1.1	p. 18
Modern Data Systems & Integration — Part 2: how data moves, data integration, APIs, backend systems †	Session 1.2	p. 23
Modern Data Systems & Integration — Part 3: batch vs real-time, OLTP vs OLAP, operational vs analytical data, why many data technologies †	Session 1.3	p. 26
Module 1 quiz concepts, summary and checklist	Session 1.4	p. 32

MODULE 2 – DATA MODELING FUNDAMENTALS

Course session / topic	Covered in	Page
Introduction to data modeling (conceptual, logical, physical) †	Session 2.1	p. 34
Entities, attributes and relationships; ER concepts †	Session 2.2	p. 37
Primary keys, foreign keys and constraints †	Session 2.3	p. 40
Relational model, schema, table design; one-to-one, one-to-many, many-to-many †	Session 2.4	p. 46
Normalization: functional dependency, anomalies, 1NF, 2NF, 3NF, BCNF †	Session 2.5	p. 51
Denormalization: when to normalize, when to denormalize †	Session 2.6	p. 58
Data warehousing: fact and dimension tables, star vs snowflake, OLTP vs OLAP †	Session 2.7	p. 62
NoSQL modeling: schema flexibility, embedding vs referencing, access patterns †	Session 2.8	p. 70
Module 2 quiz concepts, summary and checklist	Session 2.9	p. 76

MODULE 3 – BASICS OF SQL (FOUNDATIONAL)

Course session / topic	Covered in	Page
(Prerequisite) The sample database used in every example	Session 3.0	p. 78
PostgreSQL Basics — Part 1 (SELECT, FROM, WHERE, DISTINCT, ORDER BY, LIMIT, OFFSET, aliases, data types)	Session 3.1	p. 81
PostgreSQL Basics — Part 2 (operators, NULL, aggregate, string, date and numeric functions, CASE)	Session 3.2	p. 90
DDL (CREATE, ALTER, DROP, TRUNCATE, constraints)	Session 3.3	p. 102
DML (INSERT, UPDATE, DELETE) plus DCL and TCL	Session 3.4	p. 108
Grouping and aggregation: GROUP BY, HAVING, WHERE vs HAVING †	Session 3.5	p. 116
Joins: inner, left, right, full, cross, self; referential integrity †	Session 3.6	p. 123
Subqueries, CTEs (including recursive) †	Session 3.7	p. 137
Window Functions & Indexing — window functions	Session 3.8	p. 151
Window Functions & Indexing — indexing	Session 3.9	p. 164
Transactions, ACID, isolation and read anomalies †	Session 3.10	p. 172
Practice programs: SQL interview patterns (22 patterns)	Session 3.11	p. 180
Module 3 quiz concepts, summary and checklist	Session 3.12	p. 205

MODULE 4 – CLOUD DATABASES & STORAGE SYSTEMS

Course session / topic	Covered in	Page
Cloud databases: managed, RDS / Cloud SQL, serverless, analytical; scaling, availability, backups, replication, failover †	Session 4.1	p. 208
SQL vs NoSQL: relational, document, key-value, graph, column-family, columnar †	Session 4.2	p. 214
NoSQL fundamentals and the CAP theorem; eventual consistency †	Session 4.3	p. 219
MongoDB: database, collection, document, field, BSON †	Session 4.4	p. 224
MongoDB CRUD: insert, find, update, delete, updateMany, deleteMany †	Session 4.5	p. 228
MongoDB indexing, aggregation, embedding vs referencing †	Session 4.6	p. 234
Data storage: database, warehouse, lake, lakehouse; structured, semi-structured, unstructured data †	Session 4.7	p. 240
Module 4 quiz concepts, summary and checklist	Session 4.8	p. 246

MODULE 5 – DATA INTEGRATION & APIS

Course session / topic	Covered in	Page
What an API is; client-server; request/response †	Session 5.1	p. 249
HTTP and REST: methods, status codes, JSON †	Session 5.2	p. 252
Designing and consuming REST APIs: pagination, rate limiting, versioning †	Session 5.3	p. 260
Authentication and authorization †	Session 5.4	p. 267
API integration and external APIs: sync vs async, failures, retries, idempotency †	Session 5.5	p. 273
Data integration †	Session 5.6	p. 279
Module 5 quiz concepts, summary and checklist	Session 5.7	p. 287

MODULE 6 – INTEGRATION PATTERNS IN THE ENTERPRISE

Course session / topic	Covered in	Page
Enterprise integration and system-to-system communication; integration patterns †	Session 6.1	p.
Synchronous vs asynchronous; API vs message queue †	Session 6.2	p.
Messaging: queues, publish/subscribe, producers, consumers †	Session 6.3	p.
Event-driven architecture †	Session 6.4	p.
Data pipelines; batch vs real-time processing †	Session 6.5	p.
ETL vs ELT †	Session 6.6	p.
Streaming †	Session 6.7	p.
Salesforce and cloud data warehouse integration †	Session 6.8	p.
Choosing an integration pattern INTERVIEW EXTENSION	Session 6.9	p.
Module 6 quiz concepts, summary and checklist	Session 6.10	p.

MODULE 7 – MODERN TRENDS & FUTURE DIRECTIONS

Course session / topic	Covered in	Page
Data mesh †	Session 7.1	p.
Serverless †	Session 7.2	p.
AI and data management †	Session 7.3	p.
Modern data systems: real-time analytics, event-driven, cloud-native, distributed, data platforms †	Session 7.4	p.
Thinking critically about trends INTERVIEW EXTENSION	Session 7.5	p.
Module 7 summary and checklist	Session 7.6	p.

FINAL INTERVIEW REVISION

Section	Page
100 Most Important Questions	p.
SQL Question Bank — Top 50 SQL Questions	p.
Top Database Questions	p.
Top Scenario Questions	p.
Rapid-Fire Questions	p.
Project Connection	p.
One-Day Revision Sheet	p.
Interview Answering Strategy	p.
Glossary · Coverage Verification	p.

QUIZZES AND PRACTICE PROGRAMS

The course modules include quizzes and practice programs. Their concepts are covered by the **Interview Questions** and **Practice Questions** boxes in each session, by Session 3.11 (SQL patterns, which mirror typical SQL practice programs) and by the SQL Question Bank in the final part.

START HERE

PwC Interview Orientation

This chapter explains what is publicly known about PwC technology interviews in India, what is only reported by candidates, and how to prepare. It does not invent "official PwC questions".

Four Kinds of Information — Keep Them Separate

Category	What it means	How much to trust it
A. Common software and database questions	Questions asked across the industry for any software or data role (joins, normalization, ACID, REST...).	Very reliable. These are the core of every technical interview.
B. Reported candidate experiences REPORTED	What individual candidates wrote about their own PwC interviews on public sites.	Useful signal, but each report is one person, one year, one role, one campus.
C. Likely topics for the role LIKELY	Topics that fit the job description of technology, data and consulting roles.	Reasonable inference, not a promise.
D. Recommended preparation RECOMMENDED	This handbook's advice on what to study and how.	Our judgement, based on A–C.

DO NOT OVER-TRUST "PWC QUESTION LISTS"

Websites that publish "PwC interview questions" are usually generic question banks. PwC has many business units (Advisory, Tax, Assurance, the Acceleration Centers) and many roles, and the process changes from year to year. Prepare the fundamentals well and you will be ready for any variation.

A. Common Software and Database Questions

These appear in almost every entry-level technical interview, including consulting firms. They are the backbone of this handbook:

- **SQL:** write a query with a join, GROUP BY with HAVING, the second-highest salary, duplicates, top N per group, a window function.
- **DBMS:** primary vs foreign key, normalization (1NF–3NF), ACID, transactions, indexes, DELETE vs TRUNCATE vs DROP.
- **Data modeling:** design tables for a small business problem (library, hospital, e-commerce).
- **APIs:** what is REST, HTTP methods, status codes, PUT vs PATCH, authentication vs authorization.
- **Programming and OOP:** the four pillars of OOP, one language in depth, basic data structures.
- **Your project:** architecture, your role, a problem you solved, what you would improve.

B. Reported Candidate Experiences

The following themes come from public interview write-ups (sources at the end of this chapter). They were collected from search summaries in October 2026; individual details vary a lot.

Theme	What candidates reported REPORTED
Process	An online assessment, then a technical interview, then a managerial or partner round, then HR. Some write-ups describe a "Launchpad" programme and a hackathon before the interviews.
Online assessment	Aptitude sections (quantitative, logical reasoning, verbal/English). Some reports also mention technical multiple-choice questions on OOP, DBMS, networking and data structures.
Technical round	Often around 30 minutes, frequently resume- and project-based. SQL joins and their types, normalization, ACID properties and basic DBMS terms appear repeatedly.
Practical SQL	One report describes being asked to create a table, alter it to add columns, create a table from existing tables and write a GROUP BY query.
Programming	Easy-to-medium problems: sort an array without a built-in sort, swap two numbers without a third variable, list vs tuple and dictionaries in Python, OOP concepts.
APIs and backend	Reports for software roles mention HTTP methods, status codes, query vs path parameters, PUT vs PATCH, idempotency, REST and SOLID principles.
Data roles	Experienced data-engineering candidates report scenario-based questions on SQL, Spark, Databricks and Power BI.
Final rounds	The partner or managerial round may be technical or about fit and attitude. HR questions include "Why PwC?".

C. Likely Topics for Technology and Data Roles

Based on the role descriptions and the reports above, a technology or data candidate should expect **LIKELY** :

1. **SQL writing on the spot**, from simple filters to joins, grouping, subqueries and window functions.
2. **DBMS concepts** asked as "what is" and "difference between" questions.
3. **Designing a small schema** and normalizing it.
4. **SQL vs NoSQL** and when to use MongoDB.
5. **How a web request flows** through frontend, API and database.
6. **Cloud and data basics**: managed databases, warehouse vs lake, ETL vs ELT.
7. **Your project**, explained with clear architecture and your contribution.
8. **Consulting behaviours**: communication, teamwork, client focus and learning attitude.

PWC'S OWN FRAMEWORK FOR BEHAVIOURAL QUESTIONS

PwC publicly describes the **PwC Professional** framework with five dimensions: **Whole leadership, Business acumen, Technical and digital, Global and inclusive**, and **Relationships**. Prepare one short story for each (use STAR: Situation, Task, Action, Result). For example: a project where you learned a new tool quickly (*Technical and digital*), or a team conflict you resolved (*Relationships*).

D. Recommended Preparation

Priority	Study	Where in this handbook
1	SQL fundamentals and the 22 interview patterns	Module 3, Session 3.11, SQL Question Bank
2	Keys, normalization, ACID, indexes, transactions	Sessions 2.3–2.5, 3.9, 3.10
3	REST, HTTP, authentication, request flow	Module 5
4	SQL vs NoSQL, MongoDB basics, CAP	Sessions 4.2–4.6
5	ETL/ELT, warehouses, lakes, queues, event-driven basics	Sessions 2.7, 4.7, Module 6
6	Modern trends, at a "can discuss sensibly" level	Module 7
7	Your project story and behavioural stories	Project Connection, Interview Answering Strategy

HOW PWC INTERVIEWS FEEL, ACCORDING TO MOST REPORTS

Fundamentals are tested more than tricks. Interviewers often start from your resume and keep asking "why?" until they find the edge of your understanding. Being honest ("I have not used that, but I understand it as...") is better than guessing.

SOURCES CONSULTED (VIA WEB SEARCH, OCTOBER 2026)

- PwC Acceleration Centers India — *Entry Level Campus Launchpad Program* and *AC India Assurance Launchpad Program* pages (jobs-ta.pwc.com). The programme page describes hands-on training in areas such as IT, SQL, data engineering and Python.
- PwC India — *Want to be a PwCite?* campus careers page (pwc.in): register → pre-placement talk → assessment → interview → final selection.
- PwC — *The PwC Professional / The skills we look for* (pwc.com).
- GeeksforGeeks — PwC India interview experiences for Technology Consultant, Technology Consulting Intern (2023) and Associate Engineer (on-campus) roles; *PwC Recruitment Process*.
- Naukri Code360 — PwC AC India interview experience (on-campus, October 2025).
- Glassdoor — PwC Associate Software Engineer interview questions (India).
- Medium — "PwC India Interview Experience" (S. Palit), a data-engineering interview.
- InterviewBit and PrepInsta — PwC interview process summaries.

These sources describe individual experiences and change over time. Treat them as signals, not as a syllabus.

PART I · MODULE 1

Introduction to Modern Data Systems

This module gives you the big picture: what a "data system" is, how a modern application stores and moves data, and why companies end up using several kinds of databases at once. The rest of the handbook zooms into each piece.

COURSE COVERAGE

- ✓ **Modern Data Systems & Integration — Part 1** †: what a data system is, how modern applications store data, applications and databases, backend systems → Session 1.1
- ✓ **Modern Data Systems & Integration — Part 2** †: how data moves between systems, data integration, APIs, data processing → Session 1.2
- ✓ **Modern Data Systems & Integration — Part 3** †: batch vs real-time, transactional vs analytical systems (OLTP vs OLAP), operational data, why many data technologies → Session 1.3
- ✓ Module quiz concepts → interview questions in every session and Session 1.4
- ✓ *Recommended prerequisite (not course content):* data, database and DBMS basics → Session 1.0

Interview priority	Why
MEDIUM PRIORITY overall	Rarely asked as "theory", but the vocabulary (OLTP, OLAP, batch, streaming, integration) appears in many answers.
HIGH PRIORITY OLTP vs OLAP, batch vs streaming	Classic comparison questions.
HIGH PRIORITY "Explain how your app works end to end"	Almost every project discussion turns into this.

HOW THIS MODULE IS TESTED

- **DEFINITION** "What is a data system / data integration?"
- **COMPARISON** "OLTP vs OLAP", "batch vs real-time"
- **HOW** "Walk me through what happens when a user places an order."
- **DESIGN QUESTION** "Which databases would you use for a food-delivery app, and why?"

Session 1.0 — Recommended Prerequisite: Data and Database Basics

WHY THIS SESSION EXISTS

The course assumes you already know what a database is. Interviewers often start right here ("What is a DBMS?"), so a ten-minute refresher pays off. This session is an **INTERVIEW EXTENSION**, not a course video.

Data vs information

Data is raw facts: 42, Pune, 2024-03-05. **Information** is data with meaning: "Order 42 was delivered to Pune on 5 March 2024." Databases store data; reports and applications turn it into information.

Metadata is "data about data": the column name `order_date`, its type `DATE`, who created the table, when it was last updated. You will meet metadata again in Module 7, where it becomes very important for AI.

Why not just use files or Excel?

Imagine a college keeps student records in Excel files, one file per department. It works for a while, then problems appear:

Problem with files	What goes wrong	How a database solves it
Duplication	The same student's phone number is typed in three files	Store each fact once and link it with keys
Inconsistency	One file is updated, the others are not	One source of truth; constraints keep it valid
Concurrent access	Two people edit the same file and overwrite each other	Locking and transactions let many users work safely
Security	Anyone with the file sees everything	Users, roles and permissions per table or row
Crash recovery	The laptop dies mid-save and the file is corrupted	Logs and transactions recover to a consistent state
Searching	Finding "all students from Pune with CGPA > 8" is slow and manual	A query language (SQL) and indexes

Database, DBMS and RDBMS

- A **database** is an organised collection of related data, for example everything about a college's students, courses and marks.
- A **DBMS (Database Management System)** is the software that stores, retrieves and protects that data: it handles queries, users, security, backups and crashes. Examples: PostgreSQL, MySQL, Oracle, SQL Server, MongoDB.
- An **RDBMS (Relational DBMS)** stores data in **tables** (relations) made of **rows** and **columns**, and links tables using **keys**. PostgreSQL, MySQL, Oracle and SQL Server are relational. MongoDB is a DBMS but *not* relational: it stores documents.

The vocabulary of a table

STUDENTS

student_id	name	city	cgpa
1	Asha Rao	Pune	8.6
2	Vikram Shah	Delhi	7.9

Term	Meaning in this table	Other names
Table	students	relation
Row	one student, e.g. 1, Asha Rao, Pune, 8.6	record, tuple
Column	city	field, attribute
Schema	the table's design: column names, types and rules	structure
Primary key	student_id : uniquely identifies each row	—

SQL (Structured Query Language) is the standard language for working with relational databases: `SELECT` to read, `INSERT`, `UPDATE` and `DELETE` to change data, `CREATE TABLE` to define structure. Module 3 covers it in depth.

“HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"A DBMS is the software that manages a database. Instead of keeping data in loose files, it lets many users store and query data safely: it gives us a query language, enforces rules like unique IDs, controls who can see what, and recovers after a crash. A relational DBMS such as PostgreSQL stores data in tables and connects them with keys. For example, in a college system we keep students and courses in separate tables and link them through enrollments."

COMMON TRAPS & MISTAKES

- Saying "SQL is a database." SQL is a *language*; PostgreSQL and MySQL are databases (DBMS software).
- Saying "MongoDB is an RDBMS." It is a DBMS, but document-oriented, not relational.
- Mixing up a *database* (the data) with a *DBMS* (the software that manages it).

INTERVIEW QUESTIONS

BASIC

Q1 DEFINITION What is a DBMS?

A Software that stores, retrieves and manages data in a database, handling queries, security, concurrency and recovery. Examples: PostgreSQL, MySQL, Oracle.

Q2 COMPARISON What is the difference between a DBMS and an RDBMS?

A An RDBMS is a type of DBMS that stores data in related tables and uses keys and SQL. Every RDBMS is a DBMS, but not every DBMS is relational (MongoDB and Redis are not).

Q3 DEFINITION What are a row, a column and a schema?

A A row is one record (one student). A column is one attribute (city). The schema is the design of the table: its columns, data types and rules.

INTERMEDIATE

Q4 **WHY** What are the advantages of a DBMS over a file system?

A Less duplication, consistent data, safe concurrent access, security and permissions, backup and crash recovery, and fast searching with a query language and indexes.

Q5 **COMPARISON** Data vs information vs metadata?

A Data is raw facts, information is data with context and meaning, and metadata describes the data itself (column names, types, owner, last-updated time).

SCENARIO-BASED

Q6 **SCENARIO** A small shop tracks orders in Excel and keeps losing changes when two staff edit at once. What would you suggest?

A Move the data into a relational database (even a small managed PostgreSQL). It gives concurrent access with transactions, one source of truth, backups, and the ability to build a simple app or report on top of it.

FOLLOW-UP / TRAP

Q7 **TRAP QUESTION** Is Excel a database?

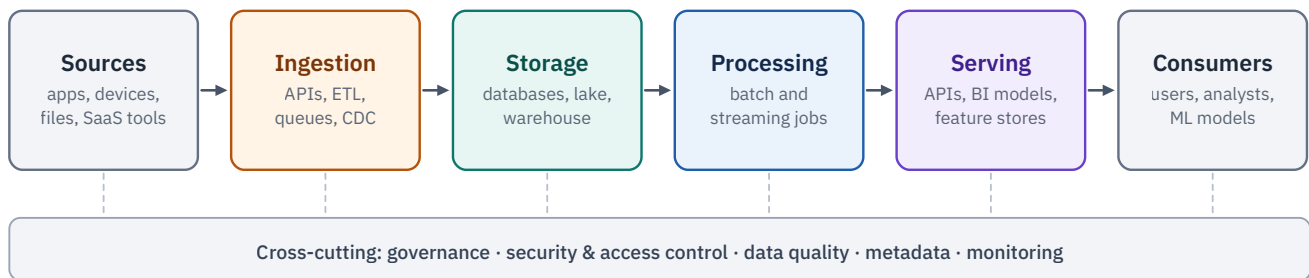
A Not really. It stores data in a grid, but it has no real concurrency control, transactions, enforced relationships or fine-grained security. It is a spreadsheet tool, fine for analysis, weak as a system of record.

Session 1.1 — Modern Data Systems & Integration (Part 1): What a Data System Is

What is a data system?

Simple meaning: a data system is *everything* an organisation uses to collect, store, process, move and serve data. It is not one database. It is the whole chain from the moment data is created (a customer taps "Pay") to the moment someone uses it (a manager reads a sales dashboard, or a model flags fraud).

Why we need the idea: in a real company, data is created in many places and used in many others. Thinking in terms of a *system* helps you see where data comes from, where it goes, what can break and who is responsible.



Data flows left to right; every stage must also be secured, monitored and documented.

Figure 1. The building blocks of a data system. Every real system has some version of each stage.

Building block	Plain-English job	Real examples
Sources	Where data is born	mobile app, website, payment gateway, IoT sensor, Excel upload, Salesforce
Ingestion	Bringing data in	REST APIs, nightly file transfers, message queues, change data capture (CDC)
Storage	Keeping data safely	PostgreSQL, MongoDB, object storage (S3), data warehouse, data lake
Processing	Cleaning, combining, calculating	SQL jobs, Spark jobs, stream processors
Serving	Making data usable	APIs, dashboards, reports, ML feature stores
Governance	Rules around all of it	access control, data quality checks, privacy, metadata catalogue, monitoring

A CITY'S WATER SYSTEM

Water is collected from rivers (**sources**), pumped through pipes (**ingestion and integration**), stored in reservoirs (**storage**), treated and filtered (**processing**), and delivered to taps (**serving**). Inspectors test quality all along the way (**governance**). If any stage fails, people at the tap notice. A data system works the same way.

How modern applications store data

Almost every app you have used (food delivery, banking, e-commerce, job portals) has the same basic shape:

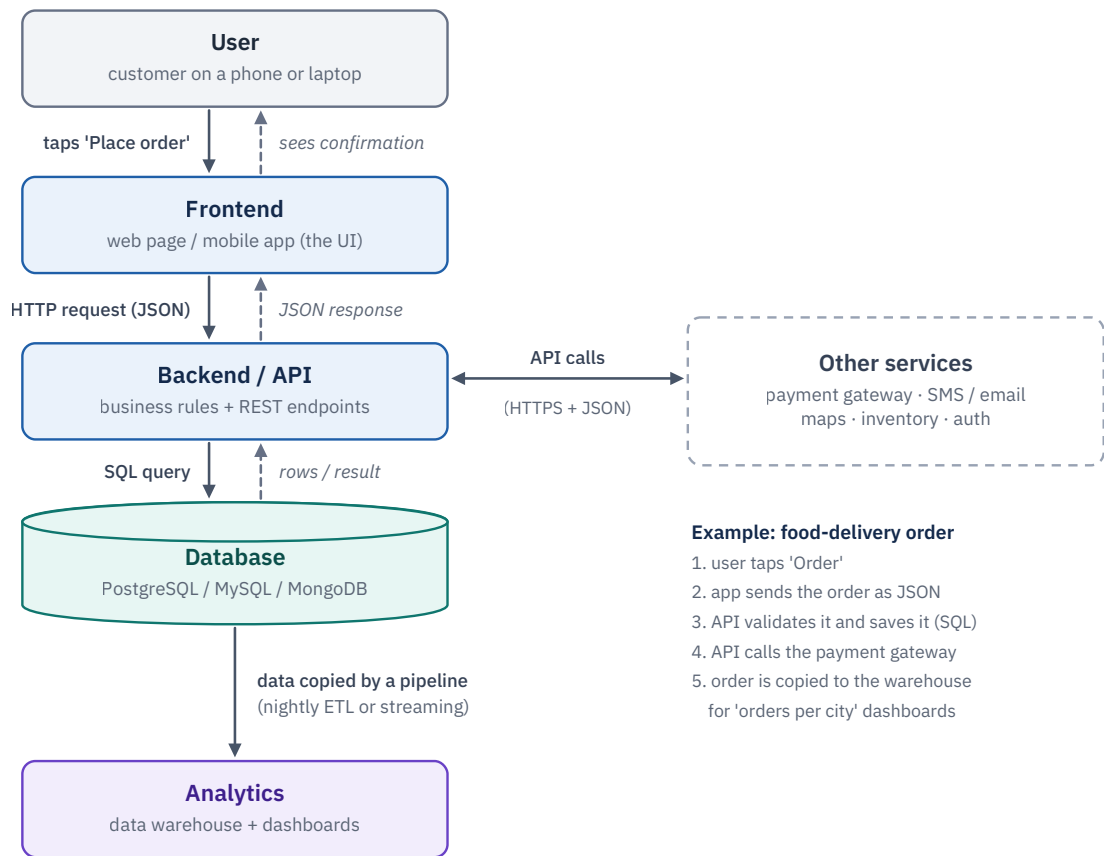


Figure 2. The typical layers of a modern application, using a food-delivery order as the example.

Layer	What it does	Example technologies
User	Interacts with the app	phone, laptop
Frontend	The screens. Collects input, shows results. Holds no permanent data of its own.	React, Angular, Android, iOS
Backend / API	The brain. Validates input, applies business rules, talks to the database and to other services. Exposes REST endpoints such as <code>POST /orders</code> .	Spring Boot, Node.js, Django, .NET
Database	The memory. Stores data permanently and safely.	PostgreSQL, MySQL, MongoDB
Other services	Specialist systems the backend calls	payment gateway, SMS/email, maps, login provider
Analytics	A copy of the data, organised for reporting and analysis	data warehouse + dashboards (Power BI, Tableau)

Why the frontend never talks to the database directly: security (you would have to put database passwords in the app), control (business rules must run on the server, where users cannot tamper with them), and flexibility (the database can change without updating every phone).

Applications and databases: how the backend talks to the database

1. The backend opens a **connection** to the database (in practice it keeps a *connection pool*, a set of ready connections, because opening one per request is slow).
2. It sends **SQL** queries, either written by hand or generated by an **ORM** (Object-Relational Mapper, such as Hibernate/JPA in Java) that turns objects into SQL.

- For changes that must happen together (create order + reduce stock), it wraps the queries in a **transaction** so either all succeed or none do.
- It turns the results into objects and then into **JSON** for the frontend.

What a backend system really does

- **Validation:** is the quantity positive? Is the email well-formed?
- **Business rules:** apply a coupon, calculate delivery charges, check stock.
- **Security:** check who the user is (authentication) and what they may do (authorization).
- **Orchestration:** call the payment gateway, then save the order, then send a notification.
- **Data access:** read and write the database efficiently and safely.

Where data lives in a modern app

One application often uses several storage technologies, each for what it does best:

Kind of data	Typical store	Why
Orders, payments, accounts	Relational database (PostgreSQL)	Needs correctness, relationships, transactions
Sessions, cached pages, OTPs	In-memory key-value store (Redis)	Extremely fast, data can expire
Product images, invoices, videos	Object storage (Amazon S3, Azure Blob)	Cheap for large files
Full-text search ("red shoes size 9")	Search engine (Elasticsearch, OpenSearch)	Ranked, fuzzy text search
Business reports	Data warehouse (Snowflake, BigQuery, Redshift)	Fast analytics over years of history
Events between services	Message broker (Kafka, RabbitMQ)	Decouples services, buffers spikes
Application logs	Log platform (ELK, CloudWatch)	Debugging and monitoring

Worked example: one food-delivery order, end to end

- You tap **Place order**. The app (frontend) sends `POST /api/orders` with JSON such as `{"restaurantId": 12, "items": [...]}`.
- The backend checks your login token, validates the items and calculates the bill.
- It calls the **payment gateway** API and waits for success.
- It saves the order in **PostgreSQL** inside a transaction.
- It publishes an `OrderPlaced` event to **Kafka**. The restaurant service, the rider-assignment service and the notification service react to it.
- Your live rider location is kept in **Redis**, because it changes every few seconds.
- Overnight (or continuously) the order is copied into the **data warehouse**, where the business sees "orders per city per hour".

MONOLITH VS MICROSERVICES (HOW ARCHITECTURE AFFECTS DATA)

A **monolith** is one application with one database. **Microservices** split the app into small services (users, orders, payments), and the usual rule is *database per service*: each service owns its data and others reach it only through its API or events. That gives teams independence, but it makes cross-service queries and consistency harder, which is exactly why integration patterns (Modules 5 and 6) matter.

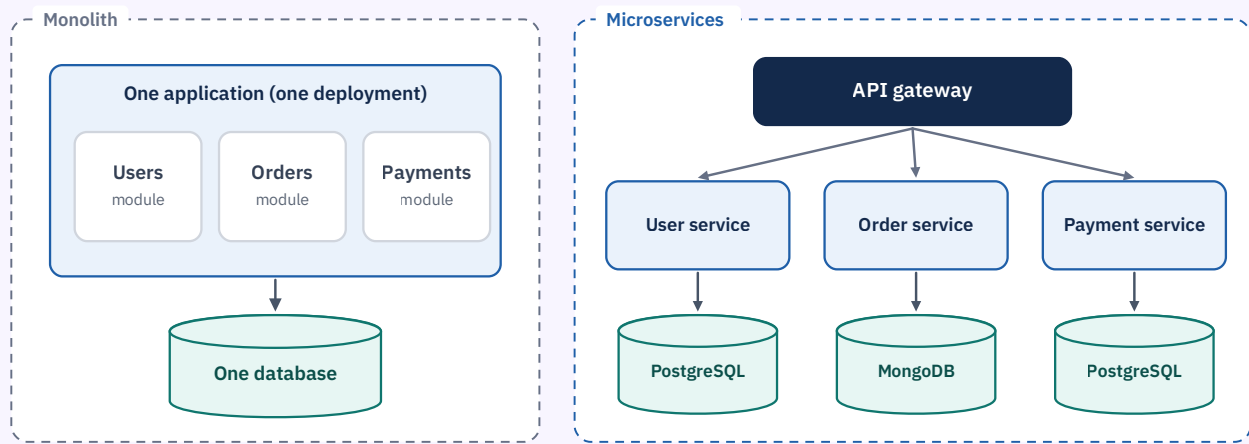


Figure 3. One shared database (monolith) vs one database per service (microservices).

USING THIS IN YOUR PROJECT STORY

If your project is a web app, describe it in these layers: "React frontend → Spring Boot REST API → PostgreSQL, with an external payment API and an export for reports." Then say which layer you built and one design decision you made in it. Interviewers love a clear map before details.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"A modern data system is the whole chain that collects, stores, processes and serves data, not just one database. A typical application has a frontend for the user, a backend API that holds the business logic, and a database that stores data safely. The backend also calls other services, like a payment gateway, and the data is copied to an analytics system for reporting. For example, in a food-delivery app the order is saved in PostgreSQL, the rider's live location sits in Redis, and the warehouse powers the city-wise sales dashboard."

COMMON TRAPS & MISTAKES

- Describing the system as "frontend + database". The backend/API layer is essential; the frontend should never connect to the database directly.
- Assuming one database does everything. Real systems mix stores (relational, cache, search, object storage, warehouse).
- Forgetting governance: security, quality and monitoring are part of the system, not extras.

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What is a data system?

A The complete set of components that collect, store, process, move and serve an organisation's data: sources, ingestion, storage, processing, serving and governance.

Q2 **DEFINITION** What does the backend of an application do?

A It validates input, applies business rules, handles authentication and authorization, talks to the database and other services, and returns responses (usually JSON) to the frontend.

Q3 WHY Why shouldn't a mobile app connect directly to the database?

- A** It would expose database credentials, let users bypass business rules, and make every schema change break installed apps. The API in the middle protects and controls access.

INTERMEDIATE**Q4 HOW What happens between a backend and a database when a request comes in?**

- A** The backend takes a connection from its pool, runs SQL (often generated by an ORM), wraps related changes in a transaction, maps rows into objects and returns JSON.

Q5 COMPARISON Where would you store product images vs order details?

- A** Images in object storage (cheap, built for large files) with only the file URL in the database; order details in a relational database, because they need structure, relationships and transactions.

Q6 DEFINITION What is a connection pool and why is it used?

- A** A set of already-open database connections that requests borrow and return. Opening a new connection for every request is slow and can overload the database.

SCENARIO-BASED**Q7 DESIGN QUESTION Sketch the architecture of a job portal.**

- A** Web/mobile frontend → REST API (job search, apply, recruiter dashboards) → PostgreSQL for users, jobs and applications; object storage for resumes; a search engine for keyword job search; email service for alerts; a warehouse for hiring analytics.

Q8 SCENARIO Your dashboard queries are slowing down the checkout page. Why might that happen and what would you do?

- A** Heavy analytical queries compete with checkout transactions on the same database. Move reporting to a read replica or, better, copy data to a warehouse (OLAP) and run dashboards there.

FOLLOW-UP / TRAP**Q9 TRAP QUESTION "Our app uses MongoDB, so we don't have a backend." Is that right?**

- A** No. The choice of database does not remove the need for a backend; something still has to validate requests, enforce rules and protect the data. Even "backend-as-a-service" products are a backend you rent.

Session 1.2 — Modern Data Systems & Integration (Part 2): How Data Moves Between Systems

What is data integration?

Simple meaning: data integration is combining data from different systems so it can be used together, or keeping several systems in sync with each other.

Why it exists: companies run many separate systems. Each team picks the best tool for its job, and soon the same customer exists in five places:

System	What it knows about customer "Aarav Patel"
E-commerce database	his orders and addresses
Payment gateway	his payments and refunds
CRM (Salesforce)	his support calls and sales opportunities
Marketing tool	which emails he opened
Mobile analytics	what he clicked in the app

Each system is a **data silo**: useful alone, but nobody can answer "Who are our most valuable customers, and are they happy?" without integrating them. A unified picture of the customer is often called **Customer 360**.

The common ways data moves

Method	How it works	Speed	Typical use
API call	One system asks another over HTTP and gets a reply	Real time	Checkout calls the payment gateway
File transfer	A system exports a CSV/Excel/JSON file; another imports it (often via SFTP or cloud storage)	Hours (batch)	Bank sends daily settlement files
Database replication / CDC	Changes in one database are copied to another, often by reading the database's change log	Seconds to minutes	Keep a reporting copy or search index fresh
Messaging / events	A system publishes a message; others consume it when ready	Near real time, asynchronous	<code>OrderPlaced</code> event triggers inventory and email
ETL / ELT pipeline	Extract from sources, transform, load into a warehouse	Minutes to hours	Nightly load of sales into the warehouse

You will study each of these properly in Modules 5 and 6. For now, notice the trade-off: **faster integration is usually more complex** to build and run.

APIs: the doors between systems

An **API (Application Programming Interface)** is a contract that says "send me a request in this format and I will reply in that format". Systems do not need to know each other's internals; they only need the contract.

THE RESTAURANT WAITER

You (the client) do not walk into the kitchen. You read the menu (API documentation), tell the waiter (API) what you want, and the waiter brings back your food (the response). The kitchen can change its ovens without changing the menu.

What "data processing" means

Between moving and storing, data is usually processed in a few standard steps:

1. **Collect** the raw data.
2. **Validate** it: are required fields present? Are dates real dates?
3. **Clean** it: remove duplicates, fix formats ("PUNE" , "Pune " → "Pune").
4. **Transform** it: join with other data, calculate totals, convert currencies.
5. **Store** it where it will be used.
6. **Serve** it to an app, report or model.

Why integration is hard

Challenge	Example	Usual fix
Different formats	One system sends XML, another expects JSON	Transformation/mapping layer
Different meanings	cust_no in ERP vs customer_id in CRM; "active customer" defined differently	Data mapping documents, master data management
Duplicates	Same customer created twice with slightly different names	Matching and de-duplication rules
Timing	Report shows yesterday's stock while the app shows today's	Agree on freshness needs; use CDC or events if needed
Failures	Target system is down during the nightly load	Retries, idempotent loads, alerts
Security and privacy	Sensitive data copied to too many places	Masking, least-privilege access, encryption

Worked example: one order touches many systems

When an e-commerce order is placed, the order system integrates with:

- the **payment gateway** (synchronous API call: we must know the payment worked);
- **inventory** (reserve stock, often through an event);
- **notifications** (send email/SMS, asynchronously; the customer should not wait for it);
- **the warehouse** (copy for reporting, in batch or by streaming);
- **the CRM** (so support staff can see the order history).

Choosing *how* each link works (API, event, file, pipeline) is the heart of integration design.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"Data integration means bringing data from different systems together so it can be used as one, or keeping those systems in sync. Companies have silos: orders in one database, customers in a CRM, payments with a gateway. We connect them using APIs for real-time requests, events or message queues for asynchronous updates, change data capture to copy database changes, and ETL or ELT pipelines to load a warehouse. The hard parts are mismatched formats and definitions, duplicates, failures and keeping the data fresh enough."

COMMON TRAPS & MISTAKES

- Thinking integration is only about APIs. Files, CDC, events and pipelines are just as common.
- Ignoring meaning: two systems can both have a "customer" field that means different things.
- Assuming real-time is always better. It costs more; many reports are fine with daily data.

INTERVIEW QUESTIONS**BASIC****Q1** **DEFINITION** What is data integration?

A Combining data from multiple sources into a consistent, usable view, or keeping multiple systems synchronised.

Q2 **DEFINITION** What is a data silo?

A Data locked inside one system or team that others cannot easily access or combine, like customer support data that never reaches the sales reports.

Q3 **DEFINITION** What is an API?

A A defined contract that lets one system request data or actions from another without knowing its internals, typically over HTTP with JSON.

INTERMEDIATE**Q4** **COMPARISON** Name four ways to move data between systems and when you would use each.

A API calls for real-time request/response (payments); file transfer for simple batch exchange (daily bank files); CDC/replication to mirror database changes (search index); events/queues for asynchronous notifications (order placed); ETL/ELT for loading warehouses.

Q5 **WHY** Why is data integration difficult?

A Different formats and definitions, duplicates, timing differences, failures and retries, and security/privacy concerns when copying data.

SCENARIO-BASED**Q6** **SCENARIO** Sales says the dashboard shows fewer orders than the app. What could be wrong?

A The pipeline may run in batch (data is stale), a load may have failed silently, filters or definitions may differ (cancelled orders excluded), or duplicates/time-zone issues. I would compare counts per day between source and warehouse to find where they diverge.

Q7 **DESIGN QUESTION** A hospital wants lab results from the lab system to appear in the doctor's app within a minute. How would you integrate them?

A Event-based or CDC integration: the lab system publishes a "ResultReady" event (or CDC captures new rows), a consumer transforms it to the app's format and stores it in the app's database. A nightly batch file would be too slow.

FOLLOW-UP / TRAP**Q8** **TRAP QUESTION** If both systems have a `customer_id`, is integrating them easy?

A Not necessarily. The IDs may come from different numbering schemes, so the same number can mean different people. You need a mapping or a shared master key (for example an email or a master customer ID).

Session 1.3 – Modern Data Systems & Integration (Part 3): Processing, OLTP vs OLAP, and Why One Database Is Not Enough

Batch processing vs real-time (stream) processing

Batch processing collects data over a period and processes it together at a scheduled time.

- *Examples:* monthly payroll, nightly sales report, end-of-day bank statement, daily load into a warehouse.
- *Why use it:* simple, efficient for large volumes, easy to re-run if something fails.

Real-time (stream) processing handles each event within seconds (or less) of it happening.

- *Examples:* card fraud detection, live delivery tracking, stock price alerts, "trending now" lists.
- *Why use it:* some decisions lose their value if they are late. A fraud alert tomorrow is useless.

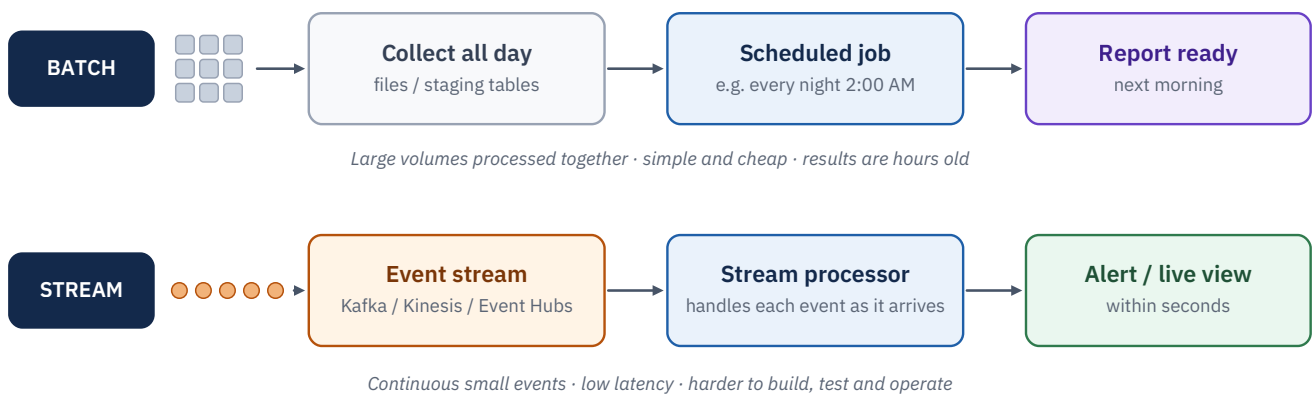


Figure 4. Batch waits and processes everything together; streaming processes each event as it arrives.

Aspect	Batch	Streaming (real-time)
When it runs	On a schedule (hourly, nightly)	Continuously
Latency (delay)	Minutes to hours	Milliseconds to seconds
Data size per run	Large chunks	One event or a small window
Complexity	Lower	Higher (ordering, late events, failures)
Cost	Usually cheaper	Usually more expensive
Re-processing	Easy: run the job again	Harder: needs replayable streams
Typical tools	SQL jobs, Spark, Airflow-scheduled jobs	Kafka, Kinesis, Flink, Spark Structured Streaming
Example	Daily sales report	Fraud detection on card swipes

Between the two sits **near-real-time** or **micro-batch** processing (for example, every 1–5 minutes), which is often "real-time enough" at a fraction of the cost.

THE INTERVIEW-WINNING QUESTION

When asked "batch or real-time?", ask back: **"How fresh does the data need to be for the decision?"** If the answer is "by tomorrow morning", batch is simpler and cheaper.

Transactional systems (OLTP) vs analytical systems (OLAP)

OLTP — Online Transaction Processing systems run the business day to day: placing orders, transferring money, booking tickets. They handle many small, fast reads and writes, and they must be correct every time.

OLAP — Online Analytical Processing systems help people *understand* the business: "revenue by city by month for three years". They run fewer but much heavier queries that scan and summarise lots of history.

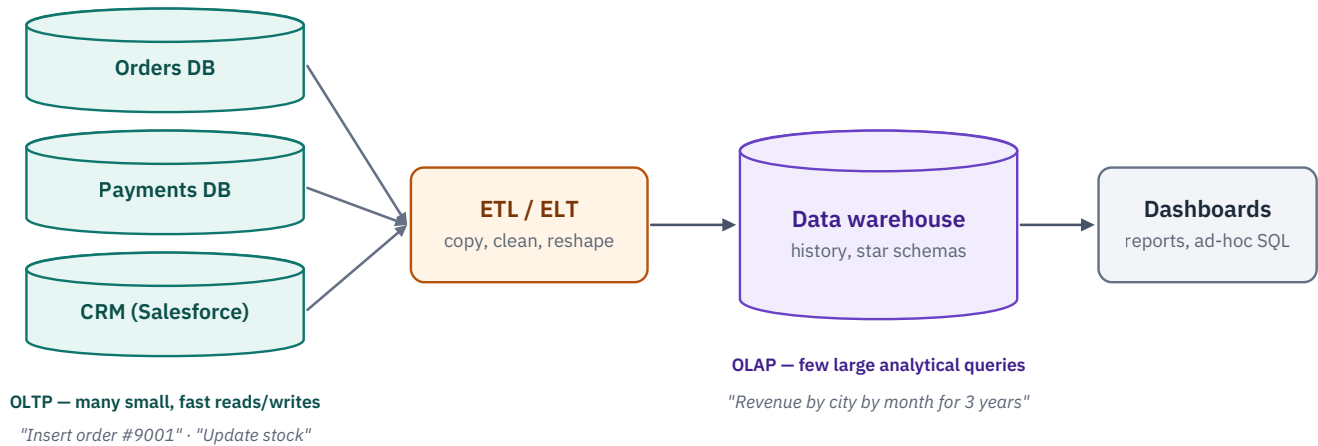


Figure 5. Operational (OLTP) databases feed an analytical (OLAP) warehouse through ETL/ELT pipelines.

Aspect	OLTP	OLAP
Purpose	Run the business	Analyse the business
Typical query	Insert order #9001; fetch one customer's cart	Total sales by region and quarter for 3 years
Query shape	Short, touches few rows	Long, scans millions of rows, aggregates
Users	Customers, staff, applications (thousands at once)	Analysts, managers, data scientists
Data	Current, up to the second	Historical, often refreshed hourly/daily
Design	Normalized (avoid duplication, protect consistency)	Denormalized star/snowflake schemas (fewer joins, fast reads)
Storage	Row-oriented	Often column-oriented
Priority	Fast writes, correctness, concurrency (ACID)	Fast reads and aggregation over large data
Examples	PostgreSQL, MySQL, Oracle, SQL Server running the app	Snowflake, BigQuery, Redshift, Azure Synapse

Why keep them separate? If a manager runs a three-year sales report on the same database that handles checkout, the report competes for CPU, memory and disk, and customers feel it as a slow checkout. Copying data into a warehouse lets each system be designed for its own job.

Operational data vs analytical data

- **Operational data** is the *current state* needed to run the business: today's stock level, the customer's current address, the open orders.
- **Analytical data** is *history and summaries* used for decisions: stock levels every day for two years, how many customers moved city, monthly revenue trends.

Operational systems often overwrite old values (the new address replaces the old one). Analytical systems usually *keep* history so trends can be studied (Module 2 shows how, with slowly changing dimensions).

Why modern systems need multiple data technologies

No single database is best at everything. Using several, each for the job it does best, is called **polyglot persistence**.

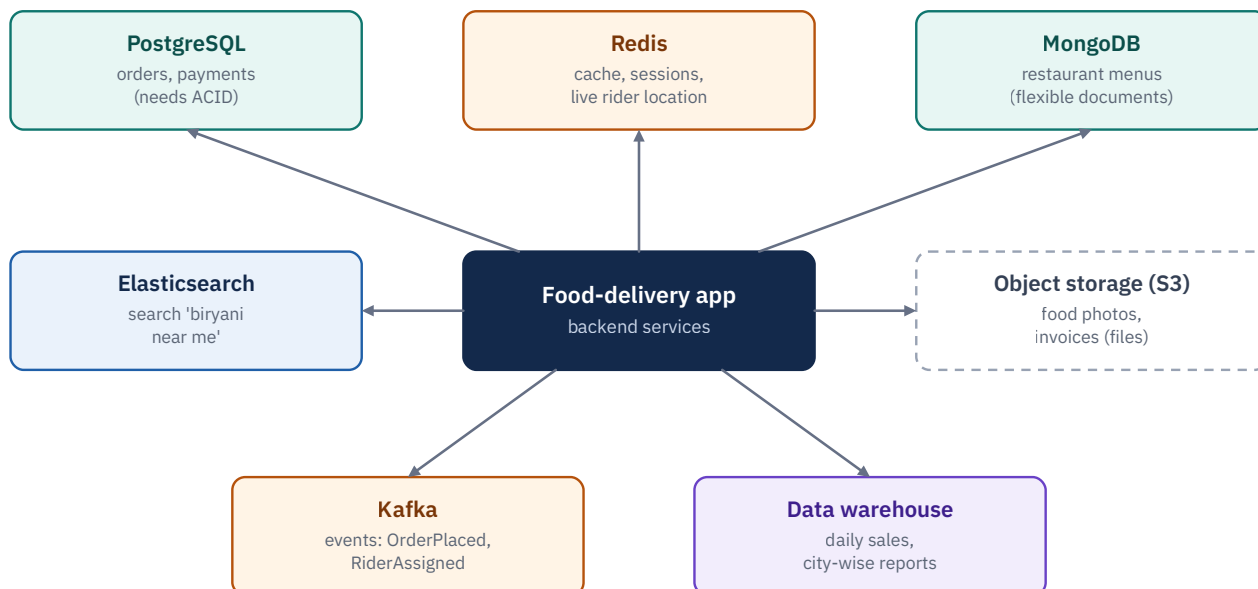


Figure 6. One food-delivery app, seven data technologies, each chosen for a specific job.

Job	Best-fit technology	Why not the relational database?
Money and orders	Relational (PostgreSQL)	— it is the right tool: ACID and relationships
Microsecond lookups, sessions, counters	Redis	A relational DB is slower and wastes resources on this
Flexible, changing product/menu attributes	Document DB (MongoDB)	Constant schema changes in a relational DB are painful
Free-text search with typos and ranking	Elasticsearch	<code>LIKE '%biryani%'</code> cannot rank results or handle typos well
Large files (images, PDFs)	Object storage	Storing large binary files in tables bloats the database
Events between services	Kafka	A table used as a queue does not scale well to many consumers
Years of history for reports	Data warehouse	Heavy scans would slow down the transactional database

A BALANCED VIEW (INTERVIEWERS LIKE THIS)

More technologies mean more to learn, monitor, secure and keep consistent. A good engineer starts simple (a single PostgreSQL database can go a long way, and it even supports JSON documents and full-text search) and adds a new store only when there is a clear need.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"OLTP systems run the business: lots of small, fast transactions like placing an order or transferring money, so they are normalized and ACID-compliant. OLAP systems analyse the business: fewer but heavy queries over years of history, like revenue by region per quarter, so they use denormalized star schemas, often in a columnar warehouse such as Snowflake or BigQuery. We keep them separate so reports don't slow down customers, and we move data from OLTP to OLAP with ETL or ELT pipelines."

COMMON TRAPS & MISTAKES

- Saying "OLAP is a type of database product." OLAP is a *workload*; many products serve it.
- Claiming real-time is always better than batch. It is costlier and more complex; choose by the freshness the business needs.
- Thinking polyglot persistence is always good. Each extra store adds operational cost.
- Mixing up "operational data" (current state) with "analytical data" (history).

INTERVIEW QUESTIONS**BASIC****Q1** **DEFINITION** What is batch processing?

A Processing data collected over a period together at a scheduled time, for example a nightly sales report or monthly payroll.

Q2 **DEFINITION** What is stream (real-time) processing?

A Processing each event continuously within seconds of it happening, for example fraud checks on card transactions or live order tracking.

Q3 **COMPARISON** What is the difference between OLTP and OLAP?

A OLTP handles many small, fast transactions that run the business and needs correctness (ACID); OLAP handles fewer, heavy analytical queries over historical data and is optimised for reading and aggregating.

INTERMEDIATE**Q4** **WHY** Why do companies keep separate OLTP and OLAP systems?

A Analytical queries scan huge amounts of data and would slow down customer transactions; the two workloads also need different designs (normalized vs star schema, row vs column storage).

Q5 **DEFINITION** What is polyglot persistence?

A Using different data storage technologies for different needs within one application, such as PostgreSQL for orders, Redis for caching and Elasticsearch for search.

Q6 **COMPARISON** Operational data vs analytical data?

A Operational data is the current state needed for daily work; analytical data is historical and summarised for decision-making, and usually keeps history instead of overwriting it.

Q7 **DEFINITION** What is micro-batch processing?

A Running small batches very frequently (every few seconds or minutes) to get near-real-time results with batch-style simplicity.

SCENARIO-BASED

Q8 **SCENARIO** A bank wants to block suspicious card transactions. Batch or streaming? Why?

A Streaming. The decision must be made before or within seconds of the transaction; a nightly batch would only find the fraud after the money is gone.

Q9 **SCENARIO** HR wants a monthly attrition report. Batch or streaming?

A Batch. Monthly freshness is enough, batch is simpler and cheaper, and it is easy to re-run.

Q10 **DESIGN QUESTION** Which data stores would you use for a ride-sharing app?

A PostgreSQL for riders, drivers, trips and payments (ACID); Redis or a geospatial store for live driver locations; Kafka for trip events; object storage for documents; a warehouse for analytics such as trips per city per hour.

FOLLOW-UP / TRAP

Q11 **TRAP QUESTION** Can PostgreSQL be used for analytics?

A Yes, for moderate data sizes PostgreSQL handles reporting well, especially on a read replica. Dedicated warehouses become worth it when data grows very large or many analysts run heavy queries.

Q12 **TRAP QUESTION** Is "real-time" always worth it?

A No. It adds cost and complexity (ordering, late events, failure handling). If the business acts on data daily, batch is the better engineering choice.

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

P1. List the six building blocks of a data system.

P2. Classify each as OLTP or OLAP: (a) withdrawing cash at an ATM, (b) a yearly sales trend chart, (c) adding an item to a cart, (d) comparing branch performance across five years.

P3. Give one example each of batch and streaming processing from daily life.

LEVEL 2 – INTERVIEW APPLICATION

P4. Explain, in under a minute, why the frontend should not connect directly to the database.

P5. A colleague wants to store product images inside a PostgreSQL table. What would you suggest instead, and why?

P6. Explain polyglot persistence and give one disadvantage.

LEVEL 3 – SCENARIO / PROBLEM SOLVING

P7. An e-commerce company's checkout becomes slow every morning at 9 AM, when managers open their dashboards. Diagnose the likely cause and propose two fixes (one quick, one long-term).

P8. Design the data flow for a food-delivery "orders per city per hour" dashboard that must be at most 5 minutes old. Which integration method would you choose?

ANSWER / EXPLANATION

P1. Sources, ingestion, storage, processing, serving, and governance (security, quality, metadata, monitoring) across all of them.

P2. (a) OLTP, (b) OLAP, (c) OLTP, (d) OLAP.

P3. Batch: your monthly credit card statement. Streaming: the live location of your cab or delivery rider.

P4. "Putting database credentials inside an app is a security risk, and users could bypass the business rules. An API in the middle validates requests, checks permissions and hides the database design, so we can change the schema without breaking every installed app."

P5. Store the images in object storage such as Amazon S3 or Azure Blob and keep only the image URL (and maybe size and type) in PostgreSQL. Object storage is cheaper for large files, keeps the database small and fast, and works with a CDN for quick delivery.

P6. Using different storage technologies for different jobs within one system (relational for orders, Redis for cache, Elasticsearch for search). Disadvantage: more systems to run, secure and monitor, and keeping data consistent across them is harder.

P7. Heavy analytical queries are running on the same OLTP database as checkout and competing for resources. Quick fix: point dashboards at a read replica. Long-term fix: load the data into a data warehouse with ETL/ELT (or streaming) and run all dashboards there.

P8. Five minutes is near-real-time, so use change data capture or order events (Kafka) feeding a stream or micro-batch job that aggregates orders by city and hour into the warehouse or a small serving table. A nightly batch would be too stale; full streaming with per-second updates is more than needed.

Session 1.4 — Module 1 Summary & Rapid Revision

MODULE 1 SUMMARY

MOST IMPORTANT CONCEPTS

- A **data system** = sources → ingestion → storage → processing → serving, with governance across all.
- A modern app = **frontend** → **backend/API** → **database**, plus other services and analytics.
- **Data integration** connects silos using APIs, files, CDC, events and ETL/ELT.
- **Batch** (scheduled, cheap, simple) vs **streaming** (continuous, low latency, complex).
- **OLTP** (run the business, many small ACID transactions) vs **OLAP** (analyse the business, heavy queries over history).
- **Polyglot persistence**: different stores for different jobs, at the cost of complexity.

WHAT TO MEMORIZE

- OLTP = transactions, normalized, row store, current data. OLAP = analytics, star schema, column store, historical data.
- Five ways data moves: API, file, CDC/replication, events/messaging, ETL/ELT.

WHAT TO UNDERSTAND (NOT MEMORIZE)

- *Why* the frontend never talks to the database directly.
- *Why* analytics is separated from transactions.
- *How* to choose batch vs streaming: by the freshness the decision needs.

MOST COMMON INTERVIEW QUESTIONS

- OLTP vs OLAP? • Batch vs streaming? • What is data integration? • Walk me through your app's architecture.
- Why use several databases?

COMMON MISTAKES

- "SQL is a database." • "Real-time is always better." • "One database for everything." • Forgetting the backend layer.

RAPID REVISION CHECKLIST

- | | |
|---|--|
| ☐ I can define a data system and its building blocks | ☐ I can compare batch and streaming with examples |
| ☐ I can draw frontend → API → database → analytics | ☐ I can compare OLTP and OLAP in a table |
| ☐ I can explain what a backend does | ☐ I can explain operational vs analytical data |
| ☐ I can define data integration and data silos | ☐ I can explain polyglot persistence and its cost |
| ☐ I can list five ways data moves between systems | ☐ I can describe my own project in layers |

PART II · MODULE 2

Data Modeling Fundamentals

Data modeling is deciding *what* data you store, *how* it is split into tables (or documents) and *how* the pieces connect. A good model makes queries simple and data trustworthy; a bad one causes duplicated, contradictory data for years. Interviewers love this module because it shows whether you can think, not just memorize.

COURSE COVERAGE

- ✓ Introduction to data modeling (conceptual, logical, physical) † → Session 2.1
- ✓ Entities, attributes, relationships, ER concepts † → Session 2.2
- ✓ Primary keys, foreign keys, constraints † → Session 2.3
- ✓ Relational model, schema, table design; one-to-one, one-to-many, many-to-many † → Session 2.4
- ✓ Normalization: functional dependency, update/insert/delete anomalies, 1NF, 2NF, 3NF, BCNF † → Session 2.5
- ✓ Denormalization; when to normalize and when to denormalize † → Session 2.6
- ✓ Data warehousing: data warehouse, fact and dimension tables, star and snowflake schemas, star vs snowflake, OLTP vs OLAP † → Session 2.7
- ✓ NoSQL modeling: why NoSQL, schema flexibility, document modeling, embedding vs referencing, access-pattern-driven design † → Session 2.8
- ✓ Module quiz concepts → interview questions in each session and Session 2.9

Topic	Priority	Typical question
Keys and constraints	HIGH PRIORITY	"Primary key vs foreign key vs unique key?"
Normalization (1NF–3NF, anomalies)	HIGH PRIORITY	"Normalize this table." "2NF vs 3NF?"
Relationships, many-to-many	HIGH PRIORITY	"How do you model students and courses?"
Star vs snowflake, facts vs dimensions	MEDIUM PRIORITY	"Design a star schema for sales."
Denormalization	MEDIUM PRIORITY	"When would you denormalize?"
NoSQL modeling, embedding vs referencing	MEDIUM PRIORITY	"Embed or reference order items in MongoDB?"

PWC ANGLE

Consulting work often starts with messy client data: spreadsheets, legacy tables and multiple systems describing the same customer differently. Being able to spot duplication, propose clean keys and explain normalization in business terms ("we store the customer's address once, so a change is made in one place") is directly useful, and it is the kind of reasoning a "design a schema for..." question is testing.

Session 2.1 — What Is Data Modeling?

Simple meaning

Data modeling is planning how data will be organised before you build the database, in the same way an architect draws a blueprint before anyone pours concrete. It answers three questions:

1. *What* things do we need to store data about? (customers, orders, products)
2. *What* do we need to know about each one? (a customer's name, email, city)
3. *How* are they connected? (a customer places many orders)

Why it matters

- **Fewer bugs in the data.** A good model makes it hard to store contradictions, like two different prices for the same product.
- **Simpler, faster queries.** When data is organised the way it will be used, queries are short and quick.
- **Cheaper changes.** Fixing a model after millions of rows exist is painful and risky.
- **A shared language.** Business people and engineers can agree on a picture before anyone writes code.

The three levels of a data model

Level	Question it answers	Who reads it	E-commerce example
Conceptual	What are the main things and how do they relate?	business stakeholders	"A Customer places Orders. An Order contains Products."
Logical	What exactly do we store about each thing? Which keys?	analysts, architects	Customer(customer_id, name, email, city); Order(order_id, customer_id, order_date, status)
Physical	How is it built in <i>this</i> database?	developers, DBAs	customers table in PostgreSQL, email VARCHAR(80) UNIQUE, index on orders(customer_id)

A logical model is independent of any product. The physical model adds the product-specific details: exact data types, indexes, partitions and storage settings.

The modeling process, step by step

1. **Understand the requirements.** What questions must the system answer? ("Which customers ordered in the last 30 days?")
2. **Find the entities**, the nouns that matter (Customer, Order, Product).
3. **List their attributes** (name, email, price).
4. **Define relationships and cardinality** (one customer, many orders).
5. **Choose keys** (primary and foreign keys).
6. **Normalize** to remove redundancy (Session 2.5).
7. **Design the physical tables:** types, constraints, indexes.
8. **Validate against access patterns:** can every important query be written simply and run fast?

Different kinds of data model

Model	Shape	Best for	Session
Relational	tables linked by keys	transactional systems (OLTP)	2.2–2.6
Dimensional	facts surrounded by dimensions (star schema)	analytics (OLAP), data warehouses	2.7
Document	JSON-like documents	flexible, nested data; read patterns known up front	2.8
Key-value / wide-column / graph	key → value; rows of sparse columns; nodes and edges	caching, huge write volumes, relationships	2.8, 4.2

WORKED EXAMPLE: A SMALL LIBRARY SYSTEM

Requirement: "Members borrow books. A book can have several authors. We must know who has which book and when it is due."

- *Entities:* Member, Book, Author, Loan.
- *Relationships:* a Member has many Loans; a Book has many Loans over time; Book ↔ Author is many-to-many.
- *Keys:* member_id, book_id, author_id; Loan(loan_id, member_id, book_id, loan_date, due_date, return_date).
- *Physical decisions:* index `loans(member_id)` for "my books" pages; `return_date` NULL means "not yet returned".

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"Data modeling is designing how data will be structured before building the database: which entities we store, what attributes they have and how they relate. We usually go from a conceptual model for the business, to a logical model with entities, attributes and keys, to a physical model for a specific database with data types and indexes. A good model avoids duplicate and inconsistent data and makes the important queries simple. For example, for e-commerce I'd model customers, orders, products and an order_items table between orders and products."

COMMON TRAPS & MISTAKES

- Jumping straight to `CREATE TABLE` without asking what questions the data must answer.
- Mixing up logical (product-independent) and physical (product-specific) models.
- Thinking modeling is only for relational databases. NoSQL needs modeling too, just driven by access patterns.

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What is data modeling?

A Planning the structure of data, meaning entities, attributes, relationships and keys, before implementing it in a database.

Q2 **COMPARISON** What are conceptual, logical and physical data models?

- A** Conceptual: high-level entities and relationships for the business. Logical: detailed attributes, keys and relationships, independent of any product. Physical: the actual tables, types, constraints and indexes in a specific DBMS.

INTERMEDIATE**Q3** **WHY** Why is data modeling important?

- A** It prevents redundancy and inconsistency, makes queries simpler and faster, reduces the cost of later changes, and gives business and technical teams a shared picture.

Q4 **HOW** What steps do you follow to model a new system?

- A** Gather requirements and key questions, identify entities, attributes and relationships, choose keys, normalize, design the physical schema, then validate it against the main queries.

SCENARIO-BASED**Q5** **DESIGN QUESTION** You are asked to model a hospital appointment system. What entities would you start with?

- A** Patient, Doctor, Department, Appointment, and possibly Prescription and Bill. Doctor belongs to one Department; Patient has many Appointments; each Appointment links one Patient and one Doctor at a time slot.

FOLLOW-UP / TRAP**Q6** **TRAP QUESTION** Does a NoSQL database mean you don't need a data model?

- A** No. The model moves into the application and is driven by access patterns. Without design you get inconsistent documents and slow queries.

Session 2.2 — Entities, Attributes and Relationships (ER Modeling)

Entities

An **entity** is a real-world "thing" we store data about: *Customer, Product, Order, Employee, Department*. Each specific customer, such as Aarav Patel, is an **entity instance**, which becomes one **row** in the table.

- A **strong entity** can be identified on its own (a Customer has a `customer_id`).
- A **weak entity** cannot exist or be identified without its owner. An *order item* only makes sense inside an order: it is identified by (`order_id`, line number or `product_id`).

Attributes

An **attribute** is a property of an entity: a customer's name, email and city.

Attribute type	Meaning	Example	How it becomes tables
Simple	cannot be split usefully	<code>email</code>	one column
Composite	made of parts	<code>address</code> = street + city + pin code	several columns (<code>street</code> , <code>city</code> , <code>pin_code</code>)
Single-valued	one value per entity	<code>date_of_birth</code>	one column
Multi-valued	many values per entity	a customer's several phone numbers	a separate table <code>customer_phones</code>
Derived	can be calculated from others	<code>age</code> from <code>date_of_birth</code> ; order total from items	usually <i>not</i> stored (or stored on purpose for speed, see Session 2.6)
Key	identifies the entity	<code>customer_id</code>	primary key

Relationships and cardinality

A **relationship** is how entities are connected: *Customer places Order, Employee works in Department*. **Cardinality** says *how many* on each side:

Cardinality	Meaning	Example
One-to-one (1:1)	one A ↔ one B	each user has one profile
One-to-many (1:N)	one A ↔ many B	one customer places many orders
Many-to-many (M:N)	many A ↔ many B	students take many courses; each course has many students

Participation (optionality) says whether the relationship is required:

- Every order *must* belong to a customer → **mandatory** (total participation). In SQL: `customer_id NOT NULL`.
- A customer *may* have zero orders → **optional** (partial participation).

Reading an ER diagram

An **ER (Entity-Relationship) diagram** draws entities as boxes and relationships as lines. Two notations are common:

- **Chen notation** (textbooks): entities as rectangles, attributes as ovals, relationships as diamonds.
- **Crow's foot notation** (industry tools): entities as boxes listing their columns; the line ends show cardinality.

Line-end symbol (crow's foot)	Meaning
(two bars)	exactly one
o (circle + bar)	zero or one
< (bar + crow's foot)	one or many
o< (circle + crow's foot)	zero or many

In this handbook the diagrams simply write **1** and **N** at the line ends:

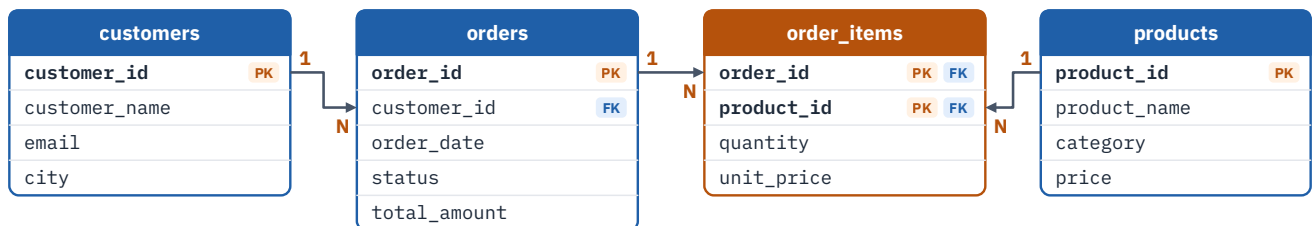


Figure 7. The e-commerce part of this handbook's sample database as an ER diagram.

Finding entities in a requirement

A practical trick: **nouns** are candidate entities or attributes; **verbs** are candidate relationships.

"Patients **book** appointments with doctors. Each doctor **belongs to** one department. A patient **can have** many appointments, and each appointment **may produce** a prescription."

- Entities: Patient, Appointment, Doctor, Department, Prescription.
- Relationships: Patient 1:N Appointment; Doctor 1:N Appointment; Department 1:N Doctor; Appointment 1:0..1 Prescription.

ENTITY OR ATTRIBUTE?

Make something its own entity when it has its own attributes or is shared. "City" can be a plain attribute of a customer, but if you need each city's state, region and delivery charges, make `cities` a table and reference it.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"An entity is a thing we store data about, like a customer or an order; each row is one instance. Attributes are its properties, like name and email. Relationships connect entities, and cardinality tells us how many: a customer places many orders, so that's one-to-many, while students and courses are many-to-many. In an ER diagram I'd draw the entities as boxes, list their keys and mark the cardinality on the lines, for example with crow's foot notation."

COMMON TRAPS & MISTAKES

- Storing a multi-valued attribute (several phone numbers) as a comma-separated string. It belongs in its own table.
- Forgetting optionality: "every order has a customer" should become `NOT NULL` on the foreign key.
- Confusing an *entity* (Customer) with an *entity instance* (Aarav).

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What is an entity? Give an example.

A A real-world object or concept we store data about, such as Customer or Product. Each instance (one customer) becomes a row.

Q2 **DEFINITION** What is an attribute?

A A property of an entity, such as a customer's name or email. It becomes a column.

Q3 **DEFINITION** What is cardinality?

A The number of instances of one entity that can relate to another: one-to-one, one-to-many or many-to-many.

INTERMEDIATE

Q4 **COMPARISON** What is the difference between a strong and a weak entity?

A A strong entity has its own identifier; a weak entity depends on an owner entity for its identity, for example an order line identified by order_id plus line number.

Q5 **COMPARISON** What are composite, multi-valued and derived attributes?

A Composite has parts (address = street, city, pin); multi-valued has several values (phone numbers); derived can be calculated (age from date of birth).

Q6 **HOW** How do you represent a multi-valued attribute in tables?

A Put it in a separate child table with a foreign key to the parent, for example
`customer_phones(customer_id, phone)`.

SCENARIO-BASED

Q7 **SCENARIO** From "Employees work in departments; each department has one manager who is also an employee", identify entities and relationships.

A Entities: Employee, Department. Relationships: Department 1:N Employee (works in); Department 1:1 Employee (managed by), which is stored as `manager_emp_id` in departments, referencing employees.

FOLLOW-UP / TRAP

Q8 **TRAP QUESTION** Should you store a customer's age?

A Usually no. Store the date of birth and calculate the age, because a stored age becomes wrong every year.

Session 2.3 — Keys and Constraints

HIGH PRIORITY This is one of the most asked topics in any DBMS interview.

Why keys exist

Keys do two jobs: they **identify each row uniquely**, and they **link tables together**. Without them, you could not reliably say "update *this* customer" or "find *this* customer's orders".

Every kind of key, on one table

Take an `employees` table where each employee has `emp_id`, `email`, `pan_number` (a tax ID that is unique per person), `name` and `dept_id`.

Key	Definition	Example
Super key	any set of columns that uniquely identifies a row	{ <code>emp_id</code> }, { <code>emp_id</code> , <code>name</code> }, { <code>email</code> }, { <code>email</code> , <code>dept_id</code> } ...
Candidate key	a <i>minimal</i> super key: remove any column and it stops being unique	{ <code>emp_id</code> }, { <code>email</code> }, { <code>pan_number</code> }
Primary key (PK)	the candidate key we <i>choose</i> as the main identifier; unique and NOT NULL	<code>emp_id</code>
Alternate key	candidate keys that were not chosen as the PK	<code>email</code> , <code>pan_number</code>
Unique key	a column (or set) declared <code>UNIQUE</code> ; no duplicates, but may allow NULL	<code>email</code>
Composite key	a key made of two or more columns	(<code>order_id</code> , <code>product_id</code>) in <code>order_items</code>
Foreign key (FK)	a column that refers to the primary (or unique) key of another table	<code>employees.dept_id</code> → <code>departments.dept_id</code>
Surrogate key	an artificial ID with no business meaning	auto-generated <code>emp_id</code> 1, 2, 3
Natural key	a key with real-world meaning	PAN number, email

Primary key rules

- Must be **unique** and **NOT NULL**.
- **One primary key per table**, though it may be composite.
- Should be **stable** (never changes) and **minimal** (no unnecessary columns).

Foreign keys and referential integrity

A **foreign key** says "this value must exist in that other table". The guarantee it provides is called **referential integrity**: there are no orphan rows, such as an order for a customer who does not exist.

CUSTOMERS (PARENT)

<code>customer_id</code>	<code>customer_name</code>	<code>email</code>
1	Aarav Patel	aarav@mail.com
2	Isha Kulkarni	isha@mail.com

ORDERS (CHILD)

order_id	customer_id	order_date	total_amount
101	1	2024-01-05	2843.00
102	2	2024-01-12	3499.00
103	1	2024-01-28	8999.00

`orders.customer_id` is a foreign key to `customers.customer_id`. Customer 1 has two orders, which is the one-to-many relationship in action. Watch PostgreSQL protect the data in the sample database. First, try to create an order for a customer who does not exist:

```
INSERT INTO orders (order_id, customer_id, order_date, status)
VALUES (200, 999, '2024-05-01', 'PENDING');
```

SQL · PostgreSQL

POSTGRESQL SAYS

ERROR insert or update on table "orders" violates foreign key constraint "orders_customer_id_fkey"
DETAIL: Key (customer_id)=(999) is not present in table "customers".

Now try to delete a customer who still has orders:

```
DELETE FROM customers WHERE customer_id = 1;
```

SQL · PostgreSQL

POSTGRESQL SAYS

ERROR update or delete on table "customers" violates foreign key constraint "orders_customer_id_fkey" on table "orders"
DETAIL: Key (customer_id)=(1) is still referenced from table "orders".

What happens to the children when a parent is deleted is controlled by the **referential action** on the foreign key:

Action	When the parent row is deleted...	Typical use
NO ACTION / RESTRICT (default)	the delete is refused while children exist	orders, payments: never lose history silently
CASCADE	the children are deleted too	a post's comments; a cart's items
SET NULL	the child's FK becomes NULL	employee's manager leaves → <code>manager_id</code> NULL
SET DEFAULT	the child's FK gets its default value	rare

The other constraints

Constraints are rules the database enforces on every insert and update, no matter which application writes the data.

Constraint	Rule	Example
NOT NULL	a value is required	<code>customer_name VARCHAR(60) NOT NULL</code>
UNIQUE	no duplicate values	<code>email VARCHAR(80) UNIQUE</code>
PRIMARY KEY	UNIQUE + NOT NULL, the row's identity	<code>customer_id INT PRIMARY KEY</code>
FOREIGN KEY	value must exist in the parent table	<code>customer_id INT REFERENCES customers(customer_id)</code>
CHECK	value must satisfy a condition	<code>price NUMERIC(10,2) CHECK (price >= 0)</code>
DEFAULT	value used when none is given	<code>status VARCHAR(12) DEFAULT 'PENDING'</code>

Each violation produces a clear error. A duplicate email:

```
INSERT INTO customers (customer_id, customer_name, email, city, signup_date)
VALUES (8, 'Aarav P', 'aarav@mail.com', 'Mumbai', '2024-05-01');
```

SQL · PostgreSQL

PostgreSQL Says

ERROR duplicate key value violates unique constraint "customers_email_key"
DETAIL: Key (email)=(aarav@mail.com) already exists.

A negative price, blocked by a `CHECK` constraint:

```
UPDATE products SET price = -10 WHERE product_id = 1;
```

SQL · PostgreSQL

PostgreSQL Says

ERROR new row for relation "products" violates check constraint "products_price_check"
DETAIL: Failing row contains (1, Wireless Mouse, Electronics, -10.00).

A missing required value:

```
INSERT INTO products (product_id, product_name, category, price)
VALUES (9, NULL, 'Electronics', 499);
```

SQL · PostgreSQL

PostgreSQL Says

ERROR null value in column "product_name" of relation "products" violates not-null constraint
DETAIL: Failing row contains (9, null, Electronics, 499.00).

Primary key vs unique key: the NULL difference

A `UNIQUE` column in PostgreSQL allows **many NULLs**, because NULL means "unknown" and two unknowns are not considered equal. A primary key allows **no NULLs**. Customer 7 already has a NULL email; adding another customer with a NULL email works:

```
INSERT INTO customers (customer_id, customer_name, email, city, signup_date)
VALUES (8, 'Test User', NULL, 'Pune', '2024-05-01');
```

SQL · PostgreSQL

```
SELECT customer_id, customer_name, email
FROM customers
WHERE email IS NULL;
```

Output

customer_id	customer_name	email
7	Reyansh Bose	NULL
8	Test User	NULL

2 rows

LINE BY LINE

Line	What it does
INSERT INTO customers (...) VALUES (8, 'Test User', NULL, ...)	adds a customer whose email is unknown (NULL)
SELECT customer_id, customer_name, email	shows these three columns
FROM customers	from the customers table
WHERE email IS NULL	only rows with no email; note IS NULL , never = NULL

The example was rolled back after it ran, so the sample data is unchanged for the next query.

DATABASE DIFFERENCES

SQL Server allows only **one** NULL in a UNIQUE column. PostgreSQL 15+ can behave the same way with `UNIQUE NULLS NOT DISTINCT`. Mentioning this shows real depth.

Comparison tables interviewers expect

	Primary key	Unique key
Purpose	the row's main identity	prevent duplicates in another column
NULLs	not allowed	allowed (PostgreSQL: many)
How many per table	one	many
Index	created automatically	created automatically
Referenced by FKs	yes (usual)	yes (allowed)

	Primary key	Foreign key
Purpose	identifies rows in <i>its own</i> table	links to rows in <i>another</i> table
Uniqueness	must be unique	can repeat (many orders per customer)
NULLs	not allowed	allowed unless declared NOT NULL
Count per table	one	any number

Surrogate vs natural keys

	Surrogate key (id 1, 2, 3 or UUID)	Natural key (email, PAN)
Meaning	none, purely technical	has business meaning
Stability	never changes	can change (people change emails)
Size and speed	small integer, fast joins	often long strings
Risk	duplicates possible unless you also add UNIQUE on the natural key	business changes break keys

Common practice: use a surrogate primary key (`INT GENERATED ALWAYS AS IDENTITY` , `BIGSERIAL` , or a UUID) and a `UNIQUE` constraint on the natural key.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"A primary key uniquely identifies each row in a table. It must be unique and not null, and there is only one per table, although it can be made of several columns. A foreign key is a column in one table that points to the primary key of another, which enforces referential integrity: you cannot create an order for a customer who doesn't exist, and you can't delete a customer who still has orders unless you choose CASCADE or SET NULL. A unique key also prevents duplicates, but it can allow NULLs and a table can have many of them."

COMMON TRAPS & MISTAKES

- "A primary key can be NULL if it's composite." No: no part of a primary key may be NULL.
- "A foreign key must be unique." No: many child rows can share the same FK value.
- "A table can have two primary keys." It has one PK; the other candidates become alternate or unique keys.
- "UNIQUE allows exactly one NULL." That depends on the database: PostgreSQL and MySQL allow many; SQL Server allows one.
- Forgetting that a FK must reference a PRIMARY KEY or UNIQUE column.

INTERVIEW QUESTIONS**BASIC****Q1** **DEFINITION** What is a primary key?

A A column or set of columns that uniquely identifies each row. It must be unique and not null, and each table has only one.

Q2 **DEFINITION** What is a foreign key?

A A column whose values must match the primary or unique key of another table. It links tables and enforces referential integrity.

Q3 **COMPARISON** Primary key vs unique key?

A Both prevent duplicates. A primary key cannot be NULL and there is only one per table; a unique key may allow NULLs and a table can have several.

Q4 **DEFINITION** What is a composite key?

A A key made of two or more columns, such as (order_id, product_id) in an order_items table.

INTERMEDIATE**Q5** **COMPARISON** Super key vs candidate key vs primary key?

A A super key is any set of columns that is unique; a candidate key is a minimal super key; the primary key is the candidate key chosen as the main identifier.

Q6 **DEFINITION** What is referential integrity?

A The guarantee that every foreign key value points to an existing parent row, so there are no orphan records.

Q7 **HOW** What happens when you delete a parent row that has child rows?

A By default the delete is rejected. With `ON DELETE CASCADE` the children are deleted too; with `SET NULL` their foreign key becomes NULL.

Q8 **COMPARISON** Surrogate key vs natural key?

- A** A surrogate key is an artificial, meaningless ID (auto-increment or UUID); a natural key has business meaning (email, PAN). Surrogates are stable and compact, so they are commonly used as the PK, with a UNIQUE constraint on the natural key.

Q9 **DEFINITION** What is a CHECK constraint?

- A** A rule that every row must satisfy, such as `CHECK (price >= 0)` or `CHECK (status IN ('PENDING', 'SHIPPED'))`.

SCENARIO-BASED**Q10** **SCENARIO** An employee table uses email as its primary key. What problems could this cause?

- A** Emails change, so the PK and every foreign key pointing to it would have to change. Emails are also long strings, which makes joins and indexes larger. Better: a surrogate `emp_id` PK plus `UNIQUE (email)`.

Q11 **SCENARIO** Users are getting orders linked to deleted customers. What is missing?

- A** A foreign key from `orders.customer_id` to `customers.customer_id` (and probably `NOT NULL`). Then the database itself blocks orphan rows.

FOLLOW-UP / TRAP**Q12** **TRAP QUESTION** Can a foreign key be NULL?

- A** Yes, unless it is declared `NOT NULL`. A NULL foreign key means "no related row", for example an employee with no manager.

Q13 **TRAP QUESTION** Can a foreign key reference a column that is not the primary key?

- A** Yes, as long as that column has a UNIQUE (or PRIMARY KEY) constraint.

Q14 **TRAP QUESTION** Can a table have no primary key?

- A** Technically yes, but it is bad practice: you cannot reliably identify, update or de-duplicate rows, and many tools require a PK.

Session 2.4 – The Relational Model, Schema and Table Design

The relational model in one picture

The relational model, proposed by E. F. Codd in 1970, stores data in **relations**, which we call tables. Its vocabulary still appears in interviews:

Formal term	Everyday term	Meaning
Relation	table	a set of rows with the same columns
Tuple	row	one record
Attribute	column	one property
Domain	data type + allowed values	e.g. <code>status</code> must be one of five words
Degree	number of columns	<code>customers</code> has degree 5
Cardinality (of a table)	number of rows	<code>customers</code> has 7 rows

Rows in a relation have **no guaranteed order**, which is why SQL needs `ORDER BY` when order matters.

Two meanings of "schema"

1. **The design** of the database: which tables, columns, types and constraints exist ("the schema of the orders table").
2. In **PostgreSQL**, a *schema* is also a **namespace (a folder) inside a database** that holds tables. The default one is `public`, so `public.orders` is the full name of `orders`. Teams use schemas such as `sales`, `hr` and `staging` to organise objects and permissions.

Turning relationships into tables

One-to-one

1:1

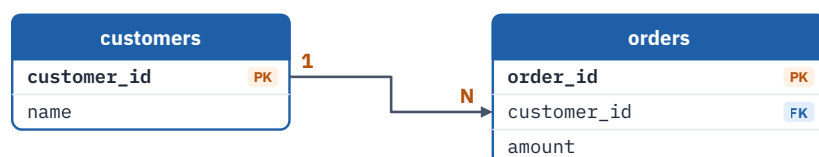
each user has exactly one profile



One-to-many

1:N

one customer places many orders



Many-to-many

M:N

students take many courses;
courses have many students

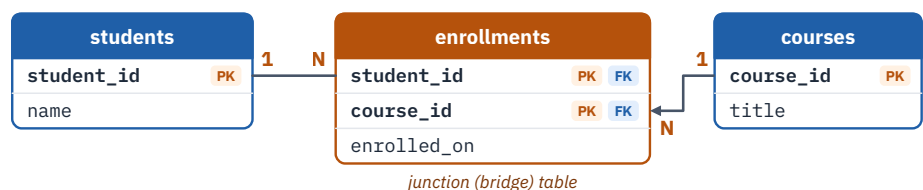


Figure 8. How each kind of relationship becomes tables and keys.

Relationship	Rule	Example
1 : N	put the foreign key on the "many" side	<code>orders.customer_id → customers</code>
1 : 1	foreign key on either side with a UNIQUE constraint , or share the same primary key	<code>user_profiles.user_id</code> is both PK and FK
M : N	create a junction (bridge) table holding both foreign keys, usually as a composite PK	<code>enrollments(student_id, course_id)</code>
Multi-valued attribute	child table	<code>customer_phones(customer_id, phone)</code>
Weak entity	child table whose key includes the owner's key	<code>order_items(order_id, product_id, ...)</code>

Building a many-to-many design for real

Students and courses are the classic many-to-many example. The block below creates the three tables with every kind of constraint, then loads a little data. It is kept for the rest of this chapter (keep):

```

CREATE TABLE students (
    student_id INT PRIMARY KEY,
    full_name  VARCHAR(60) NOT NULL,
    email      VARCHAR(80) UNIQUE,
    city       VARCHAR(40) NOT NULL DEFAULT 'Unknown'
);

CREATE TABLE courses (
    course_id  VARCHAR(10) PRIMARY KEY,
    title      VARCHAR(80) NOT NULL,
    credits    INT NOT NULL CHECK (credits BETWEEN 1 AND 6)
);

CREATE TABLE enrollments (
    student_id INT NOT NULL REFERENCES students(student_id) ON DELETE CASCADE,
    course_id  VARCHAR(10) NOT NULL REFERENCES courses(course_id),
    enrolled_on DATE NOT NULL,
    grade      CHAR(1) CHECK (grade IN ('A','B','C','D','F')),
    PRIMARY KEY (student_id, course_id)
);

INSERT INTO students VALUES
    (1, 'Asha Rao', 'asha@uni.edu', 'Pune'),
    (2, 'Vikram Shah', 'vikram@uni.edu', 'Delhi'),
    (3, 'Neha Iyer', NULL, 'Chennai');

INSERT INTO courses VALUES
    ('DB101', 'Database Systems', 4),
    ('PY101', 'Python Programming', 3),
    ('DS201', 'Data Structures', 4);

INSERT INTO enrollments VALUES
    (1, 'DB101', '2024-07-01', 'A'),
    (1, 'PY101', '2024-07-01', 'B'),
    (2, 'DB101', '2024-07-02', NULL),
    (3, 'DS201', '2024-07-03', 'A'),
    (3, 'DB101', '2024-07-03', 'B');

```

LINE BY LINE

Part of the code	What it means
<code>student_id INT PRIMARY KEY</code>	every student has a unique, non-null ID
<code>email VARCHAR(80) UNIQUE</code>	no two students share an email (NULL allowed)
<code>DEFAULT 'Unknown'</code>	used if no city is supplied
<code>CHECK (credits BETWEEN 1 AND 6)</code>	rejects impossible credit values
<code>REFERENCES students(student_id) ON DELETE CASCADE</code>	each enrollment must point to a real student; deleting a student removes their enrollments
<code>PRIMARY KEY (student_id, course_id)</code>	a composite key: the same student cannot enroll in the same course twice

Now the junction table lets us answer "who takes what" with two joins:

```
SELECT s.full_name, c.title, e.grade
FROM enrollments e
JOIN students s ON s.student_id = e.student_id
JOIN courses c ON c.course_id = e.course_id
ORDER BY s.full_name, c.title;
```

SQL · POSTGRESQL

OUTPUT

full_name	title	grade
Asha Rao	Database Systems	A
Asha Rao	Python Programming	B
Neha Iyer	Data Structures	A
Neha Iyer	Database Systems	B
Vikram Shah	Database Systems	NULL

5 rows

And `ON DELETE CASCADE` in action: deleting Asha removes her two enrollments automatically.

```
DELETE FROM students WHERE student_id = 1;
```

SQL · POSTGRESQL

```
SELECT student_id, course_id, grade
FROM enrollments
ORDER BY student_id, course_id;
```

OUTPUT

student_id	course_id	grade
2	DB101	NULL
3	DB101	B
3	DS201	A

3 rows

Table design best practices

Practice	Why
Every table has a primary key	rows can be identified, updated and joined reliably
Use the right data types: <code>NUMERIC(10,2)</code> for money, <code>DATE</code> / <code>TIMESTAMP</code> for time, <code>BOOLEAN</code> for flags	<code>FLOAT</code> rounds money ($0.1 + 0.2 \neq 0.3$); dates stored as text cannot be compared or validated
One fact per column, one value per cell	no "Mouse, Keyboard" lists; they break searching and violate 1NF
Declare foreign keys and <code>NOT NULL</code> where required	the database enforces correctness for every application
Consistent naming (<code>snake_case</code> , clear names like <code>customer_id</code>)	readable queries, fewer mistakes
Don't store what you can calculate cheaply (age, totals) unless you have a reason	avoids stale values (see Session 2.6 for when to break this rule)
Add audit columns such as <code>created_at</code> , <code>updated_at</code>	debugging, incremental loads, history

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"To model a many-to-many relationship, like students and courses, I create a junction table, say `enrollments` , with two foreign keys: one to students and one to courses. The pair is the composite primary key, so a student can't enroll twice in the same course, and the table can also hold attributes of the relationship itself, like the enrollment date and grade. One-to-many is simpler: the foreign key goes on the many side, so `orders` gets a `customer_id` ."

COMMON TRAPS & MISTAKES

- Putting the foreign key on the wrong side of a 1:N relationship (for example, an `order_id` column in `customers`).
- Modeling M:N with comma-separated IDs in one column instead of a junction table.
- Using `FLOAT` for currency.
- Storing dates as text (`'05/03/2024'`): is that 5 March or 3 May?

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What is a relation in the relational model?

A A table: a set of tuples (rows) that share the same attributes (columns).

Q2 **HOW** How do you implement a one-to-many relationship?

A Put a foreign key on the "many" side that references the primary key of the "one" side, e.g. `orders.customer_id` → `customers.customer_id` .

Q3 **HOW** How do you implement a many-to-many relationship?

A With a junction table holding foreign keys to both tables, usually forming a composite primary key, e.g. `enrollments(student_id, course_id)` .

INTERMEDIATE**Q4** **HOW** How do you implement a one-to-one relationship?

A Put a foreign key with a UNIQUE constraint in one table, or make the child's primary key also a foreign key to the parent (`user_profiles.user_id`).

Q5 **DEFINITION** What is a schema in PostgreSQL?

A A namespace inside a database that groups tables, views and functions; the default is `public` . "Schema" also means the overall design of the tables.

Q6 **WHY** Why should money not be stored as FLOAT?

A Floating-point numbers are binary approximations and produce rounding errors; `NUMERIC` / `DECIMAL` stores exact values.

SCENARIO-BASED**Q7** **DESIGN QUESTION** Design tables for a library where books can have multiple authors and members borrow books.

A `books(book_id PK, title, isbn UNIQUE)` , `authors(author_id PK, name)` , `book_authors(book_id FK, author_id FK, PK both)` , `members(member_id PK, name, email UNIQUE)` , `loans(loan_id PK, member_id FK, book_id FK, loan_date, due_date, return_date NULL)` .

Q8 **SCENARIO** A colleague stores `course_ids` as "DB101,PY101" in the students table. What's wrong and how do you fix it?

A It breaks 1NF, cannot be indexed or joined properly, and allows invalid IDs. Replace it with an `enrollments` junction table with foreign keys to both sides.

FOLLOW-UP / TRAP**Q9** **TRAP QUESTION** Are rows in a table stored in insertion order?

A You must not rely on it. A relation is an unordered set; use `ORDER BY` whenever order matters.

Q10 **TRAP QUESTION** Can a junction table have extra columns?

A Yes. Attributes of the relationship itself, such as enrollment date and grade, belong there.

Session 2.5 — Normalization

HIGH PRIORITY Expect at least one normalization question in almost every DBMS interview.

Simple meaning

Normalization means splitting data into sensible tables so the same fact is not stored over and over. Each fact lives in exactly one place, and tables are linked with keys.

Why: the three anomalies

Look at this "everything in one sheet" table. It works, until you try to change it:

ORDER_DETAILS_FLAT (UNNORMALIZED)

order_id	order_date	customer_name	customer_phone	customer_city	product_name	unit_price	qty
101	2024-01-05	Aarav Patel	98200 11111	Mumbai	Wireless Mouse	799	2
101	2024-01-05	Aarav Patel	98200 11111	Mumbai	Notebook Pack	249	5
103	2024-01-28	Aarav Patel	98200 11111	Mumbai	Office Chair	8999	1
102	2024-01-12	Isha Kulkarni	98900 22222	Pune	Mechanical Keyboard	3499	1

Anomaly	What goes wrong here
Update anomaly	Aarav changes his phone number. It is stored in three rows; miss one and the data contradicts itself.
Insert anomaly	We cannot record a new product, "Monitor Arm", until somebody orders it, because there is no order_id to put in the row.
Delete anomaly	If order 102 is deleted, we also lose the only record that Isha exists <i>and</i> that a keyboard costs 3499.

All three come from the same root cause: **one table is storing facts about several different things** (orders, customers, products).

Functional dependency: the idea behind normalization

$X \rightarrow Y$ (read "X determines Y") means: **if you know X, you know exactly one Y.**

- `customer_id` $\rightarrow$ `customer_name`, `customer_phone`, `customer_city`
- `product_id` $\rightarrow$ `product_name`, `unit_price`
- `order_id` $\rightarrow$ `order_date`, `customer_id`
- `(order_id, product_id)` $\rightarrow$ `qty`

Three kinds of dependency matter:

Kind	Meaning	Example
Full	a column depends on the <i>whole</i> key	<code>qty</code> depends on <code>(order_id, product_id)</code> together
Partial	a column depends on only <i>part</i> of a composite key	<code>product_name</code> depends only on <code>product_id</code> , not on <code>order_id</code>
Transitive	a non-key column depends on another non-key column	<code>order_id</code> $\rightarrow$ <code>customer_id</code> $\rightarrow$ <code>customer_city</code>

Normalization removes partial and transitive dependencies step by step.

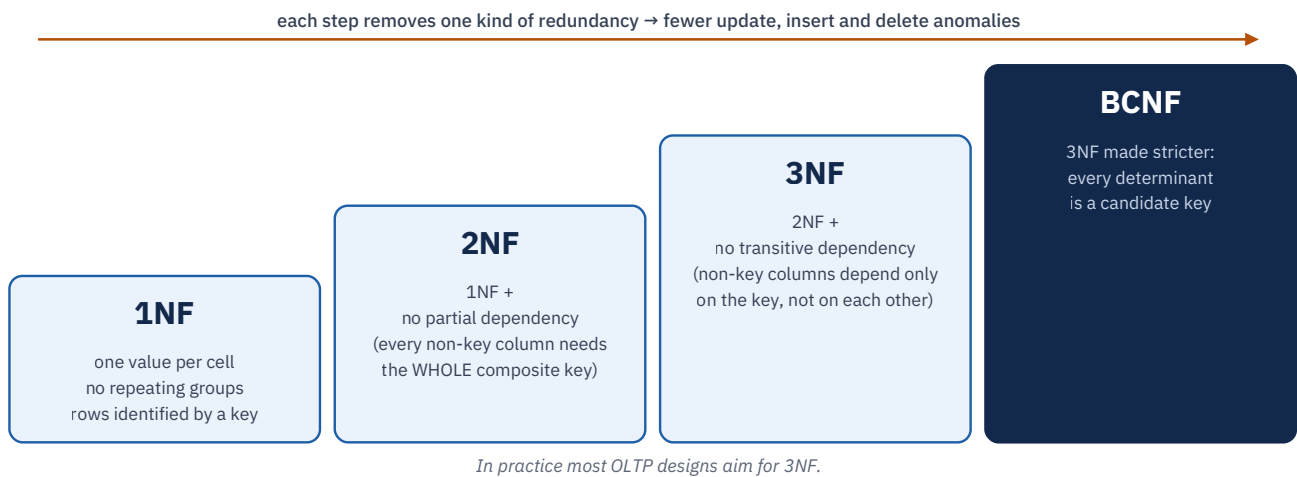


Figure 9. The normal forms build on each other; each removes one kind of redundancy.

First Normal Form (1NF)

Rule: every cell holds **one atomic value**, there are **no repeating groups**, and each row can be identified by a key.

Before (not 1NF), one cell holds a list:

order_id	customer_name	products
101	Aarav Patel	Wireless Mouse, Notebook Pack

After (1NF), one product per row. The key becomes (order_id, product_name), or better (order_id, product_id):

order_id	customer_name	product_name
101	Aarav Patel	Wireless Mouse
101	Aarav Patel	Notebook Pack

Second Normal Form (2NF)

Rule: 1NF **and** no partial dependency. Every non-key column must depend on the **whole** primary key. (This only matters when the key is composite.)

In `order_lines(order_id, product_id, qty, product_name, unit_price)`, the key is (order_id, product_id), but `product_name` and `unit_price` depend only on `product_id`. Move them out:

order_lines (order_id, product_id, qty)	products (product_id, product_name, unit_price)
(101, 1, 2), (101, 6, 5), (103, 4, 1)	(1, Wireless Mouse, 799), (6, Notebook Pack, 249), (4, Office Chair, 8999)

Third Normal Form (3NF)

Rule: 2NF **and** no transitive dependency. Non-key columns depend **only on the key**, not on other non-key columns.

In `orders(order_id, order_date, customer_id, customer_city)`, `customer_city` depends on `customer_id`, which is not the key. Move customer details to `customers`:

orders (order_id, order_date, customer_id)	customers (customer_id, customer_name, phone, city)
(101, 2024-01-05, 1), (103, 2024-01-28, 1)	(1, Aarav Patel, 98200 11111, Mumbai)

A memory line many interviewers like: *every non-key attribute must depend on "the key, the whole key, and nothing but the key".*

Boyce–Codd Normal Form (BCNF)

Rule: for every functional dependency $X \rightarrow Y$, **X must be a super key**. BCNF is a stricter 3NF; the difference shows up only when a table has overlapping candidate keys.

Example: `tutoring(student, subject, teacher)`. Each teacher teaches exactly one subject (`teacher → subject`), and each student has one teacher per subject. The candidate keys are `(student, subject)` and `(student, teacher)`. The table is in 3NF, but `teacher → subject` has a determinant (`teacher`) that is not a super key, so it is **not** in BCNF. The fix is to split it into `teacher_subject(teacher, subject)` and `student_teacher(student, teacher)`.

4NF AND 5NF

4NF removes multi-valued dependencies (independent lists stored in one table, like an employee's skills and languages in the same table). **5NF** deals with join dependencies. They are rarely asked at entry level; knowing their names and the one-line idea is enough.

The summary table to memorize

Normal form	Rule	Removes	Quick test
1NF	atomic values, no repeating groups, has a key	multi-valued cells	"Any cell with a list?"
2NF	1NF + no partial dependency	facts about <i>part</i> of a composite key	"Does a column depend on only part of the key?"
3NF	2NF + no transitive dependency	facts about non-key columns	"Does a column depend on another non-key column?"
BCNF	every determinant is a super key	remaining anomalies with overlapping keys	"Is there an $X \rightarrow Y$ where X isn't a key?"

The result: the handbook's sample database is the normalized version

Normalizing `order_details_flat` gives exactly the four tables this handbook uses everywhere: `customers`, `orders`, `products` and `order_items`. Nothing is lost: a join rebuilds the flat view whenever we need it. This is called a **lossless decomposition**.

```

SELECT o.order_id, o.order_date, c.customer_name, c.city,
       p.product_name, i.unit_price, i.quantity
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
JOIN order_items i ON i.order_id = o.order_id
JOIN products p ON p.product_id = i.product_id
WHERE o.order_id IN (101, 102, 103)
ORDER BY o.order_id, p.product_name;

```

SQL · POSTGRES SQL

OUTPUT

order_id	order_date	customer_name	city	product_name	unit_price	quantity
101	2024-01-05	Aarav Patel	Mumbai	Notebook Pack	249.00	5
101	2024-01-05	Aarav Patel	Mumbai	Wireless Mouse	799.00	2
102	2024-01-12	Isha Kulkarni	Pune	Mechanical Keyboard	3499.00	1
103	2024-01-28	Aarav Patel	Mumbai	Office Chair	8999.00	1

4 rows

LINE BY LINE

Line	What it does
FROM orders o	start from orders; o is a short alias
JOIN customers c ON c.customer_id = o.customer_id	attach each order's customer
JOIN order_items i ON i.order_id = o.order_id	attach the order's lines (one row per product)
JOIN products p ON p.product_id = i.product_id	attach product names
WHERE o.order_id IN (101, 102, 103)	only three orders, to keep the output short
ORDER BY ...	predictable order for reading

Aarav's phone or city now lives in **one** row of `customers` ; changing it fixes every order at once.

Advantages and disadvantages

Advantages	Disadvantages
No update, insert or delete anomalies	More tables, so queries need joins
Less storage wasted on duplicates	Very heavy reporting queries can get slower
Data stays consistent	Over-normalizing can make the design hard to use
Easier to change one fact	Requires understanding of the data's dependencies

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"Normalization is a design technique that reduces duplicate data and prevents update, insert and delete anomalies. We split a table so that each fact is stored once and connect the pieces with keys. In 1NF every cell has one value; in 2NF every non-key column depends on the whole key; in 3NF non-key columns depend only on the key, not on each other. For example, instead of repeating a customer's phone and city on every order, I keep a customers table and store just the customer_id in orders. For OLTP systems I normally aim for 3NF."

COMMON TRAPS & MISTAKES

- Saying 2NF applies to every table. Partial dependency is only possible with a **composite** key; a table with a single-column key in 1NF is automatically in 2NF.
- Confusing 2NF (partial dependency on part of the key) with 3NF (dependency between non-key columns).
- Claiming normalization always makes queries faster. Writes and consistency improve; complex reads may need more joins.
- Thinking "more normalization is always better". Warehouses deliberately denormalize.

INTERVIEW QUESTIONS**BASIC**

Q1 **DEFINITION** What is normalization?

A Organising data into related tables to remove redundancy and avoid insert, update and delete anomalies, linking the tables with keys.

Q2 DEFINITION What is 1NF?

A Every column holds atomic (single) values, there are no repeating groups, and each row is uniquely identifiable.

Q3 DEFINITION What is a functional dependency?

A $X \rightarrow Y$ means a value of X determines exactly one value of Y, e.g. `customer_id` $\rightarrow$ `customer_name`.

Q4 DEFINITION What are update, insert and delete anomalies?

A Update: the same fact stored in many rows becomes inconsistent when only some are changed. Insert: you cannot add a fact without unrelated data. Delete: removing a row unintentionally removes another fact.

INTERMEDIATE**Q5 COMPARISON What is the difference between 2NF and 3NF?**

A 2NF removes partial dependencies (a column depending on part of a composite key). 3NF removes transitive dependencies (a non-key column depending on another non-key column).

Q6 COMPARISON What is the difference between 3NF and BCNF?

A BCNF requires every determinant to be a super key. 3NF allows an exception when the dependent column is part of a candidate key, so a table can be in 3NF but not in BCNF when it has overlapping candidate keys.

Q7 DEFINITION What is a transitive dependency? Give an example.

A $A \rightarrow B$ and $B \rightarrow C$, so $A \rightarrow C$ indirectly through a non-key column. Example: `emp_id` $\rightarrow$ `dept_id` $\rightarrow$ `dept_name` in an employees table.

Q8 DEFINITION What is a lossless decomposition?

A Splitting a table into smaller tables such that joining them back gives exactly the original data, with no lost or extra rows.

SCENARIO-BASED**Q9 SCENARIO You have a table `orders(order_id, customer_name, customer_email, product, price, qty)`. How would you redesign it?**

A Split it into `customers(customer_id, name, email UNIQUE)`, `products(product_id, name, price)`, `orders(order_id, customer_id FK, order_date)` and `order_items(order_id FK, product_id FK, qty, unit_price, PK(order_id, product_id))`.

Q10 SCENARIO `employees(emp_id, emp_name, dept_id, dept_name, dept_location)`: which normal form is violated?

A 3NF, because `dept_name` and `dept_location` depend on `dept_id` (a non-key column). Move them into `departments(dept_id, dept_name, dept_location)`.

Q11 SCENARIO In `enrollments(student_id, course_id, course_title, grade)`, what's wrong?

A `course_title` depends only on `course_id`, which is part of the composite key, so it is a partial dependency and violates 2NF. Move it to `courses`.

FOLLOW-UP / TRAP**Q12 TRAP QUESTION Can normalization ever hurt performance?**

A Yes. Read-heavy reports may need many joins. That is why analytical systems denormalize, and OLTP systems sometimes keep a few derived columns or materialized views.

Q13 TRAP QUESTION Is a table with a single-column primary key always in 2NF?

A If it is already in 1NF, yes. Partial dependency needs a composite key.

Q14 TRAP QUESTION Up to which normal form do you normalize in practice?

A Usually 3NF (sometimes BCNF) for transactional systems; beyond that the benefit is rarely worth the complexity.

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

P1. In one sentence each, define 1NF, 2NF and 3NF.

P2. Name the anomaly: "We cannot add a new course to the database until a student enrolls in it."

P3. Write two functional dependencies that hold in the sample `employees` table.

LEVEL 2 – INTERVIEW APPLICATION

P4. Is this table in 1NF? `students(student_id, name, phone_numbers)` where `phone_numbers = "98200 11111, 98900 22222"`. Fix it.

P5. Explain to an interviewer why `order_items(order_id, product_id, qty, product_name)` is not in 2NF.

P6. Give an example of a transitive dependency from a hospital database.

LEVEL 3 – SCENARIO / PROBLEM SOLVING

P7. Normalize this table to 3NF and list the final tables with their keys: `invoice(invoice_no, invoice_date, customer_id, customer_name, customer_gstin, item_code, item_desc, item_rate, qty)`.

P8. A table `exam_results(student_id, subject, marks, teacher_name, teacher_phone)` is causing problems when teachers change phone numbers. Explain the anomaly and redesign it.

ANSWER / EXPLANATION

P1. 1NF: every cell holds a single atomic value and rows are uniquely identifiable. 2NF: in 1NF, and every non-key column depends on the whole primary key (no partial dependency). 3NF: in 2NF, and no non-key column depends on another non-key column (no transitive dependency).

P2. Insert anomaly.

P3. `emp_id → emp_name, email, dept_id, salary, hire_date, city` (`emp_id` is the primary key, so it determines every column), and `email → emp_id` (`email` is unique, so knowing it identifies the employee).

P4. No, the phone column holds a list. Create `student_phones(student_id FK, phone, PRIMARY KEY (student_id, phone))` and remove the list column from `students`.

P5. "The key is the pair (`order_id, product_id`), but `product_name` depends only on `product_id`, which is part of the key. That's a partial dependency, so it violates 2NF. If a product is renamed we'd have to update every order line. Product name belongs in the products table."

P6. `appointments(appointment_id, doctor_id, doctor_department)`: `appointment_id → doctor_id → doctor_department`. The department depends on the doctor, not on the appointment.

P7. Dependencies: `invoice_no → invoice_date, customer_id`; `customer_id → customer_name, customer_gstin`; `item_code → item_desc, item_rate`; (`invoice_no, item_code`) $\rightarrow$ `qty`. Final tables: `customers(customer_id PK, customer_name, customer_gstin UNIQUE)`; `items(item_code PK, item_desc, item_rate)`; `invoices(invoice_no PK, invoice_date, customer_id FK)`; `invoice_lines(invoice_no FK, item_code FK, qty, rate_charged, PK(invoice_no, item_code))`. Storing `rate_charged` on the line is deliberate: the price on an old invoice must not change when today's rate changes.

P8. Teacher details are repeated on every result row, so a phone change must be applied to many rows: an update anomaly. If a teacher's only result row is deleted, their phone is lost: a delete anomaly. Redesign: `teachers(teacher_id PK, teacher_name, teacher_phone)`, `subjects(subject_id PK, subject_name, teacher_id FK)` (if each subject has one teacher), `exam_results(student_id FK, subject_id FK, marks, PK(student_id, subject_id))`.

Session 2.6 — Denormalization: When and Why

Simple meaning

Denormalization means deliberately storing some data more than once, or storing pre-calculated results, so that reading becomes faster or simpler. It is a conscious trade: you accept extra storage and extra work on writes in exchange for quicker reads.

Normalization is the default for transactional systems. Denormalization is a targeted exception, applied where the reads justify it.

Why we denormalize

- **Joins cost time at scale.** Joining five large tables for every page view adds up.
- **Read-heavy workloads.** If a value is read a million times and changed once a day, pre-computing it makes sense.
- **Reporting and analytics.** Analysts want wide, simple tables (Session 2.7).
- **Distributed and NoSQL databases** often cannot join efficiently, so related data is stored together (Session 2.8).

Common denormalization techniques

Technique	Example	Cost you accept
Store a derived value	<code>orders.total_amount</code> instead of summing <code>order_items</code> every time	must be kept in sync when items change
Copy a column into another table	store <code>customer_name</code> on the invoice	name can drift from the customer table, which is sometimes <i>wanted</i> : the invoice should show the name at the time of sale
Summary (aggregate) table	<code>daily_sales(date, revenue)</code>	refresh jobs, possible staleness
Materialized view	a stored, refreshable query result	refresh cost; data only as fresh as the last refresh
Star schema	wide dimension tables in a warehouse	duplication inside dimensions

Example 1: a derived column in the sample database

The sample database itself is slightly denormalized: `orders.total_amount` repeats information that could be computed from `order_items`. That makes listing orders fast, but now we must check that the two never disagree. This query looks for any order whose stored total differs from its items:

```
SELECT o.order_id,
       o.total_amount,
       SUM(i.quantity * i.unit_price) AS items_total
FROM orders o
JOIN order_items i ON i.order_id = o.order_id
GROUP BY o.order_id, o.total_amount
HAVING o.total_amount <> SUM(i.quantity * i.unit_price);
```

SQL · POSTGRESQL

OUTPUT

order_id	total_amount	items_total
----------	--------------	-------------

0 rows

Zero rows means the copies are consistent. In production this check (or a trigger that recalculates the total) is part of the price of denormalizing.

Example 2: a materialized view

A **view** is a saved query that runs every time you read it. A **materialized view** stores the query's *result*, so reading it is as cheap as reading a table, until you refresh it.

```
CREATE MATERIALIZED VIEW monthly_sales AS
SELECT date_trunc('month', order_date)::date AS month,
       COUNT(*) AS orders,
       SUM(total_amount) AS revenue
FROM orders
WHERE status <> 'CANCELLED'
GROUP BY 1;

SELECT * FROM monthly_sales ORDER BY month;
```

SQL · POSTGRESQL

OUTPUT

month	orders	revenue
2024-01-01	3	15341.00
2024-02-01	2	5284.00
2024-03-01	4	38391.00
2024-04-01	4	42730.00

4 rows

LINE BY LINE

Line	What it does
CREATE MATERIALIZED VIEW monthly_sales AS	saves the result of the query below as a stored object
date_trunc('month', order_date)::date AS month	cuts each date down to the first day of its month, e.g. 2024-01-28 → 2024-01-01
COUNT(*) AS orders, SUM(total_amount) AS revenue	number of orders and revenue per month
WHERE status <> 'CANCELLED'	cancelled orders are not revenue
GROUP BY 1	group by the first column in the SELECT list (the month)
SELECT * FROM monthly_sales	reading it now costs no aggregation at all

When new orders arrive, `REFRESH MATERIALIZED VIEW monthly_sales;` recomputes it, typically on a schedule.

Normalize or denormalize? A decision table

Situation	Lean towards	Why
OLTP: orders, payments, bookings	Normalize (3NF)	correctness and cheap updates matter most
Write-heavy data	Normalize	each write touches one place
Read-heavy pages with stable data (product listing)	Denormalize selectively	fewer joins per page view
Data warehouse / BI	Denormalize (star schema)	simple, fast analytical queries
NoSQL document store	Denormalize (embed)	joins are limited; read patterns drive design
Legal or historical snapshot (invoice)	Copy on purpose	the value at that moment must be preserved

	Normalization	Denormalization
Goal	remove redundancy, protect consistency	speed up reads, simplify queries
Number of tables	more	fewer / wider
Writes	simple and cheap	more work: several copies to update
Reads	may need many joins	fewer joins
Risk	slower complex reads	inconsistent copies, more storage
Typical home	OLTP	OLAP, caches, NoSQL

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"Denormalization is intentionally adding redundancy, like storing an order total or copying a customer's name, to make reads faster or simpler. It is the opposite trade-off to normalization: reads get cheaper, but writes get more expensive and we risk copies going out of sync, so we need triggers, refresh jobs or checks. I'd normalize transactional data to 3NF first, and then denormalize specific read-heavy paths, use materialized views or summary tables, or use a star schema in the warehouse."

COMMON TRAPS & MISTAKES

- Treating denormalization as "not knowing normalization". A good answer says you normalize first, then denormalize *on purpose*, with a reason.
- Forgetting the cost: someone must keep the copies in sync.
- Confusing a view (re-runs every time) with a materialized view (stored result, needs refresh).

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What is denormalization?

A Deliberately adding redundant or pre-computed data to a design to make reads faster or simpler, at the cost of extra storage and more complex writes.

Q2 **COMPARISON** **View vs materialized view?**

- A** A view stores only the query and runs it on every read. A materialized view stores the result physically, so it is fast to read but must be refreshed to show new data.

INTERMEDIATE**Q3** **WHY** **When would you denormalize?**

- A** For read-heavy workloads where joins are a bottleneck, for reporting and warehouses, for NoSQL stores without efficient joins, and when a historical snapshot must be preserved (an invoice's price).

Q4 **HOW** **How do you keep denormalized data consistent?**

- A** Update all copies in the same transaction, use triggers, schedule refresh jobs for summary tables and materialized views, and run reconciliation checks that compare copies with the source.

SCENARIO-BASED**Q5** **SCENARIO** **Your product listing page joins six tables and is slow. What would you consider?**

- A** First check indexes and the query plan. If the joins are still the bottleneck, add a denormalized read model: a materialized view or a cached JSON document per product, refreshed when products change.

Q6 **SCENARIO** **Should an invoice store the product price or always look it up from the products table?**

- A** Store the price charged on the invoice line. Prices change; an old invoice must keep showing what was actually charged.

FOLLOW-UP / TRAP**Q7** **TRAP QUESTION** **Is denormalization the same as having no normalization?**

- A** No. Denormalization is a deliberate step taken *after* designing a normalized model, for a specific performance or simplicity reason.

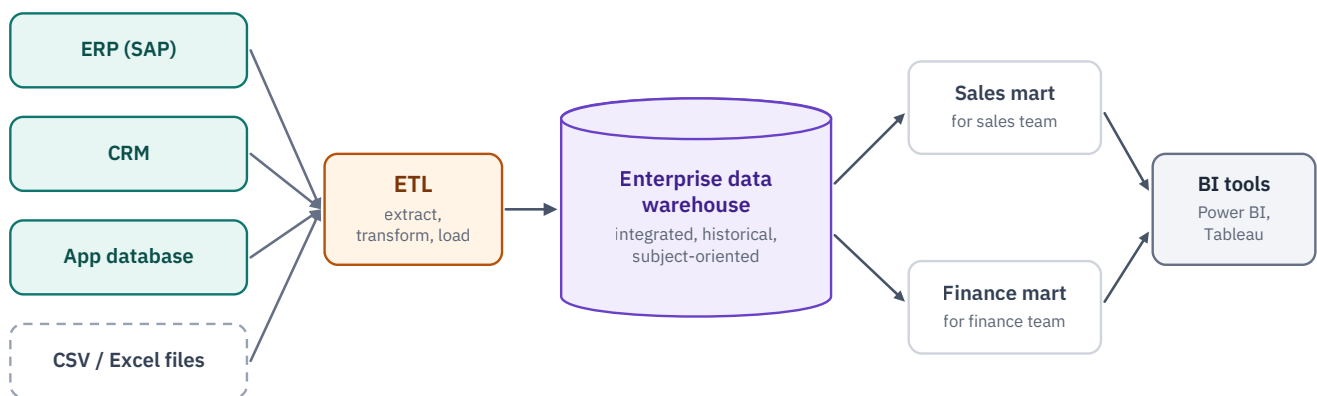
Session 2.7 — Data Warehousing and Dimensional Modeling

What is a data warehouse?

Simple meaning: a data warehouse is a central database built for **analysis**, not for running the business. It collects data from many operational systems, cleans and integrates it, and keeps years of history, so people can answer questions like "How did sales in each region change over the last three years?"

The classic definition (Bill Inmon) describes four properties. In plain words:

Property	Meaning
Subject-oriented	organised around business subjects (sales, customers, inventory), not around applications
Integrated	data from many sources made consistent (same codes, formats and definitions)
Time-variant	keeps history, so you can see how things changed over time
Non-volatile	data is loaded and read; it is not constantly updated or deleted like OLTP data



Schema-on-write: data is cleaned and modelled BEFORE it is loaded.

Figure 10. Classic warehouse architecture: sources → ETL → warehouse → data marts → BI tools.

- **Staging area:** a landing zone where raw extracts are cleaned before loading.
- **Data mart:** a smaller, focused slice of the warehouse for one department, such as a sales mart or finance mart.
- **BI tools:** Power BI, Tableau or Looker read from the warehouse or the marts.

Dimensional modeling: facts and dimensions

Warehouses are usually designed with **dimensional modeling** (Ralph Kimball's approach). Every analytical question has the same shape: *a number, sliced by some context*. "Revenue (number) by month (when), by product category (what), by city (where)."

- A **fact table** stores the **numbers** (measures) of a business event, plus foreign keys to its context. Example: `fact_sales(date_key, product_key, customer_key, quantity, sales_amount)`.
- A **dimension table** stores the **descriptive context**: who, what, where, when. Examples: `dim_date`, `dim_product`, `dim_customer`, `dim_store`.

	Fact table	Dimension table
Contains	measures (amount, quantity) + foreign keys	descriptive attributes (name, category, city)
Shape	very many rows, few columns (long and thin)	fewer rows, many columns (short and wide)
Changes	new rows appended for every event	changes slowly (a customer moves city)
Example row	(20240105, product 1, customer 1, qty 2, ₹1598)	(product 1, Wireless Mouse, Electronics)
Typical question it answers	"how much / how many?"	"by what / for whom / when / where?"

Grain: the first decision

The **grain** is what one row of the fact table represents, for example "one row per product per order line". Decide the grain *before* anything else; it determines which dimensions and measures make sense. Mixing grains (some rows per line, some per order) produces double counting.

Types of measures INTERVIEW EXTENSION

Type	Can you SUM it across everything?	Example
Additive	yes	sales amount, quantity
Semi-additive	across some dimensions, not time	account balance (summing balances across days is meaningless)
Non-additive	no	ratios and percentages (profit margin): recompute from components

Star schema

In a **star schema**, one fact table sits in the middle and each dimension connects to it directly, like the points of a star. Dimensions are **denormalized**: `dim_product` holds the product name, category and brand together.

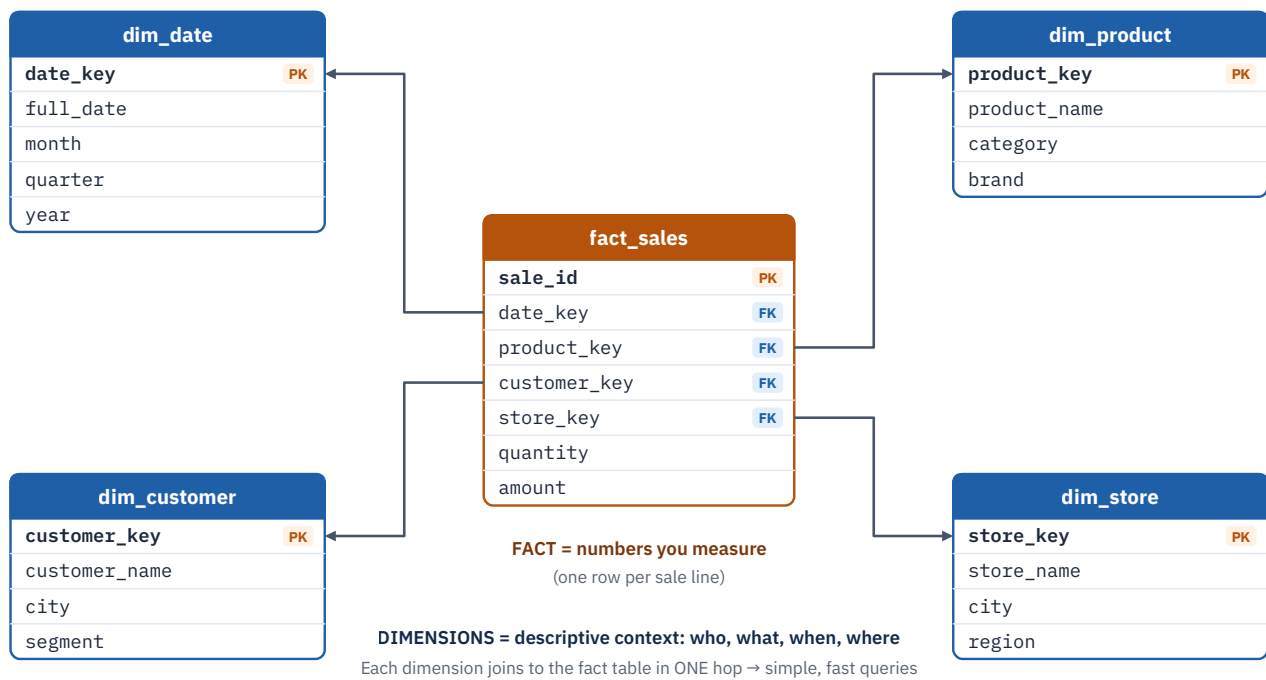


Figure 11. A star schema: every dimension is one join away from the fact table.

Building and querying a tiny star schema for real

The sample database is an OLTP design. The block below performs a mini ETL: it reshapes the OLTP tables into a star schema. In a real warehouse, a pipeline would do this, and the keys would be generated surrogate keys. Here the source IDs are reused to keep it short.

```
CREATE TABLE dim_date AS
SELECT DISTINCT
    to_char(order_date, 'YYYYMMDD')::int AS date_key,
    order_date AS full_date,
    to_char(order_date, 'Mon') AS month_name,
    EXTRACT(MONTH FROM order_date)::int AS month_no,
    EXTRACT(QUARTER FROM order_date)::int AS quarter,
    EXTRACT(YEAR FROM order_date)::int AS year
FROM orders;
ALTER TABLE dim_date ADD PRIMARY KEY (date_key);

CREATE TABLE dim_product AS
SELECT product_id AS product_key, product_name, category
FROM products;
ALTER TABLE dim_product ADD PRIMARY KEY (product_key);

CREATE TABLE dim_customer AS
SELECT customer_id AS customer_key, customer_name, city
FROM customers;
ALTER TABLE dim_customer ADD PRIMARY KEY (customer_key);

CREATE TABLE fact_sales AS
SELECT to_char(o.order_date, 'YYYYMMDD')::int AS date_key,
    i.product_id AS product_key,
    o.customer_id AS customer_key,
    i.quantity,
```

SQL · POSTGRESQL

```

        i.quantity * i.unit_price           AS sales_amount
FROM orders o
JOIN order_items i ON i.order_id = o.order_id
WHERE o.status <> 'CANCELLED';

```

The grain of `fact_sales` is one row per order line (cancelled orders excluded). A typical analytical question is "revenue and units by month and category":

```

SELECT d.month_name,
       p.category,
       SUM(f.sales_amount) AS revenue,
       SUM(f.quantity)    AS units
FROM fact_sales f
JOIN dim_date   d ON d.date_key   = f.date_key
JOIN dim_product p ON p.product_key = f.product_key
GROUP BY d.month_no, d.month_name, p.category
ORDER BY d.month_no, revenue DESC;

```

OUTPUT

month_name	category	revenue	units
Jan	Furniture	8999.00	1
Jan	Electronics	5097.00	3
Jan	Stationery	1245.00	5
Feb	Electronics	2798.00	2
Feb	Stationery	2486.00	14
Mar	Furniture	24998.00	2
Mar	Electronics	13393.00	7
Apr	Furniture	33997.00	3
Apr	Electronics	5498.00	2
Apr	Stationery	3235.00	15

10 rows

LINE BY LINE

Line	What it does
FROM fact_sales f	start from the numbers
JOIN dim_date d ... / JOIN dim_product p ...	each dimension is exactly one join away; that is the "star"
SUM(f.sales_amount) AS revenue, SUM(f.quantity) AS units	the measures, aggregated
GROUP BY d.month_no, d.month_name, p.category	slice by month and category; month_no is included so we can sort months correctly
ORDER BY d.month_no, revenue DESC	January first; inside each month, the biggest category first

Every star-schema query looks like this: **join the fact to the dimensions you need, filter and group by dimension attributes, aggregate the measures.**

Snowflake schema

A **snowflake schema** normalizes the dimensions into sub-dimensions: `dim_product` → `dim_category` → `dim_department`. The diagram branches out like a snowflake.

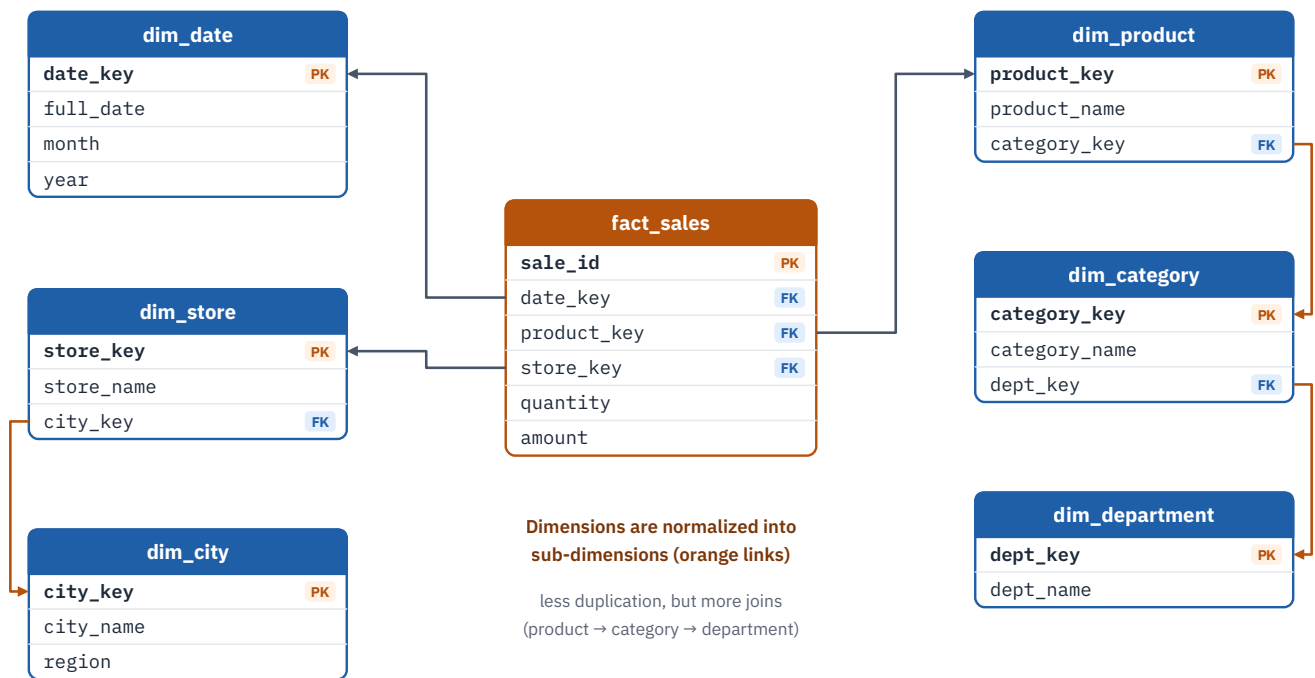


Figure 12. A snowflake schema: dimensions are split into further normalized tables.

	Star schema	Snowflake schema
Dimensions	denormalized, one table each	normalized into several tables
Joins per query	fewer (one hop)	more (several hops)
Query speed	usually faster	usually slower
Storage	some duplication inside dimensions	less duplication
Ease of use for analysts and BI tools	simpler	more complex
Maintenance of dimension data	updates may touch many rows	updates touch one place
Most common choice	yes, the default in most warehouses	when dimensions are huge or shared hierarchies matter

OLTP database vs data warehouse

	Operational database (OLTP)	Data warehouse (OLAP)
Purpose	run day-to-day transactions	analyse history and trends
Design	normalized (3NF)	dimensional (star/snowflake)
Data	current state	historical, time-stamped
Workload	many small reads and writes	few large reads and aggregations
Updates	constant inserts, updates, deletes	periodic loads, mostly appends
Users	applications, customers, staff	analysts, managers, data scientists
Examples	PostgreSQL, MySQL, Oracle	Snowflake, BigQuery, Redshift, Synapse

Slowly changing dimensions (SCD) INTERVIEW EXTENSION

What should happen in the warehouse when a customer moves from Mumbai to Pune? That is the **slowly changing dimension** problem.

Type	What happens	History kept?	Example use
Type 0	never change the value	original only	date of birth
Type 1	overwrite the old value	no	fixing a spelling mistake
Type 2	add a new row with validity dates; old row is closed	full history	customer city, for correct historical reports
Type 3	keep a "previous value" column	one level	<code>current_city</code> , <code>previous_city</code>

Type 2 is the one interviewers ask about. Each version of the customer gets its own surrogate key:

```
CREATE TABLE dim_customer_scd2 (
    customer_sk    INT PRIMARY KEY,          -- surrogate key: one per version
    customer_id    INT NOT NULL,             -- business key from the source system
    customer_name  VARCHAR(60) NOT NULL,
    city           VARCHAR(40) NOT NULL,
    valid_from     DATE NOT NULL,
    valid_to       DATE,                    -- NULL = still the current version
    is_current     BOOLEAN NOT NULL
);

INSERT INTO dim_customer_scd2 VALUES
(1, 1, 'Aarav Patel', 'Mumbai', '2023-11-05', '2024-02-29', false),
(2, 1, 'Aarav Patel', 'Pune', '2024-03-01', NULL, true),
(3, 2, 'Isha Kulkarni', 'Pune', '2023-12-12', NULL, true);
```

SQL · POSTGRES SQL

"Which city was Aarav in on 15 January 2024?" Sales made then should still count for Mumbai:

```
SELECT customer_sk, customer_name, city
FROM dim_customer_scd2
WHERE customer_id = 1
    AND DATE '2024-01-15' >= valid_from
    AND (valid_to IS NULL OR DATE '2024-01-15' <= valid_to);
```

SQL · POSTGRES SQL

OUTPUT

customer_sk	customer_name	city
1	Aarav Patel	Mumbai

1 row

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"A data warehouse is a central store designed for analysis. It integrates data from many operational systems and keeps history. We usually model it dimensionally: fact tables hold the measurable events, like sales amount and quantity, with foreign keys to dimension tables that describe the context, such as date, product, customer and store. In a star schema each dimension joins directly to the fact table, which keeps queries simple and fast. A snowflake schema normalizes the dimensions further, saving space but adding joins. The first design decision is the grain, what one fact row means."

COMMON TRAPS & MISTAKES

- Putting descriptive text (product names) in the fact table instead of a dimension.
- Forgetting to define the grain, which leads to double counting.

- Summing a semi-additive measure such as account balance over time.
- Saying a snowflake schema is "more advanced, therefore better". The star is the usual default.
- Confusing a data warehouse (curated, modelled, schema-on-write) with a data lake (raw files, schema-on-read). See Session 4.7.

INTERVIEW QUESTIONS

BASIC

Q1 DEFINITION What is a data warehouse?

A central, integrated, historical store of data designed for analysis and reporting, separate from the operational systems.

Q2 COMPARISON What is the difference between a fact table and a dimension table?

A fact table holds measures (numbers) of business events plus foreign keys; a dimension table holds descriptive attributes (who, what, when, where) used to filter and group the facts.

Q3 DEFINITION What is a star schema?

A dimensional design with one central fact table connected directly to denormalized dimension tables.

INTERMEDIATE

Q4 COMPARISON Star schema vs snowflake schema?

In a star, dimensions are denormalized single tables, so there are fewer joins and faster, simpler queries. In a snowflake, dimensions are normalized into sub-tables, so there is less redundancy but more joins.

Q5 DEFINITION What is the grain of a fact table?

The level of detail one row represents, for example one row per order line or one row per store per day.

Q6 DEFINITION What is a data mart?

A subject- or department-specific subset of the warehouse, such as a sales mart.

Q7 COMPARISON OLTP database vs data warehouse?

OLTP is normalized, holds current data and handles many small transactions; a warehouse is dimensional, holds history and handles large analytical queries.

SCENARIO-BASED

Q8 DESIGN QUESTION Design a star schema for a food-delivery company's order analytics.

A Fact: `fact_order_line(date_key, time_key, customer_key, restaurant_key, item_key, rider_key, quantity, item_amount, delivery_fee, delivery_minutes)` at the grain of one row per item ordered. Dimensions: date, time of day, customer (city, segment), restaurant (cuisine, area), menu item (category, veg/non-veg), rider.

Q9 SCENARIO A customer moved from Mumbai to Pune, and old sales now show under Pune. What went wrong and how do you fix it?

A The dimension was overwritten (SCD Type 1). Use SCD Type 2: close the old row with a `valid_to` date, insert a new row with a new surrogate key, and point new facts at the new key. Old facts keep pointing to the Mumbai version.

FOLLOW-UP / TRAP

Q10 TRAP QUESTION Why do warehouses use surrogate keys instead of source system IDs?

A To integrate several sources whose IDs may clash, to keep multiple versions of the same entity (SCD Type 2), and to stay independent of changes in source systems.

Q11 **TRAP QUESTION** Can you sum an account balance across months?

A No. Balance is semi-additive; sum it across accounts for one day, but use the closing or average balance across time.

Session 2.8 — NoSQL Data Modeling

Why NoSQL exists (the modeling view)

Relational databases are excellent, but some problems push against them:

- **Huge scale:** billions of rows and very high write rates, spread across many servers.
- **Changing structure:** products with completely different attributes (a laptop has RAM; a T-shirt has a size).
- **Known, simple access patterns:** "get a user's whole cart by user ID" a million times a second.

NoSQL databases relax some relational features (fixed schemas, joins, sometimes strict consistency) to gain flexibility and horizontal scale. Session 4.2 compares the database types; here we focus on **how you model data** in them, especially in document databases such as MongoDB.

Schema flexibility (and its catch)

- Relational: **schema-on-write**. The table structure is fixed first; every row must fit.
- Document stores: **flexible schema**. Documents in the same collection can have different fields.

```
// Two products in the same "products" collection, with different fields
db.products.insertMany([
  { _id: 1, name: "ThinkPad E14", category: "Laptop",
    price: 62990, specs: { ram_gb: 16, cpu: "Ryzen 5" } },
  { _id: 2, name: "Cotton Tee", category: "Apparel",
    price: 499, sizes: ["S", "M", "L"], colour: "navy" }
])
```

MONGODB SHELL (MONGOSH)

The catch: **"schemaless" does not mean "no design"**. The schema moves into your application code. Without discipline you get `"price": "499"` in one document and `"price": 499` in another. MongoDB supports **schema validation** rules to prevent this.

Embedding vs referencing

The central decision in document modeling: put related data **inside** one document (embed), or keep it in a separate document and store its ID (reference)?

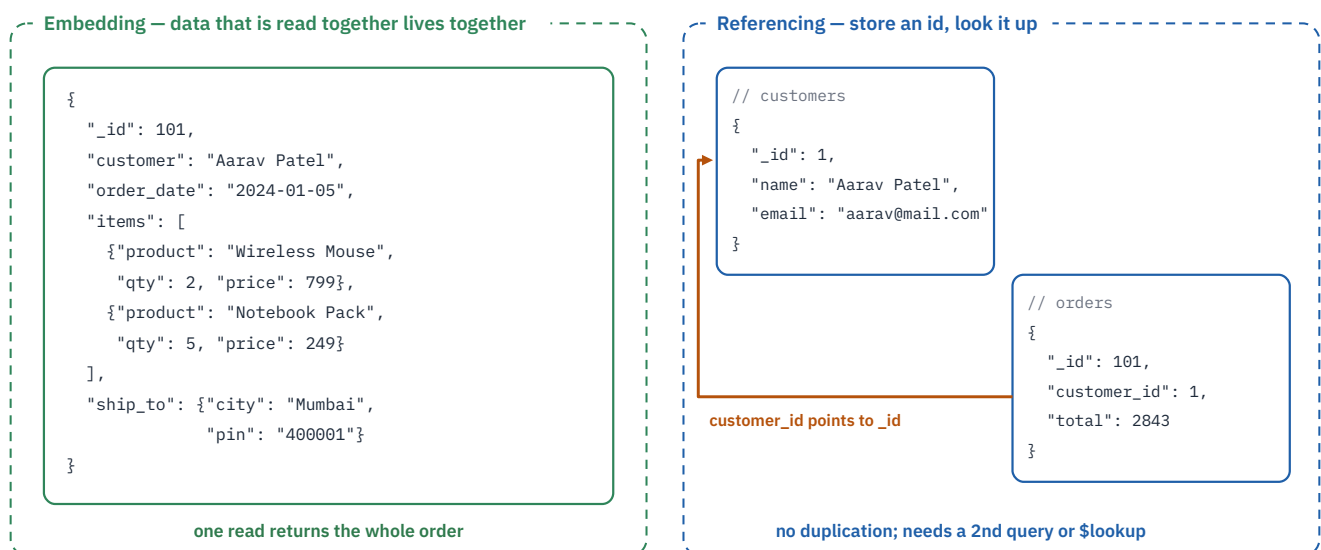


Figure 13. Embedding keeps data that is read together in one document; referencing links separate documents by ID.

	Embedding	Referencing
How	nested object or array inside the parent	store the other document's <code>_id</code>
Reads	one query returns everything	two queries, or a <code>\$lookup</code> (a join)
Writes	update one document (atomic)	update the shared document once, everywhere sees it
Duplication	data may be repeated across parents	no duplication
Growth	risky if the array grows without limit (MongoDB documents max out at 16 MB)	fine for large or unbounded sets
Use when	"contains" relationships, one-to-few, read together, owned by the parent	many-to-many, large or unbounded sets, data shared and updated independently

Rules of thumb:

- Order and its line items → **embed** (always read together, owned by the order, bounded).
- Order and customer → **reference** the customer (customer data is shared by many orders and changes independently). Optionally copy the name and address *as of the order* (denormalization for history).
- Blog post and comments → embed the latest few comments for fast display, and keep all comments in their own collection (a hybrid).
- Students and courses (many-to-many) → reference by ID on one or both sides.

Access-pattern-driven design

Relational modeling starts from the **data** ("what are the entities?") and normalizes. NoSQL modeling starts from the **queries** ("what will the app ask for, how often?") and shapes documents so the most frequent queries need a single read.

1. List the access patterns, for example: "show order with items" (very frequent), "list a customer's last 10 orders" (frequent), "monthly revenue report" (rare, can run in a warehouse).
2. Design documents so each frequent pattern is one read.
3. Add indexes for the filters you use (`customer_id`, `order_date`).
4. Accept some duplication where it saves joins, and decide how copies will be updated.

	Relational mindset	Document (NoSQL) mindset
Starting point	entities and normalization	access patterns (queries)
Duplication	avoid it	acceptable when it saves reads
Relationships	foreign keys + joins	embedding, or references with application-side joins
Schema	fixed, enforced by the DB	flexible, enforced by the app (plus optional validation)
Changing questions later	easy: new joins	harder: may need remodeling

A FOOD-DELIVERY ORDER AS ONE MONGODB DOCUMENT

MONGODB SHELL (MONGOSH)

```
{
  "_id": "ORD-9001",
  "customer": { "customer_id": 42, "name": "Diya Joshi" }, // snapshot copy
  "restaurant_id": 17, // reference
  "items": [ // embedded
    { "name": "Paneer Biryani", "qty": 1, "price": 249 },
    { "name": "Gulab Jamun", "qty": 2, "price": 60 }
  ],
  "delivery": { "address": "HSR Layout, Bengaluru", "status": "OUT_FOR_DELIVERY" },
  "total": 369,
  "created_at": ISODate("2024-04-19T19:42:00Z")
}
```

One read renders the whole order screen. The restaurant is referenced because its menu and details change independently; the customer's name is copied as it was at order time.

KEY-VALUE AND WIDE-COLUMN MODELING IN ONE PARAGRAPH

In key-value and wide-column stores (DynamoDB, Cassandra), you design around a **partition key** (which server holds the data, e.g. `customer_id`) and often a **sort key** (order inside the partition, e.g. `order_date`). Queries that do not use the partition key are expensive, so the table is shaped around exactly the queries you need. Some teams even store several entity types in one table ("single-table design").

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"In NoSQL, especially document databases like MongoDB, we model around access patterns instead of normalizing first. The main decision is embedding versus referencing. I embed data that belongs to the parent and is read with it, like an order's line items, because one read returns everything and the update is atomic. I reference data that is shared, large, unbounded or updated independently, like the customer or a restaurant, to avoid duplication and huge documents. Flexible schema doesn't mean no schema; the structure lives in the application, ideally backed by validation rules."

COMMON TRAPS & MISTAKES

- "NoSQL means no schema, so no design is needed." The design moves into the application and the access patterns.
- Embedding arrays that grow forever (all of a user's activity logs in the user document) and hitting the 16 MB document limit.
- Using MongoDB exactly like a relational database, with every relationship as a reference and `$lookup` everywhere. You lose the main benefit.
- Forgetting that duplicated (embedded) data must be updated in several places if it changes.

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What does "schema flexibility" mean in NoSQL?

A Records (documents) in the same collection can have different fields, and the structure can evolve without migrating a fixed table schema.

Q2 **COMPARISON** **Embedding vs referencing in MongoDB?**

- A** Embedding stores related data inside the parent document (one read, atomic updates, possible duplication); referencing stores the related document's ID (no duplication, but extra queries or `$lookup`).

INTERMEDIATE**Q3** **HOW** **How do you decide whether to embed or reference?**

- A** Embed when the data is owned by the parent, read together, and bounded (one-to-few). Reference when it is shared, many-to-many, large or unbounded, or updated independently.

Q4 **DEFINITION** **What is access-pattern-driven design?**

- A** Designing the data model starting from the application's most frequent queries, so each of them can be served efficiently, often with a single read.

Q5 **WHY** **Why is denormalization common in NoSQL?**

- A** Many NoSQL systems have limited or expensive joins, and they are optimised for fast single-key reads at scale, so related data is stored together.

SCENARIO-BASED**Q6** **DESIGN QUESTION** **Model a blog with posts and thousands of comments per post in MongoDB.**

- A** Store posts in a `posts` collection, possibly embedding the latest five comments for quick display; store all comments in a separate `comments` collection with `post_id` and an index on (`post_id`, `created_at`). Embedding every comment would make documents unbounded.

Q7 **SCENARIO** **A product catalogue has very different attributes per category. SQL or document model?**

- A** A document model fits well because each product can carry its own attributes. In PostgreSQL, a JSONB column for category-specific attributes is a good middle ground.

FOLLOW-UP / TRAP**Q8** **TRAP QUESTION** **Does MongoDB support joins?**

- A** Yes, the `$lookup` stage in the aggregation pipeline performs a left-outer-join-like operation, but heavy reliance on it usually means the model should embed more.

Q9 **TRAP QUESTION** **Is there a limit to embedding?**

- A** Yes. A MongoDB document can be at most 16 MB, and large, ever-growing arrays also make updates and reads slower.

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

P1. Give one example each of an additive and a semi-additive measure.

P2. Which schema has fewer joins: star or snowflake?

P3. True or false: in MongoDB, two documents in the same collection must have the same fields.

LEVEL 2 – INTERVIEW APPLICATION

P4. Explain the difference between a view and a materialized view, and when you would use each.

P5. You must store a user and their three saved addresses in MongoDB. Embed or reference? Why?

P6. Name three things that go into a dimension table and three that go into a fact table for a retail store.

LEVEL 3 – SCENARIO / PROBLEM SOLVING

P7. Design a star schema for a hospital that wants to analyse "number of appointments and revenue by doctor, department, month and patient age group". State the grain.

P8. Write a SQL query on this chapter's `fact_sales`, `dim_customer` and `dim_date` tables that returns revenue per customer city per quarter.

ANSWER / EXPLANATION

P1. Additive: sales amount (sum across any dimension). Semi-additive: inventory level or account balance (sum across products or stores, but not across days).

P2. The star schema.

P3. False. MongoDB has a flexible schema; documents can differ unless validation rules restrict them.

P4. A view is a saved query that runs on every read, so it is always current but costs the full query each time; use it to simplify or secure access. A materialized view stores the result, so it is fast to read but stale until refreshed; use it for expensive aggregations read often, such as dashboard summaries.

P5. Embed. The addresses belong to the user, there are only a few, and they are displayed together with the user, so one read and atomic updates are ideal.

P6. Dimension: product name and category, store city and region, the date's month and weekday. Fact: quantity sold, sales amount, discount amount.

P7. Grain: one row per appointment. Fact: `fact_appointment(date_key, doctor_key, department_key, patient_key, appointment_count = 1, consultation_fee, billed_amount)`. Dimensions: `dim_date` (day, month, quarter, year), `dim_doctor` (name, specialization, seniority), `dim_department` (name, building), `dim_patient` (gender, age_group, city). If age group changes over time, either store it in the fact at appointment time or use SCD Type 2.

P8. One possible answer:

```
SELECT c.city,
       d.year,
       d.quarter,
       SUM(f.sales_amount) AS revenue
FROM fact_sales f
JOIN dim_customer c ON c.customer_key = f.customer_key
JOIN dim_date      d ON d.date_key    = f.date_key
GROUP BY c.city, d.year, d.quarter
ORDER BY d.year, d.quarter, revenue DESC;
```

SQL · POSTGRESQL

OUTPUT

city	year	quarter	revenue
Mumbai	2024	1	36840.00
Ahmedabad	2024	1	6998.00
Bengaluru	2024	1	6796.00
Pune	2024	1	5985.00
Delhi	2024	1	2397.00
Pune	2024	2	17998.00
Delhi	2024	2	15999.00
Mumbai	2024	2	5498.00
Ahmedabad	2024	2	3235.00

9 rows

All sample orders fall in Q1 and Q2 of 2024, so there are only two quarters.

Session 2.9 — Module 2 Summary & Rapid Revision

MODULE 2 SUMMARY

MOST IMPORTANT CONCEPTS

- **Data modeling** = entities, attributes, relationships and keys; conceptual → logical → physical.
- **Keys:** primary (unique + not null, one per table), foreign (points to a PK/unique key, enforces referential integrity), unique, composite, candidate, super, surrogate vs natural.
- **Relationships:** 1:N → FK on the many side; 1:1 → FK + UNIQUE; M:N → junction table.
- **Normalization:** removes update/insert/delete anomalies. 1NF atomic values; 2NF no partial dependency; 3NF no transitive dependency; BCNF every determinant is a super key.
- **Denormalization:** deliberate redundancy for faster reads (derived columns, summary tables, materialized views, star schemas).
- **Warehousing:** facts (measures) + dimensions (context); grain first; star (default) vs snowflake; SCD Type 2 keeps history.
- **NoSQL modeling:** access-pattern-driven; embed what is owned and read together, reference what is shared or unbounded.

WHAT TO MEMORIZE

- "The key, the whole key, and nothing but the key" (1NF, 2NF, 3NF).
- PK vs FK vs UNIQUE differences, including NULL rules.
- Star = denormalized dimensions, fewer joins; snowflake = normalized dimensions, more joins.

WHAT TO UNDERSTAND

- *Why* anomalies happen (one table storing facts about several things).
- *When* to break normalization, and what it costs.
- *How* the grain controls a fact table's correctness.

MOST COMMON INTERVIEW QUESTIONS

- Normalize this table. • 2NF vs 3NF? • PK vs unique key? • Can a FK be NULL? • How to model many-to-many? • Star vs snowflake? • Fact vs dimension? • Embed or reference?

COMMON MISTAKES

- 2NF for single-column keys • FK on the wrong side • lists in a column • FLOAT for money • facts holding descriptive text • "NoSQL needs no design".

RAPID REVISION CHECKLIST

- | | |
|---|---|
| ☐ I can explain conceptual vs logical vs physical models | ☐ I can model 1:1, 1:N and M:N relationships |
| ☐ I can identify entities and relationships from a paragraph | ☐ I can name the three anomalies with examples |
| ☐ I know every type of key with an example | ☐ I can normalize a table to 3NF step by step |
| ☐ I can explain PK vs FK vs UNIQUE (and NULL rules) | ☐ I can explain BCNF in one sentence |
| ☐ I know the referential actions (CASCADE, SET NULL...) | ☐ I can justify when to denormalize |
| | ☐ I can design a star schema and state its grain |
| | ☐ I can compare star and snowflake schemas |
| | ☐ I can explain SCD Type 2 |
| | ☐ I can decide between embedding and referencing |

PART III • MODULE 3

Basics of SQL (Foundational)

SQL is the single most tested skill in data and software interviews, and it is where preparation pays off fastest. This module teaches SQL from the first SELECT to window functions, indexing and transactions, then drills the 22 query patterns that interviewers reuse again and again.

Every query in this module was **executed on PostgreSQL while this PDF was built**, and the output you see is the real output. Type the queries yourself: reading SQL and writing SQL are different skills.

COURSE COVERAGE

- ✓ **PostgreSQL Basics — Part 1:** PostgreSQL and psql, data types, SELECT, FROM, WHERE, DISTINCT, ORDER BY, LIMIT, OFFSET, aliases → Session 3.1
- ✓ **PostgreSQL Basics — Part 2:** comparison and logical operators, IN, BETWEEN, LIKE, NULL, aggregate, string, date and numeric functions, CASE → Session 3.2
- ✓ **DDL:** CREATE, ALTER, DROP, TRUNCATE, constraints → Session 3.3
- ✓ **DML:** INSERT, UPDATE, DELETE; also DCL (GRANT, REVOKE) and TCL (COMMIT, ROLLBACK) → Session 3.4
- ✓ Grouping and aggregation: GROUP BY, HAVING, WHERE vs HAVING † → Session 3.5
- ✓ Joins: inner, left, right, full outer, cross, self; referential integrity † → Session 3.6
- ✓ Subqueries and CTEs: scalar, correlated, in WHERE / FROM / SELECT; WITH; recursive CTE † → Session 3.7
- ✓ **Window Functions & Indexing:** OVER, PARTITION BY, ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, running and moving totals → Session 3.8; indexes, composite, clustered vs non-clustered → Session 3.9
- ✓ Transactions: COMMIT, ROLLBACK, ACID, isolation, dirty / non-repeatable / phantom reads † → Session 3.10
- ✓ Practice programs → the 22 SQL interview patterns, Session 3.11
- ✓ Module quiz concepts → interview questions in each session and Session 3.12

Topic	Priority
SELECT basics, filtering, NULL handling	HIGH PRIORITY
GROUP BY / HAVING, joins	HIGH PRIORITY MUST KNOW
Subqueries, CTEs	HIGH PRIORITY
Window functions (ROW_NUMBER, RANK, DENSE_RANK, LAG/LEAD)	HIGH PRIORITY
Indexing, transactions, ACID	HIGH PRIORITY
DDL, DML, DELETE vs TRUNCATE vs DROP	HIGH PRIORITY
Recursive CTEs, isolation levels	MEDIUM PRIORITY

PWC ANGLE

Public PwC interview write-ups repeatedly mention SQL joins and their types, GROUP BY queries, creating and altering tables, and DBMS basics such as ACID and normalization (see the PwC Interview Orientation chapter). Data-engineering candidates also report window functions and CTEs. If you master this module and the patterns in Session 3.11, you are covered for that range.

Session 3.0 – Meet the Sample Database

All examples in Modules 2 and 3 use one small, realistic database with an HR part, an e-commerce part and a few extra tables. Knowing its data makes every output easy to check. Bookmark this session.

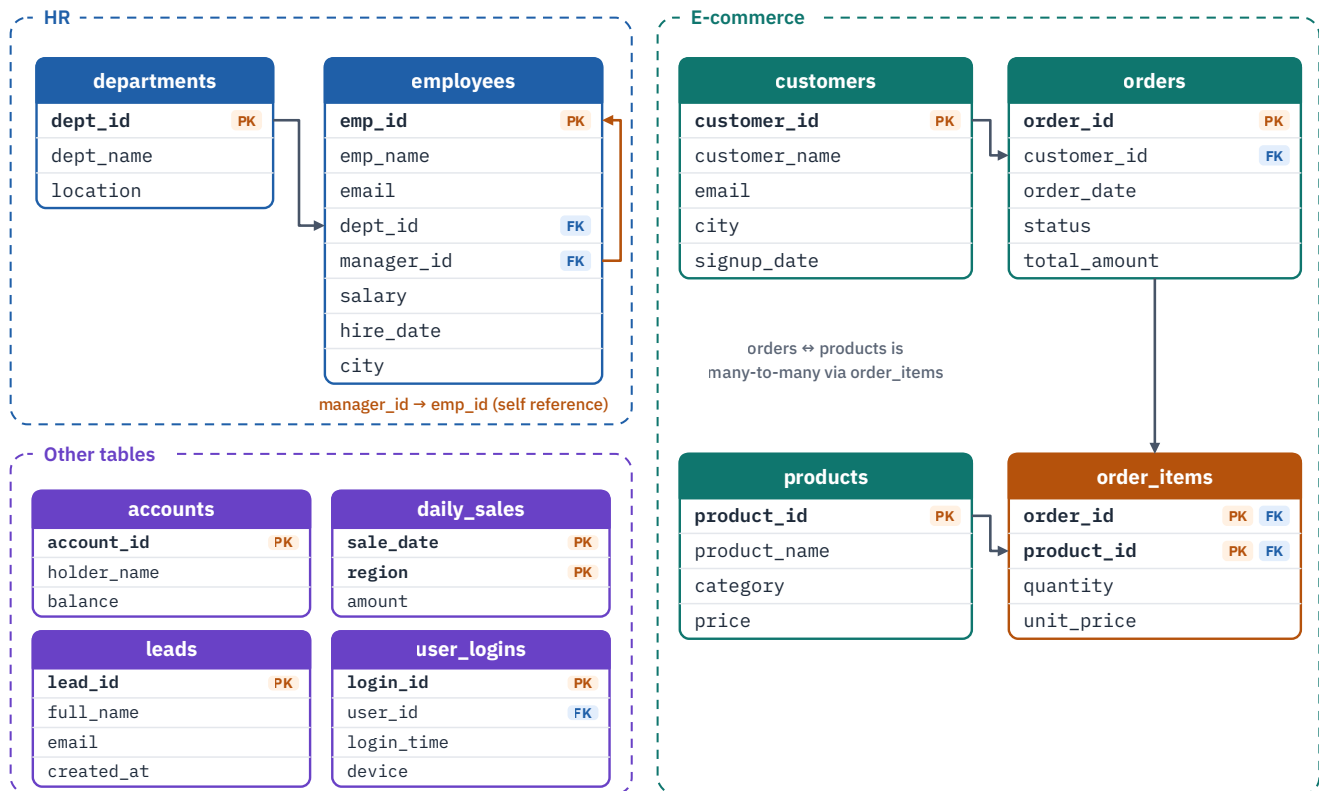


Figure 14. The sample database. Arrows point from a foreign key to the key it references.

Table	Rows	What it holds	Used for
departments	5	department name and city	joins, departments with no employees
employees	14	name, email, department, manager, salary, hire date, city	almost everything: filters, grouping, ranking, self joins
customers	7	e-commerce customers	joins, customers with no orders
products	8	catalogue with category and price	joins, products never sold
orders	14	order header: customer, date, status, total	dates, running totals, latest per customer
order_items	20	order lines: product, quantity, price	multi-table joins, many-to-many
accounts	3	bank balances	transactions and ACID
leads	7	marketing leads, with duplicates	finding and removing duplicates
daily_sales	14	sales per region per day	window functions
user_logins	10	app logins per customer	latest record, date analysis

Deliberate quirks (they make interview patterns possible):

- The **Legal** department has no employees.
- **Farhan Ali** (emp 13) has no department and no email (NULLs).
- Salaries contain **ties**: Rahul and Sneha both earn 95000; Karan and Meera both earn 70000.

- Customers **Saanvi** (6) and **Reyansh** (7) have never ordered; product **Monitor Arm** (8) was never sold.
- Order IDs **107** and **112** are missing (deleted orders).
- The `leads` table contains the same email several times.

The data you will see in outputs

```
SELECT * FROM employees ORDER BY emp_id;
```

SQL · POSTGRESQL

OUTPUT

emp_id	emp_name	email	dept_id	manager_id	salary	hire_date	city
1	Arjun Mehta	arjun.mehta@company.com	1	NULL	150000.00	2015-04-01	Bengaluru
2	Priya Sharma	priya.sharma@company.com	1	1	120000.00	2017-06-15	Bengaluru
3	Rahul Verma	rahul.verma@company.com	1	2	95000.00	2019-01-10	Pune
4	Sneha Iyer	sneha.iyer@company.com	1	2	95000.00	2020-03-23	Bengaluru
5	Vikram Rao	vikram.rao@company.com	1	2	80000.00	2021-07-01	Hyderabad
6	Ananya Das	ananya.das@company.com	2	1	110000.00	2016-09-12	Mumbai
7	Karan Malhotra	karan.malhotra@company.com	2	6	70000.00	2020-11-02	Mumbai
8	Meera Nair	meera.nair@company.com	2	6	70000.00	2022-02-14	Kochi
9	Rohan Gupta	rohan.gupta@company.com	3	1	65000.00	2018-05-21	Kolkata
10	Divya Menon	divya.menon@company.com	3	9	52000.00	2023-01-09	Kolkata
11	Aditya Singh	aditya.singh@company.com	4	1	90000.00	2019-08-19	Delhi
12	Neha Kapoor	neha.kapoor@company.com	4	11	60000.00	2021-12-06	Delhi
13	Farhan Ali	NULL	NULL	1	55000.00	2024-06-03	Pune
14	Pooja Reddy	pooja.reddy@company.com	1	3	72000.00	2024-02-12	Hyderabad

14 rows

```
SELECT * FROM departments ORDER BY dept_id;
```

SQL · POSTGRESQL

OUTPUT

dept_id	dept_name	location
1	Engineering	Bengaluru
2	Finance	Mumbai
3	HR	Kolkata
4	Marketing	Delhi
5	Legal	Hyderabad

5 rows

```
SELECT * FROM customers ORDER BY customer_id;
```

SQL · POSTGRESQL

OUTPUT

customer_id	customer_name	email	city	signup_date
1	Aarav Patel	aarav@mail.com	Mumbai	2023-11-05
2	Isha Kulkarni	isha@mail.com	Pune	2023-12-12
3	Kabir Khan	kabir@mail.com	Delhi	2024-01-03
4	Diya Joshi	diya@mail.com	Bengaluru	2024-01-20
5	Vivaan Shah	vivaan@mail.com	Ahmedabad	2024-02-08
6	Saanvi Rao	saanvi@mail.com	Chennai	2024-02-25
7	Reyansh Bose	NULL	Kolkata	2024-03-15

7 rows

```
SELECT * FROM products ORDER BY product_id;
```

SQL · POSTGRESQL

OUTPUT

product_id	product_name	category	price
1	Wireless Mouse	Electronics	799.00
2	Mechanical Keyboard	Electronics	3499.00
3	USB-C Hub	Electronics	1999.00
4	Office Chair	Furniture	8999.00
5	Standing Desk	Furniture	15999.00
6	Notebook Pack	Stationery	249.00
7	Gel Pen Set	Stationery	149.00
8	Monitor Arm	Furniture	2499.00

8 rows

```
SELECT * FROM orders ORDER BY order_id;
```

SQL · POSTGRESQL

OUTPUT

order_id	customer_id	order_date	status	total_amount
101	1	2024-01-05	DELIVERED	2843.00
102	2	2024-01-12	DELIVERED	3499.00
103	1	2024-01-28	DELIVERED	8999.00
104	3	2024-02-02	CANCELLED	15999.00
105	4	2024-02-14	DELIVERED	2798.00
106	2	2024-02-20	DELIVERED	2486.00
108	5	2024-03-03	SHIPPED	6998.00
109	1	2024-03-10	DELIVERED	24998.00
110	3	2024-03-18	DELIVERED	2397.00
111	4	2024-03-29	RETURNED	3998.00
113	2	2024-04-04	SHIPPED	17998.00
114	5	2024-04-11	PENDING	3235.00
115	1	2024-04-19	PENDING	5498.00
116	3	2024-04-19	DELIVERED	15999.00

14 rows

RUNNING THE EXAMPLES YOURSELF

Install PostgreSQL (or use any hosted PostgreSQL), create a database and load the script that ships with this handbook: `psql -d handbook -f handbook/sql/sample_db.sql`. The script drops and recreates all tables, so you can reload it whenever you want fresh data.

Session 3.1 — PostgreSQL Basics (Part 1)

What is PostgreSQL?

PostgreSQL ("Postgres") is a free, open-source relational database known for correctness and features: full ACID transactions, rich SQL support (CTEs, window functions), JSON support (`JSONB`), extensions and strong standards compliance. Companies use it for everything from startup apps to large enterprise systems, and every major cloud offers a managed version (Amazon RDS / Aurora, Google Cloud SQL, Azure Database for PostgreSQL).

How PostgreSQL organises things

cluster (server) → databases → schemas → tables, views, functions

- One PostgreSQL **server** can host many **databases** (`handbook` , `payroll`).
- Each database contains **schemas** (namespaces), `public` by default.
- Schemas contain **tables**, **views**, **sequences** and **functions**.

psql: the command-line client

`psql` is PostgreSQL's terminal client. Commands starting with a backslash are **psql meta-commands**: they are handled by the client, are not SQL, and do not end with a semicolon.

Command	What it does
<code>psql -h localhost -U postgres -d handbook</code>	connect to a database
<code>\l</code>	list databases
<code>\c handbook</code>	connect to another database
<code>\dt</code>	list tables in the current schema
<code>\d employees</code>	describe a table: columns, types, indexes, constraints
<code>\dn / \du</code>	list schemas / list users (roles)
<code>\i file.sql</code>	run a SQL script
<code>\x</code>	toggle expanded display (one column per line)
<code>\timing</code>	show how long each query takes
<code>\? / \h SELECT</code>	help on meta-commands / help on a SQL command
<code>\q</code>	quit

Common PostgreSQL data types

Category	Types	Notes
Whole numbers	<code>SMALLINT</code> , <code>INTEGER (INT)</code> , <code>BIGINT</code>	pick by range; IDs for big tables → <code>BIGINT</code>
Auto-numbering	<code>SERIAL</code> / <code>BIGSERIAL</code> , or <code>INT GENERATED ALWAYS AS IDENTITY</code>	<code>IDENTITY</code> is the SQL-standard, recommended form
Exact decimals	<code>NUMERIC(p, s)</code> / <code>DECIMAL(p, s)</code>	money: <code>NUMERIC(10,2)</code>
Approximate	<code>REAL</code> , <code>DOUBLE PRECISION</code>	scientific values, never money
Text	<code>VARCHAR(n)</code> , <code>TEXT</code> , <code>CHAR(n)</code>	<code>CHAR(n)</code> pads with spaces; <code>TEXT</code> has no length limit
True/false	<code>BOOLEAN</code>	<code>true</code> , <code>false</code> , or <code>NULL</code>
Date and time	<code>DATE</code> , <code>TIME</code> , <code>TIMESTAMP</code> , <code>TIMESTAMPTZ</code> , <code>INTERVAL</code>	<code>TIMESTAMPTZ</code> stores a moment in time with time-zone handling
Others	<code>UUID</code> , <code>JSON</code> / <code>JSONB</code> , arrays (<code>INT[]</code>)	<code>JSONB</code> is binary JSON that can be indexed

SQL is made of sub-languages

Sub-language	Purpose	Commands	Session
DQL (Data Query Language)	read data	<code>SELECT</code>	3.1–3.8
DDL (Data Definition Language)	define structure	<code>CREATE</code> , <code>ALTER</code> , <code>DROP</code> , <code>TRUNCATE</code>	3.3
DML (Data Manipulation Language)	change data	<code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code>	3.4
DCL (Data Control Language)	permissions	<code>GRANT</code> , <code>REVOKE</code>	3.4
TCL (Transaction Control Language)	transactions	<code>BEGIN</code> , <code>COMMIT</code> , <code>ROLLBACK</code> , <code>SAVEPOINT</code>	3.4, 3.10

Some textbooks count `SELECT` as part of DML; either answer is accepted if you explain it.

SELECT and FROM: reading columns

```
SELECT emp_name, city, salary
FROM employees
ORDER BY emp_id
LIMIT 5;
```

SQL · POSTGRESQL

OUTPUT

emp_name	city	salary
Arjun Mehta	Bengaluru	150000.00
Priya Sharma	Bengaluru	120000.00
Rahul Verma	Pune	95000.00
Sneha Iyer	Bengaluru	95000.00
Vikram Rao	Hyderabad	80000.00

5 rows

- `SELECT` lists the columns you want; `FROM` names the table.
- `SELECT *` returns every column. It is fine for exploring, but avoid it in application code: it fetches more data than needed and breaks when columns change.

Expressions and aliases

A SELECT can compute new values. **Aliases** (AS) give columns and tables readable names.

```
SELECT e.emp_name,
       e.salary,
       e.salary * 12 AS annual_salary,
       e.salary * 12 * 0.10 AS annual_bonus
FROM employees AS e
ORDER BY annual_salary DESC
LIMIT 4;
```

SQL · POSTGRESQL

OUTPUT

emp_name	salary	annual_salary	annual_bonus
Arjun Mehta	150000.00	1800000.00	180000.0000
Priya Sharma	120000.00	1440000.00	144000.0000
Ananya Das	110000.00	1320000.00	132000.0000
Sneha Iyer	95000.00	1140000.00	114000.0000

4 rows

LINE BY LINE

Line	What it does
SELECT e.emp_name, e.salary,	two existing columns; e. says they come from the table aliased e
e.salary * 12 AS annual_salary	a calculated column, renamed with an alias
e.salary * 12 * 0.10 AS annual_bonus	another calculation: 10% of annual salary
FROM employees AS e	table alias: e now stands for employees (the AS is optional)
ORDER BY annual_salary DESC	ORDER BY can use a column alias, because it runs after SELECT
LIMIT 4	keep only the first four rows of the sorted result

An alias containing spaces or capitals needs double quotes: AS "Annual Salary". Single quotes are for **string values**, double quotes are for **names**.

WHERE: filtering rows

WHERE keeps only the rows for which the condition is true.

Operator	Meaning	Example
=	equal	city = 'Pune'
<> or !=	not equal	status <> 'CANCELLED'
<, >, <=, >=	comparisons	salary >= 90000

```
SELECT emp_name, city, salary
FROM employees
WHERE salary >= 90000
ORDER BY salary DESC;
```

SQL · POSTGRESQL

OUTPUT

emp_name	city	salary
Arjun Mehta	Bengaluru	150000.00
Priya Sharma	Bengaluru	120000.00
Ananya Das	Mumbai	110000.00
Rahul Verma	Pune	95000.00
Sneha Iyer	Bengaluru	95000.00
Aditya Singh	Delhi	90000.00

6 rows

Text comparisons are **case-sensitive** in PostgreSQL: `city = 'pune'` finds nothing. Text and date literals go in **single quotes**.

Why you cannot use a SELECT alias in WHERE

```
SELECT emp_name, salary * 12 AS annual_salary
FROM employees
WHERE annual_salary > 1000000;
```

SQL · POSTGRESQL

POSTGRESQL SAYS

ERROR column "annual_salary" does not exist

PostgreSQL evaluates **WHERE before SELECT**, so the alias does not exist yet. Repeat the expression (`WHERE salary * 12 > 1000000`) or use a subquery/CTE. The full logical order is explained in Session 3.5.

DISTINCT: removing duplicate rows

```
SELECT DISTINCT city
FROM employees
ORDER BY city;
```

SQL · POSTGRESQL

OUTPUT

city
Bengaluru
Delhi
Hyderabad
Kochi
Kolkata
Mumbai
Pune

7 rows

DISTINCT applies to the **whole row of selected columns**. `SELECT DISTINCT dept_id, city` returns each unique *combination* of department and city.

DISTINCT ON (POSTGRESQL ONLY)

`SELECT DISTINCT ON (dept_id) dept_id, emp_name, salary FROM employees ORDER BY dept_id, salary DESC;` keeps the **first row per department** according to the ORDER BY, i.e. the top earner in each department. It is a handy shortcut for "top 1 per group" (Pattern 12 in Session 3.11).

ORDER BY: sorting

```
SELECT emp_name, dept_id, salary
FROM employees
ORDER BY dept_id ASC NULLS FIRST, salary DESC
LIMIT 8;
```

SQL · POSTGRESQL

OUTPUT

emp_name	dept_id	salary
Farhan Ali	NULL	55000.00
Arjun Mehta	1	150000.00
Priya Sharma	1	120000.00
Rahul Verma	1	95000.00
Sneha Iyer	1	95000.00
Vikram Rao	1	80000.00
Pooja Reddy	1	72000.00
Ananya Das	2	110000.00

8 rows

- `ASC` (default) or `DESC`, per column; later columns break ties.
- `NULLS FIRST` / `NULLS LAST` controls where NULLs go. In PostgreSQL, NULLs sort as if **larger** than every value: last in ascending order, first in descending order.
- Without `ORDER BY`, the row order is not guaranteed, even if it looks stable.

LIMIT and OFFSET: top-N and pagination

```
SELECT emp_id, emp_name
FROM employees
ORDER BY emp_id
LIMIT 5 OFFSET 5;      -- page 2 when each page has 5 rows
```

SQL · POSTGRESQL

OUTPUT

emp_id	emp_name
6	Ananya Das
7	Karan Malhotra
8	Meera Nair
9	Rohan Gupta
10	Divya Menon

5 rows

- `LIMIT n` returns at most n rows; `OFFSET k` skips the first k rows. Page p of size n uses `OFFSET (p - 1) * n`.
- Always pair them with `ORDER BY`, otherwise "page 2" is undefined.
- The SQL-standard spelling is `OFFSET 5 ROWS FETCH FIRST 5 ROWS ONLY`. SQL Server uses `TOP`, Oracle uses `FETCH FIRST` (or `ROWNUM` in old versions).
- A large `OFFSET` (page 10,000) is slow, because the database still reads and throws away all skipped rows. Big apps use **keyset pagination**: `WHERE emp_id > :last_seen_id ORDER BY emp_id LIMIT 5`.

Comments and style

```
-- single-line comment
/* multi-line
   comment */
SELECT emp_name      -- one column per line is easy to read
FROM employees
WHERE dept_id = 1;    -- keywords in capitals is a common convention
```

SQL · POSTGRESQL

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"A basic SQL query has SELECT for the columns I want, FROM for the table, WHERE to filter rows, ORDER BY to sort, and LIMIT to restrict the number of rows. I can compute expressions and name them with aliases. One subtle point is that the database logically runs FROM and WHERE before SELECT, so a column alias defined in SELECT can't be used in WHERE, but it can be used in ORDER BY. And without ORDER BY, the order of rows is never guaranteed."

COMMON TRAPS & MISTAKES

- Using a SELECT alias in WHERE: error, because WHERE runs first.
- Expecting a fixed row order without ORDER BY.
- LIMIT without ORDER BY for "top N": you get *some* N rows, not the top ones.
- Thinking DISTINCT works on one column when several are selected; it de-duplicates whole rows.
- Double quotes for strings: WHERE city = "Pune" looks for a *column* named Pune.

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What does DISTINCT do?

A It removes duplicate rows from the result, considering all selected columns together.

Q2 **HOW** How do you get the 5 highest-paid employees?

A SELECT emp_name, salary FROM employees ORDER BY salary DESC LIMIT 5;

Q3 **DEFINITION** What is an alias?

A A temporary name for a column or table in a query, created with AS, e.g. salary * 12 AS annual_salary or FROM employees e.

Q4 **COMPARISON** CHAR vs VARCHAR vs TEXT in PostgreSQL?

A CHAR(n) is fixed length and pads with spaces; VARCHAR(n) is variable length with a maximum; TEXT is variable length without a limit. In PostgreSQL VARCHAR and TEXT perform the same.

INTERMEDIATE

Q5 **WHY** Why can't you use a column alias in the WHERE clause?

A WHERE is evaluated before SELECT in the logical processing order, so the alias doesn't exist yet. Repeat the expression or wrap the query in a subquery or CTE.

Q6 **HOW** How do you implement pagination in SQL?

A ORDER BY a unique column with LIMIT page_size OFFSET (page - 1) * page_size. For large tables, keyset pagination (WHERE id > last_id ORDER BY id LIMIT n) is faster.

Q7 **COMPARISON** **TIMESTAMP vs TIMESTAMPTZ?**

A **TIMESTAMP** stores a date and time without a time zone. **TIMESTAMPTZ** stores an absolute moment (internally in UTC) and converts it to the session's time zone on display, which is safer for global apps.

Q8 **HOW** **Where do NULLs appear when you sort in PostgreSQL?**

A Last in ascending order and first in descending order by default; you can override this with `NULLS FIRST` or `NULLS LAST`.

SCENARIO-BASED**Q9** **SQL PROBLEM** **List all distinct cities where employees earning more than 70000 work, alphabetically.**

A `SELECT DISTINCT city FROM employees WHERE salary > 70000 ORDER BY city;`

Q10 **SCENARIO** **Page 50,000 of a product listing is extremely slow. Why, and what would you do?**

A `OFFSET` makes the database read and discard all earlier rows. Switch to keyset (cursor) pagination using the last seen sort key, with an index on that key.

FOLLOW-UP / TRAP**Q11** **TRAP QUESTION** **Does `SELECT DISTINCT dept_id, city FROM employees` return unique departments?**

A No, it returns unique (dept_id, city) combinations, so a department appears once per distinct city.

Q12 **TRAP QUESTION** **Is `\dt` a SQL command?**

A No, it is a psql meta-command handled by the client. It works only in psql, not from an application.

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

P1. Write a query that shows each product's name and price, most expensive first.

P2. Write a query that lists the distinct product categories.

P3. What does `LIMIT 3 OFFSET 6` return?

LEVEL 2 – INTERVIEW APPLICATION

P4. Show the name and monthly salary of employees in Bengaluru, plus a column `monthly_tax` equal to 20% of salary, highest salary first.

P5. Return page 3 of customers ordered by `customer_id`, with 2 customers per page.

LEVEL 3 – SCENARIO / PROBLEM SOLVING

P6. A query `SELECT emp_name, salary*12 AS yearly FROM employees WHERE yearly > 1200000;` fails. Fix it in two different ways.

P7. Return the three most recently hired employees, showing name, city and hire date.

ANSWER / EXPLANATION**P1.**

```
SELECT product_name, price
FROM products
ORDER BY price DESC;
```

SQL · POSTGRESQL

OUTPUT

product_name	price
Standing Desk	15999.00
Office Chair	8999.00
Mechanical Keyboard	3499.00
Monitor Arm	2499.00
USB-C Hub	1999.00
Wireless Mouse	799.00
Notebook Pack	249.00
Gel Pen Set	149.00

8 rows

P2.

```
SELECT DISTINCT category
FROM products
ORDER BY category;
```

SQL · POSTGRESQL

OUTPUT

category
Electronics
Furniture
Stationery

3 rows

P3. It skips the first 6 rows of the (ordered) result and returns the next 3, i.e. rows 7, 8 and 9. Without ORDER BY, which rows those are is not defined.

P4.

```
SELECT emp_name,
       salary,
       salary * 0.20 AS monthly_tax
FROM employees
WHERE city = 'Bengaluru'
ORDER BY salary DESC;
```

SQL · POSTGRESQL

OUTPUT

emp_name	salary	monthly_tax
Arjun Mehta	150000.00	30000.0000
Priya Sharma	120000.00	24000.0000
Sneha Iyer	95000.00	19000.0000

3 rows

P5. Page 3 with 2 per page skips $(3 - 1) \times 2 = 4$ rows:

```
SELECT customer_id, customer_name
FROM customers
ORDER BY customer_id
LIMIT 2 OFFSET 4;
```

SQL · POSTGRESQL

OUTPUT

customer_id	customer_name
5	Vivaan Shah
6	Saanvi Rao

2 rows

P6. (a) Repeat the expression: `WHERE salary * 12 > 1200000`. (b) Compute it in a subquery first:

```
SELECT emp_name, yearly
FROM (SELECT emp_name, salary * 12 AS yearly FROM employees) AS t
WHERE yearly > 1200000;
```

SQL · POSTGRESQL

OUTPUT

emp_name	yearly
Arjun Mehta	1800000.00
Priya Sharma	1440000.00
Ananya Das	1320000.00

3 rows

P7.

```
SELECT emp_name, city, hire_date
FROM employees
ORDER BY hire_date DESC
LIMIT 3;
```

SQL · POSTGRESQL

OUTPUT

emp_name	city	hire_date
Farhan Ali	Pune	2024-06-03
Pooja Reddy	Hyderabad	2024-02-12
Divya Menon	Kolkata	2023-01-09

3 rows

Session 3.2 — PostgreSQL Basics (Part 2)

Combining conditions: AND, OR, NOT

- **AND** : both conditions must be true.
- **OR** : at least one must be true.
- **NOT** : reverses a condition.

Precedence trap: **AND** is evaluated before **OR**, just as multiplication comes before addition. "Employees in Pune or Mumbai earning more than 60000", written without parentheses:

```
SELECT emp_name, city, salary
FROM employees
WHERE city = 'Pune' OR city = 'Mumbai' AND salary > 60000
ORDER BY emp_name;
```

SQL · POSTGRESQL

OUTPUT

emp_name	city	salary
Ananya Das	Mumbai	110000.00
Farhan Ali	Pune	55000.00
Karan Malhotra	Mumbai	70000.00
Rahul Verma	Pune	95000.00

4 rows

Farhan (Pune, 55000) sneaked in, because the database read it as `city = 'Pune' OR (city = 'Mumbai' AND salary > 60000)`. With parentheses:

```
SELECT emp_name, city, salary
FROM employees
WHERE (city = 'Pune' OR city = 'Mumbai') AND salary > 60000
ORDER BY emp_name;
```

SQL · POSTGRESQL

OUTPUT

emp_name	city	salary
Ananya Das	Mumbai	110000.00
Karan Malhotra	Mumbai	70000.00
Rahul Verma	Pune	95000.00

3 rows

Rule: whenever you mix AND and OR, add parentheses.

IN, BETWEEN and LIKE

```
SELECT emp_name, city
FROM employees
WHERE city IN ('Pune', 'Kochi', 'Delhi')
ORDER BY city, emp_name;
```

SQL · PostgreSQL

OUTPUT

emp_name	city
Aditya Singh	Delhi
Neha Kapoor	Delhi
Meera Nair	Kochi
Farhan Ali	Pune
Rahul Verma	Pune

5 rows

`IN (list)` is shorthand for several ORs. `NOT IN` excludes a list (careful with NULLs, see Session 3.7).

```
SELECT emp_name, hire_date
FROM employees
WHERE hire_date BETWEEN '2019-01-01' AND '2020-12-31'
ORDER BY hire_date;
```

SQL · PostgreSQL

OUTPUT

emp_name	hire_date
Rahul Verma	2019-01-10
Aditya Singh	2019-08-19
Sneha Iyer	2020-03-23
Karan Malhotra	2020-11-02

4 rows

`BETWEEN a AND b` is **inclusive** at both ends (`>= a AND <= b`).

BETWEEN WITH TIMESTAMPS

For a `TIMESTAMP` column, `BETWEEN '2024-04-01' AND '2024-04-30'` stops at **midnight at the start of 30 April**, so almost the whole last day is missed. The safe pattern is a half-open range: `login_time >= '2024-04-01' AND login_time < '2024-05-01'`.

`LIKE` matches text patterns: `%` means "any number of characters" (including none), `_` means "exactly one character".

Pattern	Matches
'A%'	starts with A
'%a'	ends with a
'%Rao%'	contains "Rao"
'_r%'	second letter is r
ILIKE 'a%'	starts with a or A (PostgreSQL's case-insensitive LIKE)

```
SELECT emp_name, email
FROM employees
WHERE emp_name LIKE '_t%'
      OR email ILIKE 'NEHA%';
```

SQL · POSTGRESQL

OUTPUT

emp_name	email
Arjun Mehta	arjun.mehta@company.com
Priya Sharma	priya.sharma@company.com
Neha Kapoor	neha.kapoor@company.com

3 rows

NULL: the value that means "unknown"

`NULL` means **missing or unknown**. It is not zero, not an empty string, and not false. This leads to **three-valued logic**: a condition can be TRUE, FALSE or **UNKNOWN**, and `WHERE` keeps only TRUE rows.

- `NULL = NULL` is not true; it is UNKNOWN. So `WHERE email = NULL` returns **nothing**:

```
SELECT emp_name FROM employees WHERE email = NULL;
```

SQL · POSTGRESQL

OUTPUT

emp_name

0 rows

- Use `IS NULL` / `IS NOT NULL` :

```
SELECT emp_name, email, dept_id
FROM employees
WHERE email IS NULL OR dept_id IS NULL;
```

SQL · POSTGRESQL

OUTPUT

emp_name	email	dept_id
Farhan Ali	NULL	NULL

1 row

Expression	Result
<code>NULL + 100</code>	<code>NULL</code> (arithmetic with <code>NULL</code> gives <code>NULL</code>)
<code>'Hi ' NULL</code>	<code>NULL</code> (concatenation with <code> </code> gives <code>NULL</code>)
<code>CONCAT('Hi ', NULL)</code>	<code>'Hi '</code> (<code>CONCAT</code> ignores <code>NULL</code> s)
<code>NULL = NULL</code>	<code>NULL</code> (unknown), never <code>TRUE</code>
<code>TRUE OR NULL</code>	<code>TRUE</code>
<code>FALSE AND NULL</code>	<code>FALSE</code>

COALESCE returns its first non-NULL argument, which is perfect for defaults. **NULLIF(a, b)** returns NULL when a equals b, which is handy to avoid division by zero: `x / NULLIF(y, 0)`.

```
SELECT emp_name,
       COALESCE(email, 'not assigned') AS email,
       COALESCE(dept_id::text, 'no department') AS department
FROM employees
WHERE emp_id IN (12, 13);
```

SQL · POSTGRESQL

OUTPUT

emp_name	email	department
Neha Kapoor	neha.kapoor@company.com	4
Farhan Ali	not assigned	no department

2 rows

Aggregate functions over the whole table

Aggregates turn many rows into one value: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`. They **ignore NULLs**, except `COUNT(*)`, which counts rows.

```
SELECT COUNT(*) AS all_rows,
       COUNT(email) AS with_email,
       COUNT(DISTINCT city) AS cities,
       SUM(salary) AS payroll,
       ROUND(AVG(salary), 2) AS avg_salary,
       MIN(hire_date) AS first_hire,
       MAX(salary) AS top_salary
FROM employees;
```

SQL · POSTGRESQL

OUTPUT

all_rows	with_email	cities	payroll	avg_salary	first_hire	top_salary
14	13	7	1184000.00	84571.43	2015-04-01	150000.00

1 row

LINE BY LINE

Expression	Meaning here
<code>COUNT(*)</code>	number of rows: 14 employees
<code>COUNT(email)</code>	rows where email is not NULL : 13, because Farhan has no email
<code>COUNT(DISTINCT city)</code>	number of different cities
<code>SUM(salary)</code>	total monthly payroll
<code>ROUND(AVG(salary), 2)</code>	average, rounded to 2 decimals
<code>MIN(hire_date)</code>	MIN and MAX also work on dates and text

`AVG(column)` ignores NULLs too: the average of (100, NULL, 200) is 150, not 100. If NULL should count as zero, write `AVG(COALESCE(column, 0))`. Grouping (one result per department, etc.) comes in Session 3.5.

String functions

Function	Example	Result
UPPER, LOWER, INITCAP	INITCAP('raHUL verma')	Rahul Verma
LENGTH	LENGTH('Pune')	4
TRIM, LTRIM, RTRIM	TRIM(' Pune ')	Pune
SUBSTRING(s FROM start FOR n)	SUBSTRING('PostgreSQL' FROM 1 FOR 4)	Post
LEFT, RIGHT	RIGHT('ORD-9001', 4)	9001
POSITION(sub IN s) / STRPOS	POSITION('@' IN 'a@b.com')	2
REPLACE	REPLACE('2024/01/05', '/', '-')	2024-01-05
CONCAT,	'Mr. ' 'Rao'	Mr. Rao
SPLIT_PART(s, sep, n)	SPLIT_PART('a@mail.com', '@', 2)	mail.com
LPAD, RPAD	LPAD('42', 5, '0')	00042
STRING_AGG(col, sep) (aggregate)	names joined with commas	Asha, Ravi

```

SELECT emp_name,
       UPPER(emp_name)           AS shouting,
       SPLIT_PART(emp_name, ' ', 1) AS first_name,
       SPLIT_PART(email, '@', 2)  AS email_domain,
       LENGTH(emp_name)          AS name_length
FROM employees
WHERE dept_id = 2;

```

SQL · POSTGRESQL

OUTPUT

emp_name	shouting	first_name	email_domain	name_length
Ananya Das	ANANYA DAS	Ananya	company.com	10
Karan Malhotra	KARAN MALHOTRA	Karan	company.com	14
Meera Nair	MEERA NAIR	Meera	company.com	10

3 rows

Date and time functions

Real queries use `CURRENT_DATE` or `NOW()`. To keep this book's outputs stable, the examples treat **31 December 2024** as "today".

Function	Example	Result
CURRENT_DATE, NOW()	today's date, current timestamp	depends on when you run it
EXTRACT(part FROM d)	EXTRACT(YEAR FROM DATE '2024-03-05')	2024
DATE_TRUNC('month', d)	DATE_TRUNC('month', TIMESTAMP '2024-03-05 10:00')	2024-03-01 00:00:00
date - date	DATE '2024-03-05' - DATE '2024-03-01'	4 (days, as an integer)
date + interval	DATE '2024-01-31' + INTERVAL '1 month'	2024-02-29 00:00:00
AGE(a, b)	AGE(DATE '2024-12-31', DATE '2015-04-01')	9 years 8 mons 30 days
TO_CHAR(d, fmt)	TO_CHAR(DATE '2024-03-05', 'DD Mon YYYY')	05 Mar 2024
TO_DATE(s, fmt) / cast	TO_DATE('05/03/2024', 'DD/MM/YYYY')	2024-03-05

```

SELECT emp_name,
       hire_date,
       EXTRACT(YEAR FROM hire_date)      AS hire_year,
       TO_CHAR(hire_date, 'Mon YYYY')    AS hired_in,
       DATE '2024-12-31' - hire_date     AS days_employed,
       AGE(DATE '2024-12-31', hire_date) AS tenure
FROM employees
WHERE emp_id IN (1, 10, 14);

```

SQL · PostgreSQL

OUTPUT

emp_name	hire_date	hire_year	hired_in	days_employed	tenure
Arjun Mehta	2015-04-01	2015	Apr 2015	3562	9 years 8 mons 30 days
Divya Menon	2023-01-09	2023	Jan 2023	722	1 year 11 mons 22 days
Pooja Reddy	2024-02-12	2024	Feb 2024	323	10 mons 19 days

3 rows

Numeric functions and the integer-division trap

Function	Example	Result
ROUND(x, d)	ROUND(84571.4286, 2)	84571.43
CEIL, FLOOR	CEIL(4.1), FLOOR(4.9)	5, 4
TRUNC(x, d)	TRUNC(4.567, 1)	4.5
ABS, POWER, SQRT	ABS(-7), POWER(2, 10)	7, 1024
MOD(a, b) or a % b	7 % 3	1

```

SELECT 5 / 2      AS int_division,
       5 / 2.0    AS decimal_division,
       5::numeric / 2 AS cast_first,
       7 % 3      AS remainder;

```

SQL · PostgreSQL

OUTPUT

int_division	decimal_division	cast_first	remainder
2	2.5000000000000000	2.5000000000000000	1

1 row

When **both** operands are integers, PostgreSQL does integer division and drops the fraction: `5 / 2 = 2`. This silently breaks percentage calculations like `delivered_count / total_count * 100`. Cast one side first: `delivered_count::numeric / total_count`.

`CAST(x AS type)` and the PostgreSQL shortcut `x::type` convert between types: `'2024-03-05'::date`, `salary::int`, `dept_id::text`.

CASE WHEN: if-then-else inside SQL

CASE evaluates conditions **in order** and returns the result of the first true one; ELSE covers everything else (without ELSE, the result is NULL).

```
SELECT emp_name,
       salary,
       CASE
         WHEN salary >= 110000 THEN 'Band A'
         WHEN salary >= 80000  THEN 'Band B'
         WHEN salary >= 60000  THEN 'Band C'
         ELSE 'Band D'
       END AS salary_band
FROM employees
ORDER BY salary DESC;
```

SQL · PostgreSQL

OUTPUT

emp_name	salary	salary_band
Arjun Mehta	150000.00	Band A
Priya Sharma	120000.00	Band A
Ananya Das	110000.00	Band A
Sneha Iyer	95000.00	Band B
Rahul Verma	95000.00	Band B
Aditya Singh	90000.00	Band B
Vikram Rao	80000.00	Band B
Pooja Reddy	72000.00	Band C
Meera Nair	70000.00	Band C
Karan Malhotra	70000.00	Band C
Rohan Gupta	65000.00	Band C
Neha Kapoor	60000.00	Band C
Farhan Ali	55000.00	Band D
Divya Menon	52000.00	Band D

14 rows

LINE BY LINE

Line	What it does
CASE	start of the conditional expression
WHEN salary >= 110000 THEN 'Band A'	checked first; 150000, 120000 and 110000 stop here
WHEN salary >= 80000 THEN 'Band B'	only reached if the first test failed, so this means 80000–109999
ELSE 'Band D'	everything that matched no WHEN
END AS salary_band	closes the CASE and names the new column

CASE also works inside `ORDER BY` (custom sort orders) and inside aggregates, the basis of **conditional aggregation** (Pattern 18):

```
SELECT COUNT(*) FILTER (WHERE status = 'DELIVERED') AS delivered,
       SUM(CASE WHEN status = 'PENDING' THEN 1 ELSE 0 END) AS pending,
       COUNT(*) AS total
FROM orders;
```

SQL · POSTGRESQL

OUTPUT

delivered	pending	total
8	2	14

1 row

`COUNT(*) FILTER (WHERE ...)` is PostgreSQL's neat syntax; `SUM(CASE WHEN ... THEN 1 ELSE 0 END)` works in every database.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"NULL in SQL means unknown, not zero or empty, so comparisons with NULL return unknown, and WHERE only keeps rows where the condition is true. That's why we must write `IS NULL` instead of `= NULL`. Aggregates like SUM and AVG ignore NULLs, and COUNT of a column counts only non-null values, while COUNT(*) counts all rows. To replace NULLs with a default, I use COALESCE, and NULLIF helps avoid division by zero."

COMMON TRAPS & MISTAKES

- `WHERE col = NULL` returns nothing; use `IS NULL`.
- Mixing AND/OR without parentheses.
- `BETWEEN` on timestamps misses most of the last day; use `>= start AND < next_day`.
- `LIKE` is case-sensitive in PostgreSQL; use `ILIKE` or `LOWER(col) LIKE ...`.
- Integer division: `5/2 = 2`.
- `COUNT(column)` silently skips NULLs; `AVG` ignores NULLs instead of counting them as zero.
- `CASE` without `ELSE` returns NULL for unmatched rows.

INTERVIEW QUESTIONS

BASIC

Q1 **COMPARISON** What is the difference between `IN` and `BETWEEN`?

A `IN` matches any value in a list (`city IN ('Pune', 'Delhi')`); `BETWEEN` matches a continuous, inclusive range (`salary BETWEEN 50000 AND 90000`).

Q2 **DEFINITION** What do `%` and `_` mean in `LIKE`?

A `%` matches any sequence of characters (including none); `_` matches exactly one character.

Q3 **HOW** How do you find rows where a column has no value?

A `WHERE column IS NULL`. Using `= NULL` never matches.

Q4 **COMPARISON** `COUNT(*)` vs `COUNT(column)`?

A `COUNT(*)` counts all rows; `COUNT(column)` counts rows where that column is not NULL.

INTERMEDIATE**Q5** **DEFINITION** What is COALESCE?

A A function returning its first non-NULL argument, used to substitute defaults: `COALESCE(phone, 'N/A')`.

Q6 **WHY** What does `SELECT 7/2` return in PostgreSQL, and why?

A 3, because both operands are integers, so integer division truncates the fraction. Use `7/2.0` or `7::numeric/2` for 3.5.

Q7 **HOW** How would you extract the domain from an email address?

A `SPLIT_PART(email, '@', 2)`, or `SUBSTRING(email FROM POSITION('@' IN email) + 1)`.

Q8 **DEFINITION** What is three-valued logic?

A SQL conditions evaluate to TRUE, FALSE or UNKNOWN (when NULL is involved). WHERE keeps only TRUE, so UNKNOWN rows are filtered out.

Q9 **HOW** How do you count delivered and pending orders in one query?

A Conditional aggregation: `SUM(CASE WHEN status='DELIVERED' THEN 1 ELSE 0 END)` and the same for PENDING, or PostgreSQL's `COUNT(*) FILTER (WHERE status = 'DELIVERED')`.

SCENARIO-BASED**Q10** **SQL PROBLEM** Find employees whose name starts with "A" or "R" and who were hired after 2018.

A `SELECT emp_name, hire_date FROM employees WHERE (emp_name LIKE 'A%' OR emp_name LIKE 'R%') AND hire_date >= '2019-01-01'`; Note the parentheses.

Q11 **SCENARIO** A report divides `delivered_orders / total_orders` and always shows 0. Why?

A Both values are integers, so integer division truncates to 0. Cast one side to numeric and use `NULLIF(total_orders, 0)` to avoid division by zero.

Q12 **SCENARIO** An average rating looks too high because unrated items have NULL. What's happening?

A AVG ignores NULLs, so only rated items are averaged. That's usually correct; if unrated must count as 0, use `AVG(COALESCE(rating, 0))`, and state the business rule clearly.

FOLLOW-UP / TRAP**Q13** **TRAP QUESTION** Is `NULL = NULL` true?

A No, it is UNKNOWN (NULL). Use `IS NULL`, or `IS NOT DISTINCT FROM` to compare two possibly-NULL values.

Q14 **TRAP QUESTION** Is an empty string the same as NULL?

A In PostgreSQL, no: `''` is a known, empty value, while NULL is unknown. (Oracle is the exception: it treats `''` as NULL.)

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

P1. List products in the Furniture or Stationery category priced under 3000.

P2. Show employees with no manager.

P3. What does `COALESCE(NULL, NULL, 'x', 'y')` return?

LEVEL 2 – INTERVIEW APPLICATION

P4. Show each order's id, status and a column `is_open` that is 'Yes' for PENDING or SHIPPED orders and 'No' otherwise.

P5. Count customers who have an email and customers who don't, in one row.

P6. List employees hired in 2021 or later whose email domain is `company.com`, showing the year they joined.

LEVEL 3 – SCENARIO / PROBLEM SOLVING

P7. Marketing wants leads whose email does not end with `mail.com`. Write it, and say what happens to rows whose email is NULL.

P8. Calculate the percentage of orders that are DELIVERED, rounded to one decimal place, avoiding integer division.

ANSWER / EXPLANATION

P1.

```
SELECT product_name, category, price
FROM products
WHERE category IN ('Furniture', 'Stationery')
      AND price < 3000
ORDER BY price;
```

SQL · POSTGRESQL

OUTPUT

product_name	category	price
Gel Pen Set	Stationery	149.00
Notebook Pack	Stationery	249.00
Monitor Arm	Furniture	2499.00

3 rows

P2.

```
SELECT emp_id, emp_name
FROM employees
WHERE manager_id IS NULL;
```

SQL · POSTGRESQL

OUTPUT

emp_id	emp_name
1	Arjun Mehta

1 row

P3. 'x', the first non-NULL argument.

P4.

```
SELECT order_id,
       status,
       CASE WHEN status IN ('PENDING', 'SHIPPED') THEN 'Yes' ELSE 'No' END AS is_open
FROM orders
ORDER BY order_id;
```

SQL · POSTGRESQL

OUTPUT

order_id	status	is_open
101	DELIVERED	No
102	DELIVERED	No
103	DELIVERED	No
104	CANCELLED	No
105	DELIVERED	No
106	DELIVERED	No
108	SHIPPED	Yes
109	DELIVERED	No
110	DELIVERED	No
111	RETURNED	No
113	SHIPPED	Yes
114	PENDING	Yes
115	PENDING	Yes
116	DELIVERED	No

14 rows

P5. COUNT(email) skips NULLs, so the difference from COUNT(*) is the number without an email:

```
SELECT COUNT(email) AS with_email,
       COUNT(*) - COUNT(email) AS without_email
FROM customers;
```

SQL · POSTGRESQL

OUTPUT

with_email	without_email
6	1

1 row

P6.

```
SELECT emp_name,
       EXTRACT(YEAR FROM hire_date) AS joined_year
FROM employees
WHERE hire_date >= '2021-01-01'
      AND SPLIT_PART(email, '@', 2) = 'company.com'
ORDER BY hire_date;
```

SQL · POSTGRESQL

OUTPUT

emp_name	joined_year
Vikram Rao	2021
Neha Kapoor	2021
Meera Nair	2022
Divya Menon	2023
Pooja Reddy	2024

5 rows

Farhan (hired 2024) is missing: his email is NULL, so the domain test is UNKNOWN and the row is filtered out.

P7. `SELECT * FROM leads WHERE email NOT LIKE '%mail.com';` In this table every lead's email ends with `mail.com`, so it returns no rows. In general, rows with a NULL email would also be excluded, because `NULL NOT LIKE ...` is UNKNOWN. Add `OR email IS NULL` if they should be included.

P8.

```
SELECT ROUND(100.0 * COUNT(*) FILTER (WHERE status = 'DELIVERED') / COUNT(*), 1) AS delivered_pct
FROM orders;
```

SQL · POSTGRES SQL

OUTPUT

delivered_pct
57.1

1 row

Multiplying by `100.0` (a decimal) first makes the whole calculation decimal, so nothing is truncated.

Session 3.3 — DDL: Data Definition Language

Simple meaning

DDL commands define and change the *structure* of the database: creating tables, adding columns, removing tables. They work on the containers, not on the data inside (DML does that, Session 3.4).

Command	What it does
CREATE	create a database object: database, schema, table, view, index, sequence
ALTER	change an existing object: add, drop or rename columns; change types; add constraints
DROP	delete an object completely, structure and data
TRUNCATE	remove all rows quickly but keep the table's structure

POSTGRESQL DETAIL THAT IMPRESSES INTERVIEWERS

In PostgreSQL, DDL is **transactional**: CREATE , ALTER and even DROP TABLE inside a transaction can be rolled back. In MySQL and Oracle most DDL commits automatically. You will see a live demonstration below.

CREATE TABLE

```
CREATE TABLE projects (
  project_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  project_name VARCHAR(80) NOT NULL,
  client VARCHAR(60) NOT NULL,
  dept_id INT REFERENCES departments(dept_id),
  budget NUMERIC(12,2) CONSTRAINT chk_budget_positive CHECK (budget > 0),
  start_date DATE NOT NULL DEFAULT DATE '2024-01-01',
  status VARCHAR(12) NOT NULL DEFAULT 'PLANNED'
);
```

SQL · POSTGRESQL

RESULT CREATE TABLE

LINE BY LINE

Line	What it means
project_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY	the database generates 1, 2, 3... automatically; it is the primary key
project_name VARCHAR(80) NOT NULL	text up to 80 characters, required
dept_id INT REFERENCES departments(dept_id)	foreign key; NULL allowed (a project may have no department yet)
CONSTRAINT chk_budget_positive CHECK (budget > 0)	a named constraint, so error messages and later changes are clearer
DEFAULT DATE '2024-01-01' / DEFAULT 'PLANNED'	values used when an INSERT does not supply them (normally you would write DEFAULT CURRENT_DATE)

SERIAL is the older PostgreSQL shortcut for auto-numbering; GENERATED ... AS IDENTITY is the SQL-standard way and is preferred today. MySQL uses AUTO_INCREMENT ; SQL Server uses IDENTITY(1,1) .

ALTER TABLE: changing structure

```
ALTER TABLE projects ADD COLUMN manager_id INT REFERENCES employees(emp_id);
ALTER TABLE projects RENAME COLUMN client TO client_name;
ALTER TABLE projects ALTER COLUMN project_name TYPE VARCHAR(120);
ALTER TABLE projects ADD CONSTRAINT uq_project_name UNIQUE (project_name);
ALTER TABLE projects ALTER COLUMN status SET DEFAULT 'ACTIVE';
```

SQL · POSTGRESQL

Statement	Effect
ADD COLUMN manager_id INT REFERENCES employees(emp_id)	new nullable column with a foreign key
RENAME COLUMN client TO client_name	renames the column; the data stays
ALTER COLUMN project_name TYPE VARCHAR(120)	widens the column (narrowing can fail if data is too long)
ADD CONSTRAINT uq_project_name UNIQUE (project_name)	adds a rule; fails if existing data already breaks it
ALTER COLUMN status SET DEFAULT 'ACTIVE'	new default for future inserts

Other common forms: DROP COLUMN , ALTER COLUMN ... SET NOT NULL / DROP NOT NULL , DROP CONSTRAINT , RENAME TO new_table_name .

You can inspect a table's structure through the standard information_schema views (psql users simply type \d projects):

```
SELECT column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_name = 'projects'
ORDER BY ordinal_position;
```

SQL · POSTGRESQL

OUTPUT

column_name	data_type	is_nullable	column_default
project_id	integer	NO	NULL
project_name	character varying	NO	NULL
client_name	character varying	NO	NULL
dept_id	integer	YES	NULL
budget	numeric	YES	NULL
start_date	date	NO	'2024-01-01'::date
status	character varying	NO	'ACTIVE'::character varying
manager_id	integer	YES	NULL

8 rows

Creating a table from an existing table (CTAS)

"Create another table with the help of existing tables" appears in reported PwC interview write-ups. **CREATE TABLE AS SELECT** copies the *result of a query* into a new table:

```
CREATE TABLE high_earners AS
SELECT emp_id, emp_name, dept_id, salary
FROM employees
WHERE salary > 90000;

SELECT * FROM high_earners ORDER BY salary DESC;
```

SQL · PostgreSQL

OUTPUT

emp_id	emp_name	dept_id	salary
1	Arjun Mehta	1	150000.00
2	Priya Sharma	1	120000.00
6	Ananya Das	2	110000.00
3	Rahul Verma	1	95000.00
4	Sneha Iyer	1	95000.00

5 rows

- CTAS copies column names, types and data, but **not** primary keys, foreign keys, defaults or indexes. Add them with ALTER TABLE if needed.
- `CREATE TABLE copy_structure (LIKE employees INCLUDING ALL);` copies only the structure, including defaults, constraints and indexes, with no rows.
- `INSERT INTO existing_table SELECT ...` copies rows into a table that already exists.

DROP and TRUNCATE

```
DROP TABLE IF EXISTS high_earners;      -- no error if it doesn't exist
DROP TABLE departments CASCADE;        -- also drops dependent foreign keys and views
(dangerous!)
TRUNCATE TABLE leads;                  -- remove all rows, keep the table
TRUNCATE TABLE projects RESTART IDENTITY; -- also reset the auto-numbering
```

SQL · PostgreSQL

PostgreSQL refuses to truncate a table that other tables reference with foreign keys, because that would leave orphan rows:

```
TRUNCATE TABLE customers;
```

SQL · PostgreSQL

POSTGRESQL SAYS

```
ERROR cannot truncate a table referenced in a foreign key constraint
DETAIL: Table "orders" references "customers".
HINT: Truncate table "orders" at the same time, or use TRUNCATE ... CASCADE.
```

`TRUNCATE customers CASCADE` would also truncate `orders`, `order_items` and `user_logins`, which is a good reason to read every CASCADE twice.

DDL inside a transaction (PostgreSQL)

Watch a dropped table come back:

SQL · PostgreSQL

```
BEGIN;
DROP TABLE leads;
ROLLBACK;

SELECT COUNT(*) AS leads_still_here FROM leads;
```

OUTPUT

leads_still_here

7

1 row

DELETE vs TRUNCATE vs DROP

MUST KNOW This comparison is asked constantly.

	DELETE	TRUNCATE	DROP
Category	DML	DDL	DDL
Removes	chosen rows (WHERE) or all rows	all rows	the whole table: rows, structure, indexes, constraints
WHERE allowed	yes	no	no
Speed on big tables	slower: row by row, each row logged	very fast: deallocates storage pages	fast
Triggers	row-level DELETE triggers fire	DELETE triggers do not fire	—
Auto-number (identity)	not reset	can be reset (RESTART IDENTITY)	gone with the table
Table still exists	yes	yes	no
Rollback	yes, inside a transaction	PostgreSQL: yes, inside a transaction; MySQL/Oracle: no (auto-commits)	PostgreSQL: yes, inside a transaction; most others: no
Use when	removing specific rows	emptying a staging table quickly	the table is no longer needed

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"DELETE is a DML command that removes rows one by one; it can take a WHERE clause, fires triggers and can be rolled back. TRUNCATE is DDL; it removes all rows at once by releasing the data pages, which makes it much faster on big tables, but there's no WHERE, row triggers don't fire, and it can reset identity columns. DROP removes the table itself, structure and all. In PostgreSQL both TRUNCATE and DROP can even be rolled back inside a transaction, whereas in MySQL or Oracle they auto-commit."

COMMON TRAPS & MISTAKES

- Saying "TRUNCATE can never be rolled back". True for MySQL and Oracle, but not for PostgreSQL inside a transaction.
- Saying "DELETE without WHERE is the same as TRUNCATE". Same final rows, but different speed, logging, triggers and identity behaviour.
- Assuming CTAS copies constraints and indexes. It copies only columns and data.

- Using `DROP ... CASCADE` casually; it silently removes dependent objects.

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What is DDL? Give examples.

A Data Definition Language defines database structure: `CREATE`, `ALTER`, `DROP` and `TRUNCATE` (some also include `RENAME` and `COMMENT`).

Q2 **COMPARISON** `DELETE` vs `TRUNCATE` vs `DROP`?

A `DELETE` removes selected or all rows (DML, `WHERE` allowed, triggers fire, slower). `TRUNCATE` removes all rows quickly but keeps the table (DDL, no `WHERE`). `DROP` removes the entire table, structure included.

Q3 **HOW** How do you add a column to an existing table?

A `ALTER TABLE employees ADD COLUMN phone VARCHAR(15);`

INTERMEDIATE

Q4 **HOW** How do you create a new table from an existing one?

A `CREATE TABLE new_table AS SELECT ... FROM old_table WHERE ...;` It copies columns and data but not keys, defaults or indexes. Use `CREATE TABLE t (LIKE old INCLUDING ALL)` to copy only the structure.

Q5 **COMPARISON** `SERIAL` vs `IDENTITY` in PostgreSQL?

A Both auto-generate numbers through a sequence. `IDENTITY` is the SQL-standard syntax, with clearer ownership and permissions; `SERIAL` is an older PostgreSQL shortcut.

Q6 **WHY** Why would `TRUNCATE` fail on a table?

A If other tables reference it with foreign keys. PostgreSQL requires those tables to be truncated too, or `TRUNCATE ... CASCADE`.

Q7 **WHY** Why name constraints explicitly?

A Error messages become readable (`chk_budget_positive`), and it's easier to drop or alter a constraint later.

SCENARIO-BASED

Q8 **SCENARIO** You need to empty a 50-million-row staging table every night before reloading it. `DELETE` or `TRUNCATE`?

A `TRUNCATE`. It's dramatically faster and generates far less log. `DELETE` would process and log every row.

Q9 **SCENARIO** You must change a column from `VARCHAR(20)` to `VARCHAR(10)` on a live table. What do you check first?

A Whether existing data fits: `SELECT MAX(LENGTH(col)) FROM t;` Narrowing fails if values are too long. Also consider locks on a big table and whether applications expect the old length.

FOLLOW-UP / TRAP

Q10 **TRAP QUESTION** Can you roll back a `DROP TABLE`?

A In PostgreSQL, yes, if it was run inside an explicit transaction that hasn't been committed. In MySQL and Oracle, DDL auto-commits, so no (outside of features like Oracle's recycle bin).

Q11 **TRAP QUESTION** Does TRUNCATE fire DELETE triggers?

A No. Row-level DELETE triggers do not fire (PostgreSQL has separate TRUNCATE triggers).

Session 3.4 — DML: Data Manipulation Language (plus DCL and TCL)

Simple meaning

DML commands change the data inside tables: `INSERT` adds rows, `UPDATE` changes rows, `DELETE` removes rows. (`SELECT` is sometimes counted as DML too, and sometimes as DQL.)

INSERT

```
INSERT INTO departments (dept_id, dept_name, location)
VALUES (6, 'Data Science', 'Bengaluru'),
       (7, 'Operations', 'Chennai')
RETURNING *;
```

SQL · PostgreSQL

OUTPUT

dept_id	dept_name	location
6	Data Science	Bengaluru
7	Operations	Chennai

2 rows

- Always list the columns: the statement keeps working if the table gets new columns, and it documents which value goes where.
- Several rows can be inserted in one statement.
- `RETURNING` (PostgreSQL) gives back the inserted rows, including generated IDs and defaults, without another query.

INSERT ... SELECT copies rows from a query:

```
INSERT INTO projects (project_name, client_name, dept_id, budget)
SELECT 'Audit automation for ' || dept_name, 'Internal', dept_id, 500000
FROM departments
WHERE location IN ('Mumbai', 'Kolkata')
RETURNING project_id, project_name, budget, status;
```

SQL · PostgreSQL

OUTPUT

project_id	project_name	budget	status
1	Audit automation for Finance	500000.00	ACTIVE
2	Audit automation for HR	500000.00	ACTIVE

2 rows

`project_id` came from the identity column and `status` from the column default ('ACTIVE', set by the ALTER in Session 3.3).

UPSERT: insert, or update if it already exists

A very common need: "add this product; if its ID already exists, update the price instead." PostgreSQL uses `ON CONFLICT`:

```
INSERT INTO products (product_id, product_name, category, price)
VALUES (1, 'Wireless Mouse', 'Electronics', 749.00),
       (9, 'Laptop Stand', 'Furniture', 1299.00)
ON CONFLICT (product_id)
DO UPDATE SET price = EXCLUDED.price
RETURNING product_id, product_name, price;
```

SQL · POSTGRESQL

OUTPUT

product_id	product_name	price
1	Wireless Mouse	749.00
9	Laptop Stand	1299.00

2 rows

LINE BY LINE

Line	What it means
VALUES (1, ..., 749.00), (9, ..., 1299.00)	product 1 already exists; product 9 is new
ON CONFLICT (product_id)	if inserting would violate the unique/primary key on product_id...
DO UPDATE SET price = EXCLUDED.price	...update the existing row instead; EXCLUDED means "the row we tried to insert"
RETURNING ...	shows the final state of both rows: one updated, one inserted

`ON CONFLICT ... DO NOTHING` simply skips duplicates. MySQL writes `ON DUPLICATE KEY UPDATE`; the SQL standard (and PostgreSQL 15+, SQL Server, Oracle) also offers `MERGE`.

UPDATE

```
UPDATE employees
SET salary = salary * 1.10
WHERE dept_id = 3
RETURNING emp_id, emp_name, salary;
```

SQL · POSTGRESQL

OUTPUT

emp_id	emp_name	salary
9	Rohan Gupta	71500.00
10	Divya Menon	57200.00

2 rows

- `SET` can change several columns: `SET salary = 70000, city = 'Pune'`.
- The new value can be an expression using the old one (`salary * 1.10`).
- **Without `WHERE`, every row is updated.** That is the most famous production accident in SQL.

Updating with data from another table (PostgreSQL's `UPDATE ... FROM`): mark every pending order of customers from Ahmedabad as SHIPPED.

```
UPDATE orders o
SET status = 'SHIPPED'
FROM customers c
WHERE c.customer_id = o.customer_id
      AND c.city = 'Ahmedabad'
      AND o.status = 'PENDING'
RETURNING o.order_id, c.customer_name, o.status;
```

SQL · POSTGRESQL

OUTPUT

order_id	customer_name	status
114	Vivaan Shah	SHIPPED

1 row

The same can be written with a subquery: `WHERE customer_id IN (SELECT customer_id FROM customers WHERE city = 'Ahmedabad')`.

DELETE

```
DELETE FROM leads
WHERE created_at < '2024-01-05'
RETURNING lead_id, full_name, created_at;
```

SQL · POSTGRESQL

OUTPUT

lead_id	full_name	created_at
1	Amit Kumar	2024-01-02
2	Sara Thomas	2024-01-03

2 rows

- `DELETE ... USING other_table` deletes based on a join, like `UPDATE ... FROM`.
- Deleting a parent row that children reference fails unless the foreign key says `ON DELETE CASCADE` or `SET NULL` (Session 2.3).

SAFE HABITS FOR UPDATE AND DELETE

1. Write the `SELECT` with the same `WHERE` first and check the rows it returns.
2. Run the change inside `BEGIN; ...` and check the row count ("UPDATE 3") before `COMMIT`.
3. Use `RETURNING` to see exactly what changed.
4. In production, prefer a **soft delete** (an `is_deleted` flag or `deleted_at` timestamp) when you may need the data back or for audit.

DCL: Data Control Language (permissions)

DCL decides **who may do what**.

```
CREATE ROLE reporting_user LOGIN PASSWORD 'change-me';
GRANT SELECT ON orders, customers TO reporting_user; -- read-only access to two tables
GRANT INSERT, UPDATE ON orders TO app_user;
REVOKE UPDATE ON orders FROM app_user; -- take a privilege back
```

SQL · POSTGRESQL

- Follow the **principle of least privilege**: each user or application gets only the permissions it needs. A reporting tool needs `SELECT`, never `DROP`.

- Privileges are usually granted to **roles**, and users are made members of roles.
- PostgreSQL also supports **row-level security** (users see only their own rows).

TCL: Transaction Control Language

TCL groups statements into **transactions** that succeed or fail as a unit. Full details, including ACID, come in Session 3.10.

Command	Meaning
BEGIN (or START TRANSACTION)	start a transaction
COMMIT	make all changes permanent
ROLLBACK	undo all changes since BEGIN
SAVEPOINT name	a checkpoint inside a transaction
ROLLBACK TO SAVEPOINT name	undo back to the checkpoint only

By default PostgreSQL runs in **autocommit** mode: each statement outside an explicit `BEGIN` is its own transaction and commits immediately.

SQL · POSTGRESQL

```
BEGIN;
UPDATE accounts SET balance = balance - 1000 WHERE account_id = 1;
SAVEPOINT after_debit;
UPDATE accounts SET balance = balance + 1000 WHERE account_id = 99; -- wrong account: 0 rows
ROLLBACK TO SAVEPOINT after_debit;
UPDATE accounts SET balance = balance + 1000 WHERE account_id = 2;
COMMIT;

SELECT * FROM accounts ORDER BY account_id;
```

OUTPUT

account_id	holder_name	balance
1	Aarav Patel	49000.00
2	Isha Kulkarni	21000.00
3	Kabir Khan	5000.00

3 rows

LINE BY LINE

Line	What happens
BEGIN;	start a transaction; nothing is permanent yet
UPDATE ... - 1000 WHERE account_id = 1;	debit Aarav
SAVEPOINT after_debit;	remember this point
UPDATE ... WHERE account_id = 99;	a mistake: no such account, 0 rows changed
ROLLBACK TO SAVEPOINT after_debit;	undo only what happened after the savepoint (the debit is kept)
UPDATE ... + 1000 WHERE account_id = 2;	credit the right account (Isha)
COMMIT;	both changes become permanent together

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"DML is the part of SQL that changes data: INSERT adds rows, UPDATE modifies them and DELETE removes them. In PostgreSQL I often use RETURNING to get back generated IDs or the changed rows, and INSERT ... ON CONFLICT for upserts. The biggest risk is an UPDATE or DELETE without a WHERE clause, so I first run a SELECT with the same condition and wrap the change in a transaction. DCL handles permissions with GRANT and REVOKE, and TCL controls transactions with BEGIN, COMMIT, ROLLBACK and SAVEPOINT."

COMMON TRAPS & MISTAKES

- UPDATE or DELETE without WHERE changes every row.
- Inserting without a column list: it breaks as soon as the table changes.
- Forgetting COMMIT after BEGIN : the changes are invisible to others and are lost when the session ends.
- Confusing ROLLBACK TO SAVEPOINT (partial undo) with ROLLBACK (undo everything).
- Granting broad rights (ALL PRIVILEGES) to application or reporting users.

INTERVIEW QUESTIONS**BASIC****Q1** **DEFINITION** What is DML?

A Data Manipulation Language: the commands that change data, namely INSERT, UPDATE and DELETE (SELECT is sometimes included).

Q2 **COMPARISON** DDL vs DML?

A DDL defines structure (CREATE, ALTER, DROP, TRUNCATE); DML changes the data inside the structure (INSERT, UPDATE, DELETE).

Q3 **DEFINITION** What are DCL and TCL?

A DCL controls permissions (GRANT, REVOKE). TCL controls transactions (BEGIN, COMMIT, ROLLBACK, SAVEPOINT).

INTERMEDIATE**Q4** **DEFINITION** What is an upsert and how is it written in PostgreSQL?

A Insert a row, or update it if a row with the same key exists: INSERT ... ON CONFLICT (key) DO UPDATE SET col = EXCLUDED.col; (DO NOTHING to skip instead).

Q5 **HOW** How do you update a table using values from another table?

A PostgreSQL: UPDATE t SET ... FROM other o WHERE o.id = t.id AND ...; Portable: a subquery in SET or WHERE. SQL Server uses UPDATE ... FROM ... JOIN; MySQL uses UPDATE t JOIN o ON ... SET ...

Q6 **DEFINITION** What does RETURNING do?

A It returns the rows affected by INSERT, UPDATE or DELETE (for example generated IDs), saving an extra SELECT.

Q7 **DEFINITION** What is a savepoint?

A A named checkpoint inside a transaction; ROLLBACK TO SAVEPOINT undoes changes after it while keeping earlier work.

SCENARIO-BASED

Q8 **SCENARIO** Someone ran `UPDATE employees SET salary = 0;` in production without a **WHERE** clause. How could it have been prevented, and how do you recover?

A Prevention: run the `SELECT` first, use explicit transactions, restrict permissions, require code review for production SQL. Recovery: if the transaction isn't committed, `ROLLBACK`; otherwise restore from a backup or point-in-time recovery, or use audit/history tables.

Q9 **SCENARIO** A nightly job inserts customers from a file; some already exist. How do you avoid duplicates?

A Use an upsert keyed on a unique business key (email or customer code): `ON CONFLICT (email) DO UPDATE` (or `DO NOTHING`), with a `UNIQUE` constraint so the database enforces it.

Q10 **DESIGN QUESTION** What permissions would you give a BI dashboard's database user?

A A read-only role with `SELECT` on only the reporting tables or views it needs (ideally a reporting schema or read replica); no `INSERT`, `UPDATE`, `DELETE` or `DDL` rights.

FOLLOW-UP / TRAP

Q11 **TRAP QUESTION** Is `TRUNCATE DML`?

A No, it is `DDL`, even though it removes data.

Q12 **TRAP QUESTION** What happens if you run `BEGIN`, an `UPDATE`, and then close the session without `COMMIT`?

A The transaction is rolled back; the update is lost.

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

P1. Write the SQL to add a nullable `phone` column (up to 15 characters) to `customers`.

P2. Which command removes all rows but keeps the table: `DELETE`, `TRUNCATE` or `DROP`? Which one can use `WHERE`?

P3. Insert a new customer, id 8, "Kiara Menon", kiara@mail.com, Kochi, signed up 1 May 2024.

LEVEL 2 – INTERVIEW APPLICATION

P4. Give every employee in Engineering (dept 1) a 5% raise and return their new salaries.

P5. Create a table `dept_payroll` from existing data, holding each department's name and total salary.

P6. Write an upsert that sets the price of product 2 to 3299, inserting the product if it does not exist.

LEVEL 3 – SCENARIO / PROBLEM SOLVING

P7. Delete all leads whose email appears more than once, but only the copies created *after* the first one. (Hint: compare each lead with an earlier lead that has the same email.)

P8. A colleague wants to empty `orders` with `TRUNCATE` before a reload. What will happen in PostgreSQL, and what are the options?

ANSWER / EXPLANATION

P1. `ALTER TABLE customers ADD COLUMN phone VARCHAR(15);`

P2. `TRUNCATE` removes all rows and keeps the table. Only `DELETE` accepts a `WHERE` clause.

P3.

```
INSERT INTO customers (customer_id, customer_name, email, city, signup_date)
VALUES (8, 'Kiara Menon', 'kiara@mail.com', 'Kochi', '2024-05-01')
RETURNING *;
```

SQL · PostgreSQL

OUTPUT

customer_id	customer_name	email	city	signup_date
8	Kiara Menon	kiara@mail.com	Kochi	2024-05-01

1 row

P4.

```
UPDATE employees
SET salary = ROUND(salary * 1.05, 2)
WHERE dept_id = 1
RETURNING emp_name, salary;
```

SQL · PostgreSQL

OUTPUT

emp_name	salary
Arjun Mehta	157500.00
Priya Sharma	126000.00
Rahul Verma	99750.00
Sneha Iyer	99750.00
Vikram Rao	84000.00
Pooja Reddy	75600.00

6 rows

P5. A CTAS with a join and GROUP BY (grouping is covered in Session 3.5):

```
CREATE TABLE dept_payroll AS
SELECT d.dept_name, SUM(e.salary) AS total_salary
FROM departments d
JOIN employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_name;

SELECT * FROM dept_payroll ORDER BY total_salary DESC;
```

SQL · PostgreSQL

OUTPUT

dept_name	total_salary
Engineering	612000.00
Finance	250000.00
Marketing	150000.00
HR	117000.00

4 rows

P6.

```
INSERT INTO products (product_id, product_name, category, price)
VALUES (2, 'Mechanical Keyboard', 'Electronics', 3299.00)
ON CONFLICT (product_id) DO UPDATE SET price = EXCLUDED.price
RETURNING *;
```

SQL · PostgreSQL

OUTPUT

product_id	product_name	category	price
2	Mechanical Keyboard	Electronics	3299.00

1 row

P7. A lead is a later duplicate if another lead with the same email has a smaller `lead_id` :

```
DELETE FROM leads l
USING leads earlier
WHERE earlier.email = l.email
AND earlier.lead_id < l.lead_id
RETURNING l.lead_id, l.full_name, l.email;
```

SQL · PostgreSQL

OUTPUT

lead_id	full_name	email
3	Amit Kumar	amit@mail.com
5	Sara Thomas	sara@mail.com
6	Amit K.	amit@mail.com

3 rows

Leads 3 and 6 (later copies of amit@mail.com) and 5 (a later copy of sara@mail.com) are removed; the first lead for each email stays. Session 3.11 (Pattern 6) shows the window-function version.

P8. It fails, because `order_items` and other tables reference `orders` through foreign keys. Options: `TRUNCATE orders, order_items;` (truncate the related tables together), `TRUNCATE orders CASCADE` (truncates every referencing table, so be careful), or load into a separate staging table and swap.

Session 3.5 — Grouping and Aggregation: GROUP BY and HAVING

HIGH PRIORITY

MUST KNOW

Simple meaning

GROUP BY puts rows that share a value into the same group, then an aggregate function produces one result per group. "How many employees per department?" "Revenue per month?" "Average salary per city?" are all GROUP BY questions.

SORTING EXAM PAPERS

A teacher has a pile of answer sheets from three classes. She sorts them into one pile per class (**GROUP BY class**), then counts each pile and finds its highest mark (**COUNT, MAX**). She ends with one line per class, not one line per student.

Your first GROUP BY

```
SELECT dept_id,
       COUNT(*)           AS employees,
       ROUND(AVG(salary), 2) AS avg_salary,
       MAX(salary)        AS max_salary
FROM employees
GROUP BY dept_id
ORDER BY dept_id;
```

SQL · POSTGRES SQL

OUTPUT

dept_id	employees	avg_salary	max_salary
1	6	102000.00	150000.00
2	3	83333.33	110000.00
3	2	58500.00	65000.00
4	2	75000.00	90000.00
NULL	1	55000.00	55000.00

5 rows

LINE BY LINE

Line	What it does
SELECT dept_id,	one output row per department, so show which department it is
COUNT(*) AS employees	how many rows (employees) fell into each group
ROUND(AVG(salary), 2) AS avg_salary	average salary of the group, two decimals
MAX(salary) AS max_salary	highest salary in the group
GROUP BY dept_id	make one group per distinct dept_id value
ORDER BY dept_id	sort the final result

Notice the last row: Farhan's dept_id is NULL, and **all NULLs form one group of their own**.

The golden rule of GROUP BY

Every column in the SELECT list must either be **in the GROUP BY** or **inside an aggregate function**. Otherwise the database cannot know which of the group's many values to show:

```
SELECT dept_id, emp_name, COUNT(*)
FROM employees
GROUP BY dept_id;
```

SQL · PostgreSQL

PostgreSQL Says

ERROR column "employees.emp_name" must appear in the GROUP BY clause or be used in an aggregate function

Engineering has six employees; which one name should appear on its single row? The database refuses to guess. (MySQL in its old, non-strict mode *did* guess, which hid bugs. Do not rely on that.)

A PostgreSQL Convenience

If you group by a table's **primary key**, PostgreSQL lets you select that table's other columns without listing them, because the key functionally determines them: `SELECT d.dept_id, d.dept_name, COUNT(e.emp_id) FROM departments d LEFT JOIN employees e ... GROUP BY d.dept_id;` is valid.

Grouping by several columns, and grouping joined data

```
SELECT d.dept_name,
       e.city,
       COUNT(*) AS headcount
FROM employees e
JOIN departments d ON d.dept_id = e.dept_id
GROUP BY d.dept_name, e.city
ORDER BY d.dept_name, headcount DESC;
```

SQL · PostgreSQL

Output

dept_name	city	headcount
Engineering	Bengaluru	3
Engineering	Hyderabad	2
Engineering	Pune	1
Finance	Mumbai	2
Finance	Kochi	1
HR	Kolkata	2
Marketing	Delhi	2

7 rows

One row per **combination** of department and city. Joins are covered properly in Session 3.6.

HAVING: filtering groups

WHERE filters **rows before** grouping. **HAVING** filters **groups after** aggregation, so it can use aggregates.

"Departments with more than two employees":

```
SELECT dept_id, COUNT(*) AS employees
FROM employees
GROUP BY dept_id
HAVING COUNT(*) > 2
ORDER BY employees DESC;
```

SQL · PostgreSQL

OUTPUT

dept_id	employees
1	6
2	3

2 rows

Using both: "Among employees hired before 2022, which cities have an average salary above 80000?"

```
SELECT city,
       COUNT(*) AS employees,
       ROUND(AVG(salary), 2) AS avg_salary
FROM employees
WHERE hire_date < '2022-01-01'      -- row filter: runs first
GROUP BY city
HAVING AVG(salary) > 80000          -- group filter: runs after grouping
ORDER BY avg_salary DESC;
```

SQL · PostgreSQL

OUTPUT

city	employees	avg_salary
Bengaluru	3	121666.67
Pune	1	95000.00
Mumbai	2	90000.00

3 rows

	WHERE	HAVING
Filters	individual rows	groups
Runs	before GROUP BY	after GROUP BY
Can use aggregates (COUNT , SUM ...)	no	yes
Works without GROUP BY	yes	yes (the whole result is one group), but rarely useful
Performance tip	filter as early as possible here	only for conditions that need aggregates

The logical order of execution

SQL is *written* in one order but *processed* in another. Knowing the processing order explains most beginner errors:

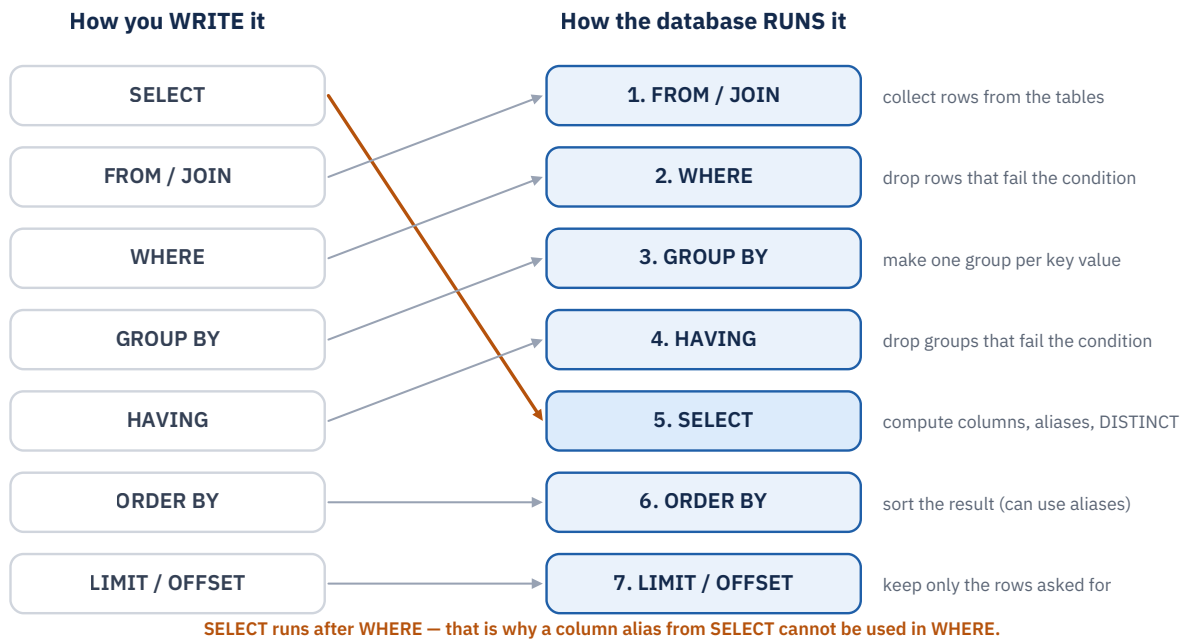


Figure 15. Written order vs logical processing order. SELECT is evaluated late.

This order explains:

- **Why WHERE cannot use SELECT aliases** (WHERE runs before SELECT).
- **Why WHERE cannot use aggregates** (groups do not exist yet).
- **Why ORDER BY can use aliases** (it runs after SELECT).
- **Why HAVING cannot use a SELECT alias in PostgreSQL** (HAVING runs before SELECT):

```
SELECT dept_id, COUNT(*) AS n
FROM employees
GROUP BY dept_id
HAVING n > 2;
```

SQL · PostgreSQL

POSTGRESQL SAYS

ERROR column "n" does not exist

Write `HAVING COUNT(*) > 2` instead. (MySQL accepts aliases in HAVING; PostgreSQL and SQL Server do not.) PostgreSQL *does* accept output-column aliases in `GROUP BY` as a convenience, but the portable habit is to repeat the expression.

THE PHYSICAL PLAN CAN DIFFER

This is the *logical* order: it defines what the result must be. The optimizer may physically do things differently (for example, push a filter into an index scan), as long as the result is the same.

Listing the members of each group

STRING_AGG joins the group's values into one string; ARRAY_AGG builds an array.

```
SELECT d.dept_name,
       COUNT(e.emp_id) AS employees,
       STRING_AGG(e.emp_name, ' ' ORDER BY e.emp_name) AS members
FROM departments d
JOIN employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY employees DESC;
```

SQL · POSTGRES SQL

OUTPUT

dept_name	employees	members
Engineering	6	Arjun Mehta, Pooja Reddy, Priya Sharma, Rahul Verma, Sneha Iyer, Vikram Rao
Finance	3	Ananya Das, Karan Malhotra, Meera Nair
HR	2	Divya Menon, Rohan Gupta
Marketing	2	Aditya Singh, Neha Kapoor

4 rows

Subtotals with ROLLUP INTERVIEW EXTENSION

GROUP BY ROLLUP (a, b) adds subtotal rows per a and a grand total. CUBE adds totals for every combination; GROUPING SETS lets you list exactly which groupings you want.

```
SELECT COALESCE(p.category, 'ALL CATEGORIES') AS category,
       SUM(i.quantity * i.unit_price) AS revenue
FROM order_items i
JOIN products p ON p.product_id = i.product_id
GROUP BY ROLLUP (p.category)
ORDER BY revenue;
```

SQL · POSTGRES SQL

OUTPUT

category	revenue
Stationery	6966.00
Electronics	26786.00
Furniture	83993.00
ALL CATEGORIES	117745.00

4 rows

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"GROUP BY splits rows into groups that share a value and lets aggregate functions like COUNT, SUM and AVG return one row per group, for example the number of employees per department. Every selected column must be grouped or aggregated. WHERE filters rows before grouping, so it can't use aggregates; HAVING filters groups after aggregation, for example departments with more than two employees. Logically the database runs FROM, WHERE, GROUP BY, HAVING, then SELECT and ORDER BY, which is also why aliases work in ORDER BY but not in WHERE."

COMMON TRAPS & MISTAKES

- Putting an aggregate in WHERE (WHERE COUNT(*) > 2): it must go in HAVING.
- Selecting a column that is neither grouped nor aggregated.
- Using HAVING for row-level filters that belong in WHERE (works, but it is slower and less clear).

- Forgetting that NULLs form their own group.
- Using a SELECT alias in HAVING in PostgreSQL.

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What does GROUP BY do?

A It groups rows with the same values in the given columns so aggregate functions return one result per group.

Q2 **COMPARISON** WHERE vs HAVING?

A WHERE filters rows before grouping and cannot use aggregates; HAVING filters groups after grouping and can use aggregates.

Q3 **SQL PROBLEM** Count employees in each city.

A `SELECT city, COUNT(*) FROM employees GROUP BY city;`

INTERMEDIATE

Q4 **WHY** Why does `SELECT dept_id, emp_name, COUNT(*) FROM employees GROUP BY dept_id` fail?

A emp_name is neither grouped nor aggregated, so there is no single value to show per department.

Q5 **HOW** What is the logical order of SQL clauses?

A FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT/OFFSET.

Q6 **HOW** Can you use HAVING without GROUP BY?

A Yes, then the whole table is one group: `SELECT COUNT(*) FROM employees HAVING COUNT(*) > 10` returns the count only if it exceeds 10. It's rarely needed.

Q7 **HOW** How are NULLs treated in GROUP BY?

A All NULLs are placed together into one group.

Q8 **DEFINITION** What does ROLLUP do?

A It adds subtotal and grand-total rows to a GROUP BY result, e.g. revenue per category plus an overall total.

SCENARIO-BASED

Q9 **SQL PROBLEM** Find cities with at least two employees and an average salary above 70000.

A `SELECT city, COUNT(*), AVG(salary) FROM employees GROUP BY city HAVING COUNT(*) >= 2 AND AVG(salary) > 70000;`

Q10 **SQL PROBLEM** Total order value per customer for delivered orders only, top spender first.

A `SELECT customer_id, SUM(total_amount) AS spent FROM orders WHERE status = 'DELIVERED' GROUP BY customer_id ORDER BY spent DESC;`

FOLLOW-UP / TRAP

Q11 **TRAP QUESTION** Is `WHERE salary > 50000` the same as `HAVING salary > 50000` in a grouped query?

A No. HAVING can only refer to grouped columns or aggregates, so the HAVING version errors unless salary is grouped. Row conditions belong in WHERE.

Q12 **TRAP QUESTION** Does `COUNT(*)` in a grouped query ever return 0?

A Not for a group produced by `GROUP BY`, since every group has at least one row. Zero counts appear only with outer joins and `COUNT(column)` (see the next session).

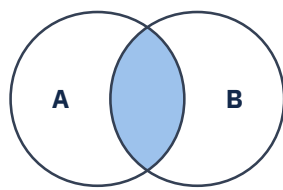
Session 3.6 – Joins

HIGH PRIORITY **MUST KNOW** Joins and their types are among the questions most often mentioned in PwC interview write-ups.

Why joins exist

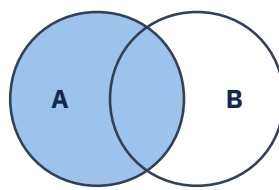
Normalization (Session 2.5) splits data into tables: employee names in one, department names in another. A **join** combines rows from two tables **based on a related column**, usually a foreign key matching a primary key, so you can see the pieces together again.

The join types at a glance



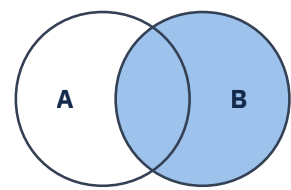
INNER JOIN

only rows that match in both



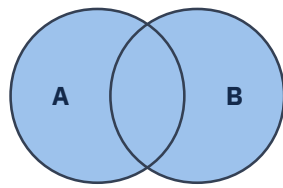
LEFT JOIN

all of A + matching rows of B



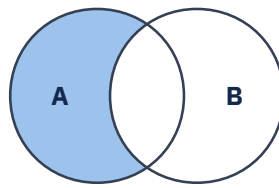
RIGHT JOIN

all of B + matching rows of A



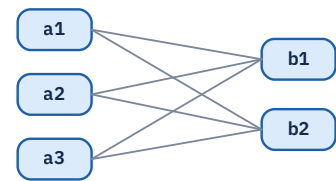
FULL OUTER JOIN

everything from A and B



LEFT JOIN ... WHERE B.key IS NULL

rows of A with NO match (anti-join)



CROSS JOIN

every A row × every B row ($3 \times 2 = 6$)

Figure 16. What each join returns. A is the left table (written first), B is the right table.

Join	Returns	Typical question
INNER JOIN	only rows with a match in both tables	"employees with their department names"
LEFT [OUTER] JOIN	all rows from the left table + matches from the right (NULLs when none)	"all customers, with orders if any"
RIGHT [OUTER] JOIN	all rows from the right table + matches from the left	same as LEFT with the tables swapped
FULL [OUTER] JOIN	all rows from both; NULLs where there is no match	"reconcile two lists: what's missing on either side?"
CROSS JOIN	every combination of rows (Cartesian product)	"every store × every day"
Self join	a table joined to itself	"employee and their manager"

INNER JOIN

```
SELECT e.emp_name, d.dept_name, d.location
FROM employees e
INNER JOIN departments d ON d.dept_id = e.dept_id
ORDER BY d.dept_name, e.emp_name;
```

SQL · POSTGRESQL

OUTPUT

emp_name	dept_name	location
Arjun Mehta	Engineering	Bengaluru
Pooja Reddy	Engineering	Bengaluru
Priya Sharma	Engineering	Bengaluru
Rahul Verma	Engineering	Bengaluru
Sneha Iyer	Engineering	Bengaluru
Vikram Rao	Engineering	Bengaluru
Ananya Das	Finance	Mumbai
Karan Malhotra	Finance	Mumbai
Meera Nair	Finance	Mumbai
Divya Menon	HR	Kolkata
Rohan Gupta	HR	Kolkata
Aditya Singh	Marketing	Delhi
Neha Kapoor	Marketing	Delhi

13 rows

LINE BY LINE

Line	What it does
FROM employees e	the left table, aliased e
INNER JOIN departments d	the right table, aliased d (INNER is optional: plain JOIN means inner join)
ON d.dept_id = e.dept_id	the join condition: pair rows whose department IDs are equal
SELECT e.emp_name, d.dept_name, d.location	columns from both tables, prefixed by alias to avoid ambiguity

Two rows are missing on purpose: **Farhan** (no department, NULL never equals anything) and the **Legal** department (no employees). An inner join keeps only matches.

LEFT JOIN

```
SELECT e.emp_name, d.dept_name
FROM employees e
LEFT JOIN departments d ON d.dept_id = e.dept_id
ORDER BY d.dept_name NULLS FIRST, e.emp_name
LIMIT 5;
```

SQL · POSTGRESQL

OUTPUT

emp_name	dept_name
Farhan Ali	NULL
Arjun Mehta	Engineering
Pooja Reddy	Engineering
Priya Sharma	Engineering
Rahul Verma	Engineering

5 rows

Every employee appears; Farhan's department columns are NULL. A LEFT JOIN answers "all of A, plus B where it exists".

Counting with a LEFT JOIN, the classic trap: employees per department, *including* departments with nobody.

```
SELECT d.dept_name,
       COUNT(*)      AS count_star_wrong,
       COUNT(e.emp_id) AS employees
FROM departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY employees DESC;
```

SQL · POSTGRESQL

OUTPUT

dept_name	count_star_wrong	employees
Engineering	6	6
Finance	3	3
HR	2	2
Marketing	2	2
Legal	1	0

5 rows

Legal shows **1** with `COUNT(*)`, because the left join produced one row for Legal (with NULL employee columns), and `COUNT(*)` counts rows. `COUNT(e.emp_id)` counts only real employees: **0**. Always count a column from the right-hand table.

RIGHT JOIN

A RIGHT JOIN keeps every row of the *right* table. It is exactly a LEFT JOIN with the tables written in the opposite order, so many teams use only LEFT JOIN for readability.

```
SELECT d.dept_name, e.emp_name
FROM employees e
RIGHT JOIN departments d ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL;
```

SQL · POSTGRESQL

OUTPUT

dept_name	emp_name
Legal	NULL

1 row

This finds departments with no employees: Legal.

FULL OUTER JOIN

```
SELECT e.emp_name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL OR d.dept_id IS NULL;
```

SQL · POSTGRESQL

OUTPUT

emp_name	dept_name
Farhan Ali	NULL
NULL	Legal

2 rows

Both kinds of unmatched rows in one query: an employee without a department *and* a department without employees. FULL OUTER JOIN is great for **reconciliation**, such as comparing yesterday's file with today's. (MySQL has no FULL JOIN; emulate it with a LEFT JOIN UNION a RIGHT JOIN.)

CROSS JOIN

```
SELECT r.region, s.size
FROM (VALUES ('North'), ('South')) AS r(region)
CROSS JOIN (VALUES ('S'), ('M'), ('L')) AS s(size)
ORDER BY r.region, s.size;
```

SQL · POSTGRESQL

OUTPUT

region	size
North	L
North	M
North	S
South	L
South	M
South	S

6 rows

2 rows × 3 rows = 6 rows. Use it deliberately for **all combinations** (every product × every month for a sales grid). Accidentally forgetting a join condition with the old comma syntax (FROM a, b) creates a cross join, which can explode into millions of rows.

SELF JOIN

A self join treats one table as two, using two aliases. Employees and their managers live in the same table:

```
SELECT e.emp_name AS employee,
       m.emp_name AS manager
FROM employees e
LEFT JOIN employees m ON m.emp_id = e.manager_id
ORDER BY m.emp_name NULLS FIRST, e.emp_name;
```

SQL · POSTGRESQL

OUTPUT

employee	manager
Arjun Mehta	NULL
Neha Kapoor	Aditya Singh
Karan Malhotra	Ananya Das
Meera Nair	Ananya Das
Aditya Singh	Arjun Mehta
Ananya Das	Arjun Mehta
Farhan Ali	Arjun Mehta
Priya Sharma	Arjun Mehta
Rohan Gupta	Arjun Mehta
Rahul Verma	Priya Sharma
Sneha Iyer	Priya Sharma
Vikram Rao	Priya Sharma
Pooja Reddy	Rahul Verma
Divya Menon	Rohan Gupta

14 rows

LINE BY LINE

Line	What it does
FROM employees e	the table in its "employee" role
LEFT JOIN employees m	the same table again in its "manager" role
ON m.emp_id = e.manager_id	an employee's manager_id points at the manager's emp_id
LEFT	keeps Arjun, who has no manager (an INNER join would drop him)

Self joins also find pairs, such as employees who work in the same city. The condition `e1.emp_id < e2.emp_id` avoids pairing someone with themselves and listing each pair twice:

```
SELECT e1.emp_name, e2.emp_name, e1.city
FROM employees e1
JOIN employees e2
  ON e1.city = e2.city
 AND e1.emp_id < e2.emp_id
WHERE e1.city IN ('Pune', 'Mumbai')
ORDER BY e1.city;
```

SQL · POSTGRESQL

OUTPUT

emp_name	emp_name	city
Ananya Das	Karan Malhotra	Mumbai
Rahul Verma	Farhan Ali	Pune

2 rows

Anti-join: rows with no match

"Customers who have never ordered" is one of the most common interview questions. A LEFT JOIN plus `IS NULL` on the right table's key:

```
SELECT c.customer_id, c.customer_name
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

SQL · POSTGRESQL

OUTPUT

customer_id	customer_name
6	Saanvi Rao
7	Reyansh Bose

2 rows

The equivalent `NOT EXISTS` form (often the clearest) appears in Session 3.7.

Joining many tables

Revenue per customer per product category needs four tables. Join them one step at a time, following the foreign keys:

```
SELECT c.customer_name,
       p.category,
       SUM(i.quantity * i.unit_price) AS revenue
FROM customers c
JOIN orders o    ON o.customer_id = c.customer_id
JOIN order_items i ON i.order_id   = o.order_id
JOIN products p  ON p.product_id  = i.product_id
WHERE o.status <> 'CANCELLED'
GROUP BY c.customer_name, p.category
ORDER BY c.customer_name, revenue DESC;
```

SQL · POSTGRESQL

OUTPUT

customer_name	category	revenue
Aarav Patel	Furniture	33997.00
Aarav Patel	Electronics	7096.00
Aarav Patel	Stationery	1245.00
Diya Joshi	Electronics	6796.00
Isha Kulkarni	Furniture	17998.00
Isha Kulkarni	Electronics	3499.00
Isha Kulkarni	Stationery	2486.00
Kabir Khan	Furniture	15999.00
Kabir Khan	Electronics	2397.00
Vivaan Shah	Electronics	6998.00
Vivaan Shah	Stationery	3235.00

11 rows

Join pitfalls that break real reports

1. Fan-out (row multiplication). Joining a one-to-many relationship repeats the "one" side once per match.

Summing a column from the "one" side then double counts:

```
SELECT c.customer_name,  
       SUM(o.total_amount) AS inflated_total    -- WRONG  
FROM   customers c  
JOIN   orders o      ON o.customer_id = c.customer_id  
JOIN   order_items i ON i.order_id   = o.order_id  
GROUP BY c.customer_name  
ORDER BY c.customer_name;
```

SQL · POSTGRESQL

OUTPUT

customer_name	inflated_total
Aarav Patel	75677.00
Diya Joshi	9594.00
Isha Kulkarni	26469.00
Kabir Khan	34395.00
Vivaan Shah	13468.00

5 rows

Aarav's real total is 42338, but each order total was repeated once per order line. Fix it by not joining tables you do not need, or by aggregating each side *before* joining:

```
SELECT c.customer_name,  
       SUM(o.total_amount) AS correct_total  
FROM   customers c  
JOIN   orders o ON o.customer_id = c.customer_id  
GROUP BY c.customer_name  
ORDER BY c.customer_name;
```

SQL · POSTGRESQL

OUTPUT

customer_name	correct_total
Aarav Patel	42338.00
Diya Joshi	6796.00
Isha Kulkarni	23983.00
Kabir Khan	34395.00
Vivaan Shah	10233.00

5 rows

2. A WHERE clause that silently turns a LEFT JOIN into an INNER JOIN. "All departments with their employees earning more than 60000":

```
SELECT d.dept_name, e.emp_name
FROM departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id
WHERE e.salary > 60000
ORDER BY d.dept_name
LIMIT 4;
```

SQL · POSTGRESQL

OUTPUT

dept_name	emp_name
Engineering	Priya Sharma
Engineering	Rahul Verma
Engineering	Sneha Iyer
Engineering	Arjun Mehta

4 rows

Legal disappeared: for Legal, `e.salary` is NULL, `NULL > 60000` is UNKNOWN, and WHERE drops the row. Put conditions on the right table **inside the ON clause** to keep all left rows:

```
SELECT d.dept_name, COUNT(e.emp_id) AS well_paid
FROM departments d
LEFT JOIN employees e
    ON e.dept_id = d.dept_id
    AND e.salary > 60000
GROUP BY d.dept_name
ORDER BY d.dept_name;
```

SQL · POSTGRESQL

OUTPUT

dept_name	well_paid
Engineering	6
Finance	3
HR	1
Legal	0
Marketing	1

5 rows

3. NULLs never match. `NULL = NULL` is not true, so rows with NULL join keys never pair up (Farhan above).

4. Ambiguous columns. If both tables have `dept_id`, writing just `dept_id` fails ("column reference is ambiguous"). Always prefix with aliases.

Referential integrity and joins

Because `orders.customer_id` is a NOT NULL foreign key, every order is guaranteed to have a matching customer, so `orders JOIN customers` never loses an order. Without the foreign key, orphan orders would silently vanish from inner-join reports. Constraints protect your query results, not just your inserts.

INNER JOIN vs LEFT JOIN

	INNER JOIN	LEFT JOIN
Rows kept	only matching pairs	all left rows, matched or not
Unmatched left rows	dropped	kept, with NULLs for right-side columns
Typical use	"orders with their customer"	"all customers, with orders if any"; finding missing matches
Row count	$\leq$ the matching combinations	$\geq$ the number of left rows
Counting	<code>COUNT(*)</code> is fine	count a right-table column, e.g. <code>COUNT(o.order_id)</code>

Two more syntax notes

- `JOIN ... USING (dept_id)` is shorthand when both columns have the same name; the column appears once in the output.
- Old comma syntax: `FROM employees e, departments d WHERE e.dept_id = d.dept_id` is an inner join written the 1980s way. Explicit `JOIN ... ON` is clearer and avoids accidental cross joins.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"A join combines rows from two tables based on a related column, usually a foreign key and a primary key. An inner join returns only rows that match in both tables. A left join returns every row from the left table plus matching rows from the right, with NULLs where there's no match; that's how I find, for example, customers with no orders, by checking that the order key is NULL. Right join is the mirror image, full outer join keeps unmatched rows from both sides, cross join gives every combination, and a self join joins a table to itself, like employees to their managers. Two things I watch for are duplicate rows when joining one-to-many before summing, and WHERE conditions on the right table that turn a left join into an inner join."

COMMON TRAPS & MISTAKES

- `COUNT(*)` with a LEFT JOIN counts unmatched rows as 1; count a right-table column instead.
- Filtering the right table in WHERE after a LEFT JOIN removes the unmatched rows; move the condition into ON.
- Summing a "one-side" column after joining a "many" table (fan-out) inflates totals.
- Saying "RIGHT JOIN is different from LEFT JOIN". It is the same operation with the tables swapped.
- Forgetting that NULL keys never match.
- Saying "a self join needs a special keyword". It is a normal join with two aliases.

INTERVIEW QUESTIONS

BASIC

Q1 DEFINITION What is a join?

- A** An operation that combines rows from two or more tables based on a related column, usually a foreign key matching a primary key.

Q2 COMPARISON INNER JOIN vs LEFT JOIN?

- A** INNER returns only rows with matches in both tables. LEFT returns all rows from the left table, with NULLs for right-side columns where there is no match.

Q3 **DEFINITION** What is a self join? Give an example.

A Joining a table to itself using two aliases, e.g. employees joined to employees to show each employee's manager.

Q4 **DEFINITION** What is a CROSS JOIN?

A The Cartesian product: every row of one table combined with every row of the other ($m \times n$ rows).

INTERMEDIATE

Q5 **SQL PROBLEM** Find customers who have never placed an order.

A `SELECT c.* FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id WHERE o.order_id IS NULL;` or with `NOT EXISTS`.

Q6 **COMPARISON** What is the difference between putting a condition in ON vs WHERE for a LEFT JOIN?

A In ON, it controls which right rows match while all left rows are kept. In WHERE, it filters after the join and removes left rows whose right side is NULL, turning it into an inner join.

Q7 **WHY** Why might a SUM be too high after joining several tables?

A One-to-many joins duplicate the "one" side's rows (fan-out), so its values are summed several times. Aggregate before joining or remove unneeded joins.

Q8 **DEFINITION** What does a FULL OUTER JOIN return, and when is it useful?

A All rows from both tables, matched where possible and with NULLs otherwise; useful for reconciliation, such as finding records missing on either side.

Q9 **HOW** If table A has 4 rows and B has 3, how many rows can each join return?

A CROSS: exactly 12. INNER: 0 to 12, depending on matches. LEFT: at least 4 (up to 12). FULL: at least $\max(4, 3)$, at most 12 (or 7 if nothing matches).

SCENARIO-BASED

Q10 **SQL PROBLEM** List each department with its number of employees, including departments with none.

A `SELECT d.dept_name, COUNT(e.emp_id) FROM departments d LEFT JOIN employees e ON e.dept_id = d.dept_id GROUP BY d.dept_name;`

Q11 **SQL PROBLEM** Show each employee with their manager's name; employees without a manager should still appear.

A `SELECT e.emp_name, m.emp_name AS manager FROM employees e LEFT JOIN employees m ON m.emp_id = e.manager_id;`

Q12 **SCENARIO** Yesterday's and today's product files are loaded into two tables. How do you find added, removed and changed products?

A FULL OUTER JOIN on product_id. Rows with only the new side are added, only the old side are removed, and both sides with different values (e.g. price) are changed.

FOLLOW-UP / TRAP

Q13 **TRAP QUESTION** Is there any difference between JOIN and INNER JOIN ?

A No. JOIN defaults to an inner join.

Q14 **TRAP QUESTION** Can a join condition use something other than equality?

A Yes, any condition: ranges (`o.order_date BETWEEN p.start AND p.end`), inequalities (`e1.emp_id < e2.emp_id`) and so on. These are called non-equi joins.

Q15 **TRAP QUESTION** Two employees have NULL dept_id . Will an inner join on dept_id pair them with each other?

A No. NULL never equals NULL, so NULL keys never match in a join.

PRACTICE QUESTIONS

LEVEL 1 – BASIC UNDERSTANDING

P1. Show each order's id, date and customer name.

P2. Count orders per status.

P3. Which join keeps all rows from both tables?

LEVEL 2 – INTERVIEW APPLICATION

P4. List products that have never been ordered.

P5. Show each department's name, number of employees and total salary, only for departments with total salary above 150000.

P6. For every employee, show their name, department name (or 'Unassigned') and manager's name (or 'None').

LEVEL 3 – SCENARIO / PROBLEM SOLVING

P7. Find the product categories each customer has bought from, as one comma-separated list per customer (exclude cancelled orders).

P8. Find pairs of employees in the same department where the first earns more than the second, showing both names and the salary difference. Limit the output to Finance.

ANSWER / EXPLANATION

P1.

```
SELECT o.order_id, o.order_date, c.customer_name
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
ORDER BY o.order_id
LIMIT 6;
```

SQL · POSTGRES SQL

OUTPUT

order_id	order_date	customer_name
101	2024-01-05	Aarav Patel
102	2024-01-12	Isha Kulkarni
103	2024-01-28	Aarav Patel
104	2024-02-02	Kabir Khan
105	2024-02-14	Diya Joshi
106	2024-02-20	Isha Kulkarni

6 rows

P2.

```
SELECT status, COUNT(*) AS orders
FROM orders
GROUP BY status
ORDER BY orders DESC;
```

SQL · POSTGRESQL

OUTPUT

status	orders
DELIVERED	8
SHIPPED	2
PENDING	2
CANCELLED	1
RETURNED	1

5 rows

P3. FULL OUTER JOIN.**P4. Anti-join from products to order_items:**

```
SELECT p.product_id, p.product_name
FROM products p
LEFT JOIN order_items i ON i.product_id = p.product_id
WHERE i.product_id IS NULL;
```

SQL · POSTGRESQL

OUTPUT

product_id	product_name
8	Monitor Arm

1 row

P5.

```
SELECT d.dept_name,
       COUNT(e.emp_id) AS employees,
       SUM(e.salary) AS total_salary
FROM departments d
JOIN employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_name
HAVING SUM(e.salary) > 150000
ORDER BY total_salary DESC;
```

SQL · POSTGRESQL

OUTPUT

dept_name	employees	total_salary
Engineering	6	612000.00
Finance	3	250000.00

2 rows

P6. Two LEFT JOINS: one to departments, one self join to managers:

SQL - POSTGRESQL

```

SELECT e.emp_name,
       COALESCE(d.dept_name, 'Unassigned') AS department,
       COALESCE(m.emp_name, 'None')       AS manager
FROM employees e
LEFT JOIN departments d ON d.dept_id = e.dept_id
LEFT JOIN employees m ON m.emp_id = e.manager_id
ORDER BY e.emp_id;

```

OUTPUT

emp_name	department	manager
Arjun Mehta	Engineering	None
Priya Sharma	Engineering	Arjun Mehta
Rahul Verma	Engineering	Priya Sharma
Sneha Iyer	Engineering	Priya Sharma
Vikram Rao	Engineering	Priya Sharma
Ananya Das	Finance	Arjun Mehta
Karan Malhotra	Finance	Ananya Das
Meera Nair	Finance	Ananya Das
Rohan Gupta	HR	Arjun Mehta
Divya Menon	HR	Rohan Gupta
Aditya Singh	Marketing	Arjun Mehta
Neha Kapoor	Marketing	Aditya Singh
Farhan Ali	Unassigned	Arjun Mehta
Pooja Reddy	Engineering	Rahul Verma

14 rows

P7.

SQL - POSTGRESQL

```

SELECT c.customer_name,
       STRING_AGG(DISTINCT p.category, ', ' ORDER BY p.category) AS categories
FROM customers c
JOIN orders o      ON o.customer_id = c.customer_id
JOIN order_items i ON i.order_id    = o.order_id
JOIN products p    ON p.product_id = i.product_id
WHERE o.status <> 'CANCELLED'
GROUP BY c.customer_name
ORDER BY c.customer_name;

```

OUTPUT

customer_name	categories
Aarav Patel	Electronics, Furniture, Stationery
Diya Joshi	Electronics
Isha Kulkarni	Electronics, Furniture, Stationery
Kabir Khan	Electronics, Furniture
Vivaan Shah	Electronics, Stationery

5 rows

DISTINCT inside STRING_AGG removes repeated categories.

P8.

SQL · POSTGRESQL

```
SELECT a.emp_name AS higher_paid,  
       b.emp_name AS lower_paid,  
       a.salary - b.salary AS difference  
FROM employees a  
JOIN employees b  
  ON a.dept_id = b.dept_id  
 AND a.salary > b.salary  
WHERE a.dept_id = 2  
ORDER BY difference DESC;
```

OUTPUT

higher_paid	lower_paid	difference
Ananya Das	Karan Malhotra	40000.00
Ananya Das	Meera Nair	40000.00

2 rows

Karan and Meera both earn 70000, so neither appears as "higher paid" than the other: the strict `>` excludes equal salaries.

Session 3.7 — Subqueries and CTEs

HIGH PRIORITY Most "medium" SQL interview questions are solved with a subquery, a CTE or a window function. Often all three work, and interviewers like to hear you compare them.

What is a subquery?

A **subquery is a query inside another query**. The inner query produces a value, a list or a table, and the outer query uses it. "Who earns more than the average?" needs the average first, and that is a subquery.

Kind	Returns	Where it is used	Example
Scalar	one value (1 row, 1 column)	WHERE, SELECT, HAVING	<code>(SELECT AVG(salary) FROM employees)</code>
Multi-row	a list of values	IN, ANY, ALL, EXISTS	<code>dept_id IN (SELECT dept_id FROM departments WHERE ...)</code>
Table (derived table)	rows and columns	FROM	<code>FROM (SELECT ...) AS t</code>
Correlated	depends on the current outer row	WHERE, SELECT	<code>salary > (SELECT AVG(salary) FROM employees x WHERE x.dept_id = e.dept_id)</code>

Scalar subquery in WHERE

```
SELECT emp_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees)
ORDER BY salary DESC;
```

SQL · POSTGRESQL

OUTPUT

emp_name	salary
Arjun Mehta	150000.00
Priya Sharma	120000.00
Ananya Das	110000.00
Rahul Verma	95000.00
Sneha Iyer	95000.00
Aditya Singh	90000.00

6 rows

LINE BY LINE

Part	What it does
<code>(SELECT AVG(salary) FROM employees)</code>	runs first and returns one number: the company average (84571.43)
<code>WHERE salary > (...)</code>	compares every employee with that single number

You cannot write `WHERE salary > AVG(salary)` : aggregates are not allowed in WHERE (Session 3.5). The subquery computes the aggregate separately.

A scalar subquery must return **at most one row**. If it returns several, PostgreSQL stops with an error:

```
SELECT emp_name
FROM employees
WHERE salary = (SELECT salary FROM employees WHERE dept_id = 2);
```

SQL · PostgreSQL

POSTGRES SQL SAYS

ERROR more than one row returned by a subquery used as an expression

Use **IN** for lists, or make the inner query return one value (MAX, LIMIT 1 or a precise filter).

Multi-row subquery with IN, ANY and ALL

Employees who work in departments located in Bengaluru or Mumbai:

```
SELECT emp_name, dept_id
FROM employees
WHERE dept_id IN (SELECT dept_id
                  FROM departments
                  WHERE location IN ('Bengaluru', 'Mumbai'))
ORDER BY dept_id, emp_name;
```

SQL · PostgreSQL

OUTPUT

emp_name	dept_id
Arjun Mehta	1
Pooja Reddy	1
Priya Sharma	1
Rahul Verma	1
Sneha Iyer	1
Vikram Rao	1
Ananya Das	2
Karan Malhotra	2
Meera Nair	2

9 rows

- **x > ANY (subquery)** : greater than **at least one** value (that is, greater than the minimum).
- **x > ALL (subquery)** : greater than **every** value (that is, greater than the maximum).

```
SELECT emp_name, salary
FROM employees
WHERE salary > ALL (SELECT salary FROM employees WHERE dept_id = 2);
```

SQL · PostgreSQL

OUTPUT

emp_name	salary
Arjun Mehta	150000.00
Priya Sharma	120000.00

2 rows

Everyone listed earns more than every Finance employee, and Finance's top salary is 110000.

Subquery in FROM (derived table)

A subquery in FROM acts like a temporary table. Give it an alias (`AS dept_avg`): most databases, and PostgreSQL before version 16, require one.

```
SELECT dept_id, avg_salary
FROM (SELECT dept_id, ROUND(AVG(salary), 2) AS avg_salary
      FROM employees
      WHERE dept_id IS NOT NULL
      GROUP BY dept_id) AS dept_avg
WHERE avg_salary > 70000
ORDER BY avg_salary DESC;
```

SQL · PostgreSQL

OUTPUT

dept_id	avg_salary
1	102000.00
2	83333.33
4	75000.00

3 rows

The outer query can filter on `avg_salary` by name, because the inner query has already turned it into a column. This is the subquery alternative to HAVING, and a way around the "can't use an alias in WHERE" rule.

Subquery in SELECT

```
SELECT emp_name,
       salary,
       (SELECT ROUND(AVG(salary), 2) FROM employees) AS company_avg,
       salary - (SELECT ROUND(AVG(salary), 2) FROM employees) AS diff_from_avg
FROM employees
WHERE dept_id = 4;
```

SQL · PostgreSQL

OUTPUT

emp_name	salary	company_avg	diff_from_avg
Aditya Singh	90000.00	84571.43	5428.57
Neha Kapoor	60000.00	84571.43	-24571.43

2 rows

Correlated subqueries

A **correlated** subquery refers to a column of the outer query, so conceptually it **re-runs for each outer row**. Classic question: "employees who earn more than the average of *their own* department".

```
SELECT e.emp_name, e.dept_id, e.salary
FROM employees e
WHERE e.salary > (SELECT AVG(x.salary)
                  FROM employees x
                  WHERE x.dept_id = e.dept_id)
ORDER BY e.dept_id;
```

SQL · POSTGRESQL

OUTPUT

emp_name	dept_id	salary
Arjun Mehta	1	150000.00
Priya Sharma	1	120000.00
Ananya Das	2	110000.00
Rohan Gupta	3	65000.00
Aditya Singh	4	90000.00

5 rows

LINE BY LINE

Part	What it does
FROM employees e	the outer query looks at one employee at a time
(SELECT AVG(x.salary) FROM employees x WHERE x.dept_id = e.dept_id)	for <i>that</i> employee, compute the average of their department; e.dept_id links inner to outer
WHERE e.salary > (...)	keep the employee if they beat their department's average

Farhan never appears: his dept_id is NULL, x.dept_id = NULL matches nothing, the average is NULL, and the comparison is UNKNOWN.

Performance: a correlated subquery can be slow on big tables if it really runs once per row, although modern optimizers often rewrite it as a join. The same question can be answered with a join to a grouped derived table, or with a window function (Session 3.8), which reads the table once:

```
SELECT emp_name, dept_id, salary
FROM (SELECT emp_name, dept_id, salary,
             AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg
      FROM employees) t
WHERE salary > dept_avg;
```

SQL · POSTGRESQL

EXISTS and NOT EXISTS

EXISTS (subquery) is true if the subquery returns **at least one row**. It does not matter what the subquery selects; SELECT 1 is the convention.

Customers who have placed at least one order (a **semi-join**):

```
SELECT c.customer_name
FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
ORDER BY c.customer_name;
```

SQL · POSTGRESQL

OUTPUT

customer_name
Aarav Patel
Diya Joshi
Isha Kulkarni
Kabir Khan
Vivaan Shah

5 rows

Customers who have never ordered (an **anti-join**):

```
SELECT c.customer_name
FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

SQL · POSTGRESQL

OUTPUT

customer_name
Saanvi Rao
Reyansh Bose

2 rows

The NOT IN + NULL trap

TRAP QUESTION "Find employees who are not managers of anyone." The natural attempt:

```
SELECT emp_name
FROM employees
WHERE emp_id NOT IN (SELECT manager_id FROM employees);
```

SQL · POSTGRESQL

OUTPUT

emp_name

0 rows

Zero rows, even though most employees manage nobody. Why? The subquery's list contains a **NULL** (Arjun has no manager). `x NOT IN (1, 2, NULL)` means `x <> 1 AND x <> 2 AND x <> NULL`, and `x <> NULL` is UNKNOWN, so the whole condition can never be TRUE. Two correct versions:

```
SELECT e.emp_name
FROM employees e
WHERE NOT EXISTS (SELECT 1 FROM employees x WHERE x.manager_id = e.emp_id)
ORDER BY e.emp_name;
```

SQL · POSTGRESQL

OUTPUT

emp_name
Divya Menon
Farhan Ali
Karan Malhotra
Meera Nair
Neha Kapoor
Pooja Reddy
Sneha Iyer
Vikram Rao

8 rows

```
-- or remove NULLs from the list explicitly
WHERE emp_id NOT IN (SELECT manager_id FROM employees WHERE manager_id IS NOT NULL)
```

SQL · POSTGRESQL

Habit: prefer `NOT EXISTS` over `NOT IN` with subqueries.

	IN	EXISTS
Checks	is the value in the list?	does at least one matching row exist?
NULLs in the subquery	<code>NOT IN</code> breaks (returns nothing)	<code>NOT EXISTS</code> is safe
Typical use	small, fixed or NULL-free lists	correlated "has / has no related rows" checks
Performance	modern optimizers often plan both the same way; <code>EXISTS</code> can stop at the first match	

Subquery or join?

Prefer a JOIN when...	Prefer a subquery when...
you need columns from both tables in the output	you only need to <i>filter</i> by the other table (<code>EXISTS</code> / <code>IN</code>)
the relationship is a straightforward key match	you need an aggregate first (compare with an average)
	it reads more naturally: "customers with no orders" → <code>NOT EXISTS</code>

CTEs: Common Table Expressions (WITH)

A **CTE** gives a name to a subquery and puts it **at the top** of the statement. The query then reads top to bottom like steps in a recipe.

```
WITH dept_stats AS (
    SELECT dept_id,
           COUNT(*) AS headcount,
           ROUND(AVG(salary), 2) AS avg_salary
    FROM employees
    WHERE dept_id IS NOT NULL
    GROUP BY dept_id
),
big_depts AS (
    SELECT * FROM dept_stats WHERE headcount >= 2
)
SELECT d.dept_name, b.headcount, b.avg_salary
FROM big_depts b
JOIN departments d ON d.dept_id = b.dept_id
ORDER BY b.avg_salary DESC;
```

SQL · PostgreSQL

OUTPUT

dept_name	headcount	avg_salary
Engineering	6	102000.00
Finance	3	83333.33
Marketing	2	75000.00
HR	2	58500.00

4 rows

LINE BY LINE

Part	What it does
WITH dept_stats AS (...)	step 1: a named result with headcount and average salary per department
big_depts AS (SELECT * FROM dept_stats ...)	step 2: a second CTE can use the first one
final SELECT ... FROM big_depts JOIN departments	step 3: the main query uses the CTEs like tables

Why CTEs are useful:

- **Readability:** complex logic becomes named steps instead of nested brackets.
- **Reuse:** a CTE can be referenced several times in the same query.
- **Debugging:** run each step on its own.
- **Recursion:** only CTEs can be recursive (below).

A CTE exists only for the duration of that one statement.

CTES AND PERFORMANCE IN POSTGRESQL

Since PostgreSQL 12, simple CTEs are inlined into the main query and optimized like subqueries. Writing `WITH x AS MATERIALIZED (...)` forces PostgreSQL to compute the CTE once and store it, which helps when an expensive CTE is referenced many times. Before version 12, every CTE was materialized, which sometimes made CTE queries slower than the subquery version.

Recursive CTEs

A **recursive CTE** refers to itself. It is the standard SQL tool for **hierarchies** (org charts, category trees, folder structures) and for generating sequences. It has two parts joined by **UNION ALL** :

1. the **anchor** member: the starting rows;
2. the **recursive** member: joins back to the CTE to find the next level, and repeats until it returns no new rows.

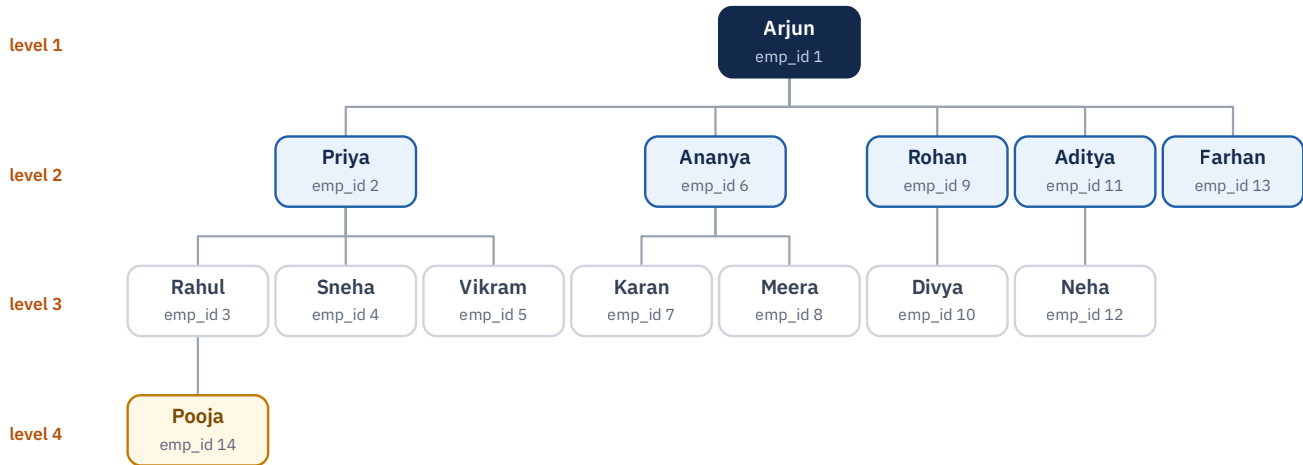


Figure 17. The reporting hierarchy stored in the employees table through manager_id.

```

WITH RECURSIVE org AS (
  -- anchor: the top of the hierarchy
  SELECT emp_id, emp_name, manager_id,
         1 AS level,
         emp_name::text AS path
  FROM employees
  WHERE manager_id IS NULL

  UNION ALL

  -- recursive step: people who report to someone already found
  SELECT e.emp_id, e.emp_name, e.manager_id,
         o.level + 1,
         o.path || ' > ' || e.emp_name
  FROM employees e
  JOIN org o ON e.manager_id = o.emp_id
)
SELECT level, emp_name, path
FROM org
ORDER BY path;

```

SQL · POSTGRES SQL

OUTPUT

level	emp_name	path
1	Arjun Mehta	Arjun Mehta
2	Aditya Singh	Arjun Mehta > Aditya Singh
3	Neha Kapoor	Arjun Mehta > Aditya Singh > Neha Kapoor
2	Ananya Das	Arjun Mehta > Ananya Das
3	Karan Malhotra	Arjun Mehta > Ananya Das > Karan Malhotra
3	Meera Nair	Arjun Mehta > Ananya Das > Meera Nair
2	Farhan Ali	Arjun Mehta > Farhan Ali
2	Priya Sharma	Arjun Mehta > Priya Sharma
3	Rahul Verma	Arjun Mehta > Priya Sharma > Rahul Verma
4	Pooja Reddy	Arjun Mehta > Priya Sharma > Rahul Verma > Pooja Reddy
3	Sneha Iyer	Arjun Mehta > Priya Sharma > Sneha Iyer
3	Vikram Rao	Arjun Mehta > Priya Sharma > Vikram Rao
2	Rohan Gupta	Arjun Mehta > Rohan Gupta
3	Divya Menon	Arjun Mehta > Rohan Gupta > Divya Menon

14 rows

LINE BY LINE

Part	What it does
<code>WITH RECURSIVE org AS (</code>	declares a CTE that may refer to itself
<code>anchor WHERE manager_id IS NULL</code>	starts from Arjun, the person with no manager (level 1)
<code>UNION ALL</code>	adds each new level's rows to the result
<code>JOIN org o ON e.manager_id = o.emp_id</code>	finds employees whose manager is already in <code>org</code> , one level deeper
<code>o.level + 1, o.path ' > ' e.emp_name</code>	carries the depth and the chain of names downward
stops when	a step finds no new employees (Pooja has no reports)

Recursion also generates sequences. Here are the first seven days of March 2024 (in PostgreSQL, `generate_series` does the same more simply):

```
WITH RECURSIVE days AS (
    SELECT DATE '2024-03-01' AS day
    UNION ALL
    SELECT (day + 1)::date FROM days WHERE day < DATE '2024-03-07'
)
SELECT day FROM days;
```

SQL · POSTGRESQL

OUTPUT

day
2024-03-01
2024-03-02
2024-03-03
2024-03-04
2024-03-05
2024-03-06
2024-03-07

7 rows

Always make sure the recursive part eventually returns no rows (a stopping condition). A cycle in the data, such as A manages B and B manages A, would otherwise loop. PostgreSQL 14+ even has a `CYCLE` clause to detect this.

CTE vs subquery vs view vs temporary table

	Subquery	CTE	View	Temporary table
Lives for	one query (inline)	one statement	permanently (until dropped)	the session
Named and reusable	no	within the statement	across queries and users	within the session
Stores data	no	no (unless materialized)	no (a materialized view does)	yes
Best for	small, one-off logic	readable multi-step queries, recursion	shared logic, security (expose only some columns)	big intermediate results reused by many queries

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"A subquery is a query nested inside another. It can return a single value for a comparison, a list for IN or EXISTS, or a whole derived table in the FROM clause. A correlated subquery refers to the outer row, like comparing each employee with their own department's average. A CTE, written with WITH, is a named subquery placed at the top, which makes multi-step logic much more readable and can be referenced more than once. Recursive CTEs handle hierarchies like an org chart. One trap I always avoid is NOT IN with a subquery that may contain NULLs; it returns no rows, so I use NOT EXISTS instead."

COMMON TRAPS & MISTAKES

- `NOT IN (subquery)` returns nothing if the subquery yields any NULL. Use `NOT EXISTS`.
- A scalar subquery that returns more than one row raises an error.
- Forgetting the alias on a derived table (`FROM (SELECT ...) AS t`): required by most databases and by PostgreSQL before version 16.
- Thinking a CTE is stored like a table; it lives only for one statement.
- A recursive CTE without a stopping condition loops until it errors.

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What is a subquery?

A A query nested inside another query (in WHERE, FROM, SELECT or HAVING) whose result is used by the outer query.

Q2 **DEFINITION** What is a correlated subquery?

A A subquery that references a column of the outer query, so it is conceptually evaluated once per outer row, e.g. comparing each employee's salary with their own department's average.

Q3 **DEFINITION** What is a CTE?

A A Common Table Expression: a named temporary result defined with WITH at the start of a statement and used by the main query.

INTERMEDIATE**Q4** **COMPARISON** **IN vs EXISTS?**

A IN checks membership in a list of values; EXISTS checks whether a correlated subquery returns any row. NOT EXISTS is safe with NULLs, while NOT IN returns nothing if the list contains NULL.

Q5 **COMPARISON** **Subquery vs CTE?**

A Logically similar. A CTE is named, defined up front, readable, reusable within the statement and can be recursive; a subquery is inline and anonymous.

Q6 **WHY** **Why might WHERE id NOT IN (SELECT parent_id FROM t) return no rows?**

A If parent_id contains a NULL, every NOT IN comparison becomes UNKNOWN. Filter the NULLs out or use NOT EXISTS.

Q7 **HOW** **How does a recursive CTE work?**

A An anchor query produces the starting rows; a recursive query joins back to the CTE to find the next level; results are combined with UNION ALL, repeating until no new rows appear.

Q8 **COMPARISON** **Correlated subquery vs window function for "above department average"?**

A Both are correct. The window function (AVG() OVER (PARTITION BY dept_id)) reads the data once and is usually faster and clearer; the correlated subquery can be costly on large tables unless the optimizer rewrites it.

SCENARIO-BASED**Q9** **SQL PROBLEM** **Find products priced above the average price of their own category.**

A `SELECT product_name, category, price FROM products p WHERE price > (SELECT AVG(price) FROM products x WHERE x.category = p.category);`

Q10 **SQL PROBLEM** **List departments that have no employees.**

A `SELECT dept_name FROM departments d WHERE NOT EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id);`

Q11 **DESIGN QUESTION** **How would you store and query a product category tree (Electronics > Computers > Laptops)?**

A Store categories(category_id, name, parent_id REFERENCES categories), an adjacency list, and query descendants or full paths with a recursive CTE.

FOLLOW-UP / TRAP**Q12** **TRAP QUESTION** **Does EXISTS (SELECT NULL FROM ...) work?**

A Yes. EXISTS only checks whether rows exist; what is selected is irrelevant.

Q13 **TRAP QUESTION** **Can a CTE be used in UPDATE or DELETE?**

A Yes, in PostgreSQL a WITH clause can precede INSERT, UPDATE and DELETE, and CTEs can even contain data-modifying statements.

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

P1. Find the product(s) with the highest price using a subquery.

P2. List customers who have placed at least one order using IN.

P3. What is wrong with `SELECT * FROM (SELECT emp_name FROM employees);` in PostgreSQL versions before 16?

LEVEL 2 — INTERVIEW APPLICATION

P4. Using a CTE, find the total revenue per customer and then show only customers whose revenue is above the average customer revenue.

P5. Find the employees who are managers (they appear as someone's manager_id) using EXISTS.

LEVEL 3 — SCENARIO / PROBLEM SOLVING

P6. Write a recursive CTE that lists everyone who reports, directly or indirectly, to Priya Sharma (emp 2), with their level below her.

P7. For each order, show the order id, its total and the customer's average order total, using a correlated subquery. Then rewrite it with a window function.

ANSWER / EXPLANATION

P1.

```
SELECT product_name, price
FROM products
WHERE price = (SELECT MAX(price) FROM products);
```

SQL - POSTGRESQL

OUTPUT

product_name	price
Standing Desk	15999.00

1 row

P2.

```
SELECT customer_name
FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders)
ORDER BY customer_name;
```

SQL - POSTGRESQL

OUTPUT

customer_name
Aarav Patel
Diya Joshi
Isha Kulkarni
Kabir Khan
Vivaan Shah

5 rows

P3. The derived table has no alias. Before PostgreSQL 16 (and in most other databases), a subquery in FROM must be named: FROM (SELECT ...) AS t. Version 16 relaxed this rule, but writing the alias is still the portable habit.

P4.

```
WITH customer_revenue AS (  
    SELECT customer_id, SUM(total_amount) AS revenue  
    FROM orders  
    WHERE status <> 'CANCELLED'  
    GROUP BY customer_id  
)  
SELECT c.customer_name, r.revenue  
FROM customer_revenue r  
JOIN customers c ON c.customer_id = r.customer_id  
WHERE r.revenue > (SELECT AVG(revenue) FROM customer_revenue)  
ORDER BY r.revenue DESC;
```

SQL · POSTGRESQL

OUTPUT

customer_name	revenue
Aarav Patel	42338.00
Isha Kulkarni	23983.00

2 rows

The CTE is used twice: once in the main query and once inside the scalar subquery.

P5.

```
SELECT m.emp_id, m.emp_name  
FROM employees m  
WHERE EXISTS (SELECT 1 FROM employees e WHERE e.manager_id = m.emp_id)  
ORDER BY m.emp_id;
```

SQL · POSTGRESQL

OUTPUT

emp_id	emp_name
1	Arjun Mehta
2	Priya Sharma
3	Rahul Verma
6	Ananya Das
9	Rohan Gupta
11	Aditya Singh

6 rows

P6.

```

WITH RECURSIVE team AS (
    SELECT emp_id, emp_name, 1 AS level
    FROM employees
    WHERE manager_id = 2          -- Priya's direct reports
    UNION ALL
    SELECT e.emp_id, e.emp_name, t.level + 1
    FROM employees e
    JOIN team t ON e.manager_id = t.emp_id
)
SELECT level, emp_name
FROM team
ORDER BY level, emp_name;

```

SQL · PostgreSQL

OUTPUT

level	emp_name
1	Rahul Verma
1	Sneha Iyer
1	Vikram Rao
2	Pooja Reddy

4 rows

P7. Correlated subquery version:

```

SELECT o.order_id,
       o.customer_id,
       o.total_amount,
       (SELECT ROUND(AVG(x.total_amount), 2)
        FROM orders x
        WHERE x.customer_id = o.customer_id) AS customer_avg
FROM orders o
ORDER BY o.customer_id, o.order_id
LIMIT 6;

```

SQL · PostgreSQL

OUTPUT

order_id	customer_id	total_amount	customer_avg
101	1	2843.00	10584.50
103	1	8999.00	10584.50
109	1	24998.00	10584.50
115	1	5498.00	10584.50
102	2	3499.00	7994.33
106	2	2486.00	7994.33

6 rows

Window function version, which reads the table once:

```

SELECT order_id, customer_id, total_amount,
       ROUND(AVG(total_amount) OVER (PARTITION BY customer_id), 2) AS customer_avg
FROM orders
ORDER BY customer_id, order_id;

```

SQL · PostgreSQL

Session 3.8 — Window Functions & Indexing (A): Window Functions

HIGH PRIORITY Window functions separate strong SQL candidates from average ones. They solve ranking, "top N per group", running totals and "compare with the previous row" problems cleanly.

Simple meaning

A window function calculates a value across a set of rows related to the current row, without collapsing those rows. GROUP BY squeezes each group into one row; a window function keeps every row and *adds* a column computed over its "window".

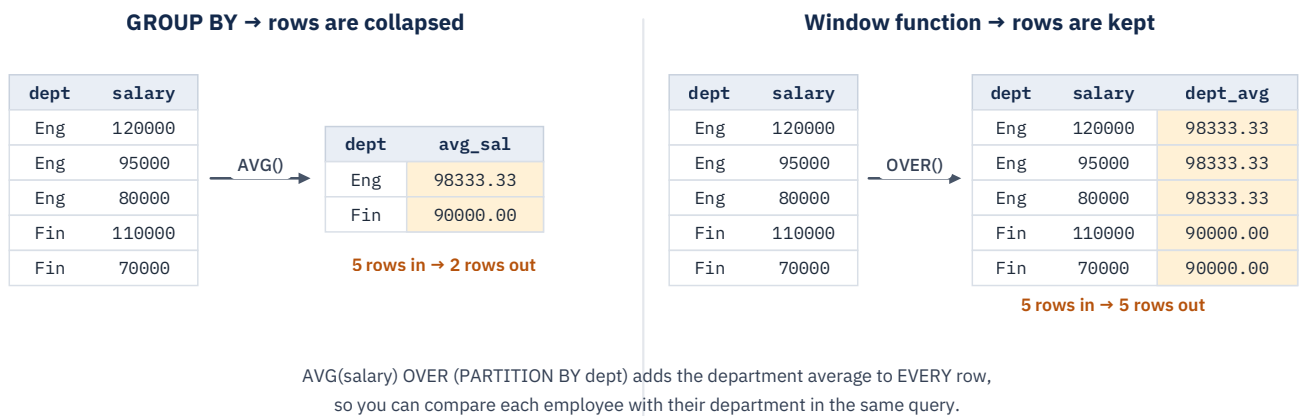


Figure 18. GROUP BY returns one row per group; a window function keeps every row and adds the group's value to each one.

The syntax

```
function_name(...) OVER (
  PARTITION BY column    -- optional: restart the calculation for each group
  ORDER BY column        -- optional: order rows inside each partition
  ROWS BETWEEN ...       -- optional: the exact frame of rows to use
)
```

SQL · POSTGRESQL

- **OVER ()** with nothing inside: the window is the **whole result set**.
- **PARTITION BY**: splits rows into groups, like GROUP BY, but **without collapsing** them.
- **ORDER BY** inside OVER: gives an order for ranking, LAG/LEAD and running totals.
- **Frame** (**ROWS BETWEEN ...**): which rows around the current row are included (moving averages).

OVER (): compare each row with the whole table

SQL · POSTGRESQL

```
SELECT emp_name,
       salary,
       ROUND(AVG(salary) OVER (), 2) AS company_avg,
       ROUND(100.0 * salary / SUM(salary) OVER (), 2) AS pct_of_payroll
FROM employees
ORDER BY salary DESC
LIMIT 5;
```

OUTPUT

emp_name	salary	company_avg	pct_of_payroll
Arjun Mehta	150000.00	84571.43	12.67
Priya Sharma	120000.00	84571.43	10.14
Ananya Das	110000.00	84571.43	9.29
Sneha Iyer	95000.00	84571.43	8.02
Rahul Verma	95000.00	84571.43	8.02

5 rows

Every row keeps its own data and also "sees" the totals. With GROUP BY, the employee rows would have disappeared.

PARTITION BY: per-group values on every row

SQL · POSTGRESQL

```
SELECT emp_name,
       dept_id,
       salary,
       ROUND(AVG(salary) OVER (PARTITION BY dept_id), 2) AS dept_avg,
       COUNT(*) OVER (PARTITION BY dept_id) AS dept_size
FROM employees
WHERE dept_id IN (2, 3, 4)
ORDER BY dept_id, salary DESC;
```

OUTPUT

emp_name	dept_id	salary	dept_avg	dept_size
Ananya Das	2	110000.00	83333.33	3
Karan Malhotra	2	70000.00	83333.33	3
Meera Nair	2	70000.00	83333.33	3
Rohan Gupta	3	65000.00	58500.00	2
Divya Menon	3	52000.00	58500.00	2
Aditya Singh	4	90000.00	75000.00	2
Neha Kapoor	4	60000.00	75000.00	2

7 rows

LINE BY LINE

Part	What it does
AVG(salary) OVER (PARTITION BY dept_id)	for each row, the average salary of that row's department
COUNT(*) OVER (PARTITION BY dept_id)	how many employees are in that row's department
WHERE dept_id IN (2, 3, 4)	runs before the window functions, so windows only see these rows

Ranking: ROW_NUMBER, RANK and DENSE_RANK

The three ranking functions differ **only in how they treat ties**. In Engineering, Rahul and Sneha both earn 95000:

```
SELECT emp_name,
       salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC, emp_id) AS row_number,
       RANK() OVER (ORDER BY salary DESC) AS rank,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank
FROM employees
WHERE dept_id = 1
ORDER BY salary DESC, emp_id;
```

SQL · POSTGRESQL

OUTPUT

emp_name	salary	row_number	rank	dense_rank
Arjun Mehta	150000.00	1	1	1
Priya Sharma	120000.00	2	2	2
Rahul Verma	95000.00	3	3	3
Sneha Iyer	95000.00	4	3	3
Vikram Rao	80000.00	5	5	4
Pooja Reddy	72000.00	6	6	5

6 rows

Function	Ties get...	After a tie...	Sequence here
ROW_NUMBER()	different numbers (arbitrary unless you add a tie-breaker)	continues normally	1, 2, 3, 4, 5, 6
RANK()	the same number	skips numbers (gaps)	1, 2, 3, 3, 5 , 6
DENSE_RANK()	the same number	no gaps	1, 2, 3, 3, 4 , 5

Which one to use:

- "Exactly one row per group" (latest order per customer, de-duplication) → ROW_NUMBER .
- "Nth highest *value*" (third-highest salary, counting ties once) → DENSE_RANK .
- "Competition ranking", like sports, where two people tied for 3rd place mean nobody is 4th → RANK .

The `emp_id` in ROW_NUMBER's ORDER BY is a **tie-breaker**: without it, which of Rahul and Sneha gets 3 and which gets 4 is not guaranteed.

Ranking within groups: PARTITION BY + ORDER BY

```
SELECT dept_id,  
       emp_name,  
       salary,  
       DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rank_in_dept  
FROM employees  
WHERE dept_id IS NOT NULL  
ORDER BY dept_id, rank_in_dept;
```

SQL · POSTGRESQL

OUTPUT

dept_id	emp_name	salary	rank_in_dept
1	Arjun Mehta	150000.00	1
1	Priya Sharma	120000.00	2
1	Rahul Verma	95000.00	3
1	Sneha Iyer	95000.00	3
1	Vikram Rao	80000.00	4
1	Pooja Reddy	72000.00	5
2	Ananya Das	110000.00	1
2	Meera Nair	70000.00	2
2	Karan Malhotra	70000.00	2
3	Rohan Gupta	65000.00	1
3	Divya Menon	52000.00	2
4	Aditya Singh	90000.00	1
4	Neha Kapoor	60000.00	2

13 rows

The ranking restarts at 1 for every department. This is the heart of "top N per group".

Filtering on a window result

Window functions are computed **after** WHERE, GROUP BY and HAVING, so they cannot appear in WHERE:

```
SELECT emp_name, salary  
FROM employees  
WHERE DENSE_RANK() OVER (ORDER BY salary DESC) <= 3;
```

SQL · POSTGRESQL

POSTGRESQL SAYS

ERROR window functions are not allowed in WHERE

Compute the window in a subquery or CTE first, then filter in the outer query. Here are the top 2 earners per department:

```
WITH ranked AS (  
    SELECT dept_id, emp_name, salary,  
           DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk  
    FROM employees  
    WHERE dept_id IS NOT NULL  
)  
SELECT dept_id, emp_name, salary, rnk  
FROM ranked  
WHERE rnk <= 2  
ORDER BY dept_id, rnk;
```

SQL · POSTGRESQL

OUTPUT

dept_id	emp_name	salary	rnk
1	Arjun Mehta	150000.00	1
1	Priya Sharma	120000.00	2
2	Ananya Das	110000.00	1
2	Meera Nair	70000.00	2
2	Karan Malhotra	70000.00	2
3	Rohan Gupta	65000.00	1
3	Divya Menon	52000.00	2
4	Aditya Singh	90000.00	1
4	Neha Kapoor	60000.00	2

9 rows

QUALIFY

Snowflake, BigQuery and Databricks support `QUALIFY`, a WHERE clause for window results: `... QUALIFY DENSE_RANK() OVER (...) <= 2`. PostgreSQL does not, so use a CTE or subquery.

LAG and LEAD: the previous and next row

`LAG(col, n, default)` reads the value from **n rows before** the current row (default `n = 1`); `LEAD` reads **n rows after**. They need an `ORDER BY` inside `OVER` to define "before" and "after".

```
SELECT sale_date,
       amount,
       LAG(amount) OVER (ORDER BY sale_date) AS prev_day,
       amount - LAG(amount) OVER (ORDER BY sale_date) AS change,
       LEAD(amount) OVER (ORDER BY sale_date) AS next_day
FROM daily_sales
WHERE region = 'North'
ORDER BY sale_date;
```

SQL · PostgreSQL

OUTPUT

sale_date	amount	prev_day	change	next_day
2024-03-01	12000.00	NULL	NULL	15000.00
2024-03-02	15000.00	12000.00	3000.00	11000.00
2024-03-03	11000.00	15000.00	-4000.00	18000.00
2024-03-04	18000.00	11000.00	7000.00	21000.00
2024-03-05	21000.00	18000.00	3000.00	17000.00
2024-03-06	17000.00	21000.00	-4000.00	23000.00
2024-03-07	23000.00	17000.00	6000.00	NULL

7 rows

LINE BY LINE

Part	What it does
<code>LAG(amount) OVER (ORDER BY sale_date)</code>	the amount from the previous date; NULL for the first row
<code>amount - LAG(amount) ...</code>	day-over-day change
<code>LEAD(amount) OVER (ORDER BY sale_date)</code>	the next date's amount; NULL for the last row

Add `PARTITION BY region` to compare within each region separately, and use the third argument for a default:

`LAG(amount, 1, 0)` returns 0 instead of NULL on the first row.

Running totals and moving averages

With `ORDER BY` inside `OVER`, aggregate functions become **cumulative**:

```
SELECT region,
       sale_date,
       amount,
       SUM(amount) OVER (PARTITION BY region ORDER BY sale_date) AS running_total,
       ROUND(AVG(amount) OVER (PARTITION BY region ORDER BY sale_date
                               ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS moving_avg_3d
FROM daily_sales
ORDER BY region, sale_date;
```

SQL · POSTGRESQL

OUTPUT

region	sale_date	amount	running_total	moving_avg_3d
North	2024-03-01	12000.00	12000.00	12000.00
North	2024-03-02	15000.00	27000.00	13500.00
North	2024-03-03	11000.00	38000.00	12666.67
North	2024-03-04	18000.00	56000.00	14666.67
North	2024-03-05	21000.00	77000.00	16666.67
North	2024-03-06	17000.00	94000.00	18666.67
North	2024-03-07	23000.00	117000.00	20333.33
South	2024-03-01	9000.00	9000.00	9000.00
South	2024-03-02	9500.00	18500.00	9250.00
South	2024-03-03	14000.00	32500.00	10833.33
South	2024-03-04	8000.00	40500.00	10500.00
South	2024-03-05	16000.00	56500.00	12666.67
South	2024-03-06	15500.00	72000.00	13166.67
South	2024-03-07	19000.00	91000.00	16833.33

14 rows

Running total

UNBOUNDED PRECEDING
→ CURRENT ROW



3-row moving avg

2 PRECEDING
→ CURRENT ROW



LAG / LEAD

previous / next row

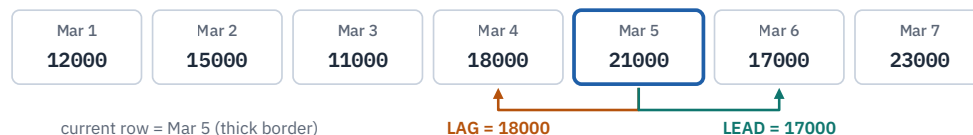


Figure 19. Window frames for the North region on 5 March: running total, 3-row moving average, and LAG/LEAD.

Frame clause	Rows included
(default with ORDER BY) RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW	from the first row of the partition up to the current row, including rows tied with it
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW	same, but counts physical rows (ties not grouped)
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW	current row + 2 before it (a 3-row moving window)
ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING	previous, current and next row (centred window)
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING	the whole partition

RANGE VS ROWS WHEN THE ORDER HAS TIES

With the default RANGE frame, rows that tie on the ORDER BY value are added to the running total **together**, so two orders on the same date both show the total *including each other*. If you want a strictly row-by-row running total, write ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW and add a tie-breaker to the ORDER BY.

Other useful window functions INTERVIEW EXTENSION

Function	Returns
NTILE(4)	splits ordered rows into 4 nearly equal buckets (quartiles)
FIRST_VALUE(col)	first value in the window (e.g. the top salary in the department)
LAST_VALUE(col)	last value in the frame. With the default frame this is the current row, a common surprise; use ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
NTH_VALUE(col, n)	the n-th value in the frame
PERCENT_RANK(), CUME_DIST()	relative rank as a fraction between 0 and 1

SQL · POSTGRES SQL

```
SELECT emp_name,
       salary,
       NTILE(4) OVER (ORDER BY salary DESC) AS salary_quartile,
       FIRST_VALUE(emp_name) OVER (PARTITION BY dept_id ORDER BY salary DESC) AS
top_earner_in_dept
FROM employees
WHERE dept_id IN (1, 2)
ORDER BY salary DESC;
```

OUTPUT

emp_name	salary	salary_quartile	top_earner_in_dept
Arjun Mehta	150000.00	1	Arjun Mehta
Priya Sharma	120000.00	1	Arjun Mehta
Ananya Das	110000.00	1	Ananya Das
Rahul Verma	95000.00	2	Arjun Mehta
Sneha Iyer	95000.00	2	Arjun Mehta
Vikram Rao	80000.00	3	Arjun Mehta
Pooja Reddy	72000.00	3	Arjun Mehta
Karan Malhotra	70000.00	4	Ananya Das
Meera Nair	70000.00	4	Ananya Das

9 rows

Window function vs GROUP BY

	GROUP BY	Window function
Output rows	one per group	one per input row
Detail columns	lost (unless grouped)	kept
Typical use	totals, counts per group	ranking, running totals, comparing a row with its group or neighbours
Can filter on the result	with HAVING	in an outer query (or QUALIFY in some warehouses)
Runs	before SELECT	during SELECT, after WHERE / GROUP BY / HAVING

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"Window functions compute values over a set of related rows without collapsing them, unlike GROUP BY. I use OVER with PARTITION BY to define the groups and ORDER BY to define the order inside each group. For ranking there are three options that differ only on ties: ROW_NUMBER gives every row a unique number, RANK gives ties the same rank and then skips numbers, and DENSE_RANK gives ties the same rank without gaps. So for the Nth highest salary I use DENSE_RANK, and for latest record per customer I use ROW_NUMBER. LAG and LEAD read the previous or next row, which is great for day-over-day changes, and SUM with ORDER BY inside OVER gives running totals. Since windows are calculated after WHERE, I filter on them in an outer query or CTE."

COMMON TRAPS & MISTAKES

- Using a window function in WHERE: not allowed; wrap the query in a CTE or subquery.
- Using ROW_NUMBER when ties should share a rank (Nth highest *salary*): use DENSE_RANK.
- ROW_NUMBER without a deterministic ORDER BY: tied rows get arbitrary numbers.
- Forgetting ORDER BY inside OVER for LAG/LEAD or running totals.
- Expecting LAST_VALUE to return the partition's last row with the default frame.
- Forgetting that WHERE runs first: filtering rows changes what the window sees.

INTERVIEW QUESTIONS

BASIC

Q1 DEFINITION What is a window function?

A A function that computes a value over a set of rows related to the current row, defined with OVER(), while still returning every row.

Q2 COMPARISON ROW_NUMBER vs RANK vs DENSE_RANK?

A ROW_NUMBER numbers rows uniquely (1, 2, 3, 4). RANK gives ties the same rank and leaves gaps (1, 2, 2, 4). DENSE_RANK gives ties the same rank without gaps (1, 2, 2, 3).

Q3 DEFINITION What does PARTITION BY do?

A It divides rows into groups for the window calculation; the function restarts for each partition, but no rows are collapsed.

Q4 DEFINITION What do LAG and LEAD do?

A LAG returns a value from a previous row and LEAD from a following row, in the order given inside OVER.

INTERMEDIATE**Q5** **COMPARISON** Window function vs GROUP BY?

A GROUP BY returns one row per group and loses detail; window functions keep every row and add group-level or ordered calculations alongside.

Q6 **SQL PROBLEM** How do you compute a running total of sales by date?

A `SUM(amount) OVER (ORDER BY sale_date)` (add `PARTITION BY region` for one running total per region).

Q7 **WHY** Why can't you filter on ROW_NUMBER() in the WHERE clause?

A Window functions are evaluated after WHERE (and GROUP BY/HAVING). Compute them in a subquery or CTE, then filter in the outer query.

Q8 **DEFINITION** What is a window frame?

A The subset of rows within the partition used for the current row's calculation, e.g. `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` for a 3-row moving average.

Q9 **HOW** How would you calculate each employee's salary as a percentage of their department's total?

A `100.0 * salary / SUM(salary) OVER (PARTITION BY dept_id)`.

SCENARIO-BASED**Q10** **SQL PROBLEM** Find the second-highest salary in each department.

A Use `DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC)` in a CTE and keep rows where the rank = 2.

Q11 **SQL PROBLEM** Show each day's sales and the percentage change from the previous day.

A `ROUND(100.0 * (amount - LAG(amount) OVER (ORDER BY sale_date)) / LAG(amount) OVER (ORDER BY sale_date), 2)`. Optionally use `NULLIF` on the denominator to avoid division by zero.

Q12 **SCENARIO** A "latest order per customer" report sometimes shows two orders for the same customer. Why, and how do you fix it?

A Probably `RANK/DENSE_RANK` on `order_date`, so ties on the same date share rank 1. Use `ROW_NUMBER` with a tie-breaker (`ORDER BY order_date DESC, order_id DESC`).

FOLLOW-UP / TRAP**Q13** **TRAP QUESTION** With the default frame, what does a running SUM do for rows with the same ORDER BY value?

A The default frame is `RANGE`, so peer rows (ties) are included together and get the same running total. Use `ROWS` for row-by-row accumulation.

Q14 **TRAP QUESTION** Can window functions be used with GROUP BY in the same query?

A Yes. The window is applied to the grouped result, e.g. `RANK() OVER (ORDER BY SUM(sales) DESC)` ranks groups by their totals.

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

P1. Number all products from cheapest to most expensive with `ROW_NUMBER`.

P2. What will `DENSE_RANK` return for the salaries 100, 90, 90, 80 (highest first)?

P3. Show each order with the previous order's date for the same customer.

LEVEL 2 — INTERVIEW APPLICATION

P4. For each region in `daily_sales`, show each day's amount and the region's total for the week on every row.

P5. Rank customers by total revenue (excluding cancelled orders) using RANK.

LEVEL 3 — SCENARIO / PROBLEM SOLVING

P6. Find, for each customer, their most expensive order, using a window function (one row per customer).

P7. Find days where North's sales were higher than on both the previous and the next day (a "local peak").

ANSWER / EXPLANATION

P1.

```
SELECT ROW_NUMBER() OVER (ORDER BY price, product_id) AS rn,
       product_name, price
FROM products;
```

SQL · POSTGRES SQL

OUTPUT

rn	product_name	price
1	Gel Pen Set	149.00
2	Notebook Pack	249.00
3	Wireless Mouse	799.00
4	USB-C Hub	1999.00
5	Monitor Arm	2499.00
6	Mechanical Keyboard	3499.00
7	Office Chair	8999.00
8	Standing Desk	15999.00

8 rows

P2. 1, 2, 2, 3 (RANK would give 1, 2, 2, 4).

P3.

```
SELECT customer_id, order_id, order_date,
       LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date, order_id) AS
prev_order_date
FROM orders
ORDER BY customer_id, order_date
LIMIT 7;
```

SQL · POSTGRES SQL

OUTPUT

customer_id	order_id	order_date	prev_order_date
1	101	2024-01-05	NULL
1	103	2024-01-28	2024-01-05
1	109	2024-03-10	2024-01-28
1	115	2024-04-19	2024-03-10
2	102	2024-01-12	NULL
2	106	2024-02-20	2024-01-12
2	113	2024-04-04	2024-02-20

7 rows

P4.

SQL · PostgreSQL

```
SELECT region, sale_date, amount,  
       SUM(amount) OVER (PARTITION BY region) AS region_week_total  
FROM daily_sales  
ORDER BY region, sale_date  
LIMIT 8;
```

OUTPUT

region	sale_date	amount	region_week_total
North	2024-03-01	12000.00	117000.00
North	2024-03-02	15000.00	117000.00
North	2024-03-03	11000.00	117000.00
North	2024-03-04	18000.00	117000.00
North	2024-03-05	21000.00	117000.00
North	2024-03-06	17000.00	117000.00
North	2024-03-07	23000.00	117000.00
South	2024-03-01	9000.00	91000.00

8 rows

Without ORDER BY inside OVER, the window is the whole partition, so every row shows the full total.

P5.

SQL · PostgreSQL

```
SELECT c.customer_name,  
       SUM(o.total_amount) AS revenue,  
       RANK() OVER (ORDER BY SUM(o.total_amount) DESC) AS revenue_rank  
FROM orders o  
JOIN customers c ON c.customer_id = o.customer_id  
WHERE o.status <> 'CANCELLED'  
GROUP BY c.customer_name  
ORDER BY revenue_rank;
```

OUTPUT

customer_name	revenue	revenue_rank
Aarav Patel	42338.00	1
Isha Kulkarni	23983.00	2
Kabir Khan	18396.00	3
Vivaan Shah	10233.00	4
Diya Joshi	6796.00	5

5 rows

The window runs after GROUP BY, so it can rank by SUM(...).

P6.

SQL · PostgreSQL

```
WITH ranked AS (  
    SELECT customer_id, order_id, total_amount,  
           ROW_NUMBER() OVER (PARTITION BY customer_id  
                               ORDER BY total_amount DESC, order_id) AS rn  
    FROM orders  
)  
SELECT customer_id, order_id, total_amount  
FROM ranked  
WHERE rn = 1  
ORDER BY customer_id;
```

OUTPUT

customer_id	order_id	total_amount
1	109	24998.00
2	113	17998.00
3	104	15999.00
4	111	3998.00
5	108	6998.00

5 rows

P7.

SQL · PostgreSQL

```
WITH n AS (  
    SELECT sale_date, amount,  
           LAG(amount) OVER (ORDER BY sale_date) AS prev_amt,  
           LEAD(amount) OVER (ORDER BY sale_date) AS next_amt  
    FROM daily_sales  
    WHERE region = 'North'  
)  
SELECT sale_date, amount  
FROM n  
WHERE amount > prev_amt AND amount > next_amt;
```

OUTPUT

sale_date	amount
2024-03-02	15000.00
2024-03-05	21000.00

2 rows

The first and last days are excluded automatically: their `prev_amt` or `next_amt` is NULL, and a comparison with NULL is not TRUE.

Session 3.9 — Window Functions & Indexing (B): Indexing

HIGH PRIORITY "What is an index? Why not index every column?" is a top-10 DBMS question.

Simple meaning

An index is a separate, sorted data structure that helps the database find rows quickly without reading the whole table.

THE INDEX AT THE BACK OF A BOOK

To find "normalization" in a 600-page book you don't read every page; you check the index, which lists the topic alphabetically with page numbers, and jump straight there. A database index does the same: it stores column values in sorted order with pointers to the rows. And like a book index, it takes extra pages and must be updated whenever the content changes.

Why indexes exist

Without an index, a query like `WHERE customer_id = 4242` forces a **full table (sequential) scan**: read every row and test it. That is fine for 14 rows and painful for 200 million. An index turns it into a few page reads.

Indexes speed up:

- `WHERE` filters on selective columns (`email = ...`, `customer_id = ...`);
- `JOIN`s on foreign keys;
- `ORDER BY ... LIMIT` (rows are already sorted in the index);
- uniqueness checks: PRIMARY KEY and UNIQUE constraints automatically create an index.

How a B-tree index works

The default index type in PostgreSQL (and most databases) is the **B-tree** (balanced tree). Values are kept sorted in a shallow tree of pages. Searching starts at the root and moves down one level at a time. Even a billion values need only around 4–5 levels, so the search cost grows **logarithmically**, not linearly. The leaf pages are linked to each other, which makes range scans (`BETWEEN`, `>`, `ORDER BY`) efficient.

Index on employees(salary)

find salary = 95000

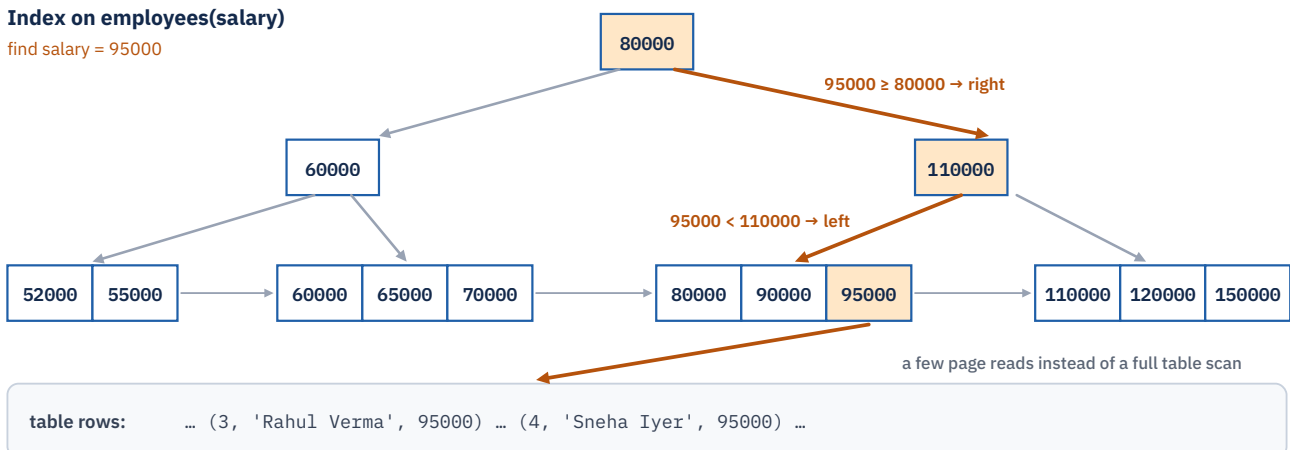


Figure 20. Searching a B-tree index: three short hops lead to the matching rows instead of scanning the whole table.

Seeing it for real: a 200,000-row table

The block below creates a larger table with deterministic data. Each customer has exactly 4 orders, and the status values are deliberately skewed: 70% DELIVERED and 10% for each other status.

```
CREATE TABLE orders_big AS
SELECT g
      ((g * 7919) % 50000 + 1) AS order_id,
      DATE '2023-01-01' + (g % 730) AS customer_id,
      ROUND(((g * 37) % 50000) / 100.0 + 100, 2) AS order_date,
      ROUND(((g * 37) % 50000) / 100.0 + 100, 2) AS amount,
      CASE WHEN g % 10 < 7 THEN 'DELIVERED'
            WHEN g % 10 = 7 THEN 'PENDING'
            WHEN g % 10 = 8 THEN 'SHIPPED'
            ELSE 'CANCELLED' END AS status
FROM generate_series(1, 200000) AS g;

ALTER TABLE orders_big ADD PRIMARY KEY (order_id);
ANALYZE orders_big;
```

Before indexing, look up one customer's orders. `EXPLAIN ANALYZE` shows how PostgreSQL actually executed the query (parallelism is switched off so the plan stays simple):

```
SET LOCAL max_parallel_workers_per_gather = 0;
EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF, SUMMARY OFF)
SELECT * FROM orders_big WHERE customer_id = 4242;
```

QUERY PLAN (EXPLAIN)

```
Seq Scan on orders_big (actual rows=4 loops=1)
  Filter: (customer_id = 4242)
  Rows Removed by Filter: 199996
```

Seq Scan ... Rows Removed by Filter: 199996 : PostgreSQL read all 200,000 rows to find 4.

Now create an index and run the same query:

```
CREATE INDEX idx_orders_big_customer ON orders_big (customer_id);
ANALYZE orders_big;
```

```
SET LOCAL max_parallel_workers_per_gather = 0;
EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF, SUMMARY OFF)
SELECT * FROM orders_big WHERE customer_id = 4242;
```

QUERY PLAN (EXPLAIN)

```
Bitmap Heap Scan on orders_big (actual rows=4 loops=1)
  Recheck Cond: (customer_id = 4242)
  Heap Blocks: exact=4
  -> Bitmap Index Scan on idx_orders_big_customer (actual rows=4 loops=1)
       Index Cond: (customer_id = 4242)
```

Bitmap Index Scan on `idx_orders_big_customer` found the 4 matching rows directly, and only 4 table pages were visited (Heap Blocks: exact=4) instead of the whole table.

LINE BY LINE

Plan line	Meaning
Seq Scan on orders_big	read the table from start to end
Rows Removed by Filter: 199996	rows read and thrown away; the cost of having no index
Bitmap Index Scan on idx_orders_big_customer	use the index to collect the locations of matching rows
Bitmap Heap Scan ... Heap Blocks: exact=4	then fetch just those table pages
Index Cond / Recheck Cond	the condition applied through the index

Why indexes slow down writes

An index is a second copy of some data, kept sorted. Every `INSERT`, every `DELETE` and every `UPDATE` of an indexed column must also update **every index** on the table. A table with ten indexes does roughly ten extra pieces of work per insert. Indexes also use disk and memory.

	Without an index	With an index
Lookup of a few rows in a big table	slow: full scan	fast: a few page reads
Range queries and <code>ORDER BY</code> on the column	sort or scan everything	read in order from the index
<code>INSERT</code> / <code>UPDATE</code> / <code>DELETE</code>	faster	slower: index maintenance
Storage	less	more (each index is extra pages)
Small tables	fine	little or no benefit

Selectivity: when the database ignores your index

An index helps when the condition picks a **small fraction** of rows. If a value matches a large share of the table, reading the whole table sequentially is cheaper than jumping between index and table thousands of times. The optimizer decides using statistics:

```
CREATE INDEX idx_orders_big_status ON orders_big (status);
ANALYZE orders_big;
```

SQL · POSTGRESQL

```
SET LOCAL max_parallel_workers_per_gather = 0;
EXPLAIN (COSTS OFF)
SELECT * FROM orders_big WHERE status = 'DELIVERED'; -- 70% of rows
```

SQL · POSTGRESQL

```
QUERY PLAN (EXPLAIN)
Seq Scan on orders_big
Filter: (status = 'DELIVERED'::text)
```

```
SET LOCAL max_parallel_workers_per_gather = 0;
EXPLAIN (COSTS OFF)
SELECT * FROM orders_big WHERE status = 'CANCELLED'; -- 10% of rows
```

SQL · POSTGRESQL

```
QUERY PLAN (EXPLAIN)
Bitmap Heap Scan on orders_big
Recheck Cond: (status = 'CANCELLED'::text)
-> Bitmap Index Scan on idx_orders_big_status
Index Cond: (status = 'CANCELLED'::text)
```

Same index, two different plans: a sequential scan for the common value, the index for the rarer one. That is why indexing a low-variety column (status, gender, a true/false flag) on its own often brings little benefit.

Composite indexes and the leftmost-prefix rule

A **composite (multi-column) index** sorts by the first column, then by the second within each first value, like a phone book sorted by surname, then first name.

```
DROP INDEX idx_orders_big_customer;      -- replaced by the composite index below
CREATE INDEX idx_orders_big_cust_date ON orders_big (customer_id, order_date);
ANALYZE orders_big;
```

SQL · POSTGRES SQL

```
SET LOCAL max_parallel_workers_per_gather = 0;
EXPLAIN (COSTS OFF)
SELECT * FROM orders_big
WHERE customer_id = 4242 AND order_date >= '2024-01-01';
```

SQL · POSTGRES SQL

QUERY PLAN (EXPLAIN)

```
Bitmap Heap Scan on orders_big
  Recheck Cond: ((customer_id = 4242) AND (order_date >= '2024-01-01'::date))
  -> Bitmap Index Scan on idx_orders_big_cust_date
      Index Cond: ((customer_id = 4242) AND (order_date >= '2024-01-01'::date))
```

```
SET LOCAL max_parallel_workers_per_gather = 0;
EXPLAIN (COSTS OFF)
SELECT * FROM orders_big WHERE order_date = '2024-01-01';
```

SQL · POSTGRES SQL

QUERY PLAN (EXPLAIN)

```
Seq Scan on orders_big
  Filter: (order_date = '2024-01-01'::date)
```

The composite index serves queries on `customer_id`, or on `customer_id and order_date`. It cannot efficiently serve a query on **only** `order_date`, the second column, just as a phone book sorted by surname cannot quickly find everyone named "Priya". This is the **leftmost-prefix rule**:

Index on (a, b, c) helps with...	Does not help (efficiently) with...
WHERE a = ?	WHERE b = ?
WHERE a = ? AND b = ?	WHERE c = ?
WHERE a = ? AND b = ? AND c = ?	WHERE b = ? AND c = ?
WHERE a = ? ORDER BY b	

Column order matters: put the column used in equality filters (and the most frequently filtered one) first, and range or sort columns after it.

Clustered vs non-clustered indexes

	Clustered index	Non-clustered (secondary) index
What it is	the table's rows are physically stored in the index order ; the index is the table	a separate structure holding keys plus pointers to the rows
How many per table	one (rows can be sorted only one way)	many
Lookup	finds the row directly	finds the pointer, then fetches the row
Examples	SQL Server clustered index; MySQL InnoDB's primary key	most other indexes
Analogy	a dictionary: the content itself is in order	the index at the back of a textbook

PostgreSQL specifics: PostgreSQL stores tables as an unordered **heap**, so *all* its indexes are non-clustered (secondary), including the primary key. The `CLUSTER` command physically reorders a table by an index **once**, but the order is not maintained for new rows. Saying this in an interview shows real depth.

PostgreSQL index types INTERVIEW EXTENSION

Type	Good for
B-tree (default)	equality and range on sortable data: numbers, text, dates
Hash	equality only
GIN	"contains" queries: JSONB keys, arrays, full-text search
GiST / SP-GiST	geometric, location and range data; nearest-neighbour searches
BRIN	very large tables whose values follow physical order (time-series logs); tiny index size

Also useful: **unique indexes**, **partial indexes** (`CREATE INDEX ... WHERE status = 'PENDING'` indexes only the rows you query), **expression indexes** (`CREATE INDEX ON customers (LOWER(email))`), and **covering indexes** (`INCLUDE (amount)`) that let PostgreSQL answer from the index alone ("index-only scan").

When indexes help and when they hurt

Indexes help when...	Indexes hurt or are wasted when...
the column is used often in WHERE, JOIN or ORDER BY	the table is small (a scan is already cheap)
the filter is selective (few matching rows)	the column has few distinct values and is queried on common values
the column is a foreign key used in joins	the table is write-heavy and rarely read
queries need sorted output with LIMIT	the query wraps the column in a function (<code>WHERE UPPER(email) = ...</code>) without an expression index
you need to enforce uniqueness	<code>LIKE '%text'</code> with a leading wildcard (B-tree can't help)
	there are too many overlapping indexes (slower writes, more storage)

Practical checklist: index foreign keys that you join on; check slow queries with `EXPLAIN ANALYZE` ; prefer one well-designed composite index over several single-column ones; remove unused indexes (`pg_stat_user_indexes` shows usage counts).

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"An index is a separate data structure, usually a B-tree, that keeps a column's values sorted with pointers to the rows, so the database can find matching rows in a few steps instead of scanning the whole table, like the index at the back of a book. It speeds up selective WHERE filters, joins and ORDER BY. The cost is that every insert, update and delete must also update the indexes, and they take extra storage, so I index the columns that are filtered or joined on frequently, not every column. With a composite index, column order matters because of the leftmost-prefix rule. And the optimizer may still choose a full scan if a condition matches a large part of the table."

COMMON TRAPS & MISTAKES

- "Indexes always make queries faster." Not for small tables, low-selectivity conditions or write-heavy workloads.
- "Index every column." Each index slows writes and uses storage.
- Expecting an index on (a, b) to help a query on b alone.
- Wrapping an indexed column in a function or casting it in WHERE, which prevents normal index use.
- Saying the primary key is a clustered index in PostgreSQL. It isn't; PostgreSQL tables are heaps.
- Forgetting that PRIMARY KEY and UNIQUE create indexes automatically, while foreign keys do **not** in PostgreSQL.

INTERVIEW QUESTIONS**BASIC****Q1** **DEFINITION** What is an index?

A A data structure (usually a B-tree) that stores column values in sorted order with pointers to rows, so the database can find rows without scanning the whole table.

Q2 **WHY** Why do indexes slow down inserts and updates?

A Every index on the table must be updated whenever a row is inserted, deleted, or has an indexed column changed, which adds extra work per write.

Q3 **DEFINITION** What is a composite index?

A An index on two or more columns, sorted by the first column, then the second, and so on.

INTERMEDIATE**Q4** **COMPARISON** Clustered vs non-clustered index?

A A clustered index stores the table rows themselves in index order (only one per table, e.g. the InnoDB primary key); a non-clustered index is a separate structure with pointers to rows (many allowed). PostgreSQL has only non-clustered indexes; its tables are heaps.

Q5 **DEFINITION** What is the leftmost-prefix rule?

A A composite index on (a, b, c) can be used for conditions on a, on a and b, or on a, b and c, but not efficiently for b or c alone.

Q6 **WHY** Why might the database not use an index that exists?

A The condition matches too many rows (low selectivity), the table is small, the column is wrapped in a function or cast, the condition is a leading-wildcard LIKE, statistics are outdated, or the query does not use the index's leading column.

Q7 HOW How do you check whether a query uses an index?

A Run `EXPLAIN` (or `EXPLAIN ANALYZE` for the actual execution) and look for Index Scan, Bitmap Index Scan or Index Only Scan instead of Seq Scan.

Q8 DEFINITION What is a covering index?

A An index that contains all the columns a query needs (in PostgreSQL via `INCLUDE`), so the query is answered from the index alone without visiting the table.

SCENARIO-BASED**Q9 SCENARIO A query `SELECT * FROM orders WHERE customer_id = ? ORDER BY order_date DESC LIMIT 10` is slow on 50 million rows. What index would you create?**

A A composite index on `(customer_id, order_date DESC)`. It filters by customer and returns rows already sorted, so the LIMIT can stop after 10.

Q10 SCENARIO Reads are fast but bulk inserts became very slow after the team added 12 indexes. What would you do?

A Review index usage (`pg_stat_user_indexes`), drop unused or redundant indexes, merge overlapping ones into composites, and for big loads consider dropping and recreating non-essential indexes around the load.

Q11 SCENARIO Users search customers by email in any case ("Aarav@Mail.com"). How do you make that fast?

A Store emails consistently, or create an expression index `CREATE INDEX ON customers (LOWER(email))` and query `WHERE LOWER(email) = LOWER(:input)`.

FOLLOW-UP / TRAP**Q12 TRAP QUESTION Does PostgreSQL automatically index foreign keys?**

A No. Primary keys and unique constraints get indexes automatically; foreign key columns do not, and should usually be indexed manually for joins and for deletes on the parent table.

Q13 TRAP QUESTION Will an index on `gender` speed up `WHERE gender = 'F'`?

A Usually not much: about half the table matches, so a sequential scan is cheaper and the optimizer will likely ignore the index.

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

P1. Write the SQL to create an index on `employees(city)`.

P2. Name two operations that become slower because of indexes.

P3. Which index does PostgreSQL create automatically when you declare a PRIMARY KEY?

LEVEL 2 – INTERVIEW APPLICATION

P4. You have an index on `(last_name, first_name)`. Which of these can use it efficiently: (a) `WHERE last_name = 'Rao'`, (b) `WHERE first_name = 'Asha'`, (c) `WHERE last_name = 'Rao' AND first_name = 'Asha'`?

P5. Explain why `WHERE EXTRACT(YEAR FROM order_date) = 2024` may not use an index on `order_date`, and rewrite it.

LEVEL 3 – SCENARIO / PROBLEM SOLVING

P6. An `orders` table receives 5,000 inserts per second and is queried mainly by `order_id` and occasionally by `customer_id` for a support screen. Which indexes would you create, and which would you avoid?

P7. A dashboard query filters `status = 'PENDING'` (2% of rows) on a 100-million-row table. Suggest an index that stays small.

ANSWER / EXPLANATION

P1. `CREATE INDEX idx_employees_city ON employees (city);`

P2. INSERTs and DELETEs (plus UPDATEs of indexed columns). Bulk loads are hit hardest.

P3. A unique B-tree index on the primary key column(s).

P4. (a) yes, (c) yes, (b) not efficiently, because `first_name` is not the leftmost column.

P5. The function is applied to every row's `order_date`, so the plain index on `order_date` can't be searched directly. Use a range instead: `WHERE order_date >= '2024-01-01' AND order_date < '2025-01-01'`, or create an expression index on `EXTRACT(YEAR FROM order_date)`.

P6. Keep the primary key index on `order_id` (it is automatic). Add one index on `customer_id`, because the support screen needs it and it is a foreign key. Avoid indexes on low-selectivity or rarely queried columns such as `status` or `amount`: each extra index costs work on all 5,000 inserts per second.

P7. A partial index: `CREATE INDEX idx_orders_pending ON orders (order_date) WHERE status = 'PENDING'`; It contains only the 2% of rows the dashboard reads, so it stays small and fast, and it does not grow with delivered orders.

Session 3.10 — Transactions and ACID

HIGH PRIORITY "What is ACID?" is one of the most common DBMS questions, and PwC write-ups mention it by name.

Simple meaning

A transaction is a group of database operations that must succeed or fail together, as one unit. Either *all* of its changes are saved, or *none* are.

The classic example is a bank transfer of ₹5000 from Aarav to Isha. It is two updates: debit Aarav, credit Isha. If the system crashes between them, money must not vanish (debited but never credited) or appear from nowhere.

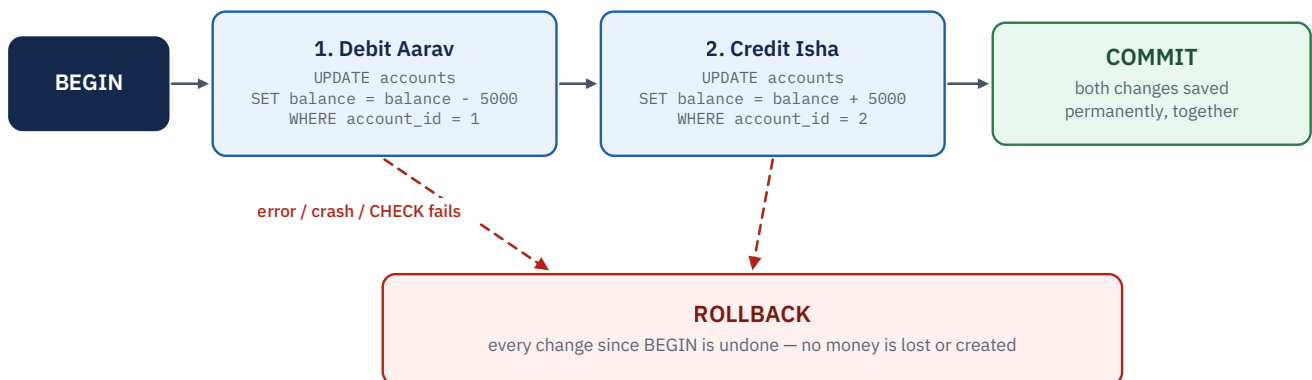


Figure 21. A transfer as one transaction: COMMIT saves both updates together; any failure rolls both back.

COMMIT and ROLLBACK in action

```

BEGIN;
UPDATE accounts SET balance = balance - 5000 WHERE account_id = 1;  -- debit Aarav
UPDATE accounts SET balance = balance + 5000 WHERE account_id = 2;  -- credit Isha
COMMIT;

SELECT account_id, holder_name, balance FROM accounts ORDER BY account_id;
  
```

SQL · POSTGRESQL

OUTPUT

account_id	holder_name	balance
1	Aarav Patel	45000.00
2	Isha Kulkarni	25000.00
3	Kabir Khan	5000.00

3 rows

LINE BY LINE

Line	What happens
BEGIN;	start a transaction: changes are now provisional
first UPDATE	Aarav's balance drops by 5000, visible only inside this transaction
second UPDATE	Isha's balance rises by 5000
COMMIT;	both changes become permanent and visible to everyone, at the same instant

Now a transfer that must fail: Kabir has only ₹5000 and tries to send ₹6000. The `CHECK (balance >= 0)` constraint rejects the debit, and the transaction is aborted:

```
BEGIN;
UPDATE accounts SET balance = balance + 6000 WHERE account_id = 2; -- credit Isha first
UPDATE accounts SET balance = balance - 6000 WHERE account_id = 3; -- debit Kabir: violates
CHECK
COMMIT;
```

SQL · POSTGRESQL

POSTGRESQL SAYS

ERROR new row for relation "accounts" violates check constraint "accounts_balance_check"
DETAIL: Failing row contains (3, Kabir Khan, -1000.00).

The credit to Isha had already succeeded, but because the transaction failed, it is undone as well. Nobody else ever saw it:

```
SELECT account_id, holder_name, balance FROM accounts ORDER BY account_id;
```

SQL · POSTGRESQL

OUTPUT

account_id	holder_name	balance
1	Aarav Patel	50000.00
2	Isha Kulkarni	20000.00
3	Kabir Khan	5000.00

3 rows

After an error inside a transaction, PostgreSQL rejects every further command with *"current transaction is aborted, commands ignored until end of transaction block"* until you issue `ROLLBACK` (or `ROLLBACK TO SAVEPOINT`, Session 3.4).

Autocommit: outside an explicit `BEGIN`, PostgreSQL treats each single statement as its own transaction and commits it immediately. Applications and frameworks (for example Spring's `@Transactional`) open explicit transactions when several statements must succeed together.

ACID: the four guarantees

Property	Simple meaning	Bank-transfer example	How databases provide it
Atomicity	all or nothing	both the debit and the credit happen, or neither does	undo information and the write-ahead log (WAL) let the DB roll back incomplete work
Consistency	every transaction moves the database from one valid state to another	the balance never goes negative; total money is unchanged	constraints (CHECK, FK, UNIQUE), triggers, and correct application logic
Isolation	concurrent transactions don't see each other's half-finished work	a report running during the transfer never sees "money in neither account"	locks and MVCC snapshots; isolation levels control how strict this is
Durability	once committed, it survives crashes and power cuts	after "Transfer successful", a server crash does not undo it	the WAL is flushed to disk before COMMIT returns; replicas and backups add more safety

A MEMORY HOOK

All or nothing · Correct state · Independent of others · Don't lose it.

TWO DIFFERENT "CONSISTENCY"S

ACID consistency means "the data obeys its rules (constraints)". **CAP consistency** (Session 4.3) means "every node in a distributed system returns the latest write". Interviewers sometimes check whether you mix them up.

What goes wrong without isolation: the read anomalies

When many transactions run at the same time, three classic problems can occur. Read each table top to bottom as time passes.

1. Dirty read: reading data another transaction has **not committed yet**.

Time	Transaction A	Transaction B
1	<code>UPDATE accounts SET balance = 0 WHERE account_id = 1;</code> (not committed)	
2		<code>SELECT balance ... WHERE account_id = 1;</code> → 0 (dirty)
3	<code>ROLLBACK;</code> (balance is 50000 again)	
4		B made a decision based on a value that never really existed

2. Non-repeatable read: reading the **same row twice** in one transaction and getting **different values**, because another transaction committed a change in between.

Time	Transaction A	Transaction B
1	<code>SELECT balance ... id 1;</code> → 50000	
2		<code>UPDATE ... SET balance = 45000 WHERE id = 1; COMMIT;</code>
3	<code>SELECT balance ... id 1;</code> → 45000 : the same query gives a different answer	

3. Phantom read: running the **same range query twice** and getting **new or missing rows**, because another transaction inserted or deleted rows that match.

Time	Transaction A	Transaction B
1	<code>SELECT COUNT(*) FROM orders WHERE status = 'PENDING';</code> → 2	
2		<code>INSERT INTO orders (... , 'PENDING');</code> <code>COMMIT;</code>
3	<code>SELECT COUNT(*) ... 'PENDING';</code> → 3 : a "phantom" row appeared	

Lost update **INTERVIEW EXTENSION** : two transactions read the same value, both compute a new value from it, and the second write overwrites the first. Two cashiers both read stock = 10 and both write 9 after selling one item each, so the stock should be 8 but shows 9.

Isolation levels

The SQL standard defines four isolation levels. Stricter levels prevent more anomalies but allow less concurrency.

Isolation level	Dirty read	Non-repeatable read	Phantom read
Read Uncommitted	possible	possible	possible
Read Committed (PostgreSQL default)	prevented	possible	possible
Repeatable Read	prevented	prevented	possible (by the standard)
Serializable	prevented	prevented	prevented

PostgreSQL specifics (good to mention):

- **Read Committed is the default.** Each *statement* sees data committed before that statement began.
- **Read Uncommitted behaves like Read Committed:** PostgreSQL never allows dirty reads.
- **Repeatable Read** gives the whole transaction one snapshot, and in PostgreSQL it **also prevents phantoms**, which is stronger than the standard requires.
- **Serializable** behaves as if transactions ran one after another. If that is impossible, one transaction fails with a *serialization failure*, and the application must **retry** it.
- MySQL InnoDB's default is Repeatable Read; SQL Server's and Oracle's default is Read Committed.

```
SHOW transaction_isolation;
```

SQL · PostgreSQL

OUTPUT

```
transaction_isolation
```

```
read committed
```

1 row

Choosing a level for a transaction:

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
```

```
-- ... statements that must see one consistent snapshot, e.g. a multi-step report
```

```
COMMIT;
```

SQL · PostgreSQL

MVCC: how PostgreSQL isolates without blocking readers INTERVIEW EXTENSION

PostgreSQL uses **MVCC (Multi-Version Concurrency Control)**: an UPDATE does not overwrite a row in place; it writes a **new version** of the row. Each transaction reads the versions that were committed as of its snapshot. As a result:

- **readers never block writers, and writers never block readers;**
- old row versions are later cleaned up by **VACUUM** (autovacuum runs this automatically).

Writers that change the **same row** still have to wait for each other (row locks).

Locks, SELECT ... FOR UPDATE and deadlocks INTERVIEW EXTENSION

To prevent lost updates in read-then-write logic, lock the row while you work with it:

```
BEGIN;
SELECT account_id, balance
FROM accounts
WHERE account_id = 1
FOR UPDATE;           -- other transactions that try to lock or update this row now wait
UPDATE accounts SET balance = balance - 1000 WHERE account_id = 1;
COMMIT;
```

SQL · POSTGRESQL

OUTPUT

account_id	balance
1	50000.00

1 row

Simpler still: make the update **atomic** in one statement

(`SET balance = balance - 1000 WHERE balance >= 1000`), so there is no gap between reading and writing.

Optimistic locking (common in web apps): keep a `version` column and update only if nobody changed the row since you read it: `UPDATE ... SET ..., version = version + 1 WHERE id = 7 AND version = 3;`. If it updates 0 rows, someone else got there first, so reload and retry.

A **deadlock** happens when two transactions each hold a lock the other needs:

Time	Transaction A	Transaction B
1	locks account 1	locks account 2
2	waits for account 2...	waits for account 1...
3	PostgreSQL detects the cycle and aborts one transaction with a deadlock error	

Prevention: always lock rows in the same order (e.g. lower `account_id` first), keep transactions short, and retry when a deadlock error occurs.

TRANSACTIONS IN A REAL APPLICATION

In a Spring Boot service, a method annotated with `@Transactional` runs inside one database transaction: if it throws an exception, everything is rolled back. Two habits interviewers like: keep transactions **short**, and never wait on a slow external call (such as a payment gateway) while holding database locks inside a transaction. Call the external service first, or use a pattern like a status column plus a retry job.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"A transaction groups operations into one unit that either fully succeeds or fully fails, like a bank transfer that debits one account and credits another. ACID describes its guarantees: Atomicity means all or nothing; Consistency means the database moves from one valid state to another, with constraints enforced; Isolation means concurrent transactions don't see each other's uncommitted work; and Durability means once committed, the change survives a crash, because it's written to the log on disk first. Isolation levels trade strictness for concurrency: Read Committed, PostgreSQL's default, prevents dirty reads; Repeatable Read also prevents non-repeatable reads; and Serializable prevents phantoms too."

COMMON TRAPS & MISTAKES

- Saying "Consistency means all replicas agree". That is CAP consistency, not ACID.

- Claiming PostgreSQL supports dirty reads at Read Uncommitted. It does not.
- Thinking a single UPDATE needs BEGIN/COMMIT to be atomic. Every single statement is already atomic (autocommit).
- Forgetting to ROLLBACK after an error inside a transaction (PostgreSQL ignores further commands).
- Long transactions holding locks while waiting for user input or external APIs.
- Mixing up non-repeatable reads (a changed *row*) and phantom reads (new or missing *rows* in a range).

INTERVIEW QUESTIONS

BASIC

Q1 DEFINITION What is a transaction?

A A sequence of database operations executed as a single unit of work: either all changes are committed or all are rolled back.

Q2 DEFINITION What is ACID?

A Atomicity (all or nothing), Consistency (valid state to valid state, constraints respected), Isolation (concurrent transactions don't interfere) and Durability (committed data survives failures).

Q3 COMPARISON COMMIT vs ROLLBACK?

A COMMIT makes the transaction's changes permanent and visible; ROLLBACK discards all changes made since the transaction began.

INTERMEDIATE

Q4 DEFINITION What is a dirty read?

A Reading data written by another transaction that has not committed yet, which might later be rolled back.

Q5 COMPARISON Non-repeatable read vs phantom read?

A A non-repeatable read is the same row returning a different value on a second read (an update by someone else); a phantom read is the same range query returning extra or missing rows (an insert or delete by someone else).

Q6 DEFINITION What are the isolation levels and PostgreSQL's default?

A Read Uncommitted, Read Committed, Repeatable Read and Serializable; PostgreSQL defaults to Read Committed (and treats Read Uncommitted as Read Committed).

Q7 HOW How does a database guarantee durability?

A It writes changes to a write-ahead log and flushes it to stable storage before confirming COMMIT, so after a crash it can replay the log; replication and backups add protection against disk loss.

Q8 DEFINITION What is a deadlock and how is it resolved?

A Two transactions each wait for a lock held by the other. The database detects the cycle and aborts one of them; applications should retry and acquire locks in a consistent order.

SCENARIO-BASED

Q9 SCENARIO Two users book the last seat on a flight at the same moment and both succeed. What went wrong and how do you fix it?

A A race condition, i.e. a lost update or check-then-act problem. Fix it with an atomic conditional update (`UPDATE seats SET booked_by = ? WHERE seat_id = ? AND booked_by IS NULL`, then check the row count), with `SELECT ... FOR UPDATE` inside a transaction, or with a unique constraint on (`flight_id`, `seat_no`).

Q10 **SCENARIO** A month-end report runs several queries and the totals don't add up because orders keep arriving. What do you do?

A Run the report in one transaction at REPEATABLE READ (or SERIALIZABLE) so every query sees the same snapshot, or run it on a replica or warehouse snapshot.

Q11 **DESIGN QUESTION** In an e-commerce checkout, which steps belong in one transaction?

A Creating the order and its items, decrementing stock, and recording the payment reference, all in one transaction. The external payment call and email sending should stay outside the database transaction, with status columns and retries to coordinate them.

FOLLOW-UP / TRAP

Q12 **TRAP QUESTION** Is a single UPDATE statement atomic without BEGIN?

A Yes. In autocommit mode each statement is its own transaction, so either all rows it targets are updated or none are.

Q13 **TRAP QUESTION** Does Serializable isolation mean transactions literally run one at a time?

A No. They run concurrently, but the database guarantees the result equals *some* serial order, aborting transactions (serialization failures) when that can't be guaranteed.

Q14 **TRAP QUESTION** Can NoSQL databases be ACID?

A Some can. MongoDB supports multi-document ACID transactions, and many others offer per-record atomicity. "NoSQL means no transactions" is outdated.

PRACTICE QUESTIONS

LEVEL 1 — BASIC UNDERSTANDING

P1. Expand ACID and give a one-line meaning for each letter.

P2. Which isolation level is PostgreSQL's default?

P3. What happens to uncommitted changes if the database server crashes?

LEVEL 2 — INTERVIEW APPLICATION

P4. Write a transaction that moves ₹2000 from Isha (account 2) to Kabir (account 3) and then shows all balances.

P5. Explain, with a timeline, how a lost update can happen when two sessions run `SELECT stock` and then `UPDATE stock = old_value - 1`.

LEVEL 3 — SCENARIO / PROBLEM SOLVING

P6. A wallet app must never let a balance go negative, even with simultaneous withdrawals. Give two database-level protections.

P7. A nightly job updates 2 million rows inside one huge transaction and blocks other users for an hour. How would you redesign it?

ANSWER / EXPLANATION

P1. Atomicity: all changes or none. Consistency: data stays valid according to its rules. Isolation: concurrent transactions don't see each other's unfinished work. Durability: committed changes survive crashes.

P2. Read Committed.

P3. They are lost (rolled back) during crash recovery; only committed transactions are restored from the write-ahead log.

P4.

SQL · PostgreSQL

```

BEGIN;
UPDATE accounts SET balance = balance - 2000 WHERE account_id = 2;
UPDATE accounts SET balance = balance + 2000 WHERE account_id = 3;
COMMIT;

SELECT account_id, holder_name, balance FROM accounts ORDER BY account_id;

```

OUTPUT

account_id	holder_name	balance
1	Aarav Patel	50000.00
2	Isha Kulkarni	18000.00
3	Kabir Khan	7000.00

3 rows

P5. Stock = 10. Session A reads 10; session B reads 10; A writes 9 and commits; B writes 9 and commits. Two items were sold, but the stock shows 9 instead of 8: A's update was lost. Fix it with an atomic `UPDATE ... SET stock = stock - 1 WHERE stock > 0`, with `SELECT ... FOR UPDATE`, or with optimistic locking on a version column.

P6. (1) A `CHECK (balance >= 0)` constraint, so any update that would go negative fails and its transaction rolls back. (2) An atomic conditional update, `UPDATE wallets SET balance = balance - :amt WHERE id = :id AND balance >= :amt`, then check that exactly 1 row was updated. Row locks (`SELECT ... FOR UPDATE`) or `SERIALIZABLE` isolation with retries are further options.

P7. Process in batches (for example 10,000 rows per transaction, keyed by id ranges) so locks are short and progress is saved; run during low traffic; make the job idempotent so a failed batch can be re-run; consider writing to a new table and swapping it in for very large changes.

Session 3.11 — SQL Interview Patterns (Practice Programs)

Interviewers reuse a small set of problem shapes. Once you recognise the shape, you know the tool. This session covers 22 patterns; each one has the idea, a tiny worked example run on the sample database, the general approach, the variations you will be asked, the difficulty, and the usual mistakes.

HOW TO ATTACK ANY SQL QUESTION IN AN INTERVIEW

1. **Restate** the question and ask about edge cases: "Should ties count once or twice? Include customers with no orders? What about NULLs?"
2. **Identify the grain** of the answer: one row per what? (customer, department, day...)
3. **Pick the pattern**: filter, aggregate, join, anti-join, rank, running total...
4. **Build it in steps** (a CTE per step), say each step out loud, then check it against a couple of rows by hand.
5. **Mention alternatives and performance** briefly: "This could also be done with a window function; an index on customer_id would help."

#	Pattern	Main tool	Difficulty
1	Filtering	WHERE, AND/OR, IN, BETWEEN, LIKE, NULL	EASY
2	Aggregation	COUNT, SUM, AVG, MIN, MAX	EASY
3	GROUP BY + HAVING	GROUP BY, HAVING	EASY
4	Joining multiple tables	JOIN chains	EASY – MEDIUM
5	Find duplicates	GROUP BY ... HAVING COUNT(*) > 1	EASY
6	Remove duplicates	ROW_NUMBER, DELETE ... USING	MEDIUM
7	Nth highest salary	DENSE_RANK, OFFSET	MEDIUM
8	Second highest salary	MAX with subquery	EASY – MEDIUM
9	Top N per group	ROW_NUMBER / DENSE_RANK + PARTITION BY	MEDIUM
10	Above the group average	window AVG, join to aggregate	MEDIUM
11	Customers with no orders	LEFT JOIN ... IS NULL, NOT EXISTS	EASY
12	Latest record per user	ROW_NUMBER, DISTINCT ON	MEDIUM
13	Finding missing records	generate_series, anti-join	MEDIUM
14	Ranking	RANK, DENSE_RANK	MEDIUM
15	Running totals	SUM() OVER (ORDER BY ...)	MEDIUM
16	Previous / next row	LAG, LEAD	MEDIUM
17	Date-based analysis	DATE_TRUNC, EXTRACT, intervals	MEDIUM
18	Conditional aggregation	SUM(CASE...), FILTER	MEDIUM
19	Subquery problems	scalar / correlated subqueries	MEDIUM
20	CTE problems	chained WITH steps	MEDIUM
21	Self joins	table joined to itself	MEDIUM
22	Window-function problems	gaps and islands, percentages	HARD

Pattern 1 – Filtering

EASY · asked in almost every interview, often as a warm-up

The pattern. Return the rows that satisfy several conditions. It looks trivial, which is exactly why mistakes with AND/OR, NULLs, ranges and case sensitivity are noticed.

General approach. Write each condition separately, combine them with parentheses, check NULL behaviour, and use half-open ranges for dates.

```
SELECT emp_name, city, salary, hire_date
FROM employees
WHERE (city = 'Bengaluru' OR city = 'Hyderabad')
      AND salary BETWEEN 70000 AND 130000
      AND hire_date >= '2019-01-01'
ORDER BY salary DESC;
```

SQL · POSTGRESQL

OUTPUT

emp_name	city	salary	hire_date
Sneha Iyer	Bengaluru	95000.00	2020-03-23
Vikram Rao	Hyderabad	80000.00	2021-07-01
Pooja Reddy	Hyderabad	72000.00	2024-02-12

3 rows

Variations interviewers ask: names starting with a letter (`LIKE 'A%'`), case-insensitive search (`ILIKE`), "not in these cities", rows with missing values (`IS NULL`), orders in the last 30 days (`order_date >= CURRENT_DATE - 30`).

Common mistakes: missing parentheses around OR; `= NULL` ; `BETWEEN` on timestamps; `NOT IN` with NULLs; quoting numbers as strings.

Pattern 2 – Aggregation

EASY · "How many...?", "What is the total / average...?"

The pattern. Summarise many rows into a few numbers.

General approach. Decide which rows count (WHERE), which aggregate answers the question, and whether duplicates should be counted once (`COUNT(DISTINCT ...)`).

```
SELECT COUNT(*)           AS orders,
       COUNT(DISTINCT customer_id) AS unique_customers,
       SUM(total_amount)    AS revenue,
       ROUND(AVG(total_amount), 2) AS avg_order_value,
       MAX(total_amount)    AS biggest_order
FROM orders
WHERE status <> 'CANCELLED'
      AND order_date >= '2024-01-01' AND order_date < '2024-04-01';  -- Q1 2024
```

SQL · POSTGRESQL

OUTPUT

orders	unique_customers	revenue	avg_order_value	biggest_order
9	5	59016.00	6557.33	24998.00

1 row

Variations: average excluding zeros or NULLs, percentage of total, minimum and maximum dates, count of non-null values.

Common mistakes: `COUNT(column)` silently skipping NULLs; integer division in averages and percentages; forgetting to exclude cancelled or test records.

Pattern 3 — GROUP BY + HAVING

EASY · reported in PwC write-ups ("write a query which uses GROUP BY")

The pattern. One result row per group, then keep only the groups that pass a condition on an aggregate.

General approach. WHERE for row filters → GROUP BY the grain → aggregate → HAVING for group filters → ORDER BY.

```
SELECT c.customer_name,
       COUNT(*)          AS orders,
       SUM(o.total_amount) AS spent
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
WHERE o.status <> 'CANCELLED'
GROUP BY c.customer_name
HAVING COUNT(*) >= 2
ORDER BY spent DESC;
```

SQL · PostgreSQL

OUTPUT

customer_name	orders	spent
Aarav Patel	4	42338.00
Isha Kulkarni	3	23983.00
Kabir Khan	2	18396.00
Vivaan Shah	2	10233.00
Diya Joshi	2	6796.00

5 rows

Variations: departments with more than N employees; categories with revenue above X; cities whose average salary exceeds the company average (HAVING with a subquery, Pattern 19).

Common mistakes: aggregates in WHERE; selecting non-grouped columns; using a SELECT alias in HAVING (PostgreSQL).

Pattern 4 — Joining Multiple Tables

EASY — **MEDIUM** · "Show X with details from Y and Z"

The pattern. Walk the foreign-key path between tables to bring together columns that live in different places.

General approach. Draw the path (customers → orders → order_items → products), join one hop at a time on the key pairs, choose INNER vs LEFT for each hop, then aggregate.

```
SELECT c.city,
       p.category,
       SUM(i.quantity * i.unit_price) AS revenue
FROM customers c
JOIN orders o      ON o.customer_id = c.customer_id
JOIN order_items i  ON i.order_id    = o.order_id
JOIN products p     ON p.product_id  = i.product_id
WHERE o.status NOT IN ('CANCELLED', 'RETURNED')
GROUP BY c.city, p.category
ORDER BY c.city, revenue DESC;
```

SQL · POSTGRESQL

OUTPUT

city	category	revenue
Ahmedabad	Electronics	6998.00
Ahmedabad	Stationery	3235.00
Bengaluru	Electronics	2798.00
Delhi	Furniture	15999.00
Delhi	Electronics	2397.00
Mumbai	Furniture	33997.00
Mumbai	Electronics	7096.00
Mumbai	Stationery	1245.00
Pune	Furniture	17998.00
Pune	Electronics	3499.00
Pune	Stationery	2486.00

11 rows

Variations: best-selling product per city, employees with department and manager names (two joins, one a self join), order lines with customer and product names.

Common mistakes: fan-out double counting (summing an order-level column after joining items); an accidental cross join from a missing condition; LEFT JOIN undone by a WHERE on the right table.

Pattern 5 — Find Duplicates

EASY · very common in data-quality and consulting contexts

The pattern. Identify values that appear more than once.

General approach. Group by the column(s) that define "duplicate" and keep groups with `COUNT(*) > 1`. To see the full duplicate rows, use a window count.

```
SELECT email, COUNT(*) AS copies
FROM leads
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY copies DESC;
```

SQL · POSTGRESQL

OUTPUT

email	copies
amit@mail.com	3
sara@mail.com	2

2 rows

Show every row that belongs to a duplicate group:

```
SELECT lead_id, full_name, email, created_at
FROM (SELECT l.*, COUNT(*) OVER (PARTITION BY email) AS copies
      FROM leads l) t
WHERE copies > 1
ORDER BY email, lead_id;
```

SQL · POSTGRESQL

OUTPUT

lead_id	full_name	email	created_at
1	Amit Kumar	amit@mail.com	2024-01-02
3	Amit Kumar	amit@mail.com	2024-01-05
6	Amit K.	amit@mail.com	2024-01-11
2	Sara Thomas	sara@mail.com	2024-01-03
5	Sara Thomas	sara@mail.com	2024-01-09

5 rows

Variations: duplicates across several columns (GROUP BY first_name, last_name, dob); case-insensitive duplicates (GROUP BY LOWER(TRIM(email))); duplicate orders within 5 minutes (self join or LAG).

Common mistakes: grouping by lead_id (every row is unique); ignoring case and whitespace differences; not agreeing on the definition of "duplicate" first.

Pattern 6 — Remove Duplicates

MEDIUM · "Delete duplicate rows but keep one"

The pattern. Keep one row per duplicate group (usually the first or the latest) and delete or ignore the rest.

General approach. Number the rows inside each group with ROW_NUMBER() OVER (PARTITION BY dup_key ORDER BY keep_rule); rows with rn > 1 are the extras.

```
DELETE FROM leads
WHERE lead_id IN (
  SELECT lead_id
  FROM (SELECT lead_id,
        ROW_NUMBER() OVER (PARTITION BY email ORDER BY created_at, lead_id) AS rn
        FROM leads) ranked
  WHERE rn > 1
)
RETURNING lead_id, full_name, email;
```

SQL · POSTGRESQL

OUTPUT

lead_id	full_name	email
3	Amit Kumar	amit@mail.com
5	Sara Thomas	sara@mail.com
6	Amit K.	amit@mail.com

3 rows

LINE BY LINE

Part	What it does
<code>ROW_NUMBER() OVER (PARTITION BY email ORDER BY created_at, lead_id)</code>	numbers each email's rows from oldest to newest: 1, 2, 3...
<code>WHERE rn > 1</code>	every row except the first per email
<code>DELETE ... WHERE lead_id IN (...)</code>	deletes exactly those rows
<code>RETURNING ...</code>	shows what was deleted (this example was rolled back afterwards)

Other ways: `DELETE ... USING` with a self join (shown in Session 3.4, P7); a de-duplicated *view* without deleting anything: `SELECT DISTINCT ON (email) * FROM leads ORDER BY email, created_at;` or `CREATE TABLE clean AS SELECT DISTINCT ...` and swap the tables.

Common mistakes: deleting *all* copies (including the one to keep); a non-deterministic keep rule (no tie-breaker); running the `DELETE` without first checking the `SELECT`; tables with no unique ID (then use `ctid` in PostgreSQL or rebuild the table).

Pattern 7 – Nth Highest Salary

MEDIUM · one of the most famous SQL interview questions

The pattern. Find the Nth largest *distinct* value. The trick is ties: two people earning 95000 should count as **one** salary level.

General approach (recommended): `DENSE_RANK()` over the value descending, then keep rank = N. It handles ties and generalises to "per group".

```
SELECT emp_name, salary
FROM (SELECT emp_name, salary,
            DENSE_RANK() OVER (ORDER BY salary DESC) AS rn
      FROM employees) ranked
WHERE rn = 4
ORDER BY emp_name;
```

SQL · POSTGRESQL

OUTPUT

emp_name	salary
Rahul Verma	95000.00
Sneha Iyer	95000.00

2 rows

The distinct salaries in order are 150000, 120000, 110000, **95000**..., so the 4th highest is 95000, and both people at that level are returned.

Alternative 1: DISTINCT + OFFSET (just the value):

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
OFFSET 3 LIMIT 1;           -- skip N-1 values
```

SQL · PostgreSQL

OUTPUT

salary
95000.00

1 row

Alternative 2: correlated subquery (works in any SQL dialect, slower on big tables): the Nth highest salary is the one that has exactly $N - 1$ distinct salaries above it.

```
SELECT DISTINCT e1.salary
FROM employees e1
WHERE 3 = (SELECT COUNT(DISTINCT e2.salary)
          FROM employees e2
          WHERE e2.salary > e1.salary);
```

SQL · PostgreSQL

OUTPUT

salary
95000.00

1 row

Variations: Nth highest per department (add `PARTITION BY dept_id`); return NULL when there is no Nth value; write it as a function with N as a parameter.

Common mistakes: `ROW_NUMBER` or `ORDER BY ... OFFSET` without `DISTINCT`, which counts ties twice and returns the wrong level; `RANK` (gaps make rank = N disappear after a tie); forgetting the "no result" case.

Pattern 8 — Second Highest Salary

EASY — **MEDIUM** · the classic special case of Pattern 7

General approach. The highest salary that is *below* the maximum:

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

SQL · PostgreSQL

OUTPUT

second_highest
120000.00

1 row

This form returns **NULL** automatically when there is no second salary (everyone earns the same), which many online judges expect. The OFFSET version returns *no row* instead; wrapping it as a scalar subquery turns that into NULL:

```
SELECT (SELECT DISTINCT salary
        FROM employees
        ORDER BY salary DESC
        OFFSET 1 LIMIT 1) AS second_highest;
```

SQL · POSTGRESQL

OUTPUT

second_highest

120000.00

1 row

Per department, with ties shown:

```
SELECT d.dept_name, r.emp_name, r.salary
FROM (SELECT emp_name, dept_id, salary,
             DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
      FROM employees) r
JOIN departments d ON d.dept_id = r.dept_id
WHERE r.rnk = 2
ORDER BY d.dept_name, r.emp_name;
```

SQL · POSTGRESQL

OUTPUT

dept_name	emp_name	salary
Engineering	Priya Sharma	120000.00
Finance	Karan Malhotra	70000.00
Finance	Meera Nair	70000.00
HR	Divya Menon	52000.00
Marketing	Neha Kapoor	60000.00

5 rows

In Finance, Karan and Meera share second place at 70000, and both are returned.

Common mistakes: ORDER BY salary DESC LIMIT 1 OFFSET 1 without DISTINCT (wrong when the top salary is tied); not stating what should happen with ties or with no second value.

Pattern 9 — Top N per Group

MEDIUM · "top 2 products in each category", "top 3 earners per department"

The pattern. Rank inside each group and keep the first N.

General approach. Aggregate if needed → `ROW_NUMBER` or `DENSE_RANK` with `PARTITION BY` group `ORDER BY` measure `DESC` → filter `<= N` in an outer query.

```
WITH product_revenue AS (
    SELECT p.category, p.product_name,
           SUM(i.quantity * i.unit_price) AS revenue
    FROM order_items i
    JOIN products p ON p.product_id = i.product_id
    JOIN orders o ON o.order_id = i.order_id
    WHERE o.status <> 'CANCELLED'
    GROUP BY p.category, p.product_name
),
ranked AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY category ORDER BY revenue DESC) AS rn
    FROM product_revenue
)
SELECT category, product_name, revenue
FROM ranked
WHERE rn <= 2
ORDER BY category, rn;
```

OUTPUT

category	product_name	revenue
Electronics	Mechanical Keyboard	13996.00
Electronics	USB-C Hub	7996.00
Furniture	Office Chair	35996.00
Furniture	Standing Desk	31998.00
Stationery	Notebook Pack	4731.00
Stationery	Gel Pen Set	2235.00

6 rows

ROW_NUMBER or DENSE_RANK? `ROW_NUMBER` returns exactly N rows per group; `DENSE_RANK` returns everyone tied at the cut-off. Ask the interviewer which they want.

LATERAL JOINS

PostgreSQL's `JOIN LATERAL (SELECT ... ORDER BY ... LIMIT 2)` runs a small "top N" subquery per row of the outer table. With the right index it is very fast for "latest 3 orders per customer".

Common mistakes: filtering on the window result in `WHERE` (not allowed); forgetting `PARTITION BY` (gives the top N overall); missing tie-breakers.

Pattern 10 — Rows Above Their Group's Average

MEDIUM · "employees earning more than their department's average"

General approach (three equivalent options): a window `AVG() OVER (PARTITION BY ...)` (Session 3.8), a correlated subquery (Session 3.7), or a join to a grouped derived table, shown here:

```
SELECT e.emp_name, e.dept_id, e.salary, a.dept_avg
FROM employees e
JOIN (SELECT dept_id, ROUND(AVG(salary), 2) AS dept_avg
      FROM employees
      GROUP BY dept_id) a ON a.dept_id = e.dept_id
WHERE e.salary > a.dept_avg
ORDER BY e.dept_id, e.salary DESC;
```

SQL · POSTGRESQL

OUTPUT

emp_name	dept_id	salary	dept_avg
Arjun Mehta	1	150000.00	102000.00
Priya Sharma	1	120000.00	102000.00
Ananya Das	2	110000.00	83333.33
Rohan Gupta	3	65000.00	58500.00
Aditya Singh	4	90000.00	75000.00

5 rows

Variations: products priced above their category's average; orders above the customer's average order; students above the class average.

Common mistakes: comparing with the *overall* average instead of the group's; forgetting that NULL group keys (Farhan) drop out of the join; using HAVING when you need row-level output.

Pattern 11 – Customers With No Orders

EASY · the anti-join; extremely common

General approach. Three standard ways; know at least two.

```
-- 1) LEFT JOIN + IS NULL
SELECT c.customer_name
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

SQL · POSTGRESQL

OUTPUT

customer_name
Saanvi Rao
Reyansh Bose

2 rows

```
-- 2) NOT EXISTS (NULL-safe and usually the clearest)
SELECT c.customer_name
FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

SQL · POSTGRESQL

OUTPUT

customer_name
Saanvi Rao
Reyansh Bose

2 rows

SQL · POSTGRESQL

```
-- 3) EXCEPT on the keys (set difference)
SELECT customer_id FROM customers
EXCEPT
SELECT customer_id FROM orders
ORDER BY customer_id;
```

OUTPUT

customer_id
6
7

2 rows

Variations: products never sold, employees who are not managers, departments without employees, users who signed up but never logged in.

Common mistakes: `NOT IN` with a subquery that can contain NULLs (returns nothing); checking `IS NULL` on a right-table column that can be NULL for matched rows (check the right table's key instead).

Pattern 12 – Latest Record per User

MEDIUM · "each customer's most recent order", "latest login per user", "current address"

General approach. `ROW_NUMBER() OVER (PARTITION BY user ORDER BY time DESC)` and keep `rn = 1`.

SQL · POSTGRESQL

```
SELECT user_id, login_time, device
FROM (SELECT l.*,
            ROW_NUMBER() OVER (PARTITION BY user_id
                                ORDER BY login_time DESC, login_id DESC) AS rn
      FROM user_logins l) t
WHERE rn = 1
ORDER BY user_id;
```

OUTPUT

user_id	login_time	device
1	2024-04-05 22:40:00	mobile
2	2024-04-04 18:20:00	mobile
3	2024-04-05 07:55:00	web
4	2024-04-04 21:10:00	web
5	2024-04-06 12:00:00	web

5 rows

PostgreSQL shortcut, DISTINCT ON:

SQL · POSTGRESQL

```
SELECT DISTINCT ON (user_id) user_id, login_time, device
FROM user_logins
ORDER BY user_id, login_time DESC;
```

OUTPUT

user_id	login_time	device
1	2024-04-05 22:40:00	mobile
2	2024-04-04 18:20:00	mobile
3	2024-04-05 07:55:00	web
4	2024-04-04 21:10:00	web
5	2024-04-06 12:00:00	web

5 rows

Another portable way: join each user to their `MAX(login_time)`. It is simple, but it returns two rows if two logins share the same latest timestamp.

Common mistakes: `GROUP BY user_id` with `MAX(login_time)` while also selecting `device` (not allowed, or picks the wrong row in lenient databases); ties without a tie-breaker; `RANK` instead of `ROW_NUMBER` when exactly one row is required.

Pattern 13 – Finding Missing Records

MEDIUM · gaps in IDs, days with no sales, expected rows that never arrived

General approach. Generate the complete *expected* set (numbers or dates) with `generate_series`, or use a calendar table, then anti-join it with the actual data.

Missing order IDs between the smallest and largest:

```
SELECT g AS missing_order_id
FROM generate_series((SELECT MIN(order_id) FROM orders),
                    (SELECT MAX(order_id) FROM orders)) AS g
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.order_id = g);
```

SQL · POSTGRESQL

OUTPUT

missing_order_id
107
112

2 rows

Days in the first week of April with no logins at all:

```
SELECT d::date AS day_without_logins
FROM generate_series(DATE '2024-04-01', DATE '2024-04-07', INTERVAL '1 day') AS d
WHERE NOT EXISTS (SELECT 1
                  FROM user_logins l
                  WHERE l.login_time::date = d::date);
```

SQL · POSTGRESQL

OUTPUT

day_without_logins
2024-04-07

1 row

Variations: months with zero revenue (show 0 instead of skipping: `LEFT JOIN` the calendar to the data and use `COALESCE(SUM(...), 0)`); employees with no timesheet for a working day; sensors that stopped reporting.

Common mistakes: grouping the data directly (days with no rows simply don't appear); comparing timestamps with dates without casting; time-zone boundaries.

Pattern 14 – Ranking

MEDIUM · "rank customers by number of orders"

General approach. Aggregate first, then rank with the function whose tie behaviour matches the question.

```
SELECT c.customer_name,
       COUNT(*)           AS orders,
       RANK()              OVER (ORDER BY COUNT(*) DESC) AS rank,
       DENSE_RANK() OVER (ORDER BY COUNT(*) DESC) AS dense_rank
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
GROUP BY c.customer_name
ORDER BY orders DESC, c.customer_name;
```

SQL · POSTGRESQL

OUTPUT

customer_name	orders	rank	dense_rank
Aarav Patel	4	1	1
Isha Kulkarni	3	2	2
Kabir Khan	3	2	2
Diya Joshi	2	4	3
Vivaan Shah	2	4	3

5 rows

Isha and Kabir tie for second place with 3 orders each: RANK then jumps to 4, while DENSE_RANK continues with 3.

Variations: rank within groups (PARTITION BY), percentile rank (`PERCENT_RANK`), quartiles (`NTILE(4)`), "show only the top 3 ranks".

Common mistakes: picking the wrong ranking function for ties; ranking before filtering out irrelevant rows; forgetting that ORDER BY inside OVER and the final ORDER BY are separate.

Pattern 15 – Running Totals

MEDIUM · cumulative revenue, running balance, cumulative signups

General approach. Aggregate to the right grain (per day), then `SUM(...) OVER (ORDER BY day)`. Add `PARTITION BY` for one running total per group.

```
WITH daily AS (
    SELECT order_date, SUM(total_amount) AS revenue
    FROM orders
    WHERE status <> 'CANCELLED'
    GROUP BY order_date
)
SELECT order_date,
       revenue,
       SUM(revenue) OVER (ORDER BY order_date
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cumulative_revenue
FROM daily
ORDER BY order_date;
```

OUTPUT

order_date	revenue	cumulative_revenue
2024-01-05	2843.00	2843.00
2024-01-12	3499.00	6342.00
2024-01-28	8999.00	15341.00
2024-02-14	2798.00	18139.00
2024-02-20	2486.00	20625.00
2024-03-03	6998.00	27623.00
2024-03-10	24998.00	52621.00
2024-03-18	2397.00	55018.00
2024-03-29	3998.00	59016.00
2024-04-04	17998.00	77014.00
2024-04-11	3235.00	80249.00
2024-04-19	21497.00	101746.00

12 rows

Two orders were placed on 19 April; aggregating per day first means each date appears once, so the running total is clean.

Variations: running total per customer (`PARTITION BY customer_id`), month-to-date totals (`PARTITION BY DATE_TRUNC('month', order_date)`), a running count, the percentage of the yearly target reached.

Common mistakes: forgetting `ORDER BY` inside `OVER` (gives the grand total on every row); `RANGE`-frame surprises with duplicate dates; mixing grains.

Pattern 16 – Previous / Next Row (LAG and LEAD)

MEDIUM · change since last time, gaps between events

General approach. `LAG(col) OVER (PARTITION BY entity ORDER BY time)` brings the previous row's value onto the current row; then subtract or compare.

Days between consecutive orders of the same customer:

```
SELECT customer_id,
       order_id,
       order_date,
       order_date - LAG(order_date) OVER (PARTITION BY customer_id
                                         ORDER BY order_date, order_id) AS days_since_prev
FROM orders
WHERE customer_id IN (1, 2)
ORDER BY customer_id, order_date;
```

OUTPUT

customer_id	order_id	order_date	days_since_prev
1	101	2024-01-05	NULL
1	103	2024-01-28	23
1	109	2024-03-10	42
1	115	2024-04-19	40
2	102	2024-01-12	NULL
2	106	2024-02-20	39
2	113	2024-04-04	44

7 rows

Date minus date gives a whole number of days in PostgreSQL. The first order of each customer has no previous row, so the gap is NULL.

Variations: month-over-month growth (Pattern 20), salary change between revisions, detecting a status change, comparing each stock price with the next day's (LEAD).

Common mistakes: missing PARTITION BY (compares different customers); no deterministic ORDER BY; dividing by a previous value that can be 0 or NULL.

Pattern 17 — Date-Based Analysis

MEDIUM · orders per month, busiest weekday, last N days

General approach. Bucket dates with DATE_TRUNC or EXTRACT / TO_CHAR, then aggregate. Filter with half-open ranges.

```
SELECT DATE_TRUNC('month', order_date)::date AS month,
       COUNT(*) AS orders,
       SUM(total_amount) AS revenue,
       ROUND(AVG(total_amount), 2) AS avg_order
FROM orders
WHERE status <> 'CANCELLED'
GROUP BY 1
ORDER BY 1;
```

OUTPUT

month	orders	revenue	avg_order
2024-01-01	3	15341.00	5113.67
2024-02-01	2	5284.00	2642.00
2024-03-01	4	38391.00	9597.75
2024-04-01	4	42730.00	10682.50

4 rows

Busiest day of the week for logins:

```
SELECT TO_CHAR(login_time, 'Dy') AS weekday,
       EXTRACT(ISODOW FROM login_time) AS day_no,
       COUNT(*) AS logins
FROM user_logins
GROUP BY 1, 2
ORDER BY day_no;
```

SQL · PostgreSQL

OUTPUT

weekday	day_no	logins
Mon	1	2
Tue	2	2
Wed	3	1
Thu	4	2
Fri	5	2
Sat	6	1

6 rows

Variations: last 7 or 30 days relative to today (`>= CURRENT_DATE - INTERVAL '30 days'`), year-over-year comparison, customer tenure (`AGE`), signups per week (`DATE_TRUNC('week', ...)`), first purchase month cohorts.

Common mistakes: `BETWEEN` with timestamps; grouping by `TO_CHAR(... 'Month')` and sorting alphabetically (April before January); ignoring time zones; using `EXTRACT(MONTH ...)` alone across several years (January 2023 and January 2024 merge).

Pattern 18 — Conditional Aggregation

MEDIUM · counts per category as columns ("pivot"), success rates

General approach. Put the condition **inside** the aggregate: `SUM(CASE WHEN cond THEN 1 ELSE 0 END)`, or `COUNT(*) FILTER (WHERE cond)` in PostgreSQL.

```
SELECT DATE_TRUNC('month', order_date)::date AS month,
       COUNT(*) FILTER (WHERE status = 'DELIVERED') AS delivered,
       COUNT(*) FILTER (WHERE status IN ('PENDING', 'SHIPPED')) AS open,
       COUNT(*) FILTER (WHERE status IN ('CANCELLED', 'RETURNED')) AS lost,
       ROUND(100.0 * COUNT(*) FILTER (WHERE status = 'DELIVERED') / COUNT(*), 1) AS delivered_pct
FROM orders
GROUP BY 1
ORDER BY 1;
```

SQL · PostgreSQL

OUTPUT

month	delivered	open	lost	delivered_pct
2024-01-01	3	0	0	100.0
2024-02-01	2	0	1	66.7
2024-03-01	2	1	1	50.0
2024-04-01	1	3	0	25.0

4 rows

The portable form of the first column is `SUM(CASE WHEN status = 'DELIVERED' THEN 1 ELSE 0 END)`.

Variations: revenue per category as columns, male/female headcount per department, pass/fail counts per subject, the share of mobile vs web logins.

Common mistakes: filtering in WHERE (removes the rows needed by the other columns); integer division in percentages; `COUNT(CASE WHEN ... THEN 1 ELSE 0 END)` (counts zeros too, because 0 is not NULL; use `SUM`, or leave out the ELSE).

Pattern 19 – Subquery Problems

MEDIUM · comparing against a computed value

General approach. Compute the reference value in a subquery (scalar, in WHERE/HAVING), or per row (correlated).

Departments whose average salary is above the company average (a subquery inside HAVING):

```
SELECT d.dept_name, ROUND(AVG(e.salary), 2) AS dept_avg
FROM employees e
JOIN departments d ON d.dept_id = e.dept_id
GROUP BY d.dept_name
HAVING AVG(e.salary) > (SELECT AVG(salary) FROM employees)
ORDER BY dept_avg DESC;
```

SQL · POSTGRESQL

OUTPUT

dept_name	dept_avg
Engineering	102000.00

1 row

Orders larger than that customer's own average order (correlated):

```
SELECT o.order_id, o.customer_id, o.total_amount
FROM orders o
WHERE o.total_amount > (SELECT AVG(x.total_amount)
                        FROM orders x
                        WHERE x.customer_id = o.customer_id)
ORDER BY o.customer_id, o.order_id;
```

SQL · POSTGRESQL

OUTPUT

order_id	customer_id	total_amount
109	1	24998.00
113	2	17998.00
104	3	15999.00
116	3	15999.00
111	4	3998.00
108	5	6998.00

6 rows

Variations: products costing more than every Stationery item (`> ALL`), employees hired before their manager, customers whose spend exceeds the average spend.

Common mistakes: a scalar subquery returning several rows; `NOT IN` with NULLs; correlated subqueries on huge tables without supporting indexes.

Pattern 20 – CTE Problems (Multi-Step Logic)

MEDIUM · "month-over-month growth", "flag months where..."

General approach. One CTE per logical step: aggregate → add the comparison (LAG) → compute the metric → filter or flag. Name each step clearly.

```

WITH monthly AS (
    SELECT DATE_TRUNC('month', order_date)::date AS month,
           SUM(total_amount) AS revenue
    FROM orders
    WHERE status <> 'CANCELLED'
    GROUP BY 1
),
with_prev AS (
    SELECT month, revenue,
           LAG(revenue) OVER (ORDER BY month) AS prev_revenue
    FROM monthly
)
SELECT month,
       revenue,
       prev_revenue,
       ROUND(100.0 * (revenue - prev_revenue) / NULLIF(prev_revenue, 0), 1) AS growth_pct,
       CASE WHEN revenue > prev_revenue THEN 'up' ELSE 'down / n.a.' END AS trend
FROM with_prev
ORDER BY month;
  
```

OUTPUT

month	revenue	prev_revenue	growth_pct	trend
2024-01-01	15341.00	NULL	NULL	down / n.a.
2024-02-01	5284.00	15341.00	-65.6	down / n.a.
2024-03-01	38391.00	5284.00	626.6	up
2024-04-01	42730.00	38391.00	11.3	up

4 rows

LINE BY LINE

Step	What it does
monthly	revenue per month (excluding cancelled orders)
with_prev	adds the previous month's revenue next to each month
final SELECT	computes growth %, protecting against division by zero with <code>NULLIF</code>

Variations: customers whose first order was in Q1 and who bought again later; top category per month; a funnel (visited → added to cart → paid) with one CTE per stage.

Common mistakes: one giant nested query nobody can read; recomputing the same aggregate in several places instead of reusing a CTE.

Pattern 21 – Self Joins

MEDIUM · employee/manager, pairs of rows, comparing rows of the same table

The classic question: "Find employees who earn more than their manager."

```
SELECT e.emp_name AS employee, e.salary,
       m.emp_name AS manager, m.salary AS manager_salary
FROM employees e
JOIN employees m ON m.emp_id = e.manager_id
WHERE e.salary > m.salary;
```

SQL · POSTGRESQL

OUTPUT

employee	salary	manager	manager_salary
0 rows			

In this data no employee out-earns their manager, so the correct answer is **an empty result**. Say so confidently in an interview: an empty result can be right. A variation that does return rows, employees based in the same city as their manager:

```
SELECT e.emp_name AS employee, m.emp_name AS manager, e.city
FROM employees e
JOIN employees m ON m.emp_id = e.manager_id
WHERE e.city = m.city
ORDER BY e.city, e.emp_name;
```

SQL · POSTGRESQL

OUTPUT

employee	manager	city
Priya Sharma	Arjun Mehta	Bengaluru
Sneha Iyer	Priya Sharma	Bengaluru
Neha Kapoor	Aditya Singh	Delhi
Divya Menon	Rohan Gupta	Kolkata
Karan Malhotra	Ananya Das	Mumbai

5 rows

And the number of direct reports per manager:

```
SELECT m.emp_name AS manager, COUNT(*) AS direct_reports
FROM employees e
JOIN employees m ON m.emp_id = e.manager_id
GROUP BY m.emp_name
ORDER BY direct_reports DESC, manager;
```

SQL · POSTGRESQL

OUTPUT

manager	direct_reports
Arjun Mehta	5
Priya Sharma	3
Ananya Das	2
Aditya Singh	1
Rahul Verma	1
Rohan Gupta	1

6 rows

Variations: pairs of employees in the same department (`e1.emp_id < e2.emp_id` to avoid duplicates); flights with a connection (`f1.dest = f2.origin`); consecutive-day comparisons in older SQL without LAG.

Common mistakes: using the same alias for both copies; joining in the wrong direction (`e.emp_id = m.manager_id` finds reports, not managers); listing each pair twice.

Pattern 22 – Window-Function Problems (Gaps and Islands)

HARD · consecutive days, streaks, sessions

The pattern. Find runs of consecutive values, for example "users who logged in on 3 or more consecutive days".

The trick. For consecutive dates, `date - ROW_NUMBER()` is **constant** within a run. Group by that constant (the "island" key):

user 1 login day	row_number	day - row_number
2024-04-01	1	2024-03-31
2024-04-02	2	2024-03-31
2024-04-03	3	2024-03-31
2024-04-05	4	2024-04-01 ← a new island

```

WITH days AS (
    SELECT DISTINCT user_id, login_time::date AS day
    FROM user_logins
),
islands AS (
    SELECT user_id, day,
           day - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY day))::int AS island
    FROM days
)
SELECT user_id,
       MIN(day) AS streak_start,
       MAX(day) AS streak_end,
       COUNT(*) AS streak_days
FROM islands
GROUP BY user_id, island
HAVING COUNT(*) >= 3;
  
```

SQL · POSTGRES SQL

OUTPUT

user_id	streak_start	streak_end	streak_days
1	2024-04-01	2024-04-03	3

1 row

LINE BY LINE

Step	What it does
<code>days</code>	one row per user per day they logged in (two logins on one day count once)
<code>ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY day)</code>	1, 2, 3... per user in date order
<code>day - (...)::int AS island</code>	constant inside a run of consecutive days; changes after a gap
<code>GROUP BY user_id, island ... HAVING COUNT(*) >= 3</code>	each group is one streak; keep streaks of 3 days or more

Customer 1 logged in on 1, 2 and 3 April, a three-day streak.

Other window-function problems: each row's percentage of its group total; cumulative distribution; the first and last value in a session; median (`PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY ...)`), strictly an ordered-set aggregate); sessionising events with LAG (a new session when the gap exceeds 30 minutes).

Common mistakes: not removing duplicate days first; forgetting PARTITION BY user; trying to filter streaks in WHERE.

“ HOW I TALK THROUGH A SQL PROBLEM IN AN INTERVIEW

"Let me make sure I understand: we want one row per department with its second-highest salary, and if two people share that salary we show both? ... OK. I'll first rank salaries inside each department with DENSE_RANK, because ties should share a rank and I don't want gaps. That goes in a CTE, since I can't filter on a window function in WHERE. Then I select the rows where the rank is 2 and join to departments for the name. If the table were large, an index on (dept_id, salary) would help. Let me quickly check it against Finance: 110000 is first, and both 70000 rows are second. Good."

INTERVIEW QUESTIONS

BASIC

Q1 **SQL PROBLEM** Find duplicate emails in a table.

A `SELECT email, COUNT(*) FROM leads GROUP BY email HAVING COUNT(*) > 1;`

Q2 **SQL PROBLEM** Find the second-highest salary.

A `SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);`
(returns NULL if there is none).

Q3 **SQL PROBLEM** List customers who never placed an order.

A `SELECT c.* FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);`

INTERMEDIATE

Q4 **SQL PROBLEM** Find the Nth highest salary.

A `DENSE_RANK` over salary descending in a subquery, keeping rank = N; or `SELECT DISTINCT salary ... ORDER BY salary DESC OFFSET N-1 LIMIT 1`.

Q5 **SQL PROBLEM** Delete duplicate rows, keeping the earliest.

A `DELETE FROM t WHERE id IN (SELECT id FROM (SELECT id, ROW_NUMBER() OVER (PARTITION BY dup_key ORDER BY created_at, id) rn FROM t) x WHERE rn > 1);`

Q6 **SQL PROBLEM** Get each user's latest login.

A `ROW_NUMBER` partitioned by user and ordered by login_time DESC, keeping rn = 1; or `DISTINCT ON (user_id) ... ORDER BY user_id, login_time DESC` in PostgreSQL.

Q7 **SQL PROBLEM** Compute a running total of daily revenue.

A Aggregate per day in a CTE, then `SUM(revenue) OVER (ORDER BY day)`.

SCENARIO-BASED

Q8 **SQL PROBLEM** Show monthly revenue and month-over-month growth percentage.

A A CTE for monthly revenue, `LAG(revenue) OVER (ORDER BY month)` for the previous month, and `growth = 100.0 * (rev - prev) / NULLIF(prev, 0)`.

Q9 **SQL PROBLEM** Find users who logged in on at least 3 consecutive days.

A Distinct user-days, then `day - ROW_NUMBER() OVER (PARTITION BY user ORDER BY day)` as an island key, then GROUP BY user and island with `HAVING COUNT(*) >= 3`.

Q10 SQL PROBLEM Top 3 products by revenue in each category.

- A** Aggregate revenue per product, then ROW_NUMBER (or DENSE_RANK for ties) partitioned by category ordered by revenue DESC, keeping rank ≤ 3 .

FOLLOW-UP / TRAP**Q11 TRAP QUESTION Why might ORDER BY salary DESC LIMIT 1 OFFSET 1 give the wrong second-highest salary?**

- A** If the top salary is shared by two people, the second row has the same top salary. Use DISTINCT or DENSE_RANK.

Q12 TRAP QUESTION Your "employees earning more than their manager" query returns no rows. Is it wrong?

- A** Not necessarily: verify the join direction and the data. An empty result is correct when no such employee exists.

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

- P1.** Find all products whose name contains "Desk" or "Chair" (case-insensitive).
P2. Count orders per customer, including customers with zero orders.
P3. Return the third-highest product price.

LEVEL 2 – INTERVIEW APPLICATION

- P4.** For each department, show the highest-paid employee (all of them if tied).
P5. Show each customer's first-ever order (id and date).
P6. Produce one row per product category with three columns: units sold in January, February and March 2024 (exclude cancelled orders).

LEVEL 3 – SCENARIO / PROBLEM SOLVING

- P7.** For each month, show revenue and the running (year-to-date) revenue, plus the share of the year-to-date total that the month represents.
P8. Find customers whose order totals strictly increased with every new order (each order larger than their previous one), considering only customers with at least two orders.

ANSWER / EXPLANATION**P1.**

```
SELECT product_name
FROM products
WHERE product_name ILIKE '%desk%' OR product_name ILIKE '%chair%';
```

SQL · POSTGRESQL

OUTPUT

product_name
Office Chair
Standing Desk

2 rows

P2. LEFT JOIN, and count a column from the right side:

```
SELECT c.customer_name, COUNT(o.order_id) AS orders
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_name
ORDER BY orders DESC, c.customer_name;
```

SQL · POSTGRESQL

OUTPUT

customer_name	orders
Aarav Patel	4
Isha Kulkarni	3
Kabir Khan	3
Diya Joshi	2
Vivaan Shah	2
Reyansh Bose	0
Saanvi Rao	0

7 rows

P3.

```
SELECT DISTINCT price
FROM products
ORDER BY price DESC
OFFSET 2 LIMIT 1;
```

SQL · POSTGRESQL

OUTPUT

price
3499.00

1 row

P4. RANK or DENSE_RANK keeps ties at rank 1:

```
SELECT dept_id, emp_name, salary
FROM (SELECT dept_id, emp_name, salary,
            RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
      FROM employees
     WHERE dept_id IS NOT NULL) t
WHERE rnk = 1
ORDER BY dept_id;
```

SQL · POSTGRESQL

OUTPUT

dept_id	emp_name	salary
1	Arjun Mehta	150000.00
2	Ananya Das	110000.00
3	Rohan Gupta	65000.00
4	Aditya Singh	90000.00

4 rows

P5.

```
SELECT DISTINCT ON (customer_id) customer_id, order_id, order_date
FROM orders
ORDER BY customer_id, order_date, order_id;
```

SQL · POSTGRESQL

OUTPUT

customer_id	order_id	order_date
1	101	2024-01-05
2	102	2024-01-12
3	104	2024-02-02
4	105	2024-02-14
5	108	2024-03-03

5 rows

P6. Conditional aggregation with a date filter inside each FILTER:

```
SELECT p.category,
       COALESCE(SUM(i.quantity) FILTER (WHERE o.order_date >= '2024-01-01' AND o.order_date
< '2024-02-01'), 0) AS jan_units,
       COALESCE(SUM(i.quantity) FILTER (WHERE o.order_date >= '2024-02-01' AND o.order_date
< '2024-03-01'), 0) AS feb_units,
       COALESCE(SUM(i.quantity) FILTER (WHERE o.order_date >= '2024-03-01' AND o.order_date
< '2024-04-01'), 0) AS mar_units
FROM order_items i
JOIN orders o ON o.order_id = i.order_id
JOIN products p ON p.product_id = i.product_id
WHERE o.status <> 'CANCELLED'
GROUP BY p.category
ORDER BY p.category;
```

SQL · POSTGRESQL

OUTPUT

category	jan_units	feb_units	mar_units
Electronics	3	2	7
Furniture	1	0	2
Stationery	5	14	0

3 rows

P7.

SQL · PostgreSQL

```
WITH monthly AS (  
    SELECT DATE_TRUNC('month', order_date)::date AS month,  
           SUM(total_amount) AS revenue  
    FROM orders  
    WHERE status <> 'CANCELLED'  
    GROUP BY 1  
)  
SELECT month,  
       revenue,  
       SUM(revenue) OVER (ORDER BY month) AS ytd_revenue,  
       ROUND(100.0 * revenue / SUM(revenue) OVER (ORDER BY month), 1) AS pct_of_ytd  
FROM monthly  
ORDER BY month;
```

OUTPUT

month	revenue	ytd_revenue	pct_of_ytd
2024-01-01	15341.00	15341.00	100.0
2024-02-01	5284.00	20625.00	25.6
2024-03-01	38391.00	59016.00	65.1
2024-04-01	42730.00	101746.00	42.0

4 rows

P8. Compare each order with the previous one using LAG; a customer qualifies if none of their orders is less than or equal to the previous order:

SQL · PostgreSQL

```
WITH seq AS (  
    SELECT customer_id, order_id, total_amount,  
           LAG(total_amount) OVER (PARTITION BY customer_id ORDER BY order_date, order_id)  
    AS prev_total  
    FROM orders  
)  
SELECT customer_id  
FROM seq  
GROUP BY customer_id  
HAVING COUNT(*) >= 2  
       AND COUNT(*) FILTER (WHERE prev_total IS NOT NULL AND total_amount <= prev_total) = 0  
ORDER BY customer_id;
```

OUTPUT

customer_id
4

1 row

Customer 4 qualifies (2798 → 3998). Every other customer has at least one order smaller than the one before.

Session 3.12 — Module 3 Summary & Rapid Revision

MODULE 3 SUMMARY

MOST IMPORTANT CONCEPTS

- **Query anatomy:** SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT. Logical order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.
- **NULL** means unknown: use `IS NULL`; aggregates ignore NULLs; `COUNT(*)` counts rows; `NOT IN` with NULLs returns nothing.
- **DDL** (CREATE, ALTER, DROP, TRUNCATE) vs **DML** (INSERT, UPDATE, DELETE) vs **DCL** (GRANT, REVOKE) vs **TCL** (COMMIT, ROLLBACK, SAVEPOINT).
- **WHERE vs HAVING:** rows before grouping vs groups after grouping.
- **Joins:** INNER (matches only), LEFT/RIGHT (keep one side), FULL (keep both), CROSS (every combination), self join (a table with itself); anti-join with `LEFT JOIN ... IS NULL` or `NOT EXISTS`.
- **Subqueries and CTEs:** scalar, multi-row, derived tables, correlated; WITH for readable steps; recursive CTEs for hierarchies.
- **Window functions:** OVER (PARTITION BY ... ORDER BY ...); ROW_NUMBER vs RANK vs DENSE_RANK; LAG/LEAD; running totals and moving averages; filter results in an outer query.
- **Indexes:** B-tree, faster reads and slower writes; selectivity; composite index leftmost-prefix rule; clustered vs non-clustered (PostgreSQL tables are heaps).
- **Transactions:** all or nothing; ACID; isolation levels vs dirty, non-repeatable and phantom reads; PostgreSQL defaults to Read Committed.

WHAT TO MEMORIZE

- ROW_NUMBER 1,2,3,4 · RANK 1,2,2,4 · DENSE_RANK 1,2,2,3.
- DELETE (DML, WHERE, slower, rollback) · TRUNCATE (DDL, all rows, fast) · DROP (removes the table).
- ACID = Atomicity, Consistency, Isolation, Durability.
- Isolation levels and the anomalies each one prevents.

WHAT TO UNDERSTAND

- Why aliases fail in WHERE but work in ORDER BY.
- Why a WHERE on the right table breaks a LEFT JOIN.
- Why fan-out inflates sums after joins.
- Why the optimizer may ignore an index.

MOST COMMON INTERVIEW QUESTIONS

- Second / Nth highest salary · duplicates (find and delete) · customers without orders · top N per group · latest record per user · running totals · WHERE vs HAVING · join types · DELETE vs TRUNCATE vs DROP · what is an index · ACID · isolation levels.

COMMON MISTAKES

- `= NULL` · AND/OR without parentheses · integer division · `COUNT(*)` with LEFT JOIN · ROW_NUMBER for Nth highest *value* · window functions in WHERE · indexing everything · forgetting COMMIT/ROLLBACK.

RAPID REVISION CHECKLIST

- | | |
|---|---|
| ☐ I can write SELECT with WHERE, ORDER BY, LIMIT and aliases | ☐ I know the string, date and numeric functions I need |
| ☐ I handle NULL correctly (IS NULL, COALESCE, COUNT) | ☐ I can write CASE WHEN and conditional aggregation |
| | ☐ I can explain DDL vs DML vs DCL vs TCL |

- ☐ I can compare DELETE, TRUNCATE and DROP
- ☐ I can write INSERT, UPDATE, DELETE and an upsert
- ☐ I can use GROUP BY with HAVING correctly
- ☐ I can recite the logical order of execution
- ☐ I can write every join type and explain the results
- ☐ I can write an anti-join two different ways
- ☐ I can write scalar, correlated and EXISTS subqueries
- ☐ I can avoid the NOT IN + NULL trap
- ☐ I can structure a query with CTEs
- ☐ I can explain a recursive CTE
- ☐ I can explain ROW_NUMBER vs RANK vs DENSE_RANK
- ☐ I can use LAG/LEAD and running totals
- ☐ I can solve Nth highest, top N per group and latest per user
- ☐ I can explain how a B-tree index speeds up reads
- ☐ I know when indexes hurt, and the leftmost-prefix rule
- ☐ I can explain clustered vs non-clustered indexes
- ☐ I can explain ACID with a bank example
- ☐ I can explain dirty, non-repeatable and phantom reads
- ☐ I know the isolation levels and PostgreSQL's default

PART IV • MODULE 4

Cloud Databases & Storage Systems

Modern data rarely lives on a single server in an office. This module explains managed cloud databases, how databases scale and stay available, the different families of databases (relational, document, key-value, wide-column, graph), the CAP theorem, MongoDB from scratch, and the storage layers used for analytics: warehouses, lakes and lakehouses.

COURSE COVERAGE

- ✓ Cloud databases: what they are, managed databases, RDS / Cloud SQL concepts, serverless databases, analytical databases, scalability, availability, backups, replication, failover † → Session 4.1
- ✓ SQL vs NoSQL: relational, document, key-value, graph, column-family and columnar; when to choose each † → Session 4.2
- ✓ NoSQL: what and why, CAP theorem (consistency, availability, partition tolerance), eventual consistency † → Session 4.3
- ✓ MongoDB: database, collection, document, field, BSON † → Session 4.4
- ✓ MongoDB CRUD: insert, find, update, delete, updateMany, deleteMany † → Session 4.5
- ✓ MongoDB indexing, aggregation, embedding vs referencing † → Session 4.6
- ✓ Data storage: database, data warehouse, data lake, data lakehouse; structured, semi-structured and unstructured data † → Session 4.7
- ✓ Module quiz concepts → interview questions in each session and Session 4.8

Topic	Priority	Typical question
SQL vs NoSQL, choosing a database	HIGH PRIORITY	"Would you use SQL or NoSQL for X, and why?"
MongoDB basics and CRUD	HIGH PRIORITY	"How do you update many documents?" "What is BSON?"
CAP theorem, eventual consistency	MEDIUM PRIORITY	"Explain CAP with an example."
Managed databases, replication, failover, backups	MEDIUM PRIORITY	"What happens when the primary fails?"
Warehouse vs lake vs lakehouse	MEDIUM PRIORITY	"Compare a data lake and a data warehouse."

PWC ANGLE

Cloud migration and data-platform modernization are large consulting service lines, so candidates are often asked to reason about trade-offs ("managed vs self-hosted", "lake vs warehouse", "why NoSQL here?") rather than to recite vendor commands. This module focuses on those concepts; product names appear only as examples.

Session 4.1 — Introduction to Cloud Databases

What is a cloud database?

Simple meaning: a cloud database is a database that runs on a cloud provider's infrastructure (AWS, Microsoft Azure, Google Cloud...) and is used over the network, instead of on servers your company buys and runs itself.

There are three main ways to run a database in the cloud:

Option	What you get	Who does the hard work	Example
Self-managed on a VM (IaaS)	a virtual machine; you install the database yourself	you: installation, patches, backups, replication	PostgreSQL installed on an EC2 or Azure VM
Managed database (DBaaS / PaaS)	a ready database endpoint	the provider runs the servers; you design and use the database	Amazon RDS, Google Cloud SQL, Azure Database for PostgreSQL
Serverless database	a database that scales (even to zero) automatically; pay per use	the provider handles capacity too	Aurora Serverless, Azure SQL serverless, DynamoDB on-demand, Neon

Managed databases and the shared responsibility model

A **managed database** means the cloud provider takes over the operational chores: provisioning hardware, installing and patching the database, automated backups, replication, failover and monitoring. You still own the **data**: schema design, queries, indexes, users and permissions.

	On-premises	DB on a cloud VM	Managed DB (RDS / Cloud SQL)	Serverless DB
Hardware, power, data centre	You	Provider	Provider	Provider
Operating system install & patches	You	You	Provider	Provider
Database install, patches, upgrades	You	You	Provider	Provider
Backups, replication, failover	You	You	Provider	Provider
Capacity planning & scaling	You	You	Provider	Provider
Schema, queries, indexes, access	You	You	You	You

Moving right = you manage less infrastructure and focus more on data and queries.

Figure 22. Who manages what. Moving right, the provider takes over more of the operational work.

Why managed databases exist: running a database well is hard work that does not differentiate most businesses. A managed service gives you, in minutes, what used to take a DBA team weeks: high availability, backups, encryption and monitoring.

Benefits	Trade-offs
Fast setup (minutes)	Can cost more than self-managed at very large scale
Automated backups and point-in-time recovery	Less control: no OS access; some extensions or settings restricted
Built-in high availability and failover	Some vendor lock-in (proprietary features, migration effort)
Easy scaling (bigger instance, read replicas)	Network latency if the app runs elsewhere
Patching and upgrades handled	Maintenance windows are on the provider's schedule

RDS and Cloud SQL, conceptually

- **Amazon RDS (Relational Database Service):** managed PostgreSQL, MySQL, MariaDB, Oracle and SQL Server. **Amazon Aurora** is AWS's cloud-native engine, compatible with MySQL and PostgreSQL, with storage replicated across availability zones.
- **Google Cloud SQL:** managed MySQL, PostgreSQL and SQL Server. **AlloyDB** and **Spanner** are Google's higher-end options.
- **Azure SQL Database / Azure Database for PostgreSQL or MySQL:** Microsoft's managed equivalents.

In an interview you rarely need commands. Say what the service does for you: "RDS runs PostgreSQL for us, with automated backups, Multi-AZ failover and read replicas; we just connect to an endpoint."

Serverless databases

A **serverless** database automatically adds and removes compute capacity as load changes, can pause when idle, and bills for what is actually used.

- **Good for:** spiky or unpredictable traffic, development and test environments, new products with unknown load.
- **Watch out for:** cold-start or resume delays after idle periods, cost surprises under constant heavy load, and limits on connections or features.

Analytical (cloud data warehouse) databases

Analytical databases are built for OLAP (Module 1): scanning and aggregating huge tables.

Product	Notable idea
Google BigQuery	serverless: no servers to size; pay per data scanned or reserved capacity
Amazon Redshift	MPP (massively parallel processing) columnar warehouse
Snowflake	storage and compute separated; many independent compute "warehouses" on the same data
Azure Synapse / Microsoft Fabric	warehouse plus lake integration in Microsoft's ecosystem

They store data in **columns** rather than rows, compress it heavily, and split each query across many machines.

Scalability: handling more load

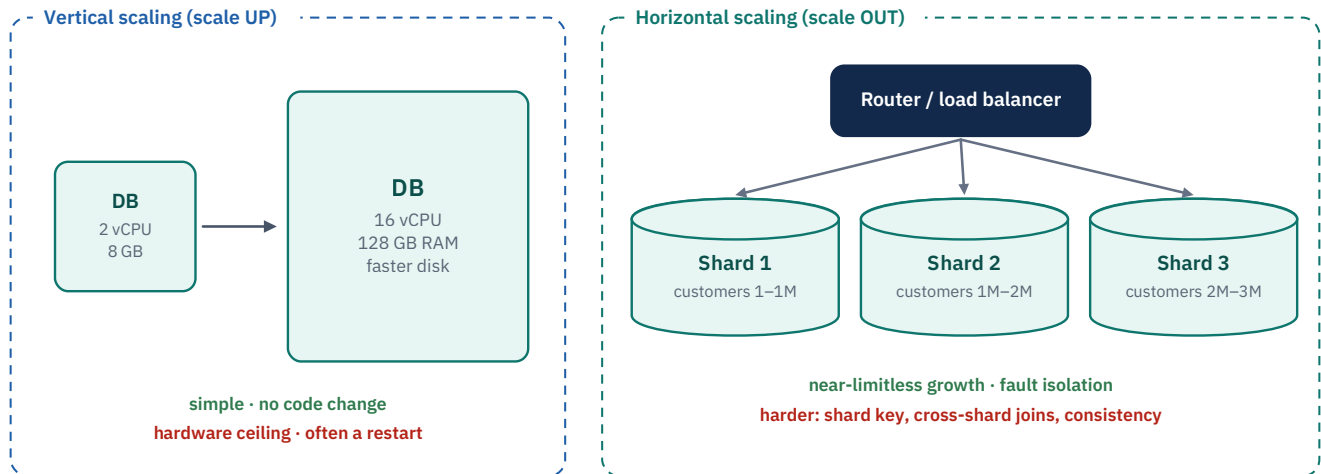


Figure 23. Vertical scaling makes one machine bigger; horizontal scaling adds more machines.

	Vertical scaling (scale up)	Horizontal scaling (scale out)
How	bigger machine: more CPU, RAM, faster disk	more machines sharing the load
Code changes	none	often needed (routing, sharding)
Limit	the biggest machine available	very high
Downtime	often a restart	can be done online
Typical for	relational databases, first step for most apps	NoSQL, distributed SQL, read replicas

Three tools for scaling relational databases:

1. **Read replicas:** copies of the database that serve read-only queries, taking load off the primary (good when reads $\gg$ writes).
2. **Partitioning:** splitting a big table into pieces *inside one database* (for example, orders partitioned by month). Queries that touch one month read only that partition.
3. **Sharding:** splitting data *across several database servers* by a **shard key** (customer_id 1-1M on shard 1...). Very scalable, but cross-shard joins and transactions become hard.

Availability: staying up

High availability (HA) means the database keeps serving even when a component fails. Clouds offer it through **multiple availability zones** (separate data centres in one region) with automatic failover.

SLA uptime	Allowed downtime per year (approx.)
99.9% ("three nines")	about 8.8 hours
99.95%	about 4.4 hours
99.99% ("four nines")	about 53 minutes

Two terms every interviewer likes **INTERVIEW EXTENSION** :

- **RPO (Recovery Point Objective):** how much data you can afford to lose, measured in time. "RPO 5 minutes" means losing at most the last 5 minutes of changes.
- **RTO (Recovery Time Objective):** how long you can afford to be down. "RTO 1 hour" means service is restored within an hour.

Backups

- **Automated snapshots:** a full copy taken daily and kept for a retention period (for example 7–35 days).
- **Point-in-time recovery (PITR):** the snapshot plus the transaction logs let you restore to any second in the retention window, for example just before someone ran `DELETE` without a `WHERE`.
- **Test your restores.** A backup you have never restored is a hope, not a plan.

Replication

Replication keeps copies of the data on other servers.

	Synchronous replication	Asynchronous replication
How	the primary waits until the replica confirms each write	the primary confirms immediately; the replica catches up shortly after
Data loss on primary failure	none (RPO ≈ 0)	possibly the last few seconds of writes
Write speed	slower (waits for the network)	faster
Typical use	standby for failover (in the same region)	read replicas, cross-region copies

Replication lag with asynchronous replicas can surprise users: someone updates their profile (on the primary), the page reloads from a replica, and the old value appears. Fixes include reading your own writes from the primary, or routing that user to the primary for a short time.

REPLICATION IS NOT A BACKUP

If someone deletes a table, replication faithfully copies the delete to every replica within seconds. Only backups (and PITR) let you go back in time.

Failover

Failover is switching to a standby when the primary fails.

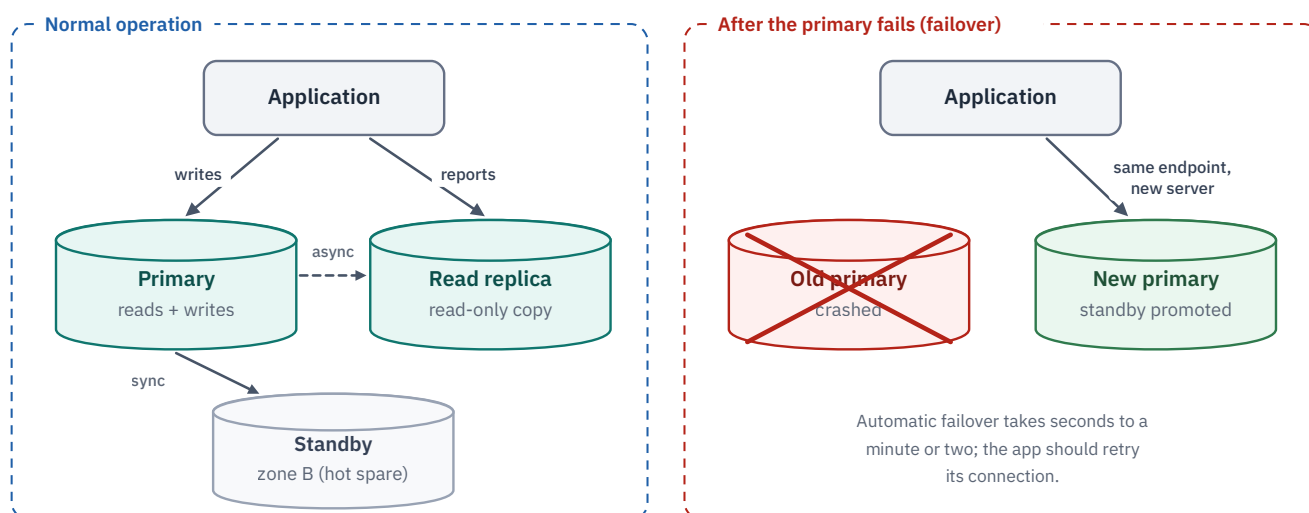


Figure 24. With Multi-AZ, a synchronous standby is promoted when the primary fails; the application reconnects to the same endpoint.

1. Health checks detect the failed primary.
2. The standby (an up-to-date synchronous copy) is **promoted** to be the new primary.
3. The database endpoint (a DNS name) is pointed at the new primary.

4. Applications reconnect. This takes from seconds to a couple of minutes, so apps should **retry connections**.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"A managed cloud database like Amazon RDS or Cloud SQL runs the database for us: the provider handles hardware, patching, backups, replication and failover, and we focus on schema, queries and access control. For scale we can move to a bigger instance, add read replicas for read-heavy traffic, partition big tables, or shard across servers. For availability we deploy a synchronous standby in another availability zone; if the primary fails, the standby is promoted and the endpoint switches over. And because replication copies mistakes too, we still rely on automated backups and point-in-time recovery."

COMMON TRAPS & MISTAKES

- Treating replicas as backups.
- Assuming "managed" means "no work". Schema design, indexing, query tuning and security remain yours.
- Confusing partitioning (within one database) with sharding (across servers).
- Forgetting replication lag when reading from replicas.
- Saying serverless means "no servers exist". They exist; you just don't manage them.

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What is a managed database?

A A cloud service where the provider operates the database (provisioning, patching, backups, replication, failover, monitoring) while the customer manages the data, schema, queries and access.

Q2 **COMPARISON** Vertical vs horizontal scaling?

A Vertical adds resources to one machine (simple, limited); horizontal adds more machines (very scalable, but more complex: sharding, routing, consistency).

Q3 **DEFINITION** What is a read replica?

A A read-only copy of the database, kept up to date by replication, used to serve read queries and reduce load on the primary.

INTERMEDIATE

Q4 **COMPARISON** Synchronous vs asynchronous replication?

A Synchronous waits for the replica to confirm each write (no data loss on failover, slower writes); asynchronous confirms immediately (faster, but recent writes can be lost and replicas lag).

Q5 **DEFINITION** What are RPO and RTO?

A RPO is the maximum acceptable data loss, measured in time; RTO is the maximum acceptable downtime before service is restored.

Q6 **COMPARISON** Partitioning vs sharding?

A Partitioning splits a table into parts within one database server; sharding distributes data across multiple servers by a shard key.

Q7 **DEFINITION** What is point-in-time recovery?

A Restoring a database to an exact moment by applying transaction logs on top of a base backup, for example to just before an accidental delete.

SCENARIO-BASED

Q8 **SCENARIO** Your app's reads are 20 times higher than writes and the database CPU is maxed out.

What would you consider?

A Add read replicas and route read-only queries to them, add caching (Redis) for hot data, check indexes and slow queries, and only then scale the primary up. Watch for replication lag on read-after-write paths.

Q9 **SCENARIO** The business needs at most 1 minute of data loss and 15 minutes of downtime. How would you set up the database?

A A managed database with a synchronous Multi-AZ standby for automatic failover (meets RTO), continuous log backup with PITR (meets RPO), alerting, and regular restore tests. For regional disasters, add an asynchronous cross-region replica.

Q10 **DESIGN QUESTION** Would you choose a serverless database for a college-fest registration site?

A Yes, it's a good fit: traffic spikes around the event and is near zero otherwise, so auto-scaling and pay-per-use keep costs low. Mention possible cold starts after idle periods.

FOLLOW-UP / TRAP

Q11 **TRAP QUESTION** We have three replicas, so do we still need backups?

A Yes. Replicas copy deletions and corruption almost instantly; backups and PITR protect against human error, bugs and ransomware.

Q12 **TRAP QUESTION** Does a managed database tune your queries for you?

A No. Some services offer recommendations, but schema design, indexing and query optimization remain the customer's responsibility.

Session 4.2 — SQL vs NoSQL: Types of Databases

HIGH PRIORITY "SQL or NoSQL, and why?" is one of the most common database interview questions.

Relational (SQL) databases, in one paragraph

Relational databases store data in **tables** with a **fixed schema**, link tables with **keys**, are queried with **SQL**, and provide **ACID transactions** and **joins**. They are the default for business data that must be correct and consistent: orders, payments, inventory, HR. Examples: PostgreSQL, MySQL, Oracle, SQL Server.

What "NoSQL" means

NoSQL ("Not only SQL") is an umbrella term for databases that do **not** use the relational table model. They typically trade some relational features (fixed schemas, joins, sometimes strict consistency) for **flexible data models**, **horizontal scaling** and **high write throughput**. There are four main families:

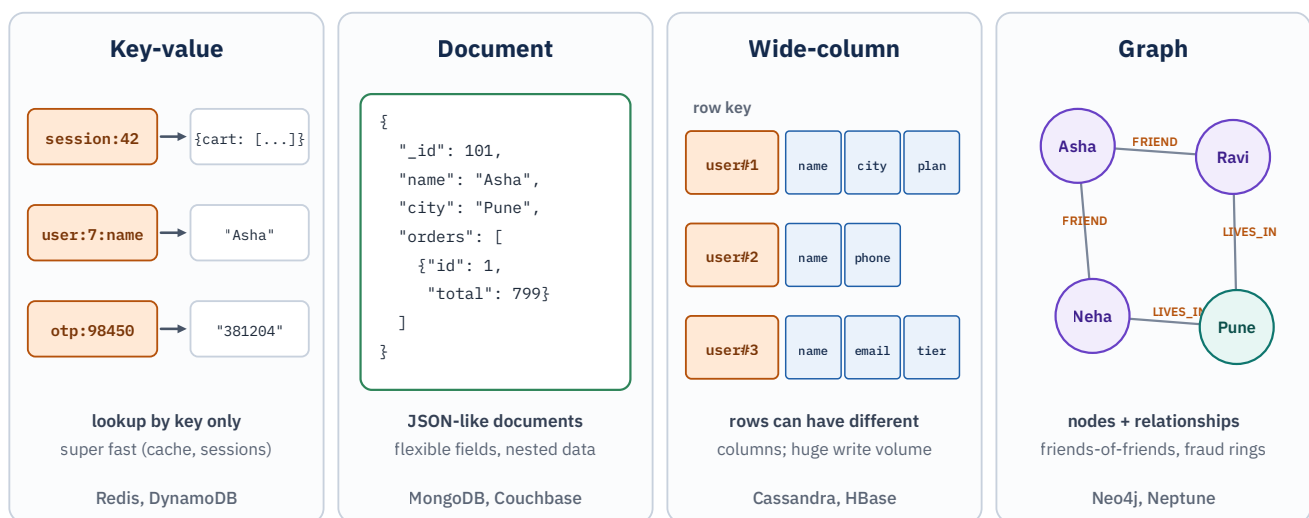


Figure 25. The four NoSQL families and the shape of the data each one stores.

Family	Data shape	Strengths	Weaknesses	Examples	Typical uses
Document	JSON-like documents in collections	flexible schema, nested data, natural fit for objects	complex joins and multi-document transactions are less natural	MongoDB, Couchbase, Firestore, Cosmos DB	catalogues, content, user profiles, mobile backends
Key-value	key → value	extremely fast, simple, scales easily	query only by key; no rich querying	Redis, DynamoDB, Memcached	caching, sessions, carts, rate limiting, leaderboards
Wide-column (column-family)	rows with flexible, sparse columns grouped in families; partitioned by key	massive write volumes, linear scale-out, multi-region	queries must follow the partition key; no joins	Cassandra, HBase, Bigtable, ScyllaDB	IoT, time-series, messaging, activity feeds
Graph	nodes and relationships (edges)	traversing relationships fast (friends-of-friends)	not built for bulk aggregation	Neo4j, Amazon Neptune	social networks, recommendations, fraud rings, knowledge graphs

Column-family vs columnar: two different ideas

These names sound alike but mean different things, and interviewers like to check:

- **Wide-column / column-family NoSQL** (Cassandra, HBase): an operational database where each *row* can have different columns, grouped into families and distributed by partition key. Built for fast writes and key-based reads.
- **Columnar storage** (Redshift, BigQuery, Snowflake, ClickHouse, Parquet files): an analytical storage layout that stores **each column's values together** on disk, instead of each row's values together.

THE SAME THREE ROWS, STORED TWO WAYS

Row-oriented (OLTP)	Column-oriented (OLAP)
[101, Aarav, Mumbai, 2843] [102, Isha, Pune, 3499] [103, Aarav, Mumbai, 8999]	order_id: [101, 102, 103] customer: [Aarav, Isha, Aarav] city: [Mumbai, Pune, Mumbai] amount: [2843, 3499, 8999]
fast to read or write one whole row (fetch order 102)	fast to scan one column for millions of rows (SUM(amount)); repeated values compress very well

SQL vs NoSQL

	SQL (relational)	NoSQL
Data model	tables, rows, columns	documents, key-value, wide-column, graph
Schema	fixed, defined up front (schema-on-write)	flexible, can vary per record
Relationships	foreign keys and joins	embedding, references, or application-side joins (graphs: native edges)
Query language	standard SQL	product-specific APIs and languages (MongoDB query language, CQL, Cypher)
Transactions	strong ACID, multi-row and multi-table	varies: often per-record atomicity; some support multi-document ACID (MongoDB)
Consistency	strong by default	often tunable; many default to eventual consistency in distributed setups
Scaling	traditionally vertical, plus read replicas (distributed SQL also scales out)	designed for horizontal scaling
Best for	structured data, complex queries, integrity-critical data	huge scale, flexible or fast-changing structure, simple access patterns
Examples	PostgreSQL, MySQL, Oracle, SQL Server	MongoDB, Redis, Cassandra, DynamoDB, Neo4j

How to choose: a decision guide

If the requirement is...	Lean towards	Because
Money, orders, bookings; correctness first	Relational (PostgreSQL)	ACID transactions, constraints, joins
Ad-hoc reporting and complex queries across entities	Relational, or a warehouse for analytics	SQL is built for this
Product catalogue with very different attributes per item	Document (MongoDB), or PostgreSQL JSONB	flexible schema, nested data
Sub-millisecond lookups, sessions, caching	Key-value (Redis)	in-memory speed
Millions of writes per second, time-series, multi-region	Wide-column (Cassandra)	linear scale-out, partitioned writes
Relationship-heavy questions ("people you may know")	Graph (Neo4j)	fast traversals
Unsure, small or medium app	Start with PostgreSQL	versatile; supports JSONB, full-text search and more

PostgreSQL can store documents too

The line between SQL and NoSQL is blurrier than it used to be. PostgreSQL's `JSONB` type stores binary JSON that can be indexed and queried:

```

CREATE TABLE product_docs (
    id INT PRIMARY KEY,
    doc JSONB NOT NULL
);

INSERT INTO product_docs VALUES
(1, '{"name": "ThinkPad E14", "category": "Laptop", "price": 62990,
    "specs": {"ram_gb": 16, "cpu": "Ryzen 5"}}'),
(2, '{"name": "Cotton Tee", "category": "Apparel", "price": 499,
    "sizes": ["S", "M", "L"], "colour": "navy"}'),
(3, '{"name": "Galaxy Buds", "category": "Audio", "price": 8999,
    "specs": {"battery_hrs": 6}}');

SELECT id,
       doc->>'name' AS name,
       (doc->>'price')::numeric AS price,
       doc->'specs'->>'ram_gb' AS ram_gb
FROM product_docs
WHERE doc ? 'specs'           -- documents that have a "specs" key
ORDER BY price DESC;

```

OUTPUT

id	name	price	ram_gb
1	ThinkPad E14	62990	16
3	Galaxy Buds	8999	NULL

2 rows

LINE BY LINE

Part	What it does
<code>doc JSONB</code>	a column holding a whole JSON document per row
<code>doc->>'name'</code>	read a key as text (<code>-></code> returns JSON, <code>->></code> returns text)
<code>(doc->>'price')::numeric</code>	cast the text to a number for sorting and maths
<code>doc->'specs' ->>'ram_gb'</code>	navigate into a nested object; NULL if the key is missing
<code>WHERE doc ? 'specs'</code>	the <code>?</code> operator tests whether a key exists

Containment queries such as `WHERE doc @> '{"category": "Apparel"}'` can use a **GIN index**. This hybrid is often the pragmatic answer: relational for core entities, JSONB for the flexible parts.

DISTRIBUTED SQL ("NEWSQL")

Google Spanner, CockroachDB, YugabyteDB and similar systems offer SQL and ACID transactions while scaling horizontally across regions. They show that "SQL cannot scale out" is no longer true; it is a question of trade-offs and cost.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"SQL databases store structured data in tables with a fixed schema, support joins and strong ACID transactions, so they're my default for business-critical data like orders and payments. NoSQL is a family of non-relational databases: document stores like MongoDB for flexible JSON data, key-value stores like Redis for very fast lookups and caching, wide-column stores like Cassandra for massive write volumes, and graph databases like Neo4j for relationship-heavy queries. They usually scale out horizontally and offer flexible schemas, sometimes with eventual consistency. I choose based on data shape, consistency needs, query patterns and scale, and many real systems use both."

COMMON TRAPS & MISTAKES

- "NoSQL is faster than SQL." It depends entirely on the workload and access pattern.
- "NoSQL has no schema, so no design." The schema moves into the application.
- "NoSQL never supports transactions." MongoDB, for example, supports multi-document ACID transactions.
- Confusing wide-column NoSQL (Cassandra) with columnar analytical storage (Redshift, Parquet).
- Choosing NoSQL "because it's modern" for data that is highly relational and integrity-critical.

INTERVIEW QUESTIONS

BASIC

Q1 **COMPARISON** What is the difference between SQL and NoSQL databases?

A SQL databases are relational: tables, fixed schema, SQL, joins, strong ACID. NoSQL databases are non-relational (document, key-value, wide-column, graph), with flexible schemas, horizontal scaling and varied consistency models.

Q2 **DEFINITION** Name the four main types of NoSQL databases with an example each.

A Document (MongoDB), key-value (Redis), wide-column (Cassandra), graph (Neo4j).

Q3 **DEFINITION** What is a key-value store used for?

A Very fast lookups by key: caching, sessions, shopping carts, rate limiting, leaderboards.

INTERMEDIATE**Q4** **COMPARISON** Wide-column store vs columnar database?

A A wide-column store (Cassandra) is an operational NoSQL database with flexible columns per row, partitioned by key. A columnar database (Redshift, BigQuery) stores each column contiguously for fast analytical scans and compression.

Q5 **WHY** When would you choose a graph database?

A When the main queries traverse relationships of variable depth (friends-of-friends, fraud rings, recommendations), which would need many expensive self-joins in SQL.

Q6 **HOW** Can a relational database store JSON?

A Yes. PostgreSQL's JSONB stores binary JSON that can be queried with operators (`->`, `->>`, `@>`, `?`) and indexed with GIN; MySQL and SQL Server also have JSON support.

SCENARIO-BASED**Q7** **SCENARIO** You need flexible schemas for a product catalogue. Would you use SQL or NoSQL?

A A document database such as MongoDB fits naturally, since each product can carry its own attributes. If the rest of the system is relational, PostgreSQL with a JSONB column for the variable attributes is a strong, simpler alternative.

Q8 **SCENARIO** Choose databases for a ride-sharing app.

A PostgreSQL for users, trips and payments (ACID); Redis for live driver locations and sessions; Cassandra or a time-series store for high-volume location history; a warehouse for analytics.

Q9 **DESIGN QUESTION** A bank asks whether to move its core ledger to a NoSQL database for scalability. What do you advise?

A Be cautious. A ledger needs strict ACID transactions, constraints and auditability. Prefer a relational or distributed SQL database, scale with partitioning, replicas or distributed SQL, and use NoSQL for peripheral workloads like caching or event history.

FOLLOW-UP / TRAP**Q10** **TRAP QUESTION** Does using MongoDB mean you can never do joins?

A No. MongoDB has `$lookup` in its aggregation pipeline; the point is that you design to need fewer joins.

Q11 **TRAP QUESTION** Is NoSQL always eventually consistent?

A No. Many NoSQL systems offer strong consistency options (MongoDB reads from the primary; DynamoDB strongly consistent reads; Cassandra quorum reads and writes).

Session 4.3 — NoSQL Fundamentals and the CAP Theorem

Why NoSQL appeared

Around the late 2000s, web companies hit limits with single-server relational databases: billions of users, constant schema changes and global traffic. NoSQL systems were designed to:

- **scale horizontally** across many cheap machines (sharding by key);
- **stay available** when machines or networks fail (replication);
- **accept flexible, evolving data** (JSON documents, sparse columns);
- **serve simple access patterns extremely fast** (lookup by key).

How distributed databases spread data

- **Partitioning / sharding:** each record's key decides which node stores it, for example `hash(user_id) mod N`. Smarter schemes (consistent hashing) avoid moving most data when nodes are added.
- **Replication:** each partition is copied to several nodes (often 3), so one node's failure loses nothing.

Once data is copied across machines connected by a network, you face the question the CAP theorem describes.

The CAP theorem

In a **distributed** data system, you would like three properties:

Letter	Property	Plain meaning
C	Consistency	every read sees the most recent write (all nodes agree)
A	Availability	every request to a working node gets a (non-error) response
P	Partition tolerance	the system keeps working even if the network splits nodes into groups that can't talk to each other

The theorem (Eric Brewer): when a network partition happens, a distributed system must choose between **consistency** and **availability**; it cannot guarantee both at that moment.

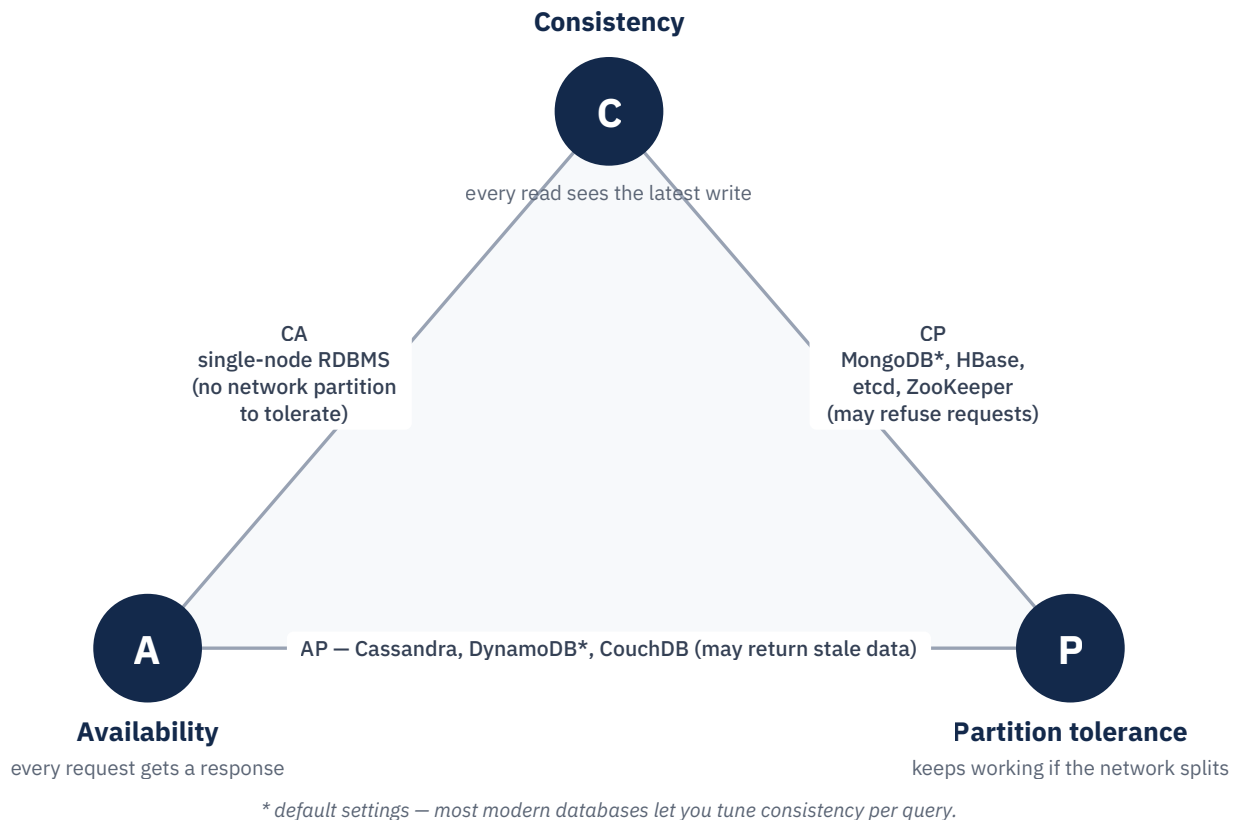


Figure 26. CAP: during a network partition, a distributed system chooses consistency (CP) or availability (AP).

A SIMPLE STORY

Two replicas of an account balance, one in Mumbai and one in Chennai. The network link between them breaks. A user in Chennai asks for the balance, which was just updated in Mumbai. - **CP choice:** Chennai refuses to answer (or returns an error) until it can confirm the latest value. Correct, but unavailable. - **AP choice:** Chennai answers with the value it has, which may be stale. Available, but possibly inconsistent.

The common misunderstanding: CAP is often phrased as "pick any two of three". In a real distributed system network partitions *will* happen, so P is not optional; the real choice is **C or A during a partition**. "CA" systems are essentially single-node databases that never face partitions.

Type	Behaviour during a partition	Examples (default settings)
CP	stays consistent; some requests fail or wait	MongoDB (writes and reads via the primary), HBase, etcd, ZooKeeper
AP	stays available; may return stale data, reconciles later	Cassandra, DynamoDB (default reads), CouchDB
CA	only possible without partitions	a single-node relational database

These labels describe *default* behaviour. Many modern databases let you tune consistency per operation.

PACELC

CAP talks only about partitions. **PACELC** adds: *if Partitioned, choose Availability or Consistency; Else (normal operation), choose Latency or Consistency*. Even without failures, waiting for replicas to agree costs time. Cassandra is PA/EL; a typical relational primary-standby setup is PC/EC.

Consistency models

Model	Guarantee	Example
Strong consistency	a read always returns the latest committed write	a bank balance after a transfer
Eventual consistency	if writes stop, all replicas <i>eventually</i> become identical; reads may be stale for a while	a social post's like count; DNS updates spreading worldwide
Read-your-writes	you always see your own updates, even if others may not yet	you change your profile picture and immediately see the new one
Causal consistency	related events are seen in order (a reply never appears before its question)	comment threads

Eventual consistency is acceptable when slightly stale data harms no one (likes, view counts, recommendations) and unacceptable when it causes real damage (double-selling the last seat, overdrawing an account).

TUNABLE CONSISTENCY WITH QUORUMS

Systems like Cassandra keep N copies of each record. A write waits for W replicas to confirm and a read asks R replicas. If $R + W > N$ (for example $N = 3, W = 2, R = 2$), every read overlaps with the latest write: strong consistency. Lower values give faster responses with weaker guarantees.

ACID vs BASE

Many NoSQL systems describe themselves with **BASE**:

ACID (typical relational)	BASE (typical distributed NoSQL)
Atomicity, Consistency, Isolation, Durability	Basically Available, Soft state, Eventually consistent
correctness first	availability and scale first
strong consistency	eventual consistency (often tunable)
pessimistic: prevent conflicts	optimistic: resolve conflicts later
bank transfers, orders	social feeds, product views, IoT telemetry

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"The CAP theorem says that in a distributed database, when a network partition happens, you have to choose between consistency, where every read returns the latest write, and availability, where every request still gets a response. Partition tolerance isn't really optional across a network, so the practical choice is CP or AP during a partition. MongoDB with default settings behaves as CP, preferring to reject writes rather than diverge, while Cassandra is AP and returns possibly stale data, then reconciles. Eventual consistency means replicas converge over time, which is fine for things like like-counts but not for bank balances."

COMMON TRAPS & MISTAKES

- "Pick any two of C, A and P." In distributed systems P is a given; the trade-off is C vs A during partitions.
- Confusing CAP consistency (replicas agree) with ACID consistency (constraints hold).
- Labelling a database permanently CP or AP. Behaviour often depends on configuration and per-query settings.
- Thinking eventual consistency means "inconsistent forever". It means replicas converge after a short delay.

INTERVIEW QUESTIONS**BASIC**

Q1 **DEFINITION** What is the CAP theorem?

A In a distributed system, during a network partition you can guarantee either consistency (latest data everywhere) or availability (every request answered), but not both.

Q2 **DEFINITION** What is eventual consistency?

A A model where replicas may be temporarily out of date, but if no new writes occur, they all converge to the same value.

Q3 **DEFINITION** What does BASE stand for?

A Basically Available, Soft state, Eventually consistent: the availability-first counterpart to ACID.

INTERMEDIATE

Q4 **COMPARISON** CP vs AP systems: give examples.

A CP systems (HBase, MongoDB with default primary reads, etcd) refuse or delay requests to stay consistent during partitions; AP systems (Cassandra, CouchDB, DynamoDB with default reads) keep answering with possibly stale data and reconcile later.

Q5 **WHY** Why can't a distributed database simply be "CA"?

A Because network partitions are unavoidable between machines. A system that cannot tolerate them stops working correctly when they occur; CA only really describes single-node systems.

Q6 **COMPARISON** ACID consistency vs CAP consistency?

A ACID consistency means transactions keep data valid according to rules and constraints. CAP consistency means all replicas return the same, most recent value.

Q7 **HOW** How does sharding work?

A Data is split by a shard key (for example hash(user_id)) so each node stores a subset; requests are routed to the node that owns the key.

SCENARIO-BASED

Q8 **SCENARIO** A social app shows like counts that differ slightly between users for a few seconds. Is this a bug?

A Probably not. It's eventual consistency, an acceptable trade-off for availability and speed in a non-critical feature.

Q9 **SCENARIO** Which consistency would you require for seat booking in a cinema app, and why?

A Strong consistency for the booking step (or a conditional, atomic write), otherwise two users could book the same seat. Browsing seat maps can tolerate slight staleness.

FOLLOW-UP / TRAP

Q10 **TRAP QUESTION** Is MongoDB CP or AP?

A With default settings (reads and writes through the primary, majority write concern), it behaves as CP. Reading from secondaries or relaxing write concerns moves it towards availability. Behaviour is configurable.

Q11 **TRAP QUESTION** Does CAP apply to a single PostgreSQL server?

A Not really. CAP is about distributed systems with replicas across a network. Once you add replicas, the trade-offs appear (e.g. synchronous vs asynchronous replication).

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

P1. Name the database family best suited to each: (a) user sessions, (b) a friends-of-friends query, (c) a product catalogue, (d) sensor readings at 1 million writes per second.

P2. What is the difference between a read replica and a standby?

P3. True or false: replication protects against accidental deletes.

LEVEL 2 – INTERVIEW APPLICATION

P4. Explain the CAP theorem using a two-city example in under a minute.

P5. Give two reasons a company might choose a managed database over running PostgreSQL on its own VM, and one reason not to.

LEVEL 3 – SCENARIO / PROBLEM SOLVING

P6. A food-delivery startup expects 10× traffic during festivals. Propose a database setup for orders, menus, sessions and analytics, with a reason for each choice.

P7. After moving reads to replicas, users complain that a new address "disappears" right after saving. Explain why and propose two fixes.

ANSWER / EXPLANATION

P1. (a) key-value (Redis), (b) graph (Neo4j), (c) document (MongoDB) or PostgreSQL JSONB, (d) wide-column (Cassandra) or a time-series database.

P2. A read replica serves read traffic (usually asynchronously updated) to reduce load. A standby exists for failover (often synchronously updated in another zone) and usually doesn't serve traffic until it is promoted.

P3. False. Replication copies the delete to every replica; you need backups or point-in-time recovery.

P4. "Imagine our data is replicated in Mumbai and Chennai and the network between them breaks. If a user in Chennai reads data that was just changed in Mumbai, the Chennai node can either refuse to answer until it can confirm the latest value, which is consistent but unavailable, or answer with what it has, which is available but possibly stale. CAP says that during such a partition you can't have both, so distributed databases choose CP or AP."

P5. For: automated backups, PITR and failover out of the box; less operational work (patching, monitoring). Against: less control (no OS or superuser access, some extensions unavailable) or higher cost at very large scale.

P6. Orders and payments: managed PostgreSQL with a Multi-AZ standby (ACID, failover) plus read replicas for order-history pages. Menus: MongoDB or PostgreSQL JSONB, because restaurant menus vary, plus a cache. Sessions and live rider location: Redis (fast, expiring keys). Analytics: a cloud warehouse such as BigQuery or Snowflake, loaded by CDC or ELT, so dashboards never touch the order database. For festivals: scale replicas and cache ahead of time, and load-test.

P7. Replication lag: the write went to the primary, but the next read hit an asynchronous replica that hadn't received it yet. Fixes: read-your-writes routing (send a user's reads to the primary for a short time after they write), read critical pages from the primary, or update the UI from the write response instead of re-reading immediately.

Session 4.4 — MongoDB: Core Concepts

HIGH PRIORITY MongoDB is the most widely used document database and the NoSQL system interviewers ask about most.

What is MongoDB?

MongoDB is a document database: instead of rows in tables, it stores **documents** (records that look like JSON) inside **collections**. A document can contain nested objects and arrays, so data that belongs together (an order and its items) can live together.

The building blocks

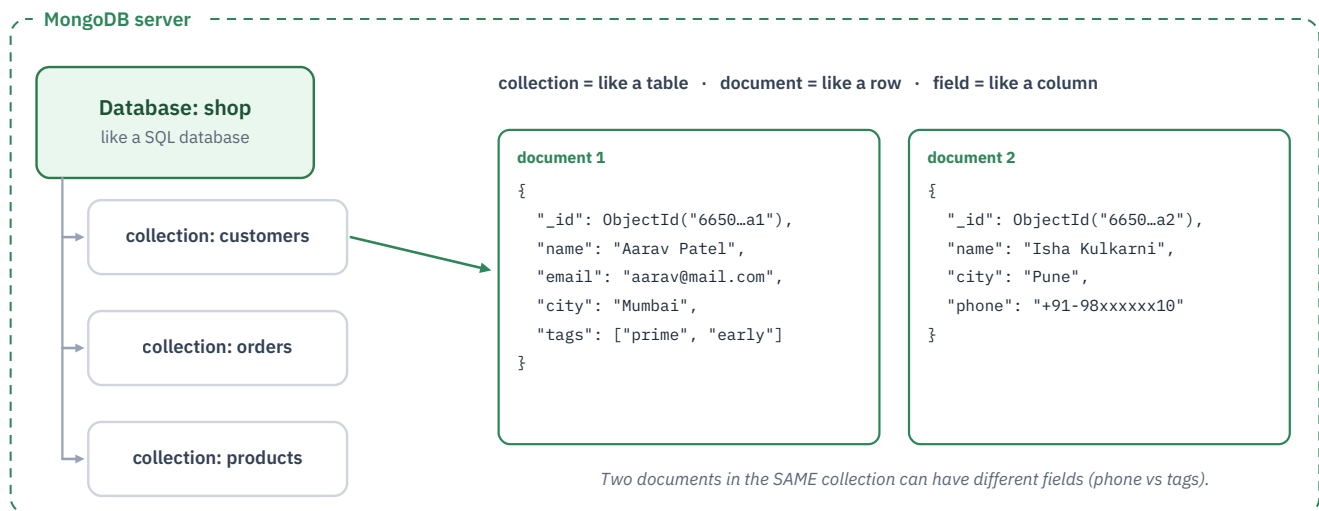


Figure 27. A MongoDB server holds databases; databases hold collections; collections hold documents made of fields.

MongoDB	Relational equivalent	Notes
database	database	e.g. shop
collection	table	e.g. customers ; no fixed schema by default
document	row	a JSON-like record, up to 16 MB
field	column	name , city ; can hold objects and arrays
<code>_id</code>	primary key	every document has a unique <code>_id</code> ; auto-generated as an <code>ObjectId</code> if you don't set one
embedded document / array	child table rows	stored <i>inside</i> the parent document
reference (<code>customer_id</code>)	foreign key	not enforced by the database
<code>\$lookup</code>	LEFT OUTER JOIN	an aggregation stage
index	index	B-tree indexes, plus text, geospatial and TTL indexes

JSON vs BSON

- **JSON** is the text format you read and write: `{"name": "Aarav", "city": "Mumbai"}`.

- **BSON (Binary JSON)** is how MongoDB actually **stores and transmits** documents. It is a binary encoding that is faster to scan and that supports **more data types** than JSON: `ObjectId`, `Date`, 32- and 64-bit integers, `Decimal128` (exact decimals for money), binary data and more.

The `_id` field and `ObjectId`

Every document needs a unique `_id`. If you don't provide one, MongoDB creates an **ObjectId**: a 12-byte value made of a **timestamp**, a random value and a counter. So ObjectIds are unique across machines without coordination, and they roughly sort by creation time.

```
{ "_id": ObjectId("665f1c2e8a4b7d0012ab34cd"), "name": "Aarav Patel" }
```

MONGODB SHELL (MONGOSH)

A tour of the shell (mongosh)

```
show dbs           // list databases
use shop           // switch to (or create on first write) the "shop" database
show collections   // list collections in the current database
db.customers.find() // read documents from the "customers" collection
```

MONGODB SHELL (MONGOSH)

A database or collection is created automatically the first time you insert into it.

Flexible schema, and how to tame it

Two documents in the same collection may have different fields: one customer has `phone`, another has `tags`. That is useful for evolving data, but production systems usually add **schema validation** so bad data is rejected:

```
db.createCollection("customers", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "city"],
      properties: {
        name: { bsonType: "string" },
        email: { bsonType: "string", pattern: "^.+@.+$" },
        city: { bsonType: "string" }
      }
    }
  }
})
```

MONGODB SHELL (MONGOSH)

Replica sets and sharding, in two lines each

- **Replica set**: a group of MongoDB servers holding the same data: one **primary** (takes writes) and several **secondaries** (copies). If the primary fails, the members **elect** a new primary automatically, which gives high availability.
- **Sharding**: splitting a collection across several replica sets by a **shard key**, so data and load spread horizontally.

MongoDB Atlas is MongoDB's managed cloud service: replica sets, backups and scaling handled for you, the same idea as RDS in Session 4.1.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"MongoDB is a document database. Data is stored as documents, JSON-like records, grouped into collections, which are roughly like tables. Documents can hold nested objects and arrays, so related data like an order and its items can be stored together and read in one go. Internally it stores documents as BSON, a binary form of JSON with extra types like ObjectId, dates and exact decimals. Every document has a unique `_id`. The schema is flexible, but we can add JSON-schema validation, and replica sets give high availability through automatic failover."

COMMON TRAPS & MISTAKES

- "MongoDB stores JSON." It stores **BSON**; JSON is the human-readable form.
- "MongoDB enforces foreign keys." It does not; references are just values your application manages.
- "Schemaless means anything goes." Use schema validation and consistent application models.
- Forgetting the 16 MB document limit when embedding ever-growing arrays.

INTERVIEW QUESTIONS

BASIC

Q1 DEFINITION What is MongoDB?

A A document-oriented NoSQL database that stores data as BSON documents in collections, with a flexible schema, rich queries, indexes, replication and sharding.

Q2 COMPARISON Collection vs table, document vs row?

A A collection groups documents like a table groups rows, but without a fixed schema; a document is a JSON-like record (with nested data) like a row.

Q3 DEFINITION What is BSON?

A Binary JSON: MongoDB's storage and wire format, which is faster to parse and supports extra types such as ObjectId, Date, Int64 and Decimal128.

INTERMEDIATE

Q4 DEFINITION What is an ObjectId?

A The default 12-byte `_id` value, containing a timestamp, a random value and a counter, so it is unique without central coordination and roughly time-ordered.

Q5 DEFINITION What is a replica set?

A A group of MongoDB nodes with the same data: one primary for writes and secondaries that replicate it, with automatic election of a new primary on failure.

Q6 HOW How can you enforce a structure in MongoDB?

A With schema validation (`$jsonSchema` validators) on the collection, plus consistent models in the application (for example Mongoose schemas in Node.js).

SCENARIO-BASED

Q7 SCENARIO A team says "MongoDB lets us skip data modeling." How do you respond?

A The flexible schema removes migrations, not design. We still need to plan documents around access patterns, decide embedding vs referencing, add indexes and validation, or we'll end up with inconsistent, slow data.

FOLLOW-UP / TRAP**Q8****TRAP QUESTION** Does MongoDB support ACID transactions?**A**

Yes. Single-document operations are always atomic, and multi-document ACID transactions are supported (replica sets since 4.0, sharded clusters since 4.2), although good document design reduces the need for them.

Session 4.5 — MongoDB: CRUD Operations

CRUD = **C**reate, **R**ead, **U**pdate, **D**elete. The examples use a small `shop` database. The results shown in comments were worked out by hand from this exact data.

ABOUT THESE EXAMPLES

This handbook's build environment could not run a MongoDB server, so unlike the SQL examples, the MongoDB outputs below were not executed automatically. The shell syntax was checked automatically, and each result was traced by hand against the sample data. Each example assumes the original data inserted below (as if run on a fresh copy), not the state left by the previous example.

Create: `insertOne` and `insertMany`

```
use shop
```

MONGODB SHELL (MONGOSH)

```
db.customers.insertOne({
  _id: 1,
  name: "Aarav Patel",
  email: "aarav@mail.com",
  city: "Mumbai",
  tags: ["prime"],
  signup_date: ISODate("2023-11-05")
})
// → { acknowledged: true, insertedId: 1 }

db.customers.insertMany([
  { _id: 2, name: "Isha Kulkarni", email: "isha@mail.com", city: "Pune", signup_date:
ISODate("2023-12-12") },
  { _id: 3, name: "Kabir Khan", email: "kabir@mail.com", city: "Delhi", tags: ["new"],
signup_date: ISODate("2024-01-03") },
  { _id: 4, name: "Diya Joshi", city: "Bengaluru", phone: "+91-98450-00000", signup_date:
ISODate("2024-01-20") }
])
// → { acknowledged: true, insertedIds: { '0': 2, '1': 3, '2': 4 } }
```

LINE BY LINE

Part	What it means
<code>use shop</code>	switch to the <code>shop</code> database (created on the first insert)
<code>db.customers.insertOne({...})</code>	insert one document into <code>customers</code> (the collection is created if needed)
<code>_id: 1</code>	we chose the ID; leave it out and MongoDB generates an ObjectId
<code>tags: ["prime"]</code>	an array field
<code>ISODate("2023-11-05")</code>	a real BSON Date, not a string
<code>insertMany([...])</code>	insert several documents in one call; note the different fields per document

We also need products and orders. Each order **embeds** its items:

```

db.products.insertMany([
  { _id: 1, name: "Wireless Mouse", category: "Electronics", price: 799, stock: 120 },
  { _id: 2, name: "Mechanical Keyboard", category: "Electronics", price: 3499, stock: 40 },
  { _id: 4, name: "Office Chair", category: "Furniture", price: 8999, stock: 15 },
  { _id: 5, name: "Standing Desk", category: "Furniture", price: 15999, stock: 5 },
  { _id: 6, name: "Notebook Pack", category: "Stationery", price: 249, stock: 500 }
])

db.orders.insertMany([
  { _id: 101, customer_id: 1, order_date: ISODate("2024-01-05"), status: "DELIVERED", total: 2843,
    items: [ { product_id: 1, name: "Wireless Mouse", qty: 2, price: 799 },
              { product_id: 6, name: "Notebook Pack", qty: 5, price: 249 } ] },
  { _id: 102, customer_id: 2, order_date: ISODate("2024-01-12"), status: "DELIVERED", total: 3499,
    items: [ { product_id: 2, name: "Mechanical Keyboard", qty: 1, price: 3499 } ] },
  { _id: 103, customer_id: 1, order_date: ISODate("2024-01-28"), status: "DELIVERED", total: 8999,
    items: [ { product_id: 4, name: "Office Chair", qty: 1, price: 8999 } ] },
  { _id: 104, customer_id: 3, order_date: ISODate("2024-02-02"), status: "CANCELLED", total:
15999,
    items: [ { product_id: 5, name: "Standing Desk", qty: 1, price: 15999 } ] },
  { _id: 115, customer_id: 1, order_date: ISODate("2024-04-19"), status: "PENDING", total: 3499,
    items: [ { product_id: 2, name: "Mechanical Keyboard", qty: 1, price: 3499 } ] }
])

```

Read: find() with filters, projection, sort and limit

The general form is `db.collection.find(filter, projection)`.

```

db.products.find({ category: "Electronics" })
// → Wireless Mouse, Mechanical Keyboard (equality filter)

db.products.find({ price: { $gt: 1000 } }, { name: 1, price: 1, _id: 0 })
// → { name: 'Mechanical Keyboard', price: 3499 }
//    { name: 'Office Chair', price: 8999 }
//    { name: 'Standing Desk', price: 15999 }

```

LINE BY LINE

Part	What it means
<code>{ category: "Electronics" }</code>	filter: like <code>WHERE category = 'Electronics'</code>
<code>{ price: { \$gt: 1000 } }</code>	an operator inside the filter: <code>price > 1000</code>
<code>{ name: 1, price: 1, _id: 0 }</code>	projection : return only name and price (<code>1</code> = include) and hide <code>_id</code> (<code>0</code> = exclude)

Operator type	Operators	Example
Comparison	<code>\$eq</code> , <code>\$ne</code> , <code>\$gt</code> , <code>\$gte</code> , <code>\$lt</code> , <code>\$lte</code> , <code>\$in</code> , <code>\$nin</code>	<code>{ city: { \$in: ["Pune", "Delhi"] } }</code>
Logical	<code>\$and</code> , <code>\$or</code> , <code>\$not</code> , <code>\$nor</code>	<code>{ \$or: [{a: 1}, {b: 2}] }</code>
Element	<code>\$exists</code> , <code>\$type</code>	<code>{ phone: { \$exists: true } }</code>
Array	<code>\$all</code> , <code>\$size</code> , <code>\$elemMatch</code>	<code>{ tags: { \$all: ["prime", "new"] } }</code>
Text pattern	<code>\$regex</code>	<code>{ name: { \$regex: "^A", \$options: "i" } }</code>

More reads:

```

// AND: two conditions in one filter object
db.products.find({ category: "Furniture", price: { $lt: 10000 } })
// → Office Chair

// OR
db.products.find({ $or: [ { category: "Stationery" }, { price: { $gte: 10000 } } ] })
// → Standing Desk, Notebook Pack

// IN
db.customers.find({ city: { $in: ["Pune", "Delhi"] } })
// → Isha Kulkarni, Kabir Khan

// field exists (flexible schema!)
db.customers.find({ phone: { $exists: true } })
// → Diya Joshi

// array contains a value
db.customers.find({ tags: "prime" })
// → Aarav Patel

// query inside embedded documents with dot notation
db.orders.find({ "items.product_id": 2 }, { _id: 1, status: 1 })
// → { _id: 102, status: 'DELIVERED' }, { _id: 115, status: 'PENDING' }

// sort, limit, skip (pagination)
db.products.find().sort({ price: -1 }).limit(2)
// → Standing Desk (15999), Office Chair (8999)

db.orders.countDocuments({ status: "DELIVERED" }) // → 3
db.products.distinct("category") // → [ 'Electronics', 'Furniture',
'Stationery' ]
db.customers.findOne({ email: "isha@mail.com" }) // → the single Isha document (or null)

```

Update: updateOne, updateMany, replaceOne

Updates take a **filter** (which documents) and an **update document** that uses **update operators**:

```

db.products.updateOne({ _id: 1 }, { $set: { price: 749 } })
// → { acknowledged: true, matchedCount: 1, modifiedCount: 1, upsertedCount: 0, ... }

db.products.updateMany({ category: "Electronics" }, { $inc: { stock: -1 } })
// → { acknowledged: true, matchedCount: 2, modifiedCount: 2, ... }

db.orders.updateMany({ status: "DELIVERED" }, { $set: { reviewed: false } })
// → matchedCount: 3, modifiedCount: 3 (adds a new field to three documents)

```

LINE BY LINE

Part	What it means
<code>updateOne({ _id: 1 }, ...)</code>	change at most one document matching the filter
<code>{ \$set: { price: 749 } }</code>	set (or add) the <code>price</code> field; other fields stay untouched
<code>updateMany({ category: "Electronics" }, ...)</code>	change every matching document
<code>{ \$inc: { stock: -1 } }</code>	add <code>-1</code> to <code>stock</code> atomically (no read-then-write race)
<code>matchedCount / modifiedCount</code>	how many documents matched, and how many actually changed

Update operator	Effect	Example
<code>\$set</code>	set or add fields	<code>{ \$set: { city: "Pune" } }</code>
<code>\$unset</code>	remove fields	<code>{ \$unset: { phone: "" } }</code>
<code>\$inc</code>	increment a number	<code>{ \$inc: { stock: -1 } }</code>
<code>\$push</code>	append to an array (creates it if missing)	<code>{ \$push: { tags: "prime" } }</code>
<code>\$addToSet</code>	append only if not already present	<code>{ \$addToSet: { tags: "prime" } }</code>
<code>\$pull</code>	remove matching values from an array	<code>{ \$pull: { tags: "new" } }</code>
<code>\$rename</code>	rename a field	<code>{ \$rename: { phone: "mobile" } }</code>

Upsert: update if found, insert otherwise:

```
db.products.updateOne(
  { _id: 9 },
  { $set: { name: "Laptop Stand", category: "Furniture", price: 1299 } },
  { upsert: true }
)
// → { matchedCount: 0, modifiedCount: 0, upsertedCount: 1, upsertedId: 9 }
```

MONGODB SHELL (MONGOSH)

replaceOne replaces the *entire* document (except `_id`) with a new one:

```
db.customers.replaceOne({ _id: 4 }, { name: "Diya Joshi", city: "Mysuru" })
// Diya's phone and signup_date are gone: only name and city remain.
```

MONGODB SHELL (MONGOSH)

Delete: deleteOne, deleteMany, drop

```
db.orders.deleteOne({ _id: 115 })
// → { acknowledged: true, deletedCount: 1 }

db.orders.deleteMany({ status: "CANCELLED" })
// → { acknowledged: true, deletedCount: 1 } (order 104)

db.orders.deleteMany({}) // ⚠ deletes EVERY document, like DELETE without WHERE
db.orders.drop()         // removes the whole collection and its indexes, like DROP TABLE
```

MONGODB SHELL (MONGOSH)

SQL to MongoDB translation

SQL	MongoDB
INSERT INTO customers VALUES (...)	db.customers.insertOne({...})
SELECT * FROM products WHERE category = 'Electronics'	db.products.find({ category: "Electronics" })
SELECT name, price FROM products WHERE price > 1000	db.products.find({ price: { \$gt: 1000 } }, { name: 1, price: 1, _id: 0 })
... WHERE city IN ('Pune', 'Delhi')	{ city: { \$in: ["Pune", "Delhi"] } }
... ORDER BY price DESC LIMIT 2	.sort({ price: -1 }).limit(2)
SELECT COUNT(*) FROM orders WHERE status = 'DELIVERED'	db.orders.countDocuments({ status: "DELIVERED" })
UPDATE products SET price = 749 WHERE id = 1	db.products.updateOne({ _id: 1 }, { \$set: { price: 749 } })
UPDATE ... WHERE category = 'Electronics' (many rows)	db.products.updateMany({ category: "Electronics" }, { ... })
DELETE FROM orders WHERE status = 'CANCELLED'	db.orders.deleteMany({ status: "CANCELLED" })
DROP TABLE orders	db.orders.drop()

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"In MongoDB, CRUD maps to insertOne and insertMany for create, find with a filter document and an optional projection for read, updateOne, updateMany and replaceOne for update, and deleteOne and deleteMany for delete. Filters use operators like \$gt, \$in and \$or, and dot notation reaches into embedded documents, for example items.product_id. Updates must use operators like \$set, \$inc or \$push, so only the named fields change; replaceOne swaps the whole document. Also, updateMany and deleteMany with an empty filter affect every document, so I double-check the filter first."

COMMON TRAPS & MISTAKES

- updateOne({_id: 1}, { price: 749 }) without \$set : modern drivers reject it ("Update document requires atomic operators"); replaceOne with a partial document silently drops the other fields.
- deleteMany({}) deletes everything.
- updateOne changes only the **first** match; use updateMany for all matches.
- Comparing dates stored as strings: use real ISODate values.
- Expecting find() results in insertion order without sort() .

INTERVIEW QUESTIONS

BASIC

Q1 **HOW** How do you insert multiple documents in MongoDB?

A db.collection.insertMany([{...}, {...}]), which returns the inserted IDs.

Q2 **COMPARISON** updateOne vs updateMany?

A updateOne modifies at most the first document matching the filter; updateMany modifies all matching documents.

Q3 **COMPARISON** **deleteOne vs deleteMany?**

A deleteOne removes the first matching document; deleteMany removes all matching documents (an empty filter `{}` removes all documents).

Q4 **DEFINITION** **What is a projection?**

A The second argument of `find()`, which chooses which fields to return: `{ name: 1, _id: 0 }`.

INTERMEDIATE**Q5** **COMPARISON** **\$set vs replaceOne?**

A `$set` changes only the named fields; `replaceOne` replaces the entire document except `_id`, dropping any fields not in the new document.

Q6 **COMPARISON** **\$push vs \$addToSet?**

A `$push` always appends (duplicates possible); `$addToSet` appends only if the value isn't already in the array.

Q7 **HOW** **How do you query a field inside an embedded document or array?**

A With dot notation, e.g. `{ "items.product_id": 2 }`, or `$elemMatch` when several conditions must match the same array element.

Q8 **DEFINITION** **What does { upsert: true } do?**

A If no document matches the filter, a new one is inserted, built from the filter and the update; otherwise the match is updated.

SCENARIO-BASED**Q9** **SCENARIO** **Increase the price of all Furniture products by 10%.**

A `db.products.updateMany({ category: "Furniture" }, { $mul: { price: 1.1 } })`.

Q10 **SCENARIO** **Two users buy the last unit of a product at the same time. How do you avoid overselling in MongoDB?**

A Use an atomic conditional update: `updateOne({ _id: 5, stock: { $gte: 1 } }, { $inc: { stock: -1 } })` and check `modifiedCount`. Only one request can succeed when stock is 1.

FOLLOW-UP / TRAP**Q11** **TRAP QUESTION** **What happens if you call `db.products.updateMany({}, { $set: { active: true } })`?**

A Every document in the collection gets `active: true`. An empty filter matches all documents.

Q12 **TRAP QUESTION** **Does `find()` return a list?**

A It returns a **cursor** that fetches documents in batches as you iterate (mongosh prints the first 20); `toArray()` loads everything into memory.

Session 4.6 — MongoDB: Indexing, Aggregation and Data Modeling

Indexes in MongoDB

The same idea as in SQL (Session 3.9): without an index MongoDB performs a **collection scan (COLLSCAN)**; with one it performs an **index scan (IXSCAN)**.

```

MONGODB SHELL (MONGOSH)
db.customers.createIndex({ email: 1 }, { unique: true }) // → 'email_1'
db.orders.createIndex({ customer_id: 1, order_date: -1 }) // →
'customer_id_1_order_date_-1' (compound)
db.products.createIndex({ name: "text" }) // text index for keyword search
db.sessions.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 }) // TTL: auto-delete
after 1 hour
db.orders.getIndexes() // list indexes

db.orders.find({ customer_id: 1 }).sort({ order_date: -1 }).explain("executionStats")
// winning plan uses stage 'IXSCAN' on customer_id_1_order_date_-1;
// totalDocsExamined: 3 (only Aarav's orders), instead of scanning every order

```

Index type	Use
Single field	{ email: 1 } (1 ascending, -1 descending)
Compound	{ customer_id: 1, order_date: -1 }; follows the leftmost-prefix rule like SQL
Multikey	created automatically when the field is an array (tags); indexes each element
Text	keyword search over string fields
TTL	documents expire automatically (sessions, OTPs, logs)
Unique / partial / sparse	enforce uniqueness; index only documents matching a condition

UNIQUE INDEXES AND MISSING FIELDS

A unique index treats a **missing** field as `null`, so only **one** document may lack the field. Diya has no email: creating the unique index works, but a second customer without an email would fail with a duplicate-key error on `null`. Use a **partial** index (`partialFilterExpression: { email: { $exists: true } }`) to enforce uniqueness only where the field exists. Compare PostgreSQL, where a **UNIQUE** column allows many **NULLs** (Session 2.3).

THE ESR RULE FOR COMPOUND INDEXES

Order the fields of a compound index as **E**quality, **S**ort, **R**ange: fields you match exactly first, then fields you sort by, then fields filtered by ranges. For `find({ customer_id: 1, total: { $gt: 1000 } }).sort({ order_date: -1 })`, the index { customer_id: 1, order_date: -1, total: 1 } follows ESR.

The aggregation pipeline

The **aggregation pipeline** is MongoDB's tool for grouping, joining and transforming data, the equivalent of SQL's **GROUP BY**, **JOIN** and computed columns. Documents flow through a sequence of **stages**, like items on an assembly line; each stage transforms the stream and passes it on.

Stage	Does	SQL equivalent
\$match	filter documents	WHERE (or HAVING, after \$group)
\$group	group and aggregate (\$sum , \$avg , \$min , \$max , \$push)	GROUP BY + aggregates
\$sort	order documents	ORDER BY
\$project / \$addFields	choose, rename or compute fields	SELECT list, computed columns
\$limit / \$skip	paginate	LIMIT / OFFSET
\$unwind	turn one document with an array into one document per element	like joining to a child table
\$lookup	pull matching documents from another collection	LEFT OUTER JOIN
\$count	count documents	COUNT(*)

Example 1: orders and revenue per customer (excluding cancelled orders)

```

db.orders.aggregate([
  { $match: { status: { $ne: "CANCELLED" } } },
  { $group: { _id: "$customer_id",
    orders: { $sum: 1 },
    revenue: { $sum: "$total" } } },
  { $sort: { revenue: -1 } }
])
// → { _id: 1, orders: 3, revenue: 15341 }    (2843 + 8999 + 3499)
//    { _id: 2, orders: 1, revenue: 3499 }

```

MONGODB SHELL (MONGOSH)

LINE BY LINE

Stage	What it does
\$match: { status: { \$ne: "CANCELLED" } }	keep orders that are not cancelled (filter early, so indexes help and less data flows on)
\$group: { _id: "\$customer_id", ... }	one output document per customer; _id is the group key; "\$total" means "the value of the total field"
orders: { \$sum: 1 }	add 1 per document, i.e. count orders
revenue: { \$sum: "\$total" }	add up the totals
\$sort: { revenue: -1 }	highest revenue first

Example 2: best-selling products, using \$unwind to open up the embedded items

```

db.orders.aggregate([
  { $match: { status: { $ne: "CANCELLED" } } },
  { $unwind: "$items" },
  { $group: { _id: "$items.name",
              units: { $sum: "$items.qty" },
              revenue: { $sum: { $multiply: ["$items.qty", "$items.price"] } } } },
  { $sort: { revenue: -1 } },
  { $limit: 3 }
])
// → { _id: 'Office Chair',      units: 1, revenue: 8999 }
//    { _id: 'Mechanical Keyboard', units: 2, revenue: 6998 }
//    { _id: 'Wireless Mouse',   units: 2, revenue: 1598 }

```

\$unwind turns order 101 (two items) into two documents, one per item, so the items can be grouped by product.

Example 3: joining with \$lookup

```

db.orders.aggregate([
  { $match: { _id: 101 } },
  { $lookup: { from: "customers", localField: "customer_id",
              foreignField: "_id", as: "customer" } },
  { $unwind: "$customer" },
  { $project: { total: 1, "customer.name": 1, "customer.city": 1 } }
])
// → { _id: 101, total: 2843, customer: { name: 'Aarav Patel', city: 'Mumbai' } }

```

\$lookup adds an array field (customer) holding the matching customer documents; \$unwind turns that one-element array into an object.

Modeling in MongoDB: embedding vs referencing, revisited

Session 2.8 introduced the idea; here is the MongoDB-specific checklist:

Question	If yes →
Is the data always read together with the parent (order + items)?	embed
Is it owned by the parent and bounded (a few addresses)?	embed
Is it shared by many parents (customer, product details)?	reference (optionally copy a snapshot)
Can it grow without limit (comments, logs, events)?	reference (separate collection)
Do you need to update it independently and often?	reference
Must several documents change atomically?	prefer embedding them in one document; otherwise use a multi-document transaction

MongoDB or PostgreSQL?

Choose MongoDB when...	Choose PostgreSQL when...
data is naturally hierarchical or varies per record	data is highly relational with many joins
access patterns are known and document-shaped	queries are ad hoc and change often (reporting)
you want flexible schema evolution	strong constraints and multi-table transactions are central
horizontal scaling via sharding is a primary need	a single primary plus replicas is enough (true for most apps)

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"Indexes in MongoDB work like in SQL: without one, a query does a collection scan; with one, an index scan, and `explain()` shows which. Compound indexes follow the leftmost-prefix idea; the ESR rule, equality then sort then range, is a good guide. For analytics MongoDB uses the aggregation pipeline, a series of stages: `$match` to filter, `$group` to aggregate, `$sort`, `$project`, `$unwind` to flatten arrays and `$lookup` to join another collection. It's basically GROUP BY and JOIN expressed as steps. For modeling, I embed what is owned and read together, like order items, and reference what is shared or unbounded, like customers or comments."

COMMON TRAPS & MISTAKES

- `$match` placed late in the pipeline: filter as early as possible so indexes can be used.
- Forgetting `$unwind` before grouping by array elements.
- `$lookup` everywhere: a sign that the model should embed more.
- Unique indexes treating missing fields as null.
- Indexing fields that are rarely queried (each index slows writes, as in SQL).

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What is the aggregation pipeline?

A A sequence of stages (`$match`, `$group`, `$sort`, `$project`, `$lookup`...) through which documents flow and are filtered, grouped and transformed, MongoDB's equivalent of SQL GROUP BY and JOIN queries.

Q2 **HOW** How do you create an index in MongoDB?

A `db.collection.createIndex({ field: 1 })` (ascending) or with options, e.g. `{ unique: true }`.

Q3 **DEFINITION** What does `$lookup` do?

A It performs a left-outer-join-like operation, adding matching documents from another collection as an array field.

INTERMEDIATE

Q4 **DEFINITION** What does `$unwind` do?

A It deconstructs an array field so each element produces its own document, typically before grouping by array elements.

Q5 **HOW** How do you check whether a query uses an index?

A `explain("executionStats")`: look for `IXSCAN` vs `COLLSCAN` and compare `totalDocsExamined` with the number of results.

Q6 DEFINITION What is a TTL index?

- A** An index on a date field with `expireAfterSeconds`, which automatically deletes documents after that time, e.g. sessions or OTPs.

Q7 DEFINITION What is a multikey index?

- A** An index on an array field; MongoDB indexes each element so queries on array values can use it.

SCENARIO-BASED**Q8 SQL PROBLEM Write the MongoDB equivalent of `SELECT city, COUNT(*) FROM customers GROUP BY city ORDER BY COUNT(*) DESC`.**

- A** `db.customers.aggregate([ { $group: { _id: "$city", count: { $sum: 1 } } }, { $sort: { count: -1 } } ]).`

Q9 DESIGN QUESTION Model a hospital's patients and their visits in MongoDB.

- A** `patients` documents with demographics and embedded allergies (small, owned); `visits` as a separate collection referencing `patient_id` (unbounded over time), indexed on `{ patient_id: 1, visit_date: -1 }`, possibly embedding a summary of the latest visit in the patient document for fast display.

FOLLOW-UP / TRAP**Q10 TRAP QUESTION Can you have two documents without an `email` field when `email` has a unique index?**

- A** Not with a normal unique index: both count as `null`, which is a duplicate. Use a partial index on documents where email exists.

Q11 TRAP QUESTION Is the aggregation pipeline order important?

- A** Yes. Stages run in order; filtering first (`$match`) reduces work and allows index use, and `$group` changes the document shape for later stages.

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

- P1.** Write a query to find all products cheaper than 1000, showing only their names.
- P2.** Increase the stock of product 4 by 10.
- P3.** Delete all orders with status "PENDING".

LEVEL 2 – INTERVIEW APPLICATION

- P4.** Add the tag "vip" to customer 2 without creating duplicates if it already exists.
- P5.** Write an aggregation that counts orders per status.
- P6.** Find orders that contain a Wireless Mouse with quantity at least 2.

LEVEL 3 – SCENARIO / PROBLEM SOLVING

- P7.** Write an aggregation that returns each customer's name with their number of non-cancelled orders, using `$lookup`.
- P8.** Searches by `{ category, price }` with a sort on price are slow. Propose an index and explain the field order.

ANSWER / EXPLANATION

P1. `db.products.find({ price: { $lt: 1000 } }, { name: 1, _id: 0 })` → Wireless Mouse, Notebook Pack.

P2. `db.products.updateOne({ _id: 4 }, { $inc: { stock: 10 } })`

P3. `db.orders.deleteMany({ status: "PENDING" })` → deletedCount: 1 (order 115).

P4. `db.customers.updateOne({ _id: 2 }, { $addToSet: { tags: "vip" } })`. `$addToSet` creates the array if it is missing and skips duplicates.

P5.

```
db.orders.aggregate([
  { $group: { _id: "$status", orders: { $sum: 1 } } },
  { $sort: { orders: -1 } }
])
// → { _id: 'DELIVERED', orders: 3 }, then CANCELLED: 1 and PENDING: 1
```

MONGODB SHELL (MONGOSH)

P6. Both conditions must hold for the **same** array element, so use `$elemMatch` :

```
db.orders.find({ items: { $elemMatch: { name: "Wireless Mouse", qty: { $gte: 2 } } } })
// → order 101
```

MONGODB SHELL (MONGOSH)

P7.

```
db.orders.aggregate([
  { $match: { status: { $ne: "CANCELLED" } } },
  { $group: { _id: "$customer_id", orders: { $sum: 1 } } },
  { $lookup: { from: "customers", localField: "_id", foreignField: "_id", as: "c" } },
  { $unwind: "$c" },
  { $project: { _id: 0, name: "$c.name", orders: 1 } },
  { $sort: { orders: -1 } }
])
// → { orders: 3, name: 'Aarav Patel' }, { orders: 1, name: 'Isha Kulkarni' }
```

MONGODB SHELL (MONGOSH)

P8. `db.products.createIndex({ category: 1, price: 1 })`. Equality field first (category), then the sort/range field (price), so MongoDB can jump to the category and read prices already in order, with no in-memory sort.

Session 4.7 — Data Storage Systems: Database, Warehouse, Lake, Lakehouse

MEDIUM PRIORITY "Data warehouse vs data lake" is a favourite comparison question for data roles, and "lakehouse" is now common in consulting conversations.

First, three kinds of data



Figure 28. Structured, semi-structured and unstructured data.

Kind	Meaning	Examples	Typical storage
Structured	fits a fixed table: known columns and types	orders, salaries, bank transactions	relational databases, warehouses
Semi-structured	has structure, but flexible and self-describing (keys and tags travel with the data)	JSON from APIs, XML, logs, CSV with varying columns	document databases, JSONB columns, data lakes
Unstructured	no predefined data model	images, PDFs, emails, audio, video, chat text	object storage, data lakes (often with metadata in a database)

Most enterprise data by volume is semi-structured or unstructured; most *business reporting* still runs on structured data. Modern platforms need to handle all three.

1. The operational database

The **database** that runs an application (OLTP, Module 1): PostgreSQL, MySQL, MongoDB. It is optimised for many small, fast, correct transactions on **current** data. It is the source of truth for the business process, but not the place for heavy analytics.

2. The data warehouse

A **data warehouse** (Session 2.7) is a central, integrated store of **cleaned, structured, historical** data, modelled for analysis (star schemas) and queried with SQL by BI tools.

- **Schema-on-write:** data is cleaned, validated and shaped *before* it is loaded.
- **Strengths:** fast and reliable SQL analytics, consistent business definitions, strong governance.
- **Limits:** expensive for huge raw data, poor at unstructured data, slower to add new sources (modelling comes first).
- **Examples:** Snowflake, BigQuery, Redshift, Azure Synapse, Teradata.

3. The data lake

A **data lake** stores **raw data of any type** (structured, semi-structured and unstructured) in its original format, in cheap **object storage** (Amazon S3, Azure Data Lake Storage, Google Cloud Storage).

- **Schema-on-read:** store first; decide the structure when you read the data.
- **Strengths:** very cheap at scale, keeps everything (useful for data science and future questions), handles any format.
- **Limits:** without governance it becomes a **data swamp**: nobody knows what is there, whether it is trustworthy, or who owns it. Classic lakes also lacked transactions, so concurrent writes and updates were painful.

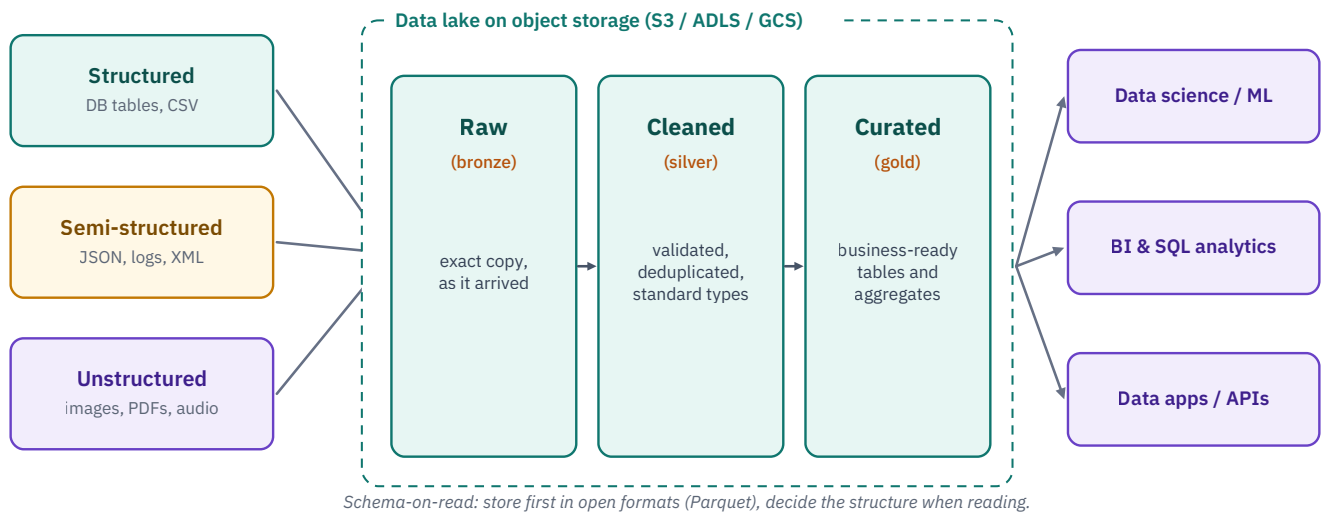


Figure 29. A data lake organised into raw, cleaned and curated zones (often called bronze, silver and gold).

Lakes are usually organised into **zones** (the "medallion" layout):

Zone	Also called	Contains
Raw	bronze	exact copies of source data, never modified, so jobs can be replayed
Cleaned	silver	validated, de-duplicated, standardised types and formats
Curated	gold	business-level tables and aggregates ready for BI and ML

FILE FORMATS IN A LAKE

CSV and **JSON** are simple but slow and large. **Parquet** and **ORC** are *columnar* formats: compressed and fast for analytics, because queries read only the columns they need. **Avro** is a row format with an embedded schema, popular for streaming. Parquet is the default choice for analytical data in lakes.

4. The data lakehouse

A **data lakehouse** combines the **cheap, open storage of a lake** with the **reliability and performance of a warehouse**, so BI, SQL analytics and machine learning all work on **one copy** of the data.

The key ingredient is an **open table format** layered on top of files in object storage:

- **Delta Lake** (Databricks), **Apache Iceberg**, **Apache Hudi**;
- they add **ACID transactions, schema enforcement and evolution, time travel** (query a table as of last Tuesday), upserts and deletes, and metadata for fast queries.

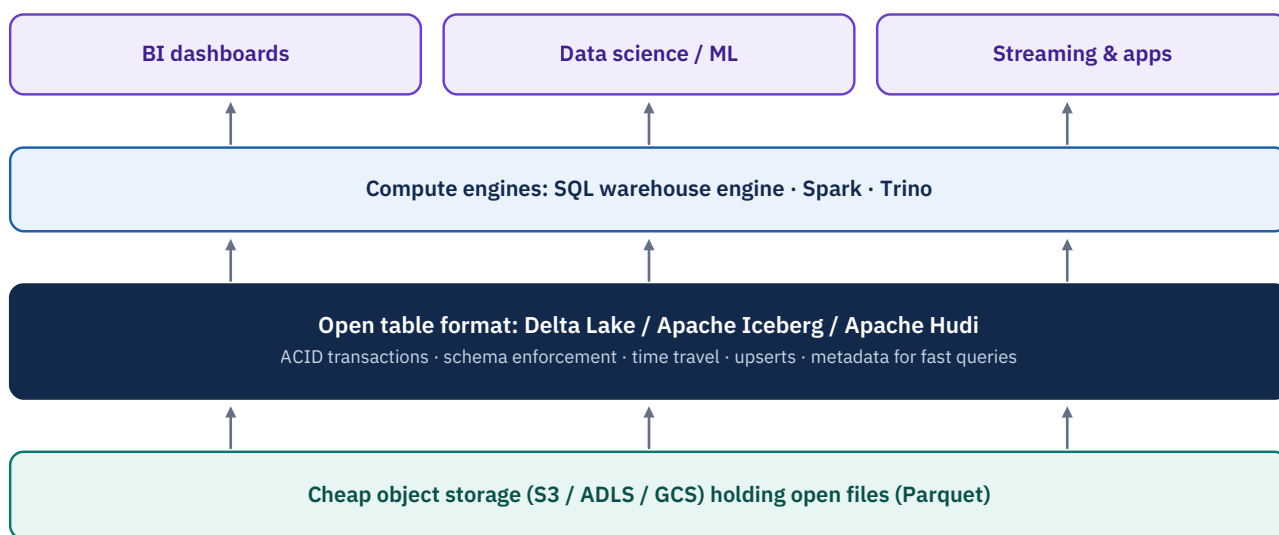


Figure 30. The lakehouse stack: open files in object storage, an open table format adding warehouse features, and engines serving BI and ML.

Why it exists: many companies ran a lake (for data science) *and* a warehouse (for BI), copying data between them with extra pipelines, cost and inconsistencies. The lakehouse aims to remove that duplication. Platforms: Databricks, Snowflake (with Iceberg tables), Microsoft Fabric (OneLake), Google BigLake.

The comparison tables interviewers ask for

	Database (OLTP)	Data warehouse
Purpose	run the application	analyse the business
Data	current, operational	historical, integrated from many sources
Design	normalized	dimensional (star/snowflake)
Workload	many small reads and writes	large scans and aggregations
Users	applications, customers	analysts, BI tools

	Data warehouse	Data lake
Data types	structured (some semi-structured)	everything: structured, semi-structured, unstructured
Schema	on write (model first)	on read (store first)
Data state	cleaned, curated	raw and refined, in zones
Storage cost	higher	low (object storage)
Users	business analysts, BI	data engineers, data scientists
Performance for SQL BI	excellent	historically weaker; depends on engine and format
Governance	strong by default	must be built deliberately, or it becomes a swamp
Typical tools	Snowflake, BigQuery, Redshift	S3/ADLS/GCS + Spark, Athena, Presto/Trino

	Data lake	Data lakehouse
Storage	object storage, open files	object storage, open files + open table format
Transactions	none (file level)	ACID tables
Updates and deletes	hard (rewrite files)	supported (MERGE, UPDATE, DELETE)
Schema	on read only	enforced, but able to evolve
BI performance	limited	warehouse-like
Time travel and versioning	manual	built in

WAREHOUSE VS LAKE VS LAKEHOUSE AT A GLANCE

	Warehouse	Lake	Lakehouse
Best at	trusted BI on structured data	cheap storage of everything, data science	one platform for BI + ML on open data
Schema	on write	on read	enforced + evolvable
ACID	yes	no	yes
Cost	\$\$\$	\$	\$--\$\$
Main risk	cost, rigidity	data swamp	platform maturity and skills

How to choose (a pragmatic view)

- **Mostly structured data and BI dashboards** → a cloud **warehouse** is the simplest and most reliable.
- **Large volumes of raw, varied data, plus ML** → a **lake** (with governance) or a **lakehouse**.
- **Want to avoid maintaining both a lake and a warehouse** → a **lakehouse**.
- Whatever you pick: a **catalogue** (what data exists), **ownership**, **quality checks** and **access control** matter more than the product.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"A database runs the application with current, transactional data. A data warehouse stores cleaned, structured, historical data modelled for analytics; it's schema-on-write, very good for BI, but more expensive and rigid. A data lake stores raw data of any type, including JSON, logs and images, cheaply in object storage, with schema-on-read; it's flexible and great for data science, but without governance it turns into a data swamp. A lakehouse adds an open table format like Delta Lake or Iceberg on top of the lake, giving ACID transactions, schema enforcement and time travel, so BI and ML can run on the same copy of the data instead of maintaining both a lake and a warehouse."

COMMON TRAPS & MISTAKES

- "A data lake is just a big database." It is file-based object storage with schema-on-read.
- "Lakes replace warehouses." Many organisations use both, or move to a lakehouse; warehouses remain excellent for BI.
- "Schema-on-read means no schema." The schema is applied when reading; someone still has to define it.
- Calling CSV files in S3 a lakehouse. Without an open table format adding ACID and metadata, it is a lake.
- Confusing **structured** with **SQL**: JSON stored in PostgreSQL is still semi-structured data.

INTERVIEW QUESTIONS**BASIC**

Q1 **COMPARISON** What is the difference between structured, semi-structured and unstructured data?

A Structured fits fixed tables (orders); semi-structured has flexible, self-describing structure (JSON, XML, logs); unstructured has no predefined model (images, PDFs, audio).

Q2 **DEFINITION** What is a data lake?

A A central repository that stores raw data of any type in its native format, usually in cheap object storage, and applies a schema when the data is read.

Q3 **COMPARISON** Data warehouse vs data lake?

A A warehouse stores cleaned, structured, modelled data (schema-on-write) for BI; a lake stores raw data of all types cheaply (schema-on-read) for flexible analysis and data science.

INTERMEDIATE

Q4 **DEFINITION** What is a data lakehouse?

A An architecture that adds warehouse features (ACID transactions, schema enforcement, performance, time travel) to data-lake storage through open table formats like Delta Lake or Iceberg, serving BI and ML from one copy of the data.

Q5 **COMPARISON** Schema-on-write vs schema-on-read?

A Schema-on-write validates and shapes data before storing it (warehouses, relational databases); schema-on-read stores raw data and applies structure at query time (lakes).

Q6 **DEFINITION** What is a data swamp and how do you prevent it?

A A lake full of undocumented, untrusted, ownerless data. Prevent it with a data catalogue and metadata, clear zones, ownership, quality checks, access control and retention rules.

Q7 **WHY** Why is Parquet preferred over CSV in data lakes?

A It is columnar and compressed, so queries read only the needed columns and much less data; it also stores a schema and types.

SCENARIO-BASED

Q8 **SCENARIO** A retailer wants sales dashboards and also to train ML models on clickstream logs and product images. What storage architecture would you propose?

A A lakehouse (or a lake plus warehouse): land raw clickstream, images and sales extracts in object storage (bronze), clean and join them (silver), and publish curated sales tables (gold) for BI, while data scientists use the same data for ML. Add a catalogue and access controls.

Q9 **SCENARIO** Analysts complain they can't find or trust data in the company's S3 lake. What would you do?

A Introduce a catalogue with documented datasets and owners, organise raw/cleaned/curated zones, add quality checks and freshness metrics, adopt a table format (Delta/Iceberg) for reliable tables, and publish a curated, governed layer for BI.

FOLLOW-UP / TRAP

Q10 **TRAP QUESTION** Is a lakehouse always better than a warehouse?

A No. For mostly structured data and BI, a cloud warehouse is often simpler and cheaper to run well. A lakehouse pays off with varied data, ML workloads, or when you want to avoid duplicating data across a lake and a warehouse.

Q11 TRAP QUESTION Is JSON stored in a relational table structured data?

- A** The *data* is semi-structured; storing it in a JSONB column doesn't change its nature. The table around it is structured.

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

- P1.** Classify as structured, semi-structured or unstructured: (a) a payroll table, (b) web server logs in JSON, (c) scanned invoices, (d) an XML feed of product prices.
- P2.** What do bronze, silver and gold zones contain?

LEVEL 2 – INTERVIEW APPLICATION

- P3.** In four lines, explain to a manager why a lakehouse could save money compared with "a lake plus a warehouse".
- P4.** Give two reasons Parquet makes analytical queries faster than CSV.

LEVEL 3 – SCENARIO / PROBLEM SOLVING

- P5.** A hospital wants to store patient records, MRI images, doctors' notes and billing data, and to run monthly financial reports plus ML research. Propose where each type of data lives and how it flows.

ANSWER / EXPLANATION

- P1.** (a) structured, (b) semi-structured, (c) unstructured, (d) semi-structured.
- P2.** Bronze: raw data exactly as received. Silver: cleaned, validated, de-duplicated, standardised data. Gold: curated business-level tables and aggregates for BI and ML.
- P3.** "Today we store data twice, in the lake and in the warehouse, and pay for pipelines that copy it between them. A lakehouse keeps one copy in cheap open storage and adds warehouse features on top, so BI and data science read the same tables. That cuts storage and pipeline costs and removes mismatches between the two copies."
- P4.** It is columnar, so a query reads only the columns it needs; and it is compressed, with stored statistics (min/max per block) that let engines skip irrelevant data.
- P5.** Patient records and billing (structured): an OLTP relational database for the hospital system, copied by CDC/ELT into a warehouse or lakehouse gold layer for monthly financial reporting. MRI images (unstructured): object storage, with metadata (patient, date, modality) in a database or catalogue. Doctors' notes (unstructured text): a lake, later processed (de-identified, NLP) into silver/gold tables for research. Governance across everything: access control, PII masking and de-identification for research, audit logs and a catalogue.

Session 4.8 — Module 4 Summary & Rapid Revision

MODULE 4 SUMMARY

MOST IMPORTANT CONCEPTS

- **Cloud database options:** self-managed VM → managed (RDS, Cloud SQL) → serverless; shared responsibility.
- **Scaling:** vertical vs horizontal; read replicas; partitioning (inside one DB) vs sharding (across servers).
- **Availability:** Multi-AZ standby, automatic failover, SLAs, RPO/RTO; backups + PITR (replication is not a backup).
- **SQL vs NoSQL:** relational (ACID, joins, fixed schema) vs document, key-value, wide-column and graph (flexible, scale-out); column-family ≠ columnar.
- **CAP:** during a partition choose consistency or availability (CP vs AP); eventual consistency; BASE vs ACID.
- **MongoDB:** database → collection → document → field; BSON; `_id` / ObjectId; CRUD (insertOne/Many, find, updateOne/Many with `$set`, `$inc`, `$push`, deleteOne/Many); indexes; aggregation pipeline (`$match`, `$group`, `$unwind`, `$lookup`); embed vs reference.
- **Storage:** structured / semi-structured / unstructured; database vs warehouse (schema-on-write) vs lake (schema-on-read) vs lakehouse (open table formats with ACID).

WHAT TO MEMORIZE

- CAP letters and the CP/AP examples; RPO = data loss, RTO = downtime.
- The MongoDB ↔ SQL terminology table.
- The warehouse / lake / lakehouse comparison.

WHAT TO UNDERSTAND

- How to choose a database from requirements (data shape, consistency, scale, queries).
- Why replication lag causes "my change disappeared".
- Why lakes become swamps and how lakehouses fix reliability.

MOST COMMON INTERVIEW QUESTIONS

- SQL vs NoSQL? · When MongoDB? · Explain CAP · What is eventual consistency? · updateOne vs updateMany · What is BSON? · Embedding vs referencing · Warehouse vs lake · What is a lakehouse? · What happens on failover?

COMMON MISTAKES

- "Replicas are backups." · "Pick 2 of 3 in CAP." · "NoSQL = no schema = no design." · `deleteMany({})` · update without `$set` · "lakes replace warehouses".

RAPID REVISION CHECKLIST

- | | |
|--|---|
| ☐ I can explain managed vs self-managed vs serverless databases | ☐ I can name the four NoSQL families with examples |
| ☐ I can draw the shared responsibility split | ☐ I can explain column-family vs columnar |
| ☐ I can compare vertical and horizontal scaling | ☐ I can explain CAP with an example |
| ☐ I can explain read replicas, partitioning and sharding | ☐ I can explain eventual consistency and BASE |
| ☐ I can explain sync vs async replication and failover | ☐ I know MongoDB's terminology and BSON |
| ☐ I can define RPO and RTO | ☐ I can write MongoDB CRUD commands |
| ☐ I know why replication is not a backup | ☐ I can write a simple aggregation pipeline |
| ☐ I can compare SQL and NoSQL in a table | ☐ I can decide between embedding and referencing |

☐ I can classify structured, semi-structured and unstructured data

☐ I can compare warehouse, lake and lakehouse

PART V • MODULE 5

Data Integration & APIs

Systems are useful only when they can talk to each other. This module explains APIs from first principles: client-server, HTTP, REST, JSON, status codes, authentication and authorization, then how to integrate with external APIs reliably (timeouts, retries, idempotency, webhooks) and how data integration works across a company.

COURSE COVERAGE

- ✓ What is an API, why APIs exist, client-server architecture, request/response † → Session 5.1
- ✓ REST, HTTP, GET, POST, PUT, PATCH, DELETE, HTTP status codes, JSON † → Session 5.2
- ✓ REST API example; pagination, rate limiting, API versioning † → Session 5.3
- ✓ Authentication and authorization † → Session 5.4
- ✓ API integration, external APIs, synchronous vs asynchronous communication, API failures, retries, idempotency † → Session 5.5
- ✓ Data integration † → Session 5.6
- ✓ Module quiz concepts → interview questions in each session and Session 5.7

Topic	Priority	Typical question
HTTP methods, PUT vs PATCH, idempotency	HIGH PRIORITY	"Difference between PUT and PATCH?" "Is POST idempotent?"
Status codes, 401 vs 403	HIGH PRIORITY	"What does 201 / 204 / 404 / 409 / 429 mean?"
REST principles, request flow	HIGH PRIORITY	"Explain what happens when the frontend calls GET / users."
Authentication vs authorization, JWT, OAuth	HIGH PRIORITY	"How does token-based authentication work?"
Retries, timeouts, idempotency keys, webhooks	MEDIUM PRIORITY	"A payment API times out. What do you do?"
Data integration methods, CDC, data quality	MEDIUM PRIORITY	"How would you integrate CRM data with the order system?"

PWC ANGLE

Reports from PwC software interviews mention HTTP status codes, HTTP methods, query vs path parameters, PUT vs PATCH, idempotency and REST (see the PwC Interview Orientation chapter). Integration is also everyday consulting work: connecting a client's ERP, CRM and web channels is a classic engagement. Be ready to explain both the protocol details and the business reason for an integration.

Session 5.1 — What Is an API? Client–Server and Request/Response

Simple meaning

An API (Application Programming Interface) is a contract that lets one piece of software ask another for data or an action, without knowing how the other side works inside. The contract says which requests you may send, in what format, and what responses you will get back.

THE WAITER AND THE MENU

In a restaurant you don't walk into the kitchen. You read the **menu** (the API documentation), tell the **waiter** (the API) what you want, and the waiter brings back your **food** (the response). The kitchen can change its staff and equipment, but as long as the menu stays the same, customers are unaffected.

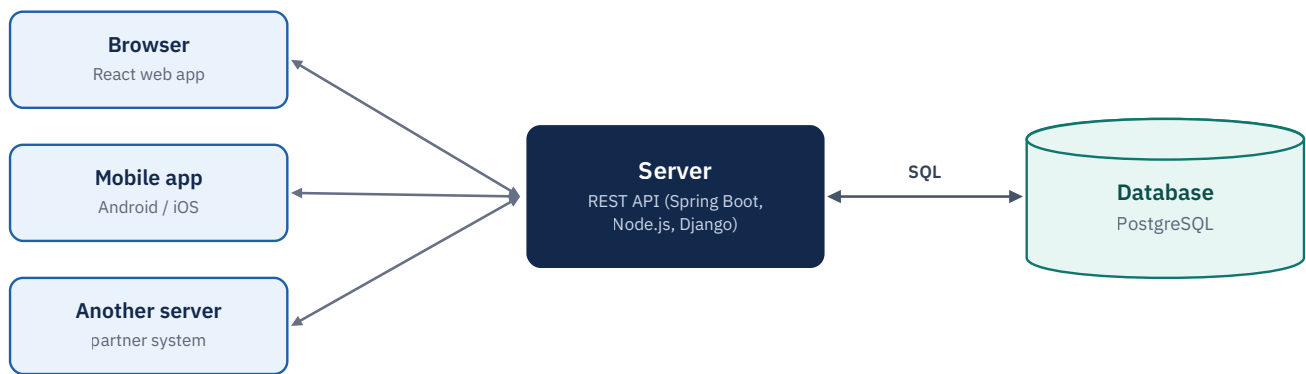
Why APIs exist

Reason	Example
Abstraction: hide complexity	a payment API hides card networks, banks, fraud checks
Reuse: build once, use everywhere	the same order API serves the website, the Android app and the iPhone app
Security: controlled access instead of direct database access	clients call <code>GET /orders/9001</code> ; they never see database credentials
Decoupling: teams change internals independently	the order service can switch databases without breaking the app
Integration: connect with partners and third parties	a food app calls a maps API for routes and an SMS API for OTPs
Business: expose capabilities as products	public APIs for weather, payments, KYC/identity verification

Kinds of APIs

- **Web APIs** over HTTP, the main topic here: **REST** (most common), **SOAP** (XML, common in older enterprise and banking systems), **GraphQL** (the client asks for exactly the fields it needs) and **gRPC** (fast, binary, service-to-service).
- **Library/SDK APIs:** the functions a library exposes, like Java's `List` interface or a payment SDK.
- **By audience:** **private** (internal, between your own services), **partner** (shared with selected companies) and **public** (anyone with a key).

Client–server architecture



Clients send requests; the server does the work and sends responses.

The client never talks to the database directly.

Figure 31. Clients send requests to a server; the server does the work, talks to the database and returns responses.

- The **client** (browser, mobile app or another server) **starts** the conversation and asks for something.
- The **server waits** for requests, applies business logic and security, talks to the database, and responds.
- Many clients share one server, and the server is the only component allowed to touch the database.
- REST servers are usually **stateless**: each request carries everything needed to process it (for example the auth token), so any server instance can handle any request, which makes scaling easy.

The request/response cycle

1. The client builds a **request**: an HTTP **method** (GET, POST...), a **URL**, **headers** (metadata such as the auth token and content type) and optionally a **body** (data, usually JSON).
2. The request travels over the network (HTTPS) to the server.
3. The server **authenticates** the caller, **validates** the input, runs the **business logic** and reads or writes the **database**.
4. The server returns a **response**: a **status code** (200, 404...), **headers** and a **body** (usually JSON).
5. The client reads the status first, then uses the body (show the data, or show an error).

EVERYDAY EXAMPLE: PAYING WITH A UPI APP

You tap "Pay ₹500". The app (client) sends an API request to its backend; the backend calls the payment network's APIs; the bank's systems debit your account; each hop replies with a status; finally your app shows "Payment successful". Every arrow in that chain is an API call with a request and a response.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"An API is a contract that lets one system request data or actions from another without knowing its internals, like a waiter taking your order to the kitchen. In client–server architecture, the client, say a mobile app, sends an HTTP request with a method, URL, headers and maybe a JSON body; the server authenticates and validates it, runs the business logic, talks to the database, and sends back a response with a status code and usually a JSON body. APIs give us reuse, security, decoupling and integration: one backend can serve the website, the mobile apps and partner systems."

COMMON TRAPS & MISTAKES

- Saying "an API is a URL". A URL identifies a resource; the API is the whole contract (operations, formats, rules).
- Thinking clients can or should connect to the database directly.
- Thinking "REST" and "API" mean the same thing. REST is one style of web API.

INTERVIEW QUESTIONS**BASIC****Q1** **DEFINITION** What is an API?

A An Application Programming Interface: a defined contract for how software components request data or actions from each other, for example over HTTP with JSON.

Q2 **DEFINITION** What is client-server architecture?

A A model where clients send requests and servers process them and return responses; the server centralises business logic, security and data access.

Q3 **WHY** Why do we need APIs?

A To expose functionality safely and reusably, decouple systems so they can change independently, and integrate with other applications and partners.

INTERMEDIATE**Q4** **DEFINITION** What does "stateless" mean for a REST server?

A The server keeps no client session between requests; each request contains all the information needed (such as the auth token), so any server instance can handle it.

Q5 **COMPARISON** REST vs SOAP vs GraphQL in one line each?

A REST: resources plus HTTP methods, usually JSON, simple and common. SOAP: XML envelopes with strict contracts (WSDL), common in older enterprise systems. GraphQL: one endpoint where clients specify exactly which fields they want.

SCENARIO-BASED**Q6** **SCENARIO** Your company has a web app and is launching mobile apps. Why build an API layer instead of letting each app talk to the database?

A One API centralises business rules and security, keeps database credentials off devices, lets all clients share the same logic, and lets the backend evolve without breaking installed apps.

FOLLOW-UP / TRAP**Q7** **TRAP QUESTION** Can one server be a client too?

A Yes. A backend is a server to the app and a client when it calls a payment gateway or another microservice.

Session 5.2 — HTTP and REST Fundamentals

HTTP in one paragraph

HTTP (HyperText Transfer Protocol) is the request/response protocol of the web. It is **stateless** (each request is independent) and text-based in its classic form. **HTTPS** is HTTP over **TLS** encryption, so nobody on the network can read or alter requests and responses. Every API carrying tokens or personal data must use HTTPS.

Anatomy of a request and a response

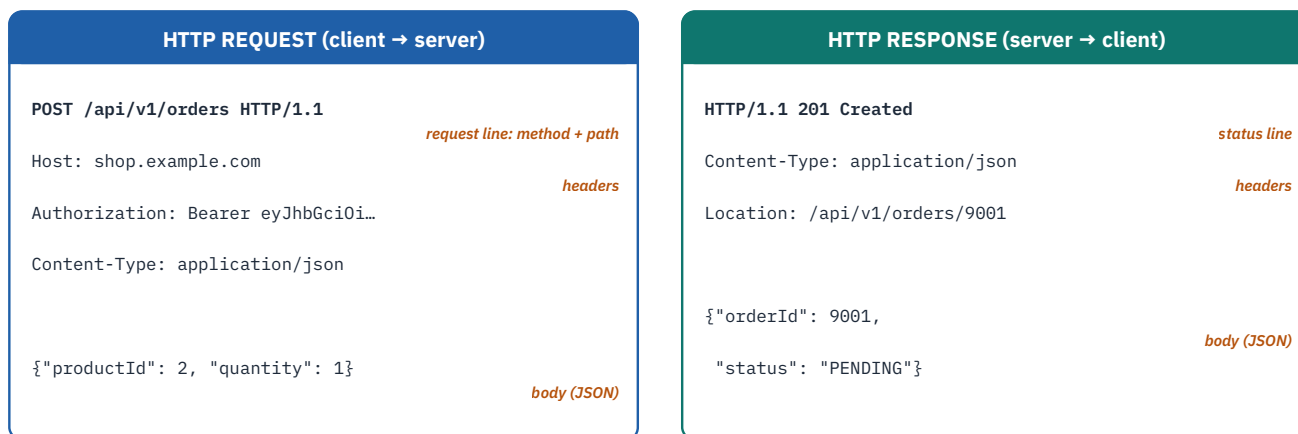


Figure 32. An HTTP request (method, path, headers, body) and its response (status line, headers, body).



LINE BY LINE

Line	Meaning
POST /api/v1/orders HTTP/1.1	method (create), path of the resource collection, protocol version
Host: shop.example.com	which server the request is for
Authorization: Bearer ...	the caller's token, proving who they are (Session 5.4)
Content-Type: application/json	the body is JSON
Accept: application/json	the client wants JSON back
blank line, then {"productId": 2, ...}	the request body
HTTP/1.1 201 Created	status: a new resource was created
Location: /api/v1/orders/9001	where the new resource lives
{"orderId": 9001, ...}	the response body describing the new order

Anatomy of a URL, and path vs query parameters

`https://api.shop.com/v1/customers/42/orders?status=PENDING&page=2`

Part	Value	Meaning
Scheme	https	protocol (HTTP over TLS)
Host	api.shop.com	the server
Path	/v1/customers/42/orders	the resource: orders of customer 42, API version 1
Path parameter	42	identifies a specific resource
Query string	?status=PENDING&page=2	filters, sorts, paginates the result

	Path parameter	Query parameter
Purpose	identify which resource	refine how to return it: filter, sort, page, search
Example	/customers/42	/customers?city=Pune&sort=name&page=2
Required?	yes, part of the resource's address	usually optional
Rule of thumb	"the thing"	"options about the thing"

The HTTP methods

Method	Purpose	Request body	Safe?	Idempotent?	Typical success code
GET	read a resource or a list	no	yes	yes	200 OK
POST	create a resource / trigger an action	yes	no	no	201 Created
PUT	replace a resource completely (or create it at a known URL)	yes (full representation)	no	yes	200 OK / 204 No Content
PATCH	update part of a resource	yes (only the changes)	no	not guaranteed	200 OK / 204 No Content
DELETE	delete a resource	usually no	no	yes	204 No Content / 200 OK
HEAD	like GET but headers only	no	yes	yes	200 OK
OPTIONS	which methods are allowed (used in CORS preflight)	no	yes	yes	204 / 200

- **Safe** means the request does not change anything on the server (read-only).
- **Idempotent** means sending the **same request once or ten times has the same effect** on the server's state.
 DELETE /orders/9001 twice still leaves order 9001 deleted (the second call may return 404, but the *state* is the same). POST /orders twice creates **two** orders, so POST is not idempotent.

CRUD	HTTP	Example
Create	POST	POST /customers
Read	GET	GET /customers/42
Update (full)	PUT	PUT /customers/42
Update (partial)	PATCH	PATCH /customers/42
Delete	DELETE	DELETE /customers/42

PUT vs PATCH

MUST KNOW Suppose customer 42 is currently:

```
{"id": 42, "name": "Isha Kulkarni", "email": "isha@mail.com", "city": "Pune"}
```

JSON

PUT /customers/42 sends the **complete new version**. Fields you leave out may be cleared:

```
{"name": "Isha Kulkarni", "email": "isha.k@mail.com", "city": "Pune"}
```

JSON

PATCH /customers/42 sends **only what changes**:

```
{"email": "isha.k@mail.com"}
```

JSON

	PUT	PATCH
Sends	the whole resource	only the fields to change
Missing fields	treated as removed or reset (replace semantics)	left unchanged
Idempotent	yes: same full document each time	not necessarily (<code>{ "op": "increment", "stock": 1 }</code> is not), though simple field sets usually are
Payload size	larger	smaller
Typical use	"save the whole form"	"change just the email"

HTTP status codes

Class	Meaning
1xx	informational (rarely seen in APIs)
2xx	success
3xx	redirection (the resource is elsewhere, or not modified)
4xx	client error : the request is wrong; fix it before retrying
5xx	server error : the server failed; retrying later may work

Code	Name	When to use it
200	OK	successful GET, PUT or PATCH with a response body
201	Created	POST created a resource (add a <code>Location</code> header)
202	Accepted	request accepted for asynchronous processing (job started)
204	No Content	success with no body (often DELETE)
301 / 302	Moved Permanently / Found	redirects
304	Not Modified	cached copy is still valid (ETag matched)
400	Bad Request	malformed request, invalid JSON, missing field
401	Unauthorized	not authenticated : missing or invalid credentials or token
403	Forbidden	authenticated but not allowed
404	Not Found	the resource does not exist
405	Method Not Allowed	e.g. DELETE on a read-only resource
409	Conflict	state conflict: duplicate email, version mismatch
415	Unsupported Media Type	sent XML where JSON was expected
422	Unprocessable Content	well-formed but semantically invalid (quantity = -5)
429	Too Many Requests	rate limit exceeded; see <code>Retry-After</code>
500	Internal Server Error	unexpected server failure (a bug)
502	Bad Gateway	a gateway or proxy got a bad response from upstream
503	Service Unavailable	overloaded or down for maintenance; retry later
504	Gateway Timeout	the upstream service did not answer in time

401 VS 403

401 Unauthorized really means **unauthenticated**: "I don't know who you are" (no token, or an expired or invalid one). **403 Forbidden** means "I know who you are, but you are not allowed to do this". A logged-in customer trying to delete a product gets 403.

JSON: the language of APIs

JSON (JavaScript Object Notation) is a lightweight text format for structured data.

```
{
  "orderId": 9001,
  "customer": {"id": 42, "name": "Isha Kulkarni"},
  "items": [
    {"product": "Mechanical Keyboard", "qty": 1, "price": 3499.00}
  ],
  "paid": true,
  "couponCode": null
}
```

JSON

- Six value types: **string** (double quotes only), **number**, **boolean** (`true` / `false`), **null**, **object** `{ }` and **array** `[ ]`.
- Keys must be double-quoted strings; no trailing commas; no comments.
- JSON has no date type: dates travel as strings, ideally ISO 8601 (`"2024-04-19T19:42:00Z"`).
- Money: send exact decimal values as strings or in the smallest unit (paise), to avoid floating-point surprises.

	JSON	XML
Syntax	<code>{"name": "Isha"}</code>	<code><name>Isha</name></code>
Size	smaller	larger (closing tags)
Readability	easy	verbose
Data types	string, number, boolean, null, object, array	text (types via schemas)
Typical use	REST APIs, web and mobile	SOAP, older enterprise and banking integrations, documents

REST: the architectural style

REST (Representational State Transfer) is a style for designing web APIs around **resources**. Its main constraints, in plain words:

Constraint	Meaning
Client–server	UI and data/logic are separated
Stateless	every request carries all the context it needs; no server-side session required
Uniform interface	resources identified by URLs, manipulated with standard HTTP methods, represented as JSON (or XML)
Cacheable	responses say whether they can be cached (<code>Cache-Control</code> , <code>ETag</code>)
Layered system	clients don't know whether they talk to the real server or a gateway or cache
(optional) HATEOAS	responses include links to related actions

Resource naming best practices:

Good	Avoid	Why
GET /customers	GET /getAllCustomers	the method already says "get"; use nouns
GET /customers/42/orders	GET /orders?customer=42 <i>as the only way</i>	nesting shows ownership (both are acceptable; be consistent)
POST /orders	POST /createOrder	no verbs in paths
/order-items (lowercase, hyphens)	/OrderItems	consistency, readability
plural collections: /products/7	mixing /product/7 and /products	predictable URLs

For actions that are not simple CRUD, use a sub-resource: POST /orders/9001/cancellation or POST /payments/55/refunds .

REST VS GRAPHQL VS GRPC

GraphQL exposes one endpoint where the client sends a query naming exactly the fields it wants, which avoids over-fetching and multiple round trips, at the cost of more complex caching and security. **gRPC** uses HTTP/2 and binary Protocol Buffers for fast, strongly typed service-to-service calls, popular inside microservice systems. REST remains the default for public and partner APIs because of its simplicity.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"REST is an architectural style where everything is a resource identified by a URL, like /customers/42, and we act on resources with standard HTTP methods: GET to read, POST to create, PUT to replace, PATCH to partially update and DELETE to remove. It's stateless, so every request carries its own context, such as the auth token. The server answers with a status code: 2xx for success, 4xx for client errors like 400, 401, 403 or 404, and 5xx for server errors. The body is usually JSON. GET, PUT and DELETE are idempotent, meaning repeating them has the same effect, while POST is not."

COMMON TRAPS & MISTAKES

- "PUT and PATCH are the same." PUT replaces the whole resource; PATCH changes parts.
- "POST is idempotent." It isn't: two POSTs usually create two resources (see idempotency keys in Session 5.5).
- Returning 200 with {"error": "not found"} instead of a proper 404.
- Confusing 401 (who are you?) with 403 (you're not allowed).
- Verbs in URLs (/getUsers) and inconsistent naming.
- Sending sensitive data in query strings (they end up in logs and browser history).

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What are the main HTTP methods?

A GET (read), POST (create), PUT (replace), PATCH (partial update), DELETE (remove), plus HEAD and OPTIONS.

Q2 COMPARISON PUT vs PATCH?

A PUT sends the complete resource and replaces it (idempotent); PATCH sends only the fields to change and leaves the rest untouched.

Q3 DEFINITION What do 2xx, 4xx and 5xx status codes mean?

A 2xx means success, 4xx means the client sent a bad or unauthorized request, and 5xx means the server failed to process a valid request.

Q4 COMPARISON Path parameter vs query parameter?

A A path parameter identifies a specific resource (`/customers/42`); query parameters filter, sort or paginate (`/customers?city=Pune&page=2`).

INTERMEDIATE**Q5 DEFINITION What does idempotent mean? Which methods are idempotent?**

A Repeating the same request has the same effect on server state as sending it once. GET, PUT, DELETE, HEAD and OPTIONS are idempotent; POST is not; PATCH may or may not be.

Q6 COMPARISON 401 vs 403?

A 401: the request lacks valid authentication (who are you?). 403: the user is authenticated but lacks permission for this action.

Q7 DEFINITION What makes an API RESTful?

A Resources addressed by URLs, standard HTTP methods with correct semantics, stateless requests, representations such as JSON, meaningful status codes, and cacheability.

Q8 HOW Which status code should a successful POST return, and what header?

A 201 Created, with a `Location` header pointing to the new resource (and usually the created resource in the body).

Q9 COMPARISON 400 vs 422?

A 400 means the request is malformed (invalid JSON, wrong types). 422 means it is well-formed but fails business validation (quantity must be positive).

SCENARIO-BASED**Q10 DESIGN QUESTION Design REST endpoints for a library: books, members and loans.**

A GET/POST `/books`, GET/PUT/PATCH/DELETE `/books/{id}`, GET/POST `/members`, GET `/members/{id}/loans`, POST `/loans` (borrow), POST `/loans/{id}/return` or PATCH `/loans/{id}` with `return_date`; filtering like GET `/books?author=...&available=true&page=2`.

Q11 SCENARIO A client sends DELETE `/orders/9001` twice because of a network retry. What should happen?

A The first call deletes the order (204); the second finds nothing and can return 404 (or 204). Either way the state is unchanged, because DELETE is idempotent.

Q12 SCENARIO Users sometimes get 503 errors during sales. What does that tell you, and what should clients do?

A The service is overloaded or temporarily unavailable, so the server side needs scaling. Clients should retry later with exponential backoff, respecting any `Retry-After` header.

FOLLOW-UP / TRAP

Q13 **TRAP QUESTION** Can a GET request have a body?

A The HTTP spec doesn't forbid it, but servers and proxies may ignore it, and GET semantics don't define it. Put filters in query parameters, or use POST for complex searches.

Q14 **TRAP QUESTION** If PATCH can be idempotent, why is it "not guaranteed"?

A Because it depends on the patch: "set email = x" is idempotent, but "append item" or "increment stock by 1" changes state again on every repeat.

Session 5.3 — Designing and Consuming REST APIs

One request, end to end

The brief's example, *Frontend* → *GET /users* → *Backend* → *Database* → *JSON response*, drawn as a sequence of steps:

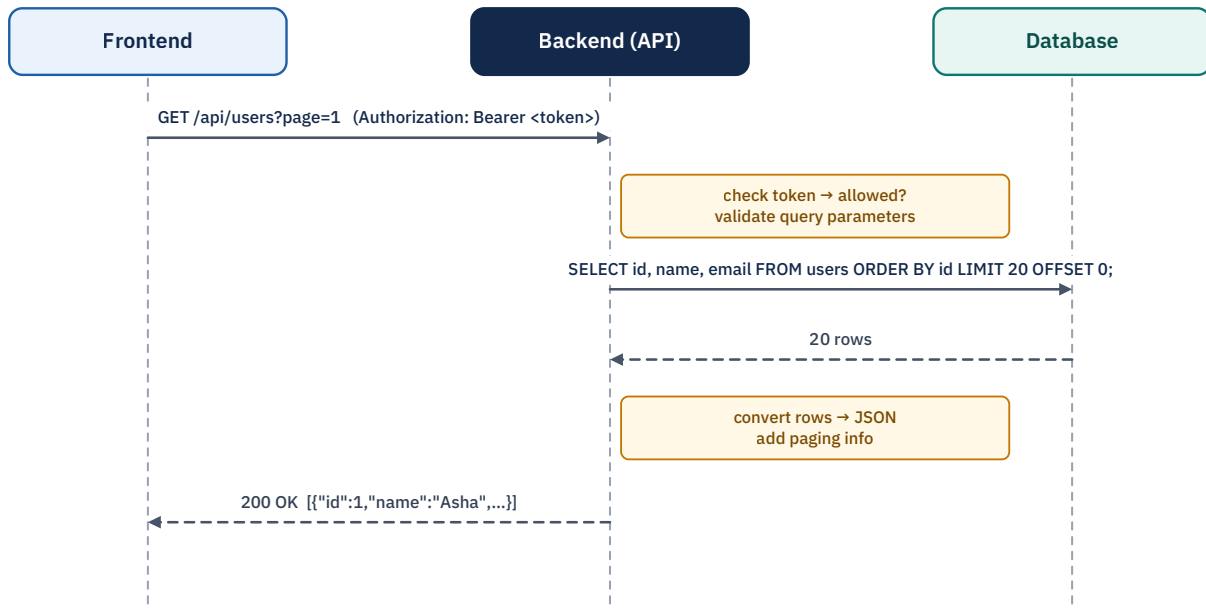


Figure 33. What happens during `GET /api/users`: authentication, validation, SQL, conversion to JSON, response.

The request the frontend sends:

```
GET /api/users?page=1&size=20 HTTP/1.1
Host: api.shop.example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Accept: application/json
```

HTTP

What a Spring Boot backend might look like (simplified):

JAVA

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping
    public Page<UserDto> listUsers(@RequestParam(defaultValue = "1") int page,
                                   @RequestParam(defaultValue = "20") int size) {
        return userService.findUsers(page, size); // runs SQL with LIMIT/OFFSET
    }

    @GetMapping("/{id}")
    public ResponseEntity<UserDto> getUser(@PathVariable long id) {
        return userService.findById(id)
            .map(ResponseEntity::ok) // 200 + JSON
            .orElse(ResponseEntity.notFound().build()); // 404
    }
}
```

LINE BY LINE

Line	What it does
@RestController	this class handles HTTP requests and returns data (JSON), not HTML pages
@RequestMapping("/api/users")	every method below lives under /api/users
@GetMapping	handles GET /api/users
@RequestParam(defaultValue = "1") int page	reads the query parameter ?page=..., defaulting to 1
userService.findUsers(page, size)	the service layer applies rules; the repository runs SELECT ... LIMIT 20 OFFSET 0
@GetMapping("/{id}") + @PathVariable long id	handles GET /api/users/42; 42 comes from the path
.map(ResponseEntity::ok)	found → 200 OK with the user as JSON
.orElse(ResponseEntity.notFound().build())	not found → 404

The response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "data": [
    {"id": 1, "name": "Asha Rao", "email": "asha@mail.com"},
    {"id": 2, "name": "Vikram Shah", "email": "vikram@mail.com"}
  ],
  "page": 1,
  "size": 20,
  "totalItems": 245,
  "next": "/api/users?page=2&size=20"
}
```

HTTP

Consuming an API from code

```
import requests

resp = requests.get(
    "https://api.shop.example.com/api/users",
    params={"page": 1, "size": 20},
    headers={"Authorization": "Bearer <token>"},
    timeout=5,
)
resp.raise_for_status()
for user in resp.json()["data"]:
    print(user["id"], user["name"])
```

PYTHON

LINE BY LINE

Line	What it does
<code>requests.get(url, params=...)</code>	builds <code>.../api/users?page=1&size=20</code> and sends a GET
<code>headers={"Authorization": ...}</code>	sends the token
<code>timeout=5</code>	give up after 5 seconds instead of hanging (Session 5.5)
<code>raise_for_status()</code>	stops here with an error if the status is 4xx or 5xx
<code>resp.json()["data"]</code>	parse the JSON body and read the list

Pagination

Never return 1 million rows in one response. Two common styles:

	Offset / page pagination	Cursor (keyset) pagination
Request	?page=3&size=20 (or ?offset=40&limit=20)	?after=last_seen_id&limit=20
SQL	ORDER BY id LIMIT 20 OFFSET 40	WHERE id > :last_id ORDER BY id LIMIT 20
Jump to page N	easy	not directly
Performance on deep pages	slow (skipped rows are still read)	fast (uses the index)
Consistency when data changes	items can shift, causing duplicates or skips	stable
Good for	admin tables, small datasets	feeds, infinite scroll, large datasets

Return paging metadata (`page` , `size` , `totalItems` , or a `next` link or cursor) so clients know how to continue.

Filtering, sorting and field selection

Use query parameters consistently: GET `/orders?status=PENDING&from=2024-04-01&sort=-order_date&fields=id,total` . A leading `-` meaning "descending" is a common convention. Validate and whitelist sort fields; never paste them into SQL as raw strings (SQL injection, Session 5.4).

API versioning

APIs change; clients (especially installed mobile apps) cannot all update at once. **Versioning** lets old clients keep working while new ones use the new contract.

Strategy	Example	Notes
URI path	<code>/api/v1/orders</code> , <code>/api/v2/orders</code>	most common; very visible and simple
Header	<code>Accept: application/vnd.shop.v2+json</code> or <code>API-Version: 2</code>	cleaner URLs; less visible
Query parameter	<code>/orders?version=2</code>	easy, but mixes versioning with filters

Rules of thumb: additive changes (a new optional field) usually don't need a new version; breaking changes (renaming or removing fields, changing types or meanings) do. Announce deprecations and keep old versions running for an agreed period.

Rate limiting

Rate limiting caps how many requests a client may make in a time window (for example 100 per minute per API key). It protects the service from overload and abuse, and enforces fair use or paid tiers.

- When the limit is exceeded, return **429 Too Many Requests**, ideally with `Retry-After` .
- Common headers: `X-RateLimit-Limit` , `X-RateLimit-Remaining` , `X-RateLimit-Reset` .
- Common algorithms: **fixed window**, **sliding window** and **token bucket** (tokens refill at a steady rate; each request spends one, which allows short bursts).
- Clients should back off and retry later, not hammer the API.

Consistent errors

Return the right status code **and** a helpful, consistent body. A widely used standard is "problem details" (RFC 9457):

```
{
  "type": "https://api.shop.example.com/errors/out-of-stock",
  "title": "Product out of stock",
  "status": 409,
  "detail": "Only 0 units of product 5 are available.",
  "instance": "/api/v1/orders"
}
```

JSON

Never return stack traces or SQL errors to clients; log them on the server with a **correlation ID** and include that ID in the response for support.

Caching INTERVIEW EXTENSION

- **Cache-Control:** `max-age=60` lets clients and CDNs reuse a response for 60 seconds.
- **ETag:** `"v7"` labels a version of the resource; the client sends **If-None-Match:** `"v7"` and gets **304 Not Modified** (no body) if nothing changed, which saves bandwidth.

Documenting and testing APIs

- **OpenAPI (Swagger):** a machine-readable description of endpoints, parameters, schemas and responses; it generates interactive docs and client code.
- **Postman** or `curl` for manual testing; automated contract and integration tests in CI.
- An **API gateway** (Kong, Apigee, AWS API Gateway, Azure API Management) often sits in front of APIs to handle authentication, rate limiting, routing, logging and versioning in one place.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"When the frontend calls `GET /api/users?page=1`, the request goes over HTTPS with a bearer token. The backend's controller receives it, the security layer validates the token, the service layer applies business rules, and the repository runs a SQL query like `SELECT ... ORDER BY id LIMIT 20 OFFSET 0`. The rows are mapped to DTOs and serialised to JSON, and the server returns 200 with the data plus paging info. For a good API I'd add pagination, filtering through query parameters, consistent error responses with proper status codes, versioning like `/v1`, and rate limiting that returns 429 when a client sends too many requests."

COMMON TRAPS & MISTAKES

- Returning unbounded lists with no pagination.
- Deep OFFSET pagination on huge tables (slow), and inconsistent pages while data changes.
- Breaking changes without a new version.
- Leaking stack traces, SQL or internal IDs in error messages.
- Clients that ignore 429 and immediately retry, making the overload worse.

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What is pagination and why is it needed?

A Returning large result sets in pages (e.g. 20 items at a time), to keep responses fast and small and protect the server and the client.

Q2 DEFINITION What is API versioning?

A Managing changes to an API's contract so old clients keep working, typically via `/v1/` in the path, a header or a query parameter.

Q3 DEFINITION What is rate limiting?

A Restricting the number of requests a client can make in a time window, returning 429 Too Many Requests when the limit is exceeded.

INTERMEDIATE**Q4 COMPARISON Offset vs cursor pagination?**

A Offset (`page` , `offset`) is simple and allows jumping to page N but is slow on deep pages and unstable when data changes. Cursor/keyset (`after=id`) is fast and stable but only moves forward and backward.

Q5 HOW What changes require a new API version?

A Breaking changes: removing or renaming fields, changing types or meanings, making optional fields required, changing URLs or behaviour. Adding optional fields or new endpoints usually doesn't.

Q6 HOW What would you include in an API error response?

A The correct status code, a stable machine-readable error type or code, a human-readable message, details about invalid fields, and a correlation/request ID, but no internal details.

Q7 DEFINITION What is OpenAPI/Swagger?

A A standard, machine-readable specification of a REST API (endpoints, parameters, schemas, responses) used to generate documentation, client SDKs and tests.

SCENARIO-BASED**Q8 SCENARIO Explain what happens when the frontend calls `GET /users` .**

A The browser sends an HTTPS GET with headers (token); the server routes it to a controller; auth is verified; the service applies rules; the repository queries the database with pagination; the rows are converted to JSON; the server returns 200 with the body; the frontend renders the list (or handles 401/500 errors).

Q9 DESIGN QUESTION Your public API is being hammered by one client's script. What do you implement?

A Per-key rate limiting (token bucket) at the gateway returning 429 with `Retry-After` , monitoring and alerts, possibly quotas by tier, and caching for expensive GET endpoints.

FOLLOW-UP / TRAP**Q10 TRAP QUESTION Is it OK to return 200 for every response and put the real status in the body?**

A No. Proper status codes let clients, proxies, monitoring and retry logic behave correctly; 200-with-error breaks all of them.

Q11 TRAP QUESTION Does adding a new optional field to a response break clients?

A Usually not, as long as clients ignore unknown fields, which they should. It's considered a non-breaking change.

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

P1. Which HTTP method and status code would you use to (a) create an order, (b) fetch order 9001, (c) delete order 9001?

P2. Identify the path and query parameters in `/api/v2/stores/17/products?category=Furniture&sort=price`.

P3. What does a 429 status mean, and what should the client do?

LEVEL 2 – INTERVIEW APPLICATION

P4. Write the JSON body for a PATCH that changes only a customer's city to "Mysuru", and explain why PUT with the same body would be risky.

P5. Design the endpoints (method + path) for a job portal: list jobs with filters, view a job, apply to a job, list my applications, withdraw an application.

LEVEL 3 – SCENARIO / PROBLEM SOLVING

P6. A mobile app shows "Something went wrong" for every error. Propose an error-response design that lets the app show useful messages.

P7. An endpoint `GET /transactions?page=50000` takes 8 seconds. Explain why, and redesign the pagination.

ANSWER / EXPLANATION

P1. (a) `POST /orders` → 201 Created; (b) `GET /orders/9001` → 200 OK (404 if missing); (c) `DELETE /orders/9001` → 204 No Content.

P2. Path parameters: `v2` (the version, part of the path) and `17` (the store ID). Query parameters: `category=Furniture` and `sort=price`.

P3. Too Many Requests: the client exceeded the rate limit. It should wait (respecting `Retry-After` if present) and retry with exponential backoff, and reduce its request rate.

P4. PATCH `/customers/42` with body `{"city": "Mysuru"}`. With PUT, the body is treated as the *complete* new representation, so fields not sent (name, email) could be cleared or reset to defaults.

P5. `GET /jobs?skill=sql&city=Pune&page=1` · `GET /jobs/{jobId}` · `POST /jobs/{jobId}/applications` · `GET /me/applications` · `DELETE /applications/{applicationId}` (or `PATCH /applications/{id}` with `{"status": "WITHDRAWN"}` to keep history).

P6. Use correct status codes (400, 401, 403, 404, 409, 422, 429, 5xx) plus a consistent JSON body: a stable code (e.g. `OUT_OF_STOCK`) the app can map to friendly messages, a human-readable message, a `fields` list for validation errors (`email: invalid format`), and a `requestId` for support. Document all codes in OpenAPI.

P7. With OFFSET pagination the database must read and discard $50,000 \times \text{page-size}$ rows before returning a page. Switch to cursor/keyset pagination: `GET /transactions?after=<last_id>&limit=50` → `WHERE id > :last_id ORDER BY id LIMIT 50`, using the primary key index, and return a `next` cursor in each response.

Session 5.4 — API Security: Authentication and Authorization

HIGH PRIORITY "Authentication vs authorization" and "how does JWT work?" are asked very often.

Two different questions

- **Authentication (AuthN): "Who are you?"** Proving identity: password, OTP, fingerprint, "Login with Google", an API key, a token.
- **Authorization (AuthZ): "What are you allowed to do?"** Checking permissions *after* identity is known: a customer may view **their own** orders; an admin may delete products.

1. Authentication — WHO are you?



2. Authorization — WHAT are you allowed to do?

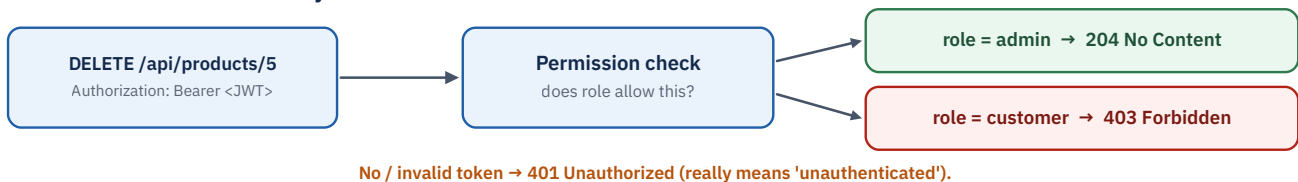


Figure 34. Authentication establishes identity and issues a token; authorization checks what that identity may do.

	Authentication	Authorization
Question	who are you?	what can you do?
Happens	first	after authentication
Based on	credentials: password, OTP, keys, certificates, tokens	roles, permissions, policies, ownership
Failure status	401 Unauthorized (really "unauthenticated")	403 Forbidden
Example	logging in with email + OTP	only admins can access <code>/admin/reports</code>
Typical tech	passwords + MFA, OAuth 2.0 / OpenID Connect, JWT	RBAC (roles), ABAC (attributes), OAuth scopes

THE OFFICE BUILDING

Showing your ID card at the reception is **authentication**: they now know who you are. Whether your card opens the server room is **authorization**: it depends on your role.

Common authentication methods for APIs

Method	How it works	Good for	Watch out
API key	a secret string sent in a header (X-API-Key: ...)	server-to-server, simple partner APIs, usage tracking	identifies an application, not a user; keep it secret; rotate it
Basic auth	Authorization: Basic base64(user:password)	internal tools, quick tests	base64 is <i>not</i> encryption: HTTPS only, avoid in production
Session cookie	server stores a session; browser sends the session ID cookie	traditional web apps	server-side state; CSRF protection needed
Bearer token (JWT)	after login the server issues a signed token; the client sends Authorization: Bearer <token>	SPAs, mobile apps, microservices	must expire; can't easily revoke before expiry; never put secrets in it
OAuth 2.0 / OpenID Connect	a trusted authorization server (Google, Azure AD/Entra ID, Okta) issues tokens	"Login with Google", delegated access, enterprise single sign-on	more moving parts; use well-tested libraries
mTLS INTERVIEW EXTENSION	both client and server present certificates	high-security service-to-service traffic (banking)	certificate management

How JWT-based authentication works

1. The user logs in (POST /auth/login with email + password, or OTP).
2. The server verifies the credentials (passwords are stored as **hashes**, e.g. bcrypt, never as plain text).
3. The server creates a **JWT (JSON Web Token)**, **signs** it with a secret or private key, and returns it.
4. The client stores it and sends it on every request: Authorization: Bearer <JWT> .
5. Each API verifies the **signature** and the **expiry**, and reads the user's ID and role from the token, with no database lookup and no server-side session (it is stateless).
6. When the short-lived access token expires (e.g. after 15–60 minutes), the client uses a longer-lived **refresh token** to get a new one.

A JWT has three base64url-encoded parts, header.payload.signature . Decoded:

```
{ "alg": "HS256", "typ": "JWT" }
```

JSON

```
{ "sub": "42", "name": "Isha Kulkarni", "role": "customer", "iat": 1713550000, "exp": 1713553600 }
```

JSON

- **Header:** the signing algorithm.
- **Payload (claims):** who the user is (sub), their role, issue time (iat) and expiry (exp).
- **Signature:** proves the token was issued by the server and not modified. Change one character of the payload and verification fails.

A JWT IS SIGNED, NOT ENCRYPTED

Anyone can base64-decode a JWT and read its payload. Never put passwords, card numbers or other secrets in it. The signature only guarantees **integrity** (no tampering), not **confidentiality**.

OAuth 2.0 in plain words

OAuth 2.0 is a delegation framework: it lets an application access resources **on a user's behalf** without ever seeing the user's password. "Allow this app to read your Google Calendar."

Role	In "Login with Google"
Resource owner	you, the user
Client	the app you are logging into
Authorization server	Google's login/consent service, which issues tokens
Resource server	the API holding your data (e.g. Google Calendar API)

The common **authorization code flow**: the app redirects you to Google → you log in and consent → Google redirects back with a short-lived **code** → the app's backend exchanges the code for an **access token** (and maybe a refresh token) → the app calls APIs with that token. **OpenID Connect (OIDC)** adds an **ID token** on top of OAuth 2.0 so the app also learns *who* you are, which is what "Login with Google" actually uses.

Authorization models

- **RBAC (Role-Based Access Control):** permissions are attached to roles (customer, support agent, admin), and users get roles. Simple and common.
- **ABAC (Attribute-Based Access Control):** decisions use attributes: "a manager may approve expenses **in their own department** under ₹50,000".
- **Ownership checks:** a customer may `GET /orders/9001` **only if order 9001 belongs to them**. Forgetting this is one of the most common API vulnerabilities (broken object-level authorization).
- **OAuth scopes:** what a token may do (`orders:read`, `orders:write`).
- Always follow **least privilege**.

Essential API security practices

Practice	Why
HTTPS everywhere	tokens and data are encrypted in transit
Hash passwords with a slow algorithm (bcrypt, Argon2)	a stolen database doesn't reveal passwords
Short-lived access tokens + refresh tokens	limits the damage of a stolen token
Validate every input; use parameterized queries	prevents SQL injection and bad data
Check authorization on every request, including object ownership	prevents users reading other users' data
Rate limiting and lockouts on login	slows brute-force attacks
Don't leak details in errors; log centrally	attackers learn less; you can investigate
Configure CORS carefully	only trusted browser origins can call the API from JavaScript

SQL injection: why parameterized queries matter

If an API builds SQL by pasting user input into a string, a malicious input becomes part of the SQL itself. Suppose the code is:

```
# DANGEROUS: user input pasted into SQL
query = f"SELECT * FROM customers WHERE email = '{email_from_user}'"
```

PYTHON

If an attacker sends the email `' OR '1'='1'`, the database receives this query and returns **every customer**:

```
SELECT customer_id, customer_name, email
FROM customers
WHERE email = '' OR '1'='1';
```

SQL · POSTGRESQL

OUTPUT

customer_id	customer_name	email
1	Aarav Patel	aarav@mail.com
2	Isha Kulkarni	isha@mail.com
3	Kabir Khan	kabir@mail.com
4	Diya Joshi	diya@mail.com
5	Vivaan Shah	vivaan@mail.com
6	Saanvi Rao	saanvi@mail.com
7	Reyansh Bose	NULL

7 rows

The fix is a **parameterized query**, where the driver sends the value separately from the SQL text, so it can never change the query's structure:

```
# SAFE: the value is passed as a parameter, not as SQL text
cur.execute("SELECT * FROM customers WHERE email = %s", (email_from_user,))
```

PYTHON

ORMs (JPA/Hibernate, Django ORM) parameterize automatically, unless you concatenate strings into native queries yourself.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"Authentication is proving who you are, for example logging in with a password and OTP; authorization is deciding what you're allowed to do once we know who you are, like only admins deleting products. If authentication fails the API returns 401; if the user is known but not allowed, it returns 403. A common setup is token-based: after login the server issues a signed JWT with the user ID, role and an expiry, and the client sends it as a Bearer token on every request. The API verifies the signature and expiry and then checks roles and ownership. For third-party login we use OAuth 2.0 with OpenID Connect, so the app never sees the user's Google password."

COMMON TRAPS & MISTAKES

- Mixing up authentication and authorization, or 401 and 403.
- "JWTs are encrypted." They are signed; the payload is readable.
- Long-lived tokens with no expiry or rotation.
- Checking the role but not **ownership** (any customer can read `/orders/{anyId}`).
- Basic auth or tokens over plain HTTP.
- Building SQL with string concatenation.

INTERVIEW QUESTIONS

BASIC

Q1 **COMPARISON** Authentication vs authorization?

A Authentication verifies identity (who you are); authorization determines permissions (what you can do). Authentication comes first.

Q2 **DEFINITION** What is an API key?

A A secret identifier sent with requests (usually in a header) that identifies the calling application, used for simple access control, quotas and tracking.

Q3 **DEFINITION** What is a JWT?

A A JSON Web Token: a compact, signed token (`header.payload.signature`) carrying claims such as user ID, role and expiry, used for stateless authentication.

INTERMEDIATE

Q4 **HOW** How does token-based authentication work?

A The user logs in; the server verifies the credentials and issues a signed, expiring token; the client sends it in the Authorization header; each API verifies the signature and expiry and reads the identity and roles from it.

Q5 **DEFINITION** What is OAuth 2.0?

A An authorization framework that lets a client app obtain limited access tokens to a user's resources from an authorization server, without handling the user's password. OpenID Connect adds identity (ID tokens) on top.

Q6 **COMPARISON** Session-based vs token-based authentication?

A Sessions store state on the server and send a session ID cookie (easy to revoke, needs shared storage to scale); tokens are self-contained and stateless (scale easily, but harder to revoke, so they must be short-lived).

Q7 **COMPARISON** RBAC vs ABAC?

A RBAC grants permissions through roles; ABAC evaluates policies on attributes of the user, the resource and the context (department, amount, time), which is more fine-grained.

Q8 **DEFINITION** What is SQL injection and how do you prevent it?

A An attack where input is interpreted as SQL because it was concatenated into a query. Prevent it with parameterized queries or prepared statements (or an ORM), input validation and least-privilege database accounts.

SCENARIO-BASED

Q9 **SCENARIO** A customer can view another customer's order by changing the ID in the URL. What's wrong and how do you fix it?

A Broken object-level authorization: the API authenticates the user but doesn't check ownership. On every request, verify the order's `customer_id` matches the token's user (or that the user has an admin role) and return 403/404 otherwise.

Q10 **DESIGN QUESTION** How would you secure an internal reporting API used by a BI tool?

A HTTPS; a service account with an API key or OAuth client-credentials token; least-privilege read-only scope; network restrictions (private network, IP allow-list); rate limiting; audit logging; read from a replica or the warehouse rather than OLTP.

FOLLOW-UP / TRAP

Q11 **TRAP QUESTION** Why is 401 named "Unauthorized" if it's about authentication?

A A historical naming quirk in the HTTP specification. In practice 401 means "not authenticated", and 403 means "authenticated but not authorized".

Q12 **TRAP QUESTION** How do you log out a user with a stateless JWT?

A You can't truly invalidate it before expiry without state. Use short expiry plus refresh-token revocation, or keep a deny-list of token IDs (jti) until they expire.

Session 5.5 — API Integration and Reliability

Integrating with external APIs

Real applications constantly call **third-party APIs**: payment gateways, SMS/email providers, maps, KYC/identity verification, GST validation, shipping partners, CRM systems. An integration usually involves:

1. **Reading the documentation and contract** (endpoints, auth, limits, error codes, sandbox).
2. **Authenticating** (API key or OAuth client credentials), keeping secrets in a vault or environment config, never in code.
3. **Mapping data** between your model and theirs (field names, formats, currencies, time zones).
4. **Handling failure**: the network *will* fail, the provider *will* be slow sometimes.
5. **Monitoring**: logs, metrics, alerts on error rates and latency.

Synchronous vs asynchronous communication

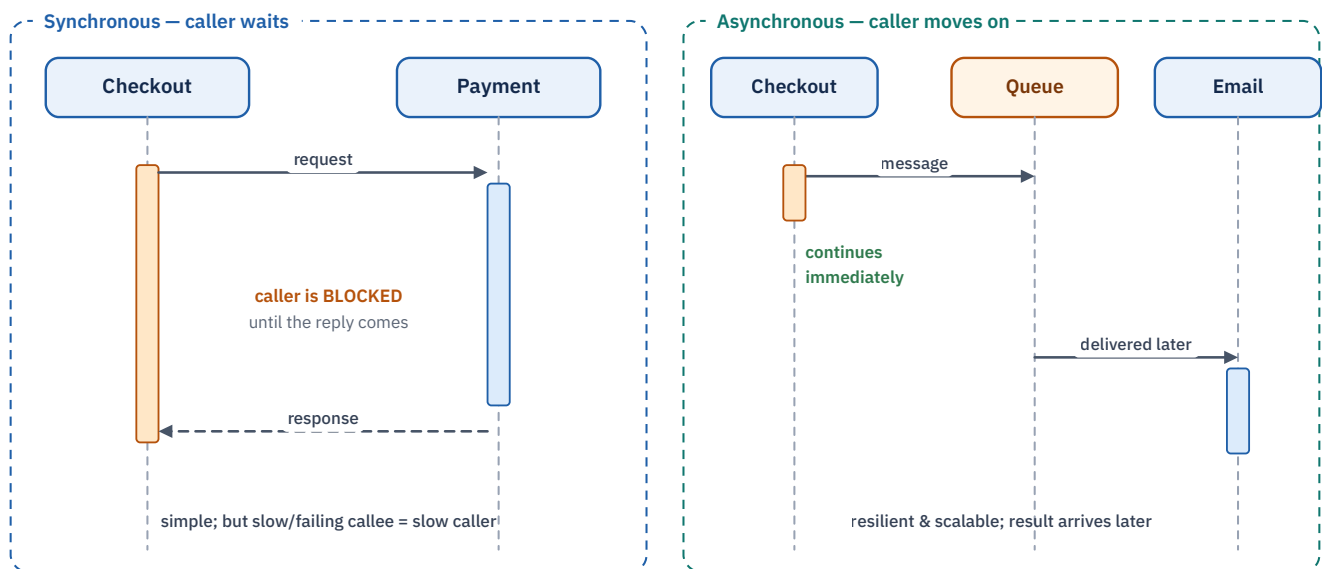


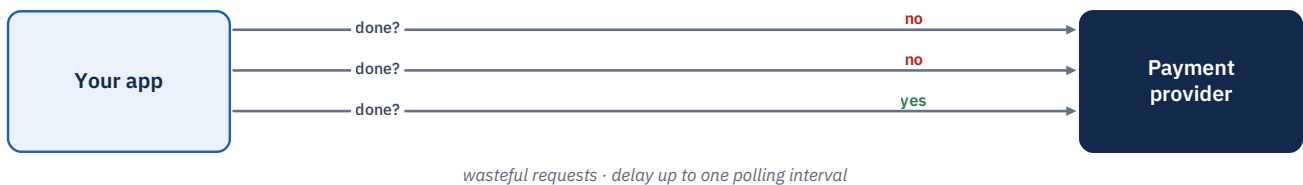
Figure 35. Synchronous calls block the caller until the reply arrives; asynchronous messages let the caller continue immediately.

	Synchronous	Asynchronous
How	the caller sends a request and waits for the response	the caller sends a message or event and continues ; the result arrives later (callback, webhook, polling, event)
Example	checkout calls the payment API and waits for approve/decline	after the order, send the confirmation email through a queue
Pros	simple, immediate result, easy to reason about	resilient (survives the other side being down), absorbs spikes, scales
Cons	caller is slowed or broken by a slow or failing dependency; tight coupling	eventual results, harder debugging, needs queues and duplicate handling
Use when	the user needs the answer now to continue	the work can happen later, or takes long (reports, emails, video processing)

A common hybrid: the API accepts a long job and returns **202 Accepted** with a job URL (`/jobs/77`); the client polls it or receives a webhook when the job is done.

Webhooks vs polling

Polling — client keeps asking



Webhook — server calls you when something happens



Figure 36. Polling repeatedly asks "is it done?"; a webhook lets the provider call you when it is.

- **Polling:** the client asks repeatedly ("is payment done?"). Simple, but wasteful and delayed.
- **Webhook:** you register a URL once; the provider sends an HTTP POST to it when an event happens ("payment.captured").

Webhook receivers must: verify the provider's **signature** (so attackers can't fake events); respond **2xx quickly** and do heavy work asynchronously; be **idempotent**, because providers retry and duplicates happen; and handle events arriving **out of order**.

When APIs fail

Failure	Example	Response
Timeout	the provider takes 40 seconds	always set connect and read timeouts; fail fast
Network error	connection reset, DNS failure	retry (if safe) with backoff
5xx	500, 502, 503, 504	retry with backoff; it may be temporary
429	rate limited	wait for <code>Retry-After</code> , then retry; reduce the request rate
4xx (other)	400, 401, 403, 404, 422	do not retry blindly: fix the request, credentials or data
Partial failure	payment succeeded but our DB write failed	idempotency, reconciliation jobs, status tracking

Retries with exponential backoff and jitter

Exponential backoff: wait 1s, 2s, 4s ... (+ random jitter) and give up after a few tries.

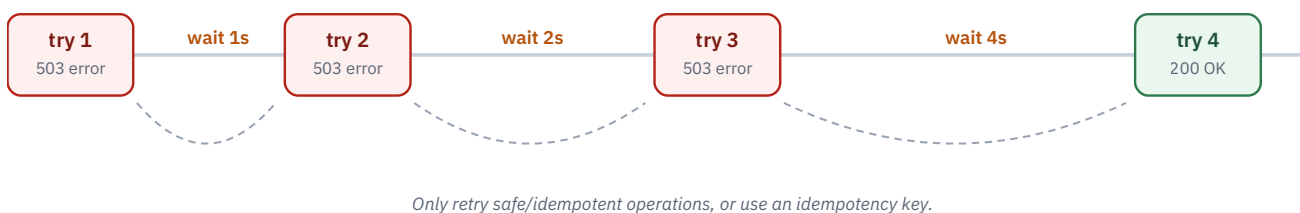


Figure 37. Retry after 1 s, 2 s, 4 s... with a little randomness (jitter), and give up after a few attempts.

- **Exponential backoff:** wait 1 s, 2 s, 4 s, 8 s between attempts, so a struggling service gets room to recover.
- **Jitter:** add randomness so thousands of clients don't all retry at the same instant (the "thundering herd").

- **Limit** the attempts, then fail gracefully (show a message, queue for later, alert).
- **Only retry operations that are safe to repeat:** idempotent requests, or POSTs protected by an idempotency key.

PYTHON

```
import random
import time

import requests

def call_with_retry(url, payload, idempotency_key, max_attempts=4):
    for attempt in range(1, max_attempts + 1):
        try:
            resp = requests.post(url, json=payload, timeout=5,
                                headers={"Idempotency-Key": idempotency_key})
            if resp.status_code < 500 and resp.status_code != 429:
                return resp  # success or a client error: don't retry
        except requests.exceptions.RequestException:
            pass  # network problem or timeout: retry
    if attempt == max_attempts:
        raise RuntimeError("payment service unavailable, try later")
    time.sleep(2 ** (attempt - 1) + random.uniform(0, 0.5))  # 1s, 2s, 4s + jitter
```

LINE BY LINE

Line	What it does
<code>for attempt in range(1, max_attempts + 1)</code>	try at most 4 times
<code>headers={"Idempotency-Key": ...}</code>	the same key on every attempt, so the server can detect repeats
<code>timeout=5</code>	never wait more than 5 seconds for one attempt
<code>if resp.status_code < 500 and resp.status_code != 429:</code> <code>return resp</code>	success (2xx) or a client error (4xx): retrying won't help, so return
<code>except requests.exceptions.RequestException</code>	timeouts and connection errors are worth retrying
<code>time.sleep(2 ** (attempt - 1) + random.uniform(0, 0.5))</code>	exponential backoff plus jitter

Idempotency and idempotency keys

Idempotency: performing an operation several times has the same effect as performing it once (Session 5.2). It is the safety net that makes retries harmless.

The danger case: you send `POST /payments` to charge ₹500, the payment succeeds, but the **response is lost** (timeout). Did it work? Retrying blindly could charge the customer twice.

The solution is an **idempotency key**: a unique ID (e.g. a UUID) the client generates per logical operation and sends with every retry. The server stores the key with the result; when the same key arrives again, it **returns the stored result instead of charging again**. Stripe, Razorpay and many payment APIs use exactly this design.

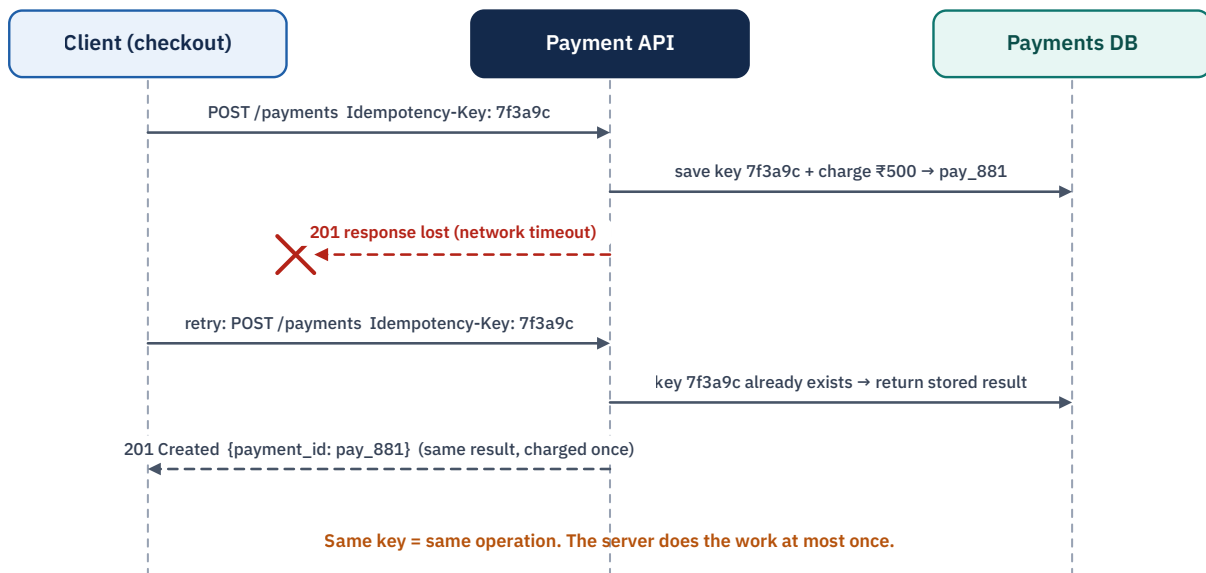


Figure 38. The response is lost, the client retries with the same key, and the server returns the saved result: the customer is charged once.

On the database side, the same idea is enforced with a unique constraint, e.g. `UNIQUE (idempotency_key)` on a payments table, plus `INSERT ... ON CONFLICT DO NOTHING` (Session 3.4).

Circuit breakers and fallbacks INTERVIEW EXTENSION

A **circuit breaker** stops calling a dependency that keeps failing: after N failures it "opens" and fails fast for a cool-down period, then lets a few test requests through. This protects your threads and the struggling service. Combine it with **fallbacks** (cached data, a default response, or "we'll notify you") and **bulkheads** (separate thread pools per dependency).

IN A REAL PROJECT

"In our order service, the call to the payment gateway has a 5-second timeout and up to three retries with exponential backoff, always sending the same idempotency key, which we also store with a unique constraint. Confirmation emails go through a queue, so a slow email provider never blocks checkout. Webhooks from the gateway are signature-verified and processed idempotently, and a nightly reconciliation job compares our payments table with the gateway's settlement report."

" HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"When integrating with an external API I assume it will sometimes be slow or fail. I set timeouts, retry only on transient errors like timeouts, 5xx or 429, using exponential backoff with jitter and a retry limit, and I never retry blindly on other 4xx errors. Retries are only safe if the operation is idempotent, so for POST requests like payments we send an idempotency key; if the response is lost and we retry, the server returns the original result instead of charging twice. For work that doesn't need an immediate answer, like emails, I prefer asynchronous messaging or webhooks over synchronous calls."

COMMON TRAPS & MISTAKES

- No timeout: one slow provider hangs every request thread.
- Retrying non-idempotent POSTs without an idempotency key leads to duplicate payments and orders.
- Retrying 400/401/403/422 errors, which will never succeed.
- Immediate, synchronized retries that create a thundering herd.

- Webhook handlers that aren't idempotent or don't verify signatures.
- Calling an external API while holding a database transaction open.

INTERVIEW QUESTIONS

BASIC

Q1 **COMPARISON** Synchronous vs asynchronous communication?

- A** Synchronous: the caller waits for the response before continuing (simple, immediate, but coupled).
Asynchronous: the caller sends a message and continues; the result comes later (resilient and scalable, but eventual).

Q2 **DEFINITION** What is a webhook?

- A** An HTTP callback: you register a URL, and the provider sends a POST request to it when an event happens.

Q3 **DEFINITION** What is exponential backoff?

- A** A retry strategy where the wait doubles after each failure (1 s, 2 s, 4 s...), usually with random jitter and a maximum number of attempts.

INTERMEDIATE

Q4 **DEFINITION** What is an idempotency key?

- A** A unique client-generated ID sent with a non-idempotent request (like a payment POST); the server stores the result per key and returns the same result for repeats, so retries don't duplicate the operation.

Q5 **HOW** Which failures should you retry?

- A** Timeouts, network errors, 5xx (especially 502, 503, 504) and 429 (after `Retry-After`). Not 400, 401, 403, 404 or 422; those need a fix, not a retry.

Q6 **COMPARISON** Webhooks vs polling?

- A** Polling repeatedly asks for status (simple but wasteful and delayed); webhooks push events when they happen (efficient, near real-time, but you must secure and idempotently process them).

Q7 **DEFINITION** What is a circuit breaker?

- A** A pattern that stops calls to a failing dependency after repeated errors and fails fast for a while, periodically testing whether it has recovered.

SCENARIO-BASED

Q8 **SCENARIO** The payment API times out. The customer clicks "Pay" again. How do you avoid charging twice?

- A** Use one idempotency key per checkout attempt, reused on retries, and a unique constraint on it in the payments table. Check the payment's status with the provider (or wait for its webhook) before creating a new charge, and show "payment pending" instead of encouraging a retry.

Q9 **DESIGN QUESTION** An order confirmation must send an email and an SMS. Should checkout call these APIs synchronously?

- A** No. Publish an `OrderPlaced` event or enqueue jobs; notification workers send email and SMS with retries. Checkout stays fast and works even if the SMS provider is down.

Q10 **SCENARIO** Your webhook endpoint receives the same "payment.captured" event three times. Is something broken?

A Not necessarily; providers retry until they get a 2xx and may send duplicates. Store processed event IDs and ignore repeats (idempotent processing), and return 2xx quickly.

FOLLOW-UP / TRAP

Q11 **TRAP QUESTION** Is it safe to retry a PUT request?

A Yes in principle, since PUT is idempotent (the same full update applied twice gives the same state). PATCH and POST need more care.

Q12 **TRAP QUESTION** Why add jitter to backoff?

A Without randomness, many clients that failed together retry at the same moments and overload the recovering service again.

Session 5.6 – Data Integration

Simple meaning, revisited

Data integration is combining data from different sources into a unified, consistent and usable form, either to keep operational systems in sync (CRM ↔ order system) or to feed analytics (everything → warehouse). Session 1.2 introduced *why*; this session covers *how*.

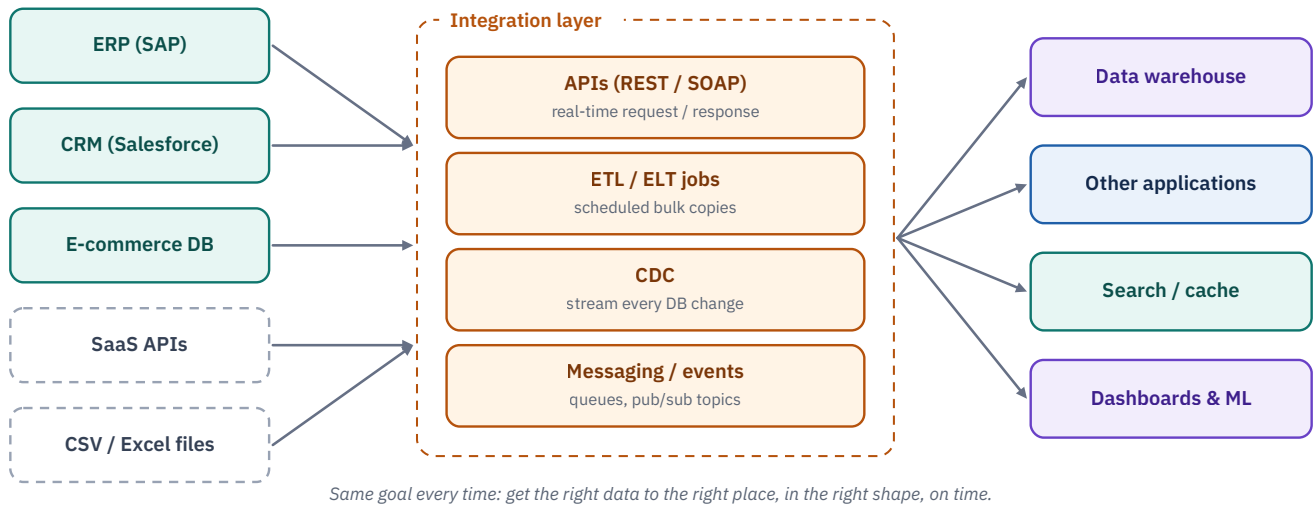


Figure 39. Many sources, one integration layer (APIs, ETL/ELT, CDC, messaging) and many targets.

Integration approaches

Approach	How it works	Latency	Typical use
API-based	systems call each other's REST/SOAP APIs	real time	create a customer in the CRM when they sign up
File-based	exports such as CSV, Excel or XML, exchanged via SFTP or cloud storage	hours (batch)	bank statements, partner catalogues, legacy systems
ETL / ELT	scheduled jobs extract, transform and load data into a target	minutes to hours	loading the warehouse every night (Module 6)
CDC (Change Data Capture)	read the source database's change log and stream every insert, update and delete	seconds	keeping a warehouse, search index or cache in sync
Messaging / events	systems publish events to a broker; others subscribe	near real time	OrderPlaced → inventory, email, analytics
Data virtualization INTERVIEW EXTENSION	query sources in place through a virtual layer, without copying	query time	quick unified views; limited by source performance

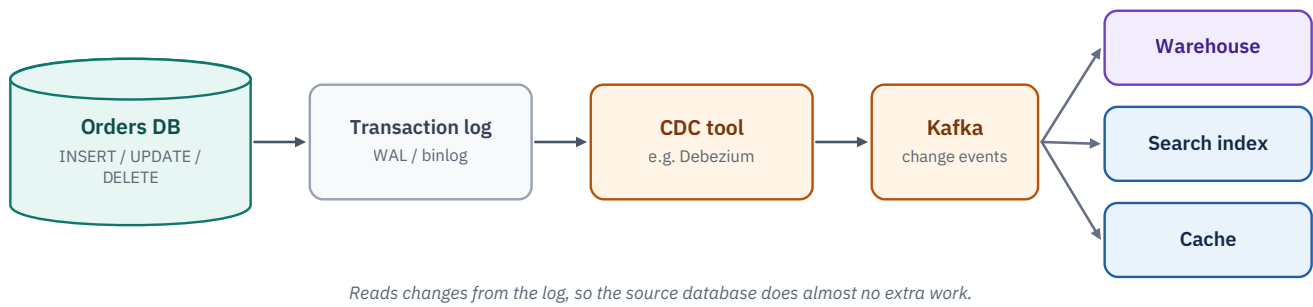


Figure 40. Change Data Capture reads the database's transaction log and streams each change to other systems.

Why CDC is popular: it captures *every* change, including deletes, in near real time, without adding load from frequent polling queries or requiring an `updated_at` column. Tools: Debezium, AWS DMS, Fivetran, Oracle GoldenGate, SQL Server CDC.

Full vs incremental loads

- **Full load:** copy everything every time. Simple, but slow and expensive as data grows.
- **Incremental load:** copy only what changed since the last run, using a **watermark** (the highest `updated_at` or ID already loaded) or CDC.

SQL · POSTGRESQL

```
-- incremental extract: the previous load finished with orders up to 31 March
SELECT order_id, order_date, status, total_amount
FROM orders
WHERE order_date > DATE '2024-03-31'
ORDER BY order_date, order_id;
```

OUTPUT

order_id	order_date	status	total_amount
113	2024-04-04	SHIPPED	17998.00
114	2024-04-11	PENDING	3235.00
115	2024-04-19	PENDING	5498.00
116	2024-04-19	DELIVERED	15999.00

4 rows

In real systems the watermark is usually an `updated_at` timestamp rather than the order date, so that *changes* to old orders (a status moving to DELIVERED) are picked up too. Loading uses **upserts** (`INSERT ... ON CONFLICT DO UPDATE` or `MERGE`) so re-running a load never creates duplicates: the load is **idempotent**.

Data formats you will meet

Format	Shape	Strengths	Weaknesses
CSV	rows of text values	universal, human-readable	no types, quoting and encoding issues, no nesting
JSON	nested key-value	APIs, flexible, readable	verbose; types limited; no schema by default
XML	nested tags	strict schemas (XSD), common in enterprise and banking	verbose, heavier to parse
Parquet	columnar, binary	compressed, fast analytics, schema included	not human-readable; for batch analytics
Avro	row-based, binary, schema	streaming (Kafka), schema evolution	not human-readable

Data mapping and transformation

Integration projects are built on a **source-to-target mapping** document. For example, CRM contacts to the warehouse customer dimension:

Source (CRM)	Target (dim_customer)	Rule
Contact.Id	crm_id	copy as is
FirstName + LastName	customer_name	concatenate with a space, trim, Title Case
Email	email	lower-case, trim; reject if invalid format
MailingCity	city	map variants (Bombay → Mumbai); unknown → 'Unknown'
CreatedDate (UTC)	signup_date	convert to IST date
IsDeleted	(filter)	exclude deleted contacts

Matching records across systems

The same customer appears in the shop database and in the CRM with slightly different data. Matching on a normalized key (here, lower-cased email) shows what matches and what exists on only one side, using the FULL OUTER JOIN from Session 3.6:

```
WITH crm_contacts (crm_id, contact_name, email) AS (
  VALUES ('C-01', 'Aarav Patel', 'AARAV@MAIL.COM'),
          ('C-02', 'Isha K.', 'isha@mail.com'),
          ('C-03', 'Rohit Sinha', 'rohit@mail.com')
)
SELECT c.customer_id, c.customer_name, k.crm_id, k.contact_name,
       CASE WHEN c.customer_id IS NULL THEN 'only in CRM'
            WHEN k.crm_id IS NULL THEN 'only in shop'
            ELSE 'matched' END AS match_status
FROM customers c
FULL OUTER JOIN crm_contacts k ON LOWER(c.email) = LOWER(k.email)
ORDER BY match_status, c.customer_id;
```

SQL · POSTGRESQL

OUTPUT

customer_id	customer_name	crm_id	contact_name	match_status
1	Aarav Patel	C-01	Aarav Patel	matched
2	Isha Kulkarni	C-02	Isha K.	matched
NULL	NULL	C-03	Rohit Sinha	only in CRM
3	Kabir Khan	NULL	NULL	only in shop
4	Diya Joshi	NULL	NULL	only in shop
5	Vivaan Shah	NULL	NULL	only in shop
6	Saanvi Rao	NULL	NULL	only in shop
7	Reyansh Bose	NULL	NULL	only in shop

8 rows

LINE BY LINE

Part	What it does
WITH crm_contacts (...) AS (VALUES ...)	a small inline table standing in for the CRM extract
FULL OUTER JOIN ... ON LOWER(c.email) = LOWER(k.email)	match on normalized email; keep unmatched rows from both sides
CASE ...	label each row: matched, only in the shop, or only in the CRM

Aarav matched despite the upper-case email, thanks to normalization. Rohit exists only in the CRM, and five shop customers have no CRM record. The output of such a query drives decisions: create the missing records, review near-matches, or build a **golden record**.

MASTER DATA MANAGEMENT (MDM)

MDM creates a single trusted "golden record" for key entities (customer, product, supplier) across systems, using matching rules (exact and fuzzy), survivorship rules (which source wins for each field), and stewardship (people who resolve conflicts).

Data quality: trust is the product

Dimension	Question	Example check
Completeness	is required data present?	customers with no email
Uniqueness	are there duplicates?	the same email several times
Validity	does data follow the rules and formats?	status in the allowed list; email pattern
Consistency	do related values agree?	order total = sum of its lines
Accuracy	does it reflect reality?	the address actually exists
Timeliness	is it fresh enough?	latest order date vs today

Checks like these can run as SQL after every load. One query can produce a small quality report:

```

SELECT 'customers: missing email' AS check_name, COUNT(*) AS failures
FROM customers WHERE email IS NULL
UNION ALL
SELECT 'leads: duplicate emails', COUNT(*) - COUNT(DISTINCT LOWER(email))
FROM leads
UNION ALL
SELECT 'orders: total does not match lines', COUNT(*)
FROM orders o
WHERE o.total_amount <> (SELECT SUM(i.quantity * i.unit_price)
                        FROM order_items i WHERE i.order_id = o.order_id)
UNION ALL
SELECT 'employees: no department', COUNT(*)
FROM employees WHERE dept_id IS NULL
UNION ALL
SELECT 'orders: dated after 2024-12-31', COUNT(*)
FROM orders WHERE order_date > DATE '2024-12-31';

```

SQL · POSTGRESQL

OUTPUT

check_name	failures
customers: missing email	1
leads: duplicate emails	3
orders: total does not match lines	0
employees: no department	1
orders: dated after 2024-12-31	0

5 rows

Non-zero failures become alerts or tickets. Tools such as dbt tests and Great Expectations automate exactly this.

“ HOW I WOULD EXPLAIN THIS IN AN INTERVIEW

"Data integration is bringing data from different systems together consistently. Depending on how fresh it needs to be, I'd use APIs for real-time system-to-system calls, file transfers for simple batch exchanges, ETL or ELT jobs for loading a warehouse, change data capture to stream database changes with low latency, or events through a message broker. The real work is in mapping and transforming fields, matching records across systems, for example on a normalized email, loading incrementally with upserts so re-runs are safe, and checking data quality: completeness, uniqueness, validity and consistency."

COMMON TRAPS & MISTAKES

- Full reloads forever: they get slower and costlier as data grows.
- Incremental loads based on creation date only, missing updates (use `updated_at` or CDC).

- Non-idempotent loads that duplicate data when a job is re-run.
- Matching records on raw values without normalizing case, spaces and formats.
- Treating data quality as a one-time cleanup instead of continuous checks.

INTERVIEW QUESTIONS

BASIC

Q1 **DEFINITION** What is Change Data Capture (CDC)?

A A technique that captures inserts, updates and deletes from a source database, usually by reading its transaction log, and streams them to other systems in near real time.

Q2 **COMPARISON** Full load vs incremental load?

A A full load copies all data every run; an incremental load copies only new or changed data since the last run, using a watermark or CDC.

Q3 **DEFINITION** Name four data quality dimensions.

A Completeness, uniqueness, validity, consistency (also accuracy and timeliness).

INTERMEDIATE

Q4 **WHY** Why prefer CDC over querying `updated_at` regularly?

A CDC captures deletes and every intermediate change, adds little load to the source, has low latency and doesn't depend on applications maintaining `updated_at` correctly.

Q5 **DEFINITION** What is a source-to-target mapping?

A A document listing each target field, its source field(s) and the transformation rule (format, lookup, default, filter), which guides building and testing integrations.

Q6 **HOW** How do you make a data load idempotent?

A Use upserts or MERGE keyed on a business key, or delete-and-reload a partition (e.g. one day) inside a transaction, so re-running produces the same result without duplicates.

SCENARIO-BASED

Q7 **SCENARIO** How would you integrate CRM customer data with the e-commerce order system?

A Agree on the master system and a matching key (normalized email or a shared customer ID); sync new and updated customers by API or CDC; map fields (names, cities, time zones); match and de-duplicate; load with upserts; add quality checks and a reconciliation report; and for analytics, land both in the warehouse and build a unified customer dimension.

Q8 **SCENARIO** A nightly load failed halfway. How do you recover without duplicates?

A Design loads to be idempotent (upserts, or replace by date partition in one transaction), keep the watermark updated only after success, then simply re-run the job for the failed window.

FOLLOW-UP / TRAP

Q9 **TRAP QUESTION** Is real-time integration always the goal?

A No. Pick the latency the business needs; batch is simpler and cheaper and is perfectly fine for many reports.

Q10 **TRAP QUESTION** Two systems both have `customer_id = 105`. Is it the same customer?

A Not necessarily: IDs are local to each system. Use a shared key or a cross-reference (mapping) table.

PRACTICE QUESTIONS**LEVEL 1 – BASIC UNDERSTANDING**

- P1.** Name five ways to move data between systems.
- P2.** Which HTTP status should an API return when a user is logged in but lacks permission?
- P3.** What are the three parts of a JWT?

LEVEL 2 – INTERVIEW APPLICATION

- P4.** Explain idempotency keys to a non-technical manager using the "double payment" example.
- P5.** Write a SQL check that returns how many leads have an email that is not lower-case.

LEVEL 3 – SCENARIO / PROBLEM SOLVING

- P6.** Design the integration between a hospital's lab system and its doctor app so results appear within a minute, survive outages and never duplicate.
- P7.** A partner sends a daily CSV of 2 million product prices. Describe a robust load process, from file arrival to the updated product table.

ANSWER / EXPLANATION

- P1.** API calls, file transfer, ETL/ELT pipelines, CDC/replication, and messaging/events (data virtualization is a sixth).
- P2.** 403 Forbidden.
- P3.** Header (algorithm), payload (claims such as user ID, role, expiry) and signature, separated by dots and base64url-encoded.
- P4.** "If our payment request times out, we don't know whether the customer was charged. So every payment attempt carries a unique reference number. If we have to resend the request, we send the same number, and the payment provider recognises it and replies 'already done', instead of charging again."
- P5.**

```
SELECT COUNT(*) AS not_lowercase
FROM leads
WHERE email <> LOWER(email);
```

SQL · PostgreSQL

OUTPUT

not_lowercase
0

1 row

All sample emails are already lower-case, so the check passes with 0.

- P6.** The lab system publishes a `ResultReady` event (or CDC captures new result rows) to a durable message broker. A consumer service transforms the result to the app's format and upserts it into the app database keyed by `result_id` (idempotent, so duplicates are harmless), then notifies the doctor. If the app database is down, messages wait in the queue and are retried; repeated failures go to a dead-letter queue with alerts. Latency is seconds, which meets the one-minute target.
- P7.** (1) The file lands in a raw storage area; record its arrival. (2) Validate: header, row count, required fields, number formats; quarantine bad rows. (3) Bulk-load it into a staging table (PostgreSQL `COPY`). (4) Transform: trim, cast, map product codes. (5) `INSERT ... ON CONFLICT (product_code) DO UPDATE` into the product

table, ideally only for changed prices. (6) Run quality checks (e.g. price > 0, no unexpected 50% swings) and a reconciliation of counts. (7) Commit, archive the file and log the run. Re-running the same file gives the same result, so the process is idempotent.

Session 5.7 — Module 5 Summary & Rapid Revision

MODULE 5 SUMMARY

MOST IMPORTANT CONCEPTS

- **API** = a contract between systems; **client-server**: clients request, servers process and respond; REST servers are stateless.
- **HTTP**: method + URL + headers + body → status + headers + body; HTTPS = HTTP over TLS.
- **Methods**: GET (read), POST (create, not idempotent), PUT (replace, idempotent), PATCH (partial), DELETE (idempotent).
- **Status codes**: 200, 201, 204 · 400, 401, 403, 404, 409, 422, 429 · 500, 502, 503, 504.
- **REST**: resources as nouns, uniform interface, statelessness, JSON, proper status codes; plus pagination, filtering, versioning, rate limiting and consistent errors.
- **AuthN vs AuthZ**: who you are vs what you may do; API keys, sessions, JWT, OAuth 2.0 / OIDC; RBAC/ABAC; ownership checks; parameterized queries against SQL injection.
- **Reliability**: timeouts, retries with exponential backoff + jitter, idempotency keys, webhooks, circuit breakers; sync vs async.
- **Data integration**: API, file, ETL/ELT, CDC, events; incremental loads with watermarks; mapping; record matching; data quality.

WHAT TO MEMORIZE

- PUT vs PATCH; 401 vs 403; which methods are idempotent.
- The key status codes and when to use each.
- JWT = header.payload.signature (signed, not encrypted).

WHAT TO UNDERSTAND

- The end-to-end flow of GET /users .
- Why retries require idempotency.
- When to choose synchronous vs asynchronous integration.

MOST COMMON INTERVIEW QUESTIONS

- What is REST? · PUT vs PATCH? · 401 vs 403? · Path vs query parameters? · Is POST idempotent? · How does JWT authentication work? · What is OAuth? · How do you handle API failures? · What is CDC?

COMMON MISTAKES

- 200 for errors · verbs in URLs · no timeouts · retrying 4xx · retrying POST without a key · JWT treated as encrypted · missing ownership checks · string-built SQL.

RAPID REVISION CHECKLIST

- | | |
|---|--|
| ☐ I can define an API and explain client-server | ☐ I can walk through GET /users end to end |
| ☐ I can describe the parts of an HTTP request and response | ☐ I can explain pagination, versioning and rate limiting |
| ☐ I can explain path vs query parameters | ☐ I can explain authentication vs authorization |
| ☐ I know each HTTP method, and whether it is safe and idempotent | ☐ I can explain how JWT and OAuth 2.0 work |
| ☐ I can explain PUT vs PATCH with an example | ☐ I can explain and prevent SQL injection |
| ☐ I know the important status codes (including 401 vs 403) | ☐ I can compare synchronous and asynchronous communication |
| ☐ I can explain REST principles and good resource naming | ☐ I can explain retries, backoff, jitter and idempotency keys |

- ☐ I can compare webhooks and polling
- ☐ I can design an incremental, idempotent load
- ☐ I can list data integration approaches and explain CDC
- ☐ I can name the data quality dimensions